tapis2pdf

Joe Stubbs

Mar 20, 2025

INTRODUCTION:

1 Getting Started **3**
- 1.1 Getting Ready . . . 4
- 1.2 Tapis Quickstart . . . 4

2 Technical Guide - Overview **13**

3 Sites, Tenancy and Authentication **15**
- 3.1 Sites . . . 15
- 3.2 Tenants . . . 16
- 3.3 Authentication . . . 17

4 Systems **25**
- 4.1 Overview . . . 25
- 4.2 Model . . . 25
- 4.3 Getting Started . . . 27
- 4.4 Minimal Definition and Restrictions . . . 34
- 4.5 Permissions . . . 35
- 4.6 Sharing . . . 35
- 4.7 Authentication Credentials . . . 36
- 4.8 Runtime . . . 40
- 4.9 Logical Batch Queue . . . 40
- 4.10 Batch Scheduler Profile . . . 40
- 4.11 Deletion . . . 41
- 4.12 System Attributes Table . . . 41
- 4.13 Credential Attributes Table . . . 52
- 4.14 KeyValuePair Attributes Table . . . 53
- 4.15 LogicalQueue Attributes Table . . . 55
- 4.16 Scheduler Profile Attributes Table . . . 56
- 4.17 Searching . . . 56
- 4.18 Sort, Limit, Select and ListType . . . 59
- 4.19 Tapis Responses . . . 61

5 Files **63**
- 5.1 Overview . . . 63
- 5.2 Basic File Operations . . . 63
- 5.3 Content . . . 69
- 5.4 Transfers . . . 70
- 5.5 Support for Globus . . . 73
- 5.6 File Permissions . . . 74
- 5.7 File Path Sharing . . . 74

5.8 PostIts . . . 76

6 Applications **83**
6.1 Overview . . . 83
6.2 Model . . . 83
6.3 Getting Started . . . 85
6.4 Minimal Definition and Restrictions . . . 90
6.5 Version . . . 90
6.6 Containerized Application . . . 90
6.7 Directory Semantics and Macros . . . 90
6.8 Data Transfer Node (DTN) Support . . . 91
6.9 Permissions . . . 91
6.10 Sharing . . . 91
6.11 Deletion . . . 91
6.12 Application Attributes Table . . . 93
6.13 JobAttributes Table . . . 95
6.14 ParameterSet Attributes Table . . . 97
6.15 ArchiveFilter Attributes Table . . . 98
6.16 Arg Attributes Table . . . 98
6.17 KeyValuePair Attributes Table . . . 100
6.18 FileInput Attributes Table . . . 101
6.19 FileInputArray Attributes Table . . . 102
6.20 Searching . . . 102
6.21 Sort, Limit, Select and ListType . . . 105
6.22 Tapis Responses . . . 107

7 Jobs **109**
7.1 Introduction to Jobs . . . 109
7.2 The Job Submission Request . . . 111
7.3 Job Execution . . . 124
7.4 Container Runtimes . . . 132
7.5 Querying Jobs . . . 137
7.6 Dedicated Search Endpoint . . . 147
7.7 Job Actions . . . 149
7.8 Job Sharing . . . 151

8 Meta **157**
8.1 Why Meta V3 . . . 157
8.2 Migration from Meta V2 to V3 . . . 157
8.3 Overview . . . 157
8.4 Getting Started . . . 158
8.5 Resources . . . 160

9 Pods **173**
9.1 Introduction to the Pods Service . . . 173
9.2 Getting Started . . . 174
9.3 Pods Quickstart . . . 176
9.4 Templates . . . 178
9.5 Neo4j . . . 179
9.6 Postgres . . . 179
9.7 Future work. Only quickstart is currently complete. . . . 179
9.8 API Reference . . . 179

10 Actors **181**
10.1 Introduction to Abaco . . . 181

10.2 Getting Started . . . 181
10.3 Abaco Quickstart . . . 183
10.4 Actor Registration . . . 187
10.5 Abaco Context & Container Runtime . . . 189
10.6 Messages, Executions, and Logs . . . 191
10.7 Database Search . . . 198
10.8 Cron Schedule . . . 203
10.9 Actor State . . . 205
10.10 Actor Sharing and Nonces . . . 206
10.11 Networks of Actors . . . 209
10.12 Autoscaling Actors . . . 213
10.13 API Reference . . . 214

11 Security **215**
11.1 Introduction to the Security Kernel . . . 215
11.2 Authorization . . . 215
11.3 Secrets . . . 216

12 Streams **219**
12.1 Projects . . . 219
12.2 Sites . . . 223
12.3 Instruments . . . 226
12.4 Variables . . . 229
12.5 Measurements . . . 232
12.6 Channels . . . 233
12.7 Templates . . . 237
12.8 Alerts . . . 239
12.9 Roles . . . 240

13 Workflows **247**
13.1 Overview . . . 247
13.2 Quick start . . . 248
13.3 Groups . . . 251
13.4 Pipelines . . . 253
13.5 Tasks . . . 257
13.6 Archives . . . 262

14 Notifications **263**
14.1 Overview . . . 263
14.2 Event Model . . . 263
14.3 Subscription . . . 264
14.4 Notification Model . . . 266
14.5 Notification Delivery . . . 267
14.6 Tables . . . 271

15 Resource Sharing **275**
15.1 Introduction to Tapis Access Controls . . . 275
15.2 Roles and Permissions . . . 275
15.3 Tapis Shared Resources . . . 276

16 Tapipy and TapisService **279**
16.1 Tapipy User Guide . . . 279
16.2 TapisService User Guide . . . 286
16.3 TapisService Modules . . . 288
16.4 Working with TapisService, Need to Knows . . . 292

17 Specialized Services - Overview **297**

18 Tapis ETL **299**
18.1 Overview . . . 299
18.2 User Guide . . . 300
18.3 Quick Start . . . 315

19 Deployment & Administration Guide - Overview **317**

20 Considerations and Prerequisites **319**
20.1 Site Type . . . 319
20.2 Kubernetes . . . 320
20.3 Environments and Capacity Planning . . . 320
20.4 Tenants & Authenticators . . . 322
20.5 Deployer . . . 323
20.6 Additional Requirements for an Initial Associate Site Deployment . . . 323

21 Tapis Deployer **325**
21.1 Installing Deployer . . . 325
21.2 Configuring Your Site . . . 326
21.3 Required Fields – All Sites . . . 327
21.4 Optional Fields – All Sites . . . 328
21.5 Generating the Tapis Deployment Script Directory . . . 329
21.6 Deployment Script Directory Structure . . . 330
21.7 Using the Deployer Control Scripts . . . 331
21.8 Initialize the Tapis Deployment . . . 331
21.9 Deploy the Primary Tapis services . . . 335
21.10 Deploy the Secondary Tapis Services . . . 337
21.11 Configuring Support for GLOBUS . . . 337
21.12 Configuring Support for TMS . . . 337
21.13 Configuring Support for Email Notifications . . . 338
21.14 Advanced Configuration Options . . . 338

22 Tapis Vault **339**
22.1 Introduction to Tapis Vault . . . 339
22.2 Deploying Vault in Kubernetes . . . 339
22.3 Deploying Vault Outside of Kubernetes . . . 340
22.4 Vault Backup . . . 345
22.5 Vault Export . . . 345
22.6 Revoke / Regenerating Vault Root Tokens . . . 346

23 Managing Secrets with SkAdmin **349**
23.1 Introduction to Tapis Secrets . . . 349
23.2 The SkAdmin Utility . . . 349

24 Tapis Administration **357**
24.1 Monitoring Tapis . . . 357
24.2 Troubleshooting Tapis Pods . . . 359
24.3 Backing Up Tapis . . . 359

25 Upgrading Tapis **361**
25.1 Breaking Deployment Changes . . . 361

26 Tapis Artifacts & Architecture **363**
26.1 API Specification . . . 363

26.2 Micro-Service Architecture 364
26.3 Artifacts and Code Repositories 364

Add your content using `reStructuredText` syntax. See the reStructuredText documentation for details.

CHAPTER

ONE

GETTING STARTED

This Getting Started guide will walk you through the initial steps of setting up the necessary accounts and installing the required software before moving to the Tapis Quickstart. If you are already using Docker Hub and the TACC Cloud APIs, feel free to jump right to the *Tapis Quickstart* or check out the Tapis Live Docs site.

- *Getting Ready*
  - *cURL or Python: Pick Your Path*
  - *Amazon S3*
  - *A TACC Account*
- *Tapis Quickstart*
  - *Python Example*
    - *Getting a Tapis Token*
    - *Register the S3 Storage with Tapis*
    - *Create an Access Credential*
    - *Access the files in our S3 Bucket*
  - *cURL Example*
    - *Getting a Tapis Token*
    - *Register the S3 Storage with Tapis*
    - *Create an Access Credential*
    - *Access the files in our S3 Bucket*
  - *Sentiment-Analysis Application Tutorial*
    - *Create A Tapis Token*
    - *Create a system*
    - *Create An Application*
    - *Application Arguments*
    - *Submitting a job*
    - *Jobs List*
    - *Jobs Output*

* *Next Steps*

1.1 Getting Ready

1.1.1 cURL or Python: Pick Your Path

This guide provides examples in both the command line web interface *cURL* and the Python SDK. cURL is a command line tool for transferring data via URLs and receiving responses; cURL is available in your terminal (for MacOS, Linux, and Unix users), and transfers data from or to a server using one of a number of internet protocols. You could also choose to use a tool like Postman which which provides a nice GUI on top of cURL.

To run the Python examples you will need a functioning Python3 environment and the Tapis Python APIs installed on your system (read more about the tapispy project at PyPi).

1.1.2 Amazon S3

In either case you will be accessing an Amazon AWS S3 storage bucket in this Tapis example. In the code we assume you have created an S3 bucket with URL *https://<your_bucket_name>.s3.amazonaws.com/* and that you have access to the bucket via an AWS user identity with `accessKey` and `accessSecret`. If you are new to Amazon's S3 service, you can get started for free to complete this tutorial. Check out their Getting Started Guide and follow the tutorial through *Step 2: Upload an object to your bucket*. Our example will list the files stored in this bucket, so you may want to upload a file to your bucket so you can verify that your code works; a small image file would do the trick. In the examples below we have created an S3 bucket called *tapisbucket* and uploaded a single file to it called *test_image.jpg*.

1.1.3 A TACC Account

The main instance of the Tapis platform is hosted at the Texas Advanced Computing Center (TACC). TACC designs and deploys some of the world's most powerful advanced computing technologies and innovative software solutions to enable researchers to answer complex questions. To follow along with this Quickstart and use the TACC-hosted Tapis platform, please create a TACC account .

1.2 Tapis Quickstart

In this guide, we will use the Tapis APIs to list files on the Amazon AWS S3 storage system using the *tacc* instance of Tapis (an instance of Tapis is called a **tenant**; since Tapis is open source any organization can install it and run their own tenant) with base URL *https://tacc.tapis.io*. We will work the example in two ways: using cURL from the command line and using a Python script. Pick the path that most closely matches your needs.

1.2.1 Python Example

Getting a Tapis Token

To begin we use our TACC account credentials to get a Tapis token from the authenticator. A valid token is needed to interact with Tapis. You will note that your token is returned with an expiration date and time. If you want to continue to use Tapis after your token expires, you will need to get a new one following the same steps.

For the purpose of this tutorial, we will be using the base_url of `https://tacc.tapis.io`. To discover other tenants, and find the details specific to that tenant, such as owner, base_url, tenant_id, etc. you can use following curl command in your terminal window.

```
curl https://admin.tapis.io/v3/tenants | jq
```

Note that `jq` is a command-line JSON formatter. Checking out their download instructions.

From here, you will use the `ctrl-F` function in your terminal window to search for your base_url. A note of caution, be sure to verify if you are using the DEV or PROD tenants; DEV tenants will be labeled with a delineation of -dev and noted in the description as such.

For more information about tenant listing and search techniques please see the Tenancy, Sites and Authentication.

In Python code below you will replace *your_tacc_username* and *your_tacc_password* with your TACC username and password, preserving the quotation marks shown in the command below. See more about the Tapis Python SDK, `tapipy`, including how to install it, from here.

This call does not produce output. However you can use the following code to see the access token you just created.

```
t.access_token
```

The output should look similar to the following; it describes the access token that was just created.

```
access_token: *very long string of alphanumeric characters*
claims: {'jti': '007fa9e6-f044-4817-a812-12292b2bdbe3', 'iss': 'https://tacc.tapis.io/v3/
↪tokens', 'sub': 'your_tacc_username', 'tapis/tenant_id': 'tacc', 'tapis/token_type':
↪'access', 'tapis/delegation': False, 'tapis/delegation_sub': None, 'tapis/username':
↪'your_tacc_username', 'tapis/account_type': 'user', 'exp': 1657686889, 'tapis/client_id
↪': None, 'tapis/grant_type': 'password'}
expires_at: 2022-07-13 04:34:49+00:00
expires_in: <function Tapis.add_claims_to_token.<locals>._expires_in at 0x10a070280>
jti: 007fa9e6-f044-4817-a812-12292b2bdbe3
original_ttl: 14400
```

Where you will have your own access token and the placeholder *your_tacc_username* will be replaced with the username you used.

Register the S3 Storage with Tapis

Now that we have an access token, we are ready to create a Tapis `system` object. Remember that the Tapis APIs are a framework for accessing a wide variety of computational resources: for this example, we will register the S3 bucket we pre-created with Amazon's AWS S3 service (if this is unfamiliar to you, refer back to the *Getting Ready* section above). This step registers the S3 bucket with Tapis so it can access the bucket on your behalf. Note that the value of `id` needs to be unique within the Tapis tenant you are using; for this example we show the string *your_tapis_system_id*, but you'll need to choose your own globally (within the Tapis tenant you are using) unique id (possibly including your user name, for example, or some other unique id; for example, something like *username.tapis_test_v1*). The values for `host` and `bucketName` are set based on the values you used when you set up your own AWS S3 bucket, as discussed above. In this example they are

```
# To register the S3 bucket with Tapis
# the description of an S3 bucket
s3_bucket = {
  "id":"your_tapis_system_id",
  "description":"Tapis Test Bucket",
  "host":"tapisbucket.s3.amazonaws.com",
  "systemType":"S3",
  "defaultAuthnMethod":"ACCESS_KEY",
  "bucketName":"tapisbucket",
  "canExec": False,
}
```

(continues on next page)

(continued from previous page)

```
# create the system in Tapis
t.systems.createSystem(**s3_bucket)
```

The output of the command will show the URL to the system you just created in the Tapis tenant.

Create an Access Credential

With our system created and linked to the S3 storage bucket, we need to create an access credential for Tapis to access our bucket on our behalf. Note that in this example we have created an access key and secret for the IAM user in our AWS instance; when substituting your own access key and secret into the code below, remember to preserve the single quotation marks enclosing these pieces of data.

```
t.systems.createUserCredential(systemId='your_tapis_system_id',
                               userName='your_tacc_username',
                               accessKey='IAM user access key that you created in the␣
↪AWS interface',
                               accessSecret='access secret from the AWS interface for␣
↪the key you created')
```

The output of the command should look similar to the text below.

```
{'result': None,
'status': 'success',
'message': 'SYSAPI_CRED_UPDATED Credential updated. jwtTenant: tacc jwtUser: your_tacc_
↪username OboTenant: tacc OboUser: your_tacc_username System: your_tapis_system_id␣
↪User: your_tacc_username',
'version': '1.2.1',
'metadata': None}
```

Access the files in our S3 Bucket

We are now able to list files in our bucket using the Files API (you can also use the API to add new files, delete files, rename files, and so on; but since this is a example, we've just selected a file listing as the example operation).

```
t.files.listFiles(systemId="your_tapis_system_id", path="/")
```

The output should include a list of all files in the bucket. For this example we only put one file in our bucket, test_image.jpg. The listing returns the name of that file along with some metadata:

```
[
group: None
lastModified: 2022-05-20T19:24:24Z
mimeType: image/jpg
name: test_image.jpg
nativePermissions: None
owner: None
path: /test_image.jpg
size: 87060
type: file
url: tapis://your_tapis_system_id/test_image.jpg]
```

1.2.2 cURL Example

As with the Python example, we will use the Tapis APIs to list files on the Amazon AWS S3 storage system using the *tacc* tenant with base URL *https://tacc.tapis.io*.

Getting a Tapis Token

To begin we use our TACC account credentials to get a Tapis token from the authenticator. A valid token is needed to interact with Tapis. You will note that your token is returned with an expiration date and time. If you want to continue to use Tapis after your token expires, you will need to get a new one following the same steps.

Type the curl command below into your terminal window, replacing *your_tacc_username* and *your_tacc_password* with your TACC user name and password, preserving the quotation marks.

```
$ curl -H "Content-type: application/json" -d '{"username": "your_tacc_username
↪", "password": "your_tacc_password", "grant_type": "password" }' https://
↪tacc.tapis.io/v3/oauth2/tokens
```

The output of this operation will look like the following (line breaks have been added for clarity in reading; your response string will not have line breaks); the phrase <*your access token string will be here*> will be replaced with your token access string, which is an 834 character alphanumeric string. Notice the expiration time in the return string.

```
{“message”:”Token created successfully.”,
 ”metadata”:{},
 ”result”:{“access_token”:{“access_token”:”<your access token string will be here>”,
    ”expires_at”:”2022-05-05T19:53:03.801252+00:00”,
    ”expires_in”:14400,”jti”:”8ef1d271-b923-49af-b2dd-ae05cc5da1ed”}},
 ”status”:”success”,
 ”version”:”dev”}
```

To work through the rest of the examples in this guide, you will need to add the token from the curl command to your environment using the variable name `JWT`. The example below shows how I added it to zsh (a bash variant); the precise method may vary with your shell:

```
$ export JWT=your_access_token_string
```

Register the S3 Storage with Tapis

Now that we have an access token, we are ready to create a Tapis `system` object. Remember that the Tapis APIs are a framework for accessing a wide variety of computational resources: for this example, we will register the S3 bucket we pre-created with Amazon's AWS S3 service (if this is unfamiliar to you, refer back to the *Getting Ready* section above). This step registers the S3 bucket with Tapis so it can access the bucket on your behalf. Note that the value of `id` needs to be unique within the Tapis tenant you are using; for this example we show the string *your_tapis_system_id*, but you'll need to choose your own globally (within the Tapis tenant you are using) unique id (possibly including your user name, for example, or some other unique id; for example, something like *username.tapis_test_v1*, but you'll need to select your own). The values for `host` and `bucketName` are set based on the you used when you set up your own AWS S3 bucket, as discussed above.

To keep the cURL command (relatively) readable, you first need to create a file in your path with the details of your S3 storage and Tapis system formatted as a JSON object; in this example we use the filename *system_s3.json*, with the following contents:

```
{
  "id":"your_tapis_system_id",
  "description":"Tapis cURL Test Bucket",
  "host":"tapisbucket.s3.amazonaws.com",
```

(continues on next page)

(continued from previous page)

```
  "systemType":"S3",
  "defaultAuthnMethod":"ACCESS_KEY",
  "bucketName":"tapisbucket",
  "canExec": False
}
```

Then you'll execute the following cURL command, being sure to specify the name of the file you created if you chose a different name:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.
↪tapis.io/v3/systems -d @system_s3.json
```

The output of the command will show the URL to the system you just created in the Tapis tenant.

```
{
  "result": {
    "url": "http://tacc.tapis.io/v3/systems/your_tapis_system_id"
  },
  "status": "success",
  "message": "SYSAPI_CREATED New system created. jwtTenant: tacc jwtUser: your_tacc_
↪username OboTenant: tacc OboUser: your_tacc_username System: your_tapis_system_id",
  "version": "1.2.3",
  "metadata": null
}
```

Create an Access Credential

With our system created and linked to the S3 storage bucket, we need to create an access credential for Tapis to access our bucket on our behalf. Note that in this example we have created an access key and secret for the IAM user in our AWS instance; when substituting your own access key and secret into the code below, remember to preserve the single quotation marks enclosing these pieces of data.

Again, to keep the cURL command (relatively) readable, you first need to create a file in your path with the details of your S3 access key formatted as a JSON object; in this example we use the filename cred_tmp.json, with the following contents:

```
{
  "accessKey":"IAM user access key that you created in the AWS interface",
  "accessSecret":"access secret from the AWS interface for the key you created"
}
```

Then you'll execute the following cURL command, being sure to specify the name of the file you created if you chose a different name:

```
$curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.
↪tapis.io/v3/systems/credential/your_tapis_system_id/user/your_tacc_username -d @cred_
↪tmp.json
```

The output of the command will show will look similar to that below.

```
{
  "result": null,
  "status": "success",
```

(continues on next page)

(continued from previous page)

```
  "message": "SYSAPI_CRED_UPDATED Credential updated. jwtTenant: tacc jwtUser: your_tacc_
↪username OboTenant: tacc OboUser: your_tacc_username System: your_tapis_system_id␣
↪User: your_tacc_username",
  "version": "1.2.3",
  "metadata": null
}
```

Access the files in our S3 Bucket

We are now able to list files in our bucket using the Files API. If you look closely at the URL you'll see that we are using the *files* access point – this URL returns a listing of the files in your bucket along with some metadata. Recall that for our example the bucket has a single image in it.

```
curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/files/ops/your_tapis_system_id/
```

The output of the command will show will look similar to that below, where the path and file name will reflect how you set up your own S3 bucket and the file(s) you put in it. We added line breaks to the output below for readability.

```
{
  "status":"success",
  "message":"ok",
  "result":
  [
    {
      "mimeType":"image/jpg",
      "type":"file",
      "owner":null,
      "group":null,
      "nativePermissions":null,
      "url":"tapis://your_tapis_system_id/test_image.jpg",
      "lastModified":"2022-05-20T19:24:24Z",
      "name":"test_image.jpg",
      "path":"/test_image.jpg",
      "size":87060
    }
  ],
  "version":"1.2.2","metadata":{}
}
```

1.2.3 Sentiment-Analysis Application Tutorial

For those of you that want to dive straight into Tapis and begin to explore it's possibilites, this is our tutorial. In this tutorial you will create a system, sentiment-analysis application, and run a job with it. For the sake of brevity and speed, we will complete this tutorial utilizing the Python SDK, Tapipy.

Pre-requisites: Active TACC account, Tapipy installed, Access to Stampede3, Frontera, or non-MFA system.

Create A Tapis Token

Next you will need to gather your access token.

```
t.access_token
```

The output should look similar to the following; it describes the access token that was just created.

```
access_token: *very long string of alphanumeric characters*
claims: {'jti': '007fa9e6-f044-4817-a812-12292b2bdbe3', 'iss': 'https://tacc.tapis.io/v3/
↪tokens', 'sub': 'your_tacc_username', 'tapis/tenant_id': 'tacc', 'tapis/token_type':
↪'access', 'tapis/delegation': False, 'tapis/delegation_sub': None, 'tapis/username':
↪'your_tacc_username', 'tapis/account_type': 'user', 'exp': 1657686889, 'tapis/client_id
↪': None, 'tapis/grant_type': 'password'}
expires_at: 2022-07-13 04:34:49+00:00
expires_in: <function Tapis.add_claims_to_token.<locals>._expires_in at 0x10a070280>
jti: 007fa9e6-f044-4817-a812-12292b2bdbe3
original_ttl: 14400
```

Where you will have your own access token and the placeholder *your_tacc_username* will be replaced with the username you used.

Create a system

Next you will need to create a system. Your system must be hosted on a machine that you can SSH to. There are a variety of authentication methods such as PASSWORD, PKI_KEYS (SSH Keys: Private, Public pair), ACCESS_KEYS (S3), and TOKENS (GLOBUS). If you are using PKI_KEYS please be aware that they will only work if MFA is NOT enabled on that system. Also, you must place the public key on that system. The Public and Private key on your system and the Public key on the host system must be formatted for one line.

```
  system_def = {
  “id”: “<YOUR_SYSTEM_ID>”,
  "description": "test system",
  "systemType": "LINUX",
  "host”:”<HOST>,
  "defaultAuthnMethod": "PASSWORD",
  "rootDir": "/",
  "canExec": True,
  "jobRuntimes": [ { "runtimeType": "SINGULARITY" } ],
  "jobWorkingDir": "workdir",
}

t.systems.createSystem(**system_def)
```

When defining a HOST, it’s important to remember that it should be defined by the URL without the “https://”

This will return:

```
url: http://tacc.tapis.io/v3/systems/<YOUR_SYSTEM_ID>
```

Create An Application

Now that you have a system to run your jobs, you must create an application. Here is an example of an application definition:

```
  app_def = {
    "id": <app_id>,
    "version": "0.2",
    "description": "Application utilizing the sentiment analysis model from Hugging Face.
↪",
    "jobType": "FORK",
    "runtime": "DOCKER",
```

(continues on next page)

(continued from previous page)

```
    "containerImage": "tapis/sentiment-analysis:1.0.0",
    "jobAttributes": {
        "parameterSet": {
            "archiveFilter": {
                "includeLaunchFiles": False
            }
        },
        "memoryMB": 1,
        "nodeCount": 1,
        "coresPerNode": 1,
        "maxMinutes": 10
    }
}
```

With a system now created, we need to register this application to make it accessible on this tenant.

```
t.apps.createAppVersion(**app_def)
```

Application Arguments

With appArgs parameter you can specify one or more command line arguments for the user application. Arguments specified in the application definition are appended to those in the submission request. Metadata can be attached to any argument:

```
# Modify Job submission arguments
  pa = {
      "parameterSet": {
      "appArgs": [
              {"arg": "--sentences"},
              {"arg": "\"This is great\" \"This is not fun\""}
          ]
      }}
```

Submitting a job

Running a job only requires 3 items: a Job name (name), an app_id, and the app_id version. If you have not specified the Execution System in your application, you will need to specify it when submitting a job.

A simple submission would look like this:

```
job_response = t.jobs.submitJob(
    name='sentiment analysis',
    appId=app_id,appVersion='0.2',
    execSystemId=system_def,
    **pa #reintroducing modified job submission arguments
  )
```

All of the Job Submission Parameters can be found here Job Submission Parameters.

Everytime a job is submitted, a unique job_id (uuid) is generated. We will use this job_id with tapipy to get the job status and download the job output.

```
  # Get job uuid from the job submission response
print("*************************************************")
job_uuid = job_response.uuid
print("Job UUID: " + job_uuid)
print("*************************************************")
```

Jobs List

When you do a jobs list now, you can see your jobUuid.

```
t.jobs.getJobList()
```

Jobs Output

To download the output of a job you need to give it the jobUuid and output path. You can download a directory in the jobs' outputPath in zip format. The outputPath is relative to archive system specified.

```
  # Download output of the job
print("Job Output file:")

print("*************************************************")
jobs_output = t.jobs.getJobOutputDownload(jobUuid=job_uuid,outputPath='stdout')
print(jobs_output)
print("*************************************************")
```

Next Steps

This concludes the Sentiment-Analysis tutorial. This tutorial was presented at PEARC'24. Please feel free to explore and read more of the Tapis documentation. If you would like to see more recent Tapis tutorials, please see Tapis Tutorials.

CHAPTER

TWO

TECHNICAL GUIDE - OVERVIEW

The Technical Guide for Tapis provides a detailed reference to the primary services in the platform.

- *Sites, Tenancy and Authentication*: Complete reference for Tapis authentication.
- *Systems*: Registering and working with remote storage and execution resources.
- *Files*: Managing data on Tapis systems.
- *Applications*: Registering and working with applications.
- *Jobs*: Running an application on a system.
- *Meta*: Storing and retrieving metadata pertaining to your projects.
- pgrest: Hosted, HTTP RESTful API to a managed Postrges database instance.
- *Pods*: Create long-lived network enabled pods for databases or servers.
- *Streams*: Managing streaming data from sensors and other instruments.
- *Actors*: Execute containerized functions ("actors") in response to events or other messages.
- *Security*: Securing the digital assets comprising your project.
- *Workflows*: Create and execute workflows in the Tapis ecosystem.
- *Resource Sharing*: Sharing Tapis resources with colleagues.
- *Tapipy and TapisService*: Tapipy and Tapisservice usage, special attributes, and examples for Python development.

CHAPTER

THREE

SITES, TENANCY AND AUTHENTICATION

3.1 Sites

Tapis supports geographically distributed deployments where different components are running in different data centers and managed by different institutions. These physically isolated installations of Tapis software are referred to as *sites*. There is a single *primary site* and zero or more *associate sites* within a Tapis installation.

3.1.1 Primary Site

The primary site in a Tapis installation runs a complete set of Tapis API services and all associated 3rd-party services, such as databases and message queues. The creation of new sites is coordinated through the primary site, and the primary site runs the unique instance of the Sites and Tenants API (see *Tenants* below) which maintain the complete registry of all sites and tenants in the installation.

The primary site of the main Tapis installation is hosted at the Texas Advanced Computing Center at the tapis.io domain.

3.1.2 Associate Sites

Associate sites are required to run the Tapis Security Kernel, a compliant Token Generator API, and one or more additional Tapis services. Each associate site is managed and operated by a separate, partner institution. For Tapis services not run at the associate site, the corresponding service at the primary site is used for requests. In this way, partner institutions can choose which Tapis services to run within their institution and leverage the primary site deployment for the rest.

3.1.3 Deployment

The official Tapis deployment tooling targets the Kubernetes container orchestration platform. The project maintains a set of deployment templates which can be used in conjunction with configuration files to deploy Tapis services. If your institution is interested in becomming a Tapis associate site please contact us.

Details about the current list of sites is available from the tenants API. For example, one can retrieve the full list of sites as follows:

With PySDK:

```
>>> t.tenants.list_sites()
```

With CURL:

```
$ curl https://admin.tapis.io/v3/sites
```

The response will look similar to the following (the response below is truncated for brevity):

```
[
 base_url: https://tapis.io
 primary: True
 services: ['systems', 'files', 'security', 'tokens', 'streams', 'authenticator', 'meta',
↪ 'actors']
 site_admin_tenant_id: admin
 site_id: tacc
 tenant_base_url_template: https://${tenant_id}.tapis.io]
 .  .  .
]
```

Each site has a `site_id` as well as a list of Tapis services it provides and the tenant ID of the administrative tenant (`admin_tenant`) associated with it.

3.2 Tenants

Tapis is a *multi-tenant* platform, meaning that different projects (or *tenants*) can have logically isolated views of the Tapis objects (i.e., the systems, files, actors, etc.) they create for their project.

Each tenant is made up of the following:

1. A base URL with which to access the tenant; by default, the base URL takes the form `https://<tenant_id>.tapis.io` where `tenant_id` is a short, unique identifier for the tenant in the Tapis system. For example, `https://tacc.tapis.io` is the base URL for the `tacc` tenant.
2. An *authenticator* providing the rules for who can authenticate in the tenant.

Additionally, each tenant is "managed" by a site.

To see the current list of tenants registered with Tapis, we can use the tenants API.

With PySDK:

```
>>> t.tenants.list_tenants()
```

With CURL:

```
$ curl https://tacc.tapis.io/v3/tenants
```

The response will look similar to the following (the response below is truncated for brevity):

```
allowable_x_tenant_ids: ['tacc']
authenticator: https://tacc.tapis.io/v3/oauth2
base_url: https://tacc.tapis.io
create_time: Thu, 02 Jul 2020 23:45:16 GMT
description: Production tenant for all TACC users.
is_owned_by_associate_site: False
last_update_time: Thu, 02 Jul 2020 23:45:16 GMT
owner: CICSupport@tacc.utexas.edu
public_key: -----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAz7rr5CsFM7rHMFs7uKIdcczn0uL4ebRMvH8pihrg1tW/
↪fp5Q+5ktltoBTfIaVDrXGF4DiCuzLsuvTG5fGElKEPPcpNqaCzD8Y1v9r3tfkoPT3Bd5KbF9f6eIwrGERMTs1kv7665pliwehz91n
↪EnSg1N3Oj1x8ktJPwbReKprHGiEDlqdyT6j58l/I+9ihR6ettkMVCq7Ho/
↪bsIrwm5gP0PjJRvaD5Flsze7P4gQT37D1c5nbLR+K6/T0QTiyQIDAQAB
-----END PUBLIC KEY-----
security_kernel: https://tacc.tapis.io/v3/security
```

(continues on next page)

(continued from previous page)

```
service_ldap_connection_id: None
tenant_id: tacc
token_service: https://tacc.tapis.io/v3/tokens
user_ldap_connection_id: tacc-all,

allowable_x_tenant_ids: ['dev']
authenticator: https://dev.tapis.io/v3/oauth2
base_url: https://dev.tapis.io
create_time: Fri, 19 Jun 2020 20:36:38 GMT
description: The dev tenant.
is_owned_by_associate_site: False
last_update_time: Fri, 19 Jun 2020 20:36:38 GMT
owner: CICSupport@tacc.utexas.edu
public_key: -----BEGIN PUBLIC KEY-----

. . .
```

Here we see the first two tenants registered in the Tapis framework, the `tacc` and `dev` tenants.

In general, the rules for authentication vary from tenant to tenant within Tapis. For example, in the `tacc` tenant, any user with a valid TACC account can authenticate and use the APIs. The `dev` tenant is a development sandbox with test accounts used by the core Tapis team.

This documentation focuses on the `tacc` tenant; however, much of what follows in the subsequent sections will be the same regardless of the tenant you are using.

3.3 Authentication

The default authenticator provided by the Tapis project is based on OAuth2, and this is the authentication mechanism in place for the `tacc` tenant. The OAuth2-based authentication services are available via the `/v3/oauth2` endpoints.

OAuth uses different *grant type flows* for generating tokens in different situations. We do not provide a comprehensive guide to OAuth2; for that, we refer the reader to the OAuth2 docs. Instead, we provide a guide to the two most common use cases for users: generating tokens for themselves using the *password* grant flow, and generating tokens on behalf of others in a web application using the *authorization code* grant flow.

In the PySDK examples that follow, we will tacitly assume the `tapipy.tapis.Tapis` object has been instantiated as the Python object `t`. There are several options in the `Tapis` constructor, but the basic options include `base_url` and `username`, for example:

```
>>> t = Tapis(base_url='https://tacc.tapis.io', username='jdoe')
```

3.3.1 Password Grant - Generating a Token For Yourself

The simplest case is that you want to generate a Tapis OAuth token for yourself; to do this you can use the *password* grant flow, providing your username and password.

Tapis v3 tries to make this process as easy as possible by providing a simplified version of the password grant flow that does not require an OAuth client (see the *OAuth Clients* section).

With PySDK:

```
>>> t = Tapis(base_url='https://tacc.tapis.io', username='apitest', password='abcd123')
>>> t.get_tokens()
```

With CURL:

```
> curl -H "Content-type: application/json" -d '{"username": "apitest", "password":
↪"abcde123", "grant_type": "password" }' \
https://tacc.tapis.io/v3/oauth2/tokens
```

In the PySDK, the access token is a first-class Python object stored on the Tapis object (the `t` in the examples above). We can inspect it

```
>>> t.access_token

access_token: eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.
↪eyJqdGkiOiJmN2I5YjE5ZS02MDk5LTRmODItYTcyMi1iNjEwYzVkMGJhMGMiLCJpc3MiOiJodHRwczovL3RhY2MudGFwaXMuaW8vd
↪alC8rRM-zNsHKcUiz3-tOJPaYtFksKb4Bit_aFE1HH_X_znnP2QkJaqc-
↪xaRoMlQu26MN72TlJE0siIN3T38xXWBGDumHUYbvnNzT-
↪7lk7AQU5MHSyCWx8IRDmTSbqmWOG8WBMCIV9Dh84mDd-X6eLJQ_cz1QqMAiI_cPgA9VVE22qDK3Lbz2pp9t0sm-
↪l9XjE5y5Im8Y0B2p0ssPD0TjW20C5yngZ4-4jowDafboKlscog9ko-adrsVIjG_r-
↪ccCUX3r8SVwQLypZFZAPKqbVzl8jt_
↪mCi30W8AYwiaYGmH7INBbHI9hO7kwJNFMuSylejFhMslxgdzGlIAyXauwg
claims: {'jti': 'f7b9b19e-6099-4f82-a722-b610c5d0ba0c', 'iss': 'https://tacc.tapis.io/v3/
↪tokens', 'sub': 'apitest@tacc', 'tapis/tenant_id': 'tacc', 'tapis/token_type': 'access
↪', 'tapis/delegation': False, 'tapis/delegation_sub': None, 'tapis/username': 'apitest
↪', 'tapis/account_type': 'user', 'exp': 1595099456, 'tapis/client_id': None, 'tapis/
↪grant_type': 'password'}
expires_at: 2020-07-18 19:10:56+00:00
expires_in: <function Tapis.set_access_token.<locals>._expires_in at 0x7f29e213c510>
jti: f7b9b19e-6099-4f82-a722-b610c5d0ba0c
original_ttl: 14400
```

What we see is that the `access_token.access_token` is a Python string representing a JSON Web Token (JWT). JWTs are cryptographically signed with the private key associated with the tenant, and anyone can validate the signature by using the corresponding public key associated with the tenant (see Tenants section above). The public key for each tenant is available from the Tenants API. The core Tapis services will validate the access token sent on a given API call using the public key associated with the tenant to verify the JWT signature.

3.3.2 Using a Token

In order to use an access token in an API request to Tapis, pass the token in as the value of the `X-Tapis-Token` header. The PySDK will automatically send the token via this header for you. In CURL examples used throughout this documentation, we assume the raw JWT string representing an access token (like the above) has been exported as a shell variable; i.e.,

```
$ export JWT=eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.
↪eyJqdGkiOiJmN2I5YjE5ZS02MDk5LTRmODItYTcyMi1iNjEwYzVkMGJhMGMiLCJpc3MiOiJodHRwczovL3RhY2MudGFwaXMuaW8vd
↪alC8rRM-zNsHKcUiz3-tOJPaYtFksKb4Bit_aFE1HH_X_znnP2QkJaqc-
↪xaRoMlQu26MN72TlJE0siIN3T38xXWBGDumHUYbvnNzT-
↪7lk7AQU5MHSyCWx8IRDmTSbqmWOG8WBMCIV9Dh84mDd-X6eLJQ_cz1QqMAiI_cPgA9VVE22qDK3Lbz2pp9t0sm-
↪l9XjE5y5Im8Y0B2p0ssPD0TjW20C5yngZ4-4jowDafboKlscog9ko-adrsVIjG_r-
↪ccCUX3r8SVwQLypZFZAPKqbVzl8jt_
↪mCi30W8AYwiaYGmH7INBbHI9hO7kwJNFMuSylejFhMslxgdzGlIAyXauwg
```

With that variable set, we can use the `-H` flag with curl to set the `X-Tapis-Token` header as follows:

```
$ curl -H "X-Tapis-Token: $JWT" ....
```

Note also the *claims* associated with the access token. These claims provide information about the token, including the user it represents (`apitest` in the above example), the tenant it belongs to (`tacc` above) when it expires, etc. Tapis tokens always include the following standard claims:

Claim	Description	Example Value
sub	The subject of the token; the subject uniquely identifies the user in a Tapis installation. The format is `user @ tenant`	apitest@tacc
exp	The expiry associated with the token.	1595099456
jti	Unique identifier for the token.	f7b9b19e-6099-4f82-a722-b610c5d0ba0c
iss	The identifier (URL) of the issuer of the JWT. For Tapis, the issuer will be a Tokens API.	https://tacc.tapis.io/v3/tokens

Additional custom claims specific to Tapis are namespaced with `tapis/` at the beginning of the claim name. The authenticator for each tenant may optionally choose to support one or more of these additional claims. The following claims are encouraged and supported by the default OAuth2 Tapis authenticator.

Claim	Description	Example Value
tapis/tenant_id	The tenant of the subject.	tacc
tapis/username	The username of the subject.	apitest
tapis/token_type	Type of token: `access` or `refresh`	access
tapis/account_type	Type of account: `user` or `service`	user
tapis/delegation	Whether a delegation flow was used to generate this token. (`true` or `false`).	false
tapis/delegation_s	For a delegation token, the subject who actually generated the token. In form `user @ tenant`	supe-ruser@tacc
tapis/client_id	The id of the OAuth client used to generate the token.	tacc.CIC.tokenapp
tapis/grant_type	The grant type used to generate the token.	authoriza-tion_code

The authenticator for your tenant may include additional claims not listed here.

3.3.3 OAuth Clients

In order to use the more advanced OAuth2 flows, including any use of the authorization code grant type, the implicit grant type, and to generate refresh tokens with the password grant type, you must generate an OAuth2 *client*. Clients in OAuth2 represent applications (for example, a web or mobile application) that will interact with the OAuth2 server to generate tokens on behalf of one or more users. Clients are created and managed using the `/v3/oauth2/clients` endpoints.

Creating Clients

To create a client, make a POST request the the Clients API. All fields are optional; if you do not pass a `client_id` or `client_key` in the request, the clients API will generate random ones for you. In order to use the `authorize_code` and `implicit` grant types you will need to set the `callback_url` when registering your client (see *Authorization Code Grant - Generating Tokens For Users*). For a complete list of available parameters, see the API live-docs for Clients.

With PySDK:

```
>>> t.authenticator.create_client(client_id='test', callback_url='https://foo.example.
↪com/oauth2/callback')
```

With CURL:

```
$ curl -H "X-Tapis-Token: $JWT" -H "Content-type: application/json" -d '{"client_id":
↪"test", "callback_url": "https://foo.example.com/oauth2/callback"}' https://tacc.tapis.
↪io/v3/oauth2/clients
```

The response will be similar to

```
callback_url: https://foo.example.com/oauth2/callback
client_id: test
client_key: WQZlQlMoxOynW
create_time: Sat, 18 Jul 2020 19:09:47 GMT
description:
display_name: https://foo.example.com/oauth2/callback
last_update_time: Sat, 18 Jul 2020 19:09:47 GMT
owner: apitest
tenant_id: tacc
```

Listing Clients

With PySDK:

```
>>> t.authenticator.list_clients()
```

With CURL:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/oauth2/clients
```

The response will be similar to

```
[
callback_url: https://foo.example.com/oauth2/callback
client_id: test
client_key: WQZlQlMoxOynW
create_time: Sat, 18 Jul 2020 19:09:47 GMT
description:
display_name: https://foo.example.com/oauth2/callback
last_update_time: Sat, 18 Jul 2020 19:09:47 GMT
owner: apitest
tenant_id: tacc]
```

Deleting Clients

You can also delete clients you are no longer using; just pass the `client_id` of the client to be deleted:

With PySDK:

```
>>> t.authenticator.delete_client(client_id='test')
```

With CURL:

```
$ curl -H "X-Tapis-Token: $JWT" -X DELETE https://tacc.tapis.io/v3/oauth2/clients/test
```

A null response is returned from a successful delete request.

3.3.4 Authorization Code Grant - Generating Tokens For Users

An important aspect of OAuth2 is that it enables applications to generate tokens on behalf of users without the applications needing to possess user credentials (i.e., passwords). In this section, we discuss using the OAuth2 *authorization code* grant type to do just that.

Assuming a Model-View-Controller (MVC) architecture, there are two controllers that must be written to support the authorization code grant type flow.

1. A controller to determine if the user already has a valid access token and direct them to the OAuth2 authorization server when they do not. This controller starts the authorization code process. To do so, it should:

- First inform the user that they will be asked to authenticate with their tenant username and password and then be asked to grant authorization to your client application.
- Redirect the user to the OAuth2 server's authorization URL. In the default Tapis authenticator, the authorization URL path is `/v3/oauth2/authorize`; for example, `https://tacc.tapis.io/v3/oauth2/authorize` in the `tacc` tenant.
- The redirect request should include the following query parameters:
  - `client_id`: the id of your client.
  - `redirect_uri`: the URI to redirect back to with the authorization code. This must match the `callback_url` parameter associated with your client.
  - `response_type`: should always have the value `code`.

2. A controller to process the authorization code returned and retrieve an access token on the user's behalf. This controller receives requests containing authorization codes from the OAuth2 server after the user has successfully authenticated with said OAuth2 server, and it immediately exchanges the code for a token.

- Responds to `GET` requests to the URL defined in the `callback_url` parameter of your client.
- Retrieves the `code` query parameter from the request.
- Makes a `POST` request to the OAuth2 server's tokens endpoint to generate a token. In the default Tapis authenticator, the tokens URL path is `/v3/oauth2/tokens`; for example, `https://tacc.tapis.io/v3/oauth2/tokens` in the `tacc` tenant. The POST body must include the following parameters:
  - `code`: the code the controller just received in the request from the OAuth2 server.
  - `redirect_uri`: should be the same as the `callback_url` parameter of your client.
  - `grant_type`: should always have the value `authorization_code`.

Note that many popular web frameworks support OAuth2 flows with minimal custom coding required.

The final step to using the authorization code grant type is to register a client (see above) with a `callback_url` parameter equal to the URL within your web application where it will handle converting authorization codes into access tokens (i.e., controller 2 above).

3.3.5 Implicit Grant - Directly Returning Tokens For Users

As mentioned in the authorization code grant section, OAuth2 provides an extra wall of security where the application does not encounter credentials of the user (i.e., passwords). In this section, we discuss using the OAuth2 *implicit* grant type.

Just like the authorization code grant type, the implicit grant type retrieves an access token on behalf of the user to login. In addition, the implicit grant type requires a redirect to the authorization server.

The difference between the two grant types is that the access token is directly delivered through the browser, rather than a code that is later exchanged for the access token. This allows the implicit grant type to be easier to set up, but much

less secure as it cannot authenticate the client. The typical use case for the implicit grant type would be for applications where the client secrets cannot be securely stored.

This is how to start the implicit flow:

- Redirect the user to the OAuth2 server's authorization URL. In the default Tapis authenticator, the authorization URL path is `/v3/oauth2/authorize`; for example, `https://tacc.tapis.io/v3/oauth2/authorize` in the `tacc` tenant.
- The redirect request should include the following query parameters:
  - `client_id`: the id of your client.
  - `redirect_uri`: the URI to redirect back to with the access token. This must match the `callback_url` parameter associated with your client.
  - `response_type`: should always have the value `token`.

Once the user completes the login process through the authorization server, the server redirects the user to the redirect uri along with the access token in the browser.

The final step is to collect the access token in the url and store it in the cookies.

The implicit grant type also requires a client (see above) with a `callback_url` parameter equal to the URL within your web application where it will handle storing the access tokens in the cookies.

Implicit Grant Tutorial

This section serves as a small tutorial to further understand how the implicit grant works and how to set it up.

The first step is to create an OAuth2 client as shown in the *OAuth Clients* section. While setting up the client, make sure to set the `callback_url` to the website that the user will be redirected to (`redirect_uri`) after successfully authenticating on the authorization server. The whole flow will not work if these two values are not identical. Additionally, make sure to remember the `client_id` as it will be needed later.

Note

The `callback_url` of the client created can be changed later on if needed, but the `client_id` cannot.

The next step is to redirect the user to the authorization server's website. The most common way is to create a button on the website that takes the user to the authorization server's website. Make sure to follow this specific formatting for the website url in order to prevent any errors from occuring:

```
https://{tenant}.tapis.io/v3/oauth2/login?client_id={client_id}&redirect_uri={redirect_
↪uri}&response_type={response_type}
```

This is the formatting that will be used for all OAuth2 authentication. Everything in curly brackets needs to be replace with the correct values to successfully set up the implicit flow. Make sure to drop the curly brackets when replacing them with their respective values.

- `{tenant}`: This is the tenant value. Examples are `tacc`, `scoped`, `dev`, etc.
- `{client_id}`: This is the client id for the client that was created in the first step.
- `{redirect_uri}`: This is the value that needs to be the same as the `callback_url`.

Important

The value that replaces this `{redirect_uri}` needs to be *URL Encoded*!! Use an online url encoder to help format the value if needed.

Example of a properly url encoded value:

```
From: https://tacc.tapis.io/v3/oauth2/login?
To:   https%3A%2F%2Ftacc.tapis.io%2Fv3%2Foauth2%2Flogin%3F
```

- `{response_type}`: This is the value that will be delivered through the browser and is dependent on the authorization type that is being used.
- For `implicit`, use `token`.
- For `authorization code`, use `code`.

Once the user has authenticated through the server, the user will automatically be redirected to the `redirect_uri` or `callback_url`. When redirected, the `access_token` (`code` for authorization code) will be returned in the url. Use a method to capture this value and store it in the cookies.

Important

Note that for both `access_token` and `code`, there are more parameters that are returned alongside these values through the url like `state`. Therefore, make sure to capture the value that is between the "*access_token*=" (or "*code*=") and "*&state*".

Finally, since the user does not need to physically see the redirect page, store the token in the cookies and redirect the user one last time to the main page.

Note

For the `authorization code` grant type, everything in this tutorial is pretty much identical. However, there is one more final step of exchanging the `code` returned through the url with the token to complete the flow.

3.3.6 The Tapis Token Web Application

Tapis provides a graphical interface via a web application that enables users to generate tokens. The Tapis Web Application is available by default for any tenant using the default Tapis authenticator, including the `tacc` tenant. The Tapis Token Web Application serves as an example of an application using the authorization code grant type.

The Tapis Token Web Application and its source code are available at the following URLs:

- Token App (`tacc` tenant): https://tacc.tapis.io/v3/oauth2/webapp
- Token App source code: https://github.com/tapis-project/authenticator

CHAPTER

FOUR

SYSTEMS

Once you are authorized to make calls to the various services, one of first things you may want to do is view storage and execution resources available to you or create your own. In Tapis a storage or execution resource is referred to as a **system**.

4.1 Overview

A Tapis system represents a server or collection of servers exposed through a single host name or IP address. Each system is associated with a specific tenant. A system can be used for the following purposes:

- Running a job, including:
  - Staging files to a system in preparation for running a job.
  - Executing a job on a system.
  - Archiving files and data on a remote system after job execution.
- Storing and retrieving files and data.

Each system is of a specific type (such as LINUX or S3) and owned by a specific user who has special privileges for the system. The system definition also includes the user that is used to access the system, referred to as *effectiveUserId*. This access user can be a static specific user (such as a service account) or dynamically specified as `${apiUserId}`. For the case of `${apiUserId}`, the username is extracted from the identity associated with the request to the service. For more information related to *effectiveUserId* please see the section below on *Effective User Id and Host Login* .

4.2 Model

At a high level a system represents the following information:

id
: A short descriptive name for the system that is unique within the tenant.

description
: An optional more verbose description for the system.

***systemType* - Type of system**
: LINUX, S3, IRODS or GLOBUS

owner
: A specific user set at system creation. By default this is the resolved value for `${apiUserId}`, the user making the request to create the system.

***host* - Host name, IP address or Globus ID**
: FQDN, IP address, Globus endpoint ID or Globus collection ID.

***enabled* - Enabled flag**
Indicates if system is currently considered active and available for use. By default this is *true*.

***effectiveUserId* - Effective User**
The username to use when accessing the system. A specific user (such as a service account) or the dynamic user `${apiUserId}`. By default this is `${apiUserId}`. For more information please see the section below on *Effective User Id and Host Login* .

***defaultAuthnMethod* - Default authentication method**
How access authentication is handled by default. Authentication method can also be specified as part of a request. Supported methods: PASSWORD, PKI_KEYS, ACCESS_KEY, TOKEN, TMS_KEYS.

***bucketName* - Bucket name**
For an S3 system this is the name of the bucket.

***rootDir* - Effective root directory**
Directory to be used when listing files or moving files to and from the system. All paths are relative to this directory when using Files to list, copy, move, mkdir, etc. For more information please see *Effective Root Directory*. May not be updated. Contact support to request a change.

***dtnSystemId* - DTN system Id**
A system that can be used during job execution as a Data Transfer Node (DTN). Use is optional. The DTN is used if the job submission request or the application defintion specify *dtnSystemInputDir* or *dtnSystemOutputDir*. *canExec* must be true. This execution system and the DTN system must have the same *rootDir* and the file system must be shared storage. Please see DTN Configuration.

canExec
Indicates if system can be used to execute jobs.

canRunBatch
Indicates if system supports running jobs using a batch scheduler. By default this is *false*.

enableCmdPrefix
Indicates if system allows a job submission request to specify a *cmdPrefix*. Since *cmdPrefix* is a free form command it is a security concern. By default this is *false*.

allowChildren
Indicates if system supports creating child systems using this system as the parent. By default this is *false*.

Job related attributes
Various attributes related to job execution such as *jobRuntimes*, *jobWorkingDir*, *batchScheduler*, *batchSchedulerProfile*, *batchLogicalQueues*

When creating a system the required attributes are: *id*, *systemType*, *host*, *defaultAuthnMethod* and *canExec*. Depending on the type of system and specific values for certain attributes there are other requirements.

Note that a system may be created as a storage-only resource (*canExec=false*) or as a system that can be used for both execution and storage (*canExec=true*).

4.2.1 Dynamically Determined System Attributes

When fetching a system or list of systems, some attributes are computed dynamically and shown in the results. The dynamically determined attributes are:

effectiveUserId
The resolved username to use when accessing the host or service. If system is defined with a dynamic user, i.e. `effectiveUserId = ${apiUserId}`, this is filled in based on the identity of the user making a request. For more information please see the section below on *Effective User Id and Host Login* .

isDynamicEffectiveUser
Indicates if *effectiveUserId* was resolved using `${apiUserId}`.

isPublic
Indicates if the system has been shared publicly. For more information please see the section below on *Sharing* .

sharedWithUsers
The list of users with whom the system has been shared.

4.3 Getting Started

Before going into further details about Systems, here we give some examples of how to create and view systems. In the examples below we assume you are using the TACC tenant with a base URL of `tacc.tapis.io` and that you have authenticated using PySDK or obtained an authorization token and stored it in the environment variable JWT, or perhaps both.

4.3.1 Creating a System

Create a local file named `system_example.json` with json similar to the following:

```
{
  "id":"tacc-sample-<userid>",
  "description":"My storage system",
  "host":"tapis-vm.tacc.utexas.edu",
  "systemType":"LINUX",
  "defaultAuthnMethod":"PKI_KEYS",
  "effectiveUserId":"${apiUserId}",
  "rootDir":"/",
  "canExec": false
}
```

where *<userid>* is replaced with your username, and your host name is updated appropriately. Note that although credentials may be included in the definition we have not done so here. For security reasons, it is recommended that login credentials be updated using a separate API call as discussed below.

Using PySDK:

```
import json
from tapipy.tapis import Tapis
t = Tapis(base_url='https://tacc.tapis.io', username='<userid>', password='************')
with open('system_example.json', 'r') as openfile:
    my_storage_system = json.load(openfile)
t.systems.createSystem(**my_storage_system)
```

Using CURL:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.
↪tapis.io/v3/systems -d @system_example.json
```

4.3.2 Effective Root Directory

Correctly defining the system attribute *rootDir* is critical because it serves as an effective root directory when referencing file paths through the Tapis Files or Jobs services. All paths are relative to this directory when using Files to list, copy, move, mkdir, etc. When creating a system there are certain restrictions for this attribute that should be kept in mind:

- Once a system is created, *rootDir* may not be updated. Contact support to request a change.
- Required for systems of type LINUX or IRODS. Must begin with `/`.

- Optional for systems of type S3 or GLOBUS.
- For S3 type systems, typically will not begin with `/`.
  - S3 keys are usually created and manipulated using URLs and do not have a leading `/`.
- Support is provided for resolving the following variables at create time: *${apiUserId}*, *${tenant}* and *${owner}*.

Use of HOST_EVAL at System Creation

There is also a special macro available for *rootDir* that may be used under certain conditions when a system is first created. The macro name is `HOST_EVAL`. The syntax for the macro is `HOST_EVAL($var)`, where `var` is the environment variable to be evaluated on the system host when the create request is made. Note that the `$` character preceding the environment variable name is optional. If after resolution the final path does not have the required leading slash (`/`) to make it an absolute path, then one will be prepended. The following conditions must be met in order to use the macro

- System must be of type LINUX
- Credentials must be provided when system is created.
- Macro `HOST_EVAL()` must only appear once and must be the first element of the path. Including a leading slash is optional.
- The *effectiveUserId* for the system must be static. Note that *effectiveUserId* may be set to `${owner}`.

Here are some examples

- HOST_EVAL($SCRATCH)
- HOST_EVAL($HOME)
- /HOST_EVAL(MY_ROOT_DIR)/scratch
- /HOST_EVAL($PROJECT_HOME)/projects/${tenant}/${owner}

4.3.3 Effective User Id and Host Login

The attribute *effectiveUserId* determines the host login user, the user used to access the underlying host or service. The attribute can be set to a static string indicating a specific user (such as a service account) or dynamically specified as `${apiUserId}`. For the case of `${apiUserId}`, the service resolves the variable by extracting the identity from the request to the service (i.e. the JWT) and applying a mapping to a host login user if such a mapping has been provided. If no mapping is provided, then the extracted identity is taken to be the host login user.

Host Login User Mapping

A mapping between a Tapis user and a host login user is created when the system has a dynamic *effectiveUserId* and the attribute *loginUser* is included when registering credentials. Please see the section below on *Registering Credentials for a System*.

For example, if my user id when logging into Tapis is `jdoe` and my host login user id is `jdoe3`, then a login user mapping would be required if the system is defined using a dynamic *effectiveUserId*. Note that if my Tapis user id happened to also be `jdoe3` then no mapping would be required.

Please note that if the system is defined using a static *effectiveUserId*, then there is no need for a mapping. In this case the *effectiveUserId* is logically independent of the Tapis identity and may be set to any valid host username value.

4.3.4 Registering Credentials for a System

Now that you have registered a system you will need to register credentials so you can use Tapis to access the host. Various authentication methods can be used to access a system. Supported methods are PASSWORD, PKI_KEYS, ACCESS_KEY, TOKEN and TMS_KEYS. The process of registering credentials can vary significantly depending

on the authentication method. For more information please see the appropriate section below under *Authentication Credentials*.

Please note that there is support for only one set of credentials per user per system. Updating credentials overwrites previously registered data.

Here we will cover registering PKI_KEYS (i.e. ssh keys) as an example. Please note that registering ssh keys requires special care when translating the generated keypair information to json format. For more information please see *Use of PKI_KEYS as credentials* under the section on *Authentication Credentials*.

Create a local file named `cred_tmp.json` with json similar to the following:

```
{
  "publicKey": "<ssh_public_key>",
  "privateKey": "<ssh_private_key>"
}
```

where <*ssh_public_key*> and <*ssh_private_key*> are replaced with your keys. The keys must be encoded on a single line with embedded newline characters. You may find the following linux command useful in converting a multi-line private key into a single line:

```
cat $privateKeyFile | awk -v ORS='\\n' '1'
```

Using PySDK:

```
t.systems.createUserCredential(systemId='tacc-sample-<userid>', userName='<userid>',␣
↪publicKey='<ssh_public_key>', privateKey='<ssh_private_key>'))
```

Using CURL:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.
↪tapis.io/v3/systems/credential/tacc-sample-<userid>/user/<userid> -d @cred_tmp.json
```

An optional attribute *loginUser* may be included in the request body in order to map the Tapis user to a username to be used when accessing the system. If the login user is not provided then there is no mapping and the Tapis user is always used when accessing the system. When a *loginUser* is provided the json would be similar to the following:

```
{
  "publicKey": "<ssh_public_key>",
  "privateKey": "<ssh_private_key>",
  "loginUser": "<linux_host_username>"
}
```

Note that credentials are stored in the Security Kernel. Only specific Tapis services are authorized to retrieve credentials.

4.3.5 Viewing Systems

Retrieving details for a system

To retrieve details for a specific system, such as the one above:

> **Note**
>
> See the section below on *Selecting* to find out how to control the amount of information returned.

Using PySDK:

```
t.systems.getSystem(systemId='tacc-sample-<userid>')
```

Using CURL:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems/tacc-sample-<userid>
```

The response should look similar to the following:


```
{
   "result": {
       "tenant": "dev",
       "id": "tacc-sample-<userid>",
       "description": "My storage system",
       "systemType": "LINUX",
       "owner": "<userid>",
       "host": "tapis-vm.tacc.utexas.edu",
       "enabled": true,
       "effectiveUserId": "<userid>",
       "defaultAuthnMethod": "PKI_KEYS",
       "authnCredential": null,
       "rootDir": "/",
       "port": 22,
       "useProxy": false,
       "proxyHost": "",
       "proxyPort": -1,
       "dtnSystemId": null,
       "canExec": false,
       "canRunBatch": false,
       "enableCmdPrefix": false,
       "allowChildren": false,
       "jobRuntimes": [],
       "jobWorkingDir": null,
       "jobEnvVariables": [],
       "jobMaxJobs": 2147483647,
       "jobMaxJobsPerUser": 2147483647,
       "batchScheduler": null,
       "batchSchedulerProfile": null,
       "batchLogicalQueues": [],
       "batchDefaultLogicalQueue": null,
       "jobCapabilities": [],
       "tags": [],
       "notes": {},
       "uuid": "f83606bf-7a1a-4ff0-9953-dd732cc07ac0",
       "deleted": false,
       "created": "2021-04-26T18:45:40.771Z",
       "updated": "2021-04-26T18:45:40.771Z"
   },
   "status": "success",
   "message": "TAPIS_FOUND System found: tacc-sample-<userid>",
   "version": "0.0.1",
   "metadata": null
}
```


Note that authnCredential is *null*. Only specific Tapis services are authorized to retrieve credentials.

Retrieving details for all systems

To see the list of systems that you own:

Using PySDK:

```
t.systems.getSystems()
```

Using CURL:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems?select=allAttributes
```

The response should contain a list of items similar to the single listing shown above.

Note

See the sections below on *Searching*, *Selecting*, *Sorting* and *Limiting* to find out how to control the amount of information returned.

4.3.6 Child Systems

Creating Child Systems

A system that has *allowChildren* set to *true* allows for creation of child systems based on it. This ability provides a way to easily clone and manage systems based on existing systems. Child systems allow a user to set only a few fields, and use all other values from an existing parent system. This can reduce the difficulty in managing systems. It allows for all child systems to be updated when the parent is updated.

To create a child system, first ensure that the system intended to serve as the parent as *allowChildren* set to *true*. Next, create a local file (for example child_system_example.json) similar to the following:

```
{
   "id": "my-child-<userid>",
   "effectiveUserId": "${apiUserId}",
   "rootDir": "/home/<userid>"
}
```

Where *<userid>* is replaced with your username. Also ensure that the root directory path is correct. Now use the create child system REST endpoint to create the child system. Let's assume that the new child system will be a child of a parent system called *parent-system*.

Using PySDK:

```
import json
from tapipy.tapis import Tapis
t = Tapis(base_url='https://tacc.tapis.io', username='<userid>', password='************')
with open('child_system_example.json', 'r') as openfile:
    child_system = json.load(openfile)
t.systems.createChildSystem(parentId="parent-system", **child_system)
```

Using CURL:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.
↪tapis.io/v3/systems/parent-system/createChildSystem -d @child_system_example.json
```

These fields are maintained independently for child systems:

Attribute	Type	Example	Notes
id	String	ds1.storage.default	• Identifier for the system. URI safe, see RFC 3986. • *tenant* + *id* must be unique. • Allowed characters: Alphanumeric [0-9a-zA-Z] and special characters [-._~].
owner	String	jdoe	• username of *owner*. • Variable references: *${apiUserId}*. Resolved at create time. • By default this is the resolved value for *${apiUserId}*.
enabled	boolean	FALSE	• Indicates if system currently enabled for use. • May be updated using the enable/disable endpoints. • By default this is *true*.
effectiveUserId	String	tg869834	• User to use when accessing the system. • May be a static string or a variable reference. • Variable references: *${apiUserId}*, *${owner}* • On output variable reference will be resolved.
rootDir	String	/home/${apiUserId}	• Required if *systemType* is LINUX or IRODS. • For LINUX or IRODS must begin with /. • Optional for S3 and GLOBUS. For S3 will typically not begin with /. • Variable references are resolved at cre-

During the creation of a child system, any of these fields may be specified except for created, updated and deleted. All other fields are taken from the parent system.

Updating a Child System

Updates are done just like any other system, however, only the following fields may be updated for a child system.

Attribute	Type	Example	Notes
effectiveUserId	String	tg869834	• User to use when accessing the system. • May be a static string or a variable reference. • Variable references: *${apiUserId}*, *${owner}* • On output variable reference will be resolved.

Some other fields can be updated through special endpoints. For example deleted and enabled are updated through the endpoints for deleting, undeleting, enabling and disabling.

Child System Operations

Most operations other than update are the same for child systems as they are for parent systems. For more information see the appropriate section of the document for the operation.

- Delete - see *Deletion*
- Undelete - see *Deletion*
- Enable - see “enabled” in *System Attributes Table*
- Disable - see “enabled” in *System Attributes Table*

Unlinking a Child System from it’s Parent System

A child system may be unlinked from it’s parent. This is a permanent operation, and cannot be undone. This will make the child a standalone system with all of it’s current settings. When the unlink happens any fields that had previously been linked to the parent will be copied to the child, and it will be as if the child was created as in independent system with those values.

If the owner of the child system wants to unlink the child from it’s parent, the owner may use the *unlinkFromParent* endpoint.

Using PySDK:

```
import json
from tapipy.tapis import Tapis
t = Tapis(base_url='https://tacc.tapis.io', username='<userid>', password='************')
t.systems.unlinkFromParent(childSystemId="<child-system-id>")
```

Using CURL:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.
↪tapis.io/v3/systems/<child-system-id>/unlinkFromParent
```

Replace <*child-system-id*> with the id of the child system.

The owner of a parent system can also decide to unlink child systems from the parent. In that case the parent system owner would use the *unlinkChildren* endpoint. The child systems to unlink may be specified in the request body. First create a json file (for example children_to_unlink.json):

```
{
   "childSystemIds":
   [
     "<child-system-1-id>",
     "<child-system-2-id>"
     ...
   ]
}
```

Using PySDK:

```
import json
from tapipy.tapis import Tapis
t = Tapis(base_url='https://tacc.tapis.io', username='<userid>', password='************')
with open('children_to_unlink.json', 'r') as openfile:
    children_to_unlink = json.load(openfile)
t.systems.unlinkChildren(parentSystemId="<parent-system-id>", **children_to_unlink)
```

Using CURL:

```
$curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.
↪tapis.io/v3/systems/<parent-system-id>/unlinkChildren -d @./children_to_unlink.json
```

Or all child systems using *all=True* (no json file required)

Using PySDK:

```
import json
from tapipy.tapis import Tapis
t = Tapis(base_url='https://tacc.tapis.io', username='<userid>', password='************')
t.systems.unlinkChildren(parentSystemId="<parent-system-id>", all=True)
```

Using CURL:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" "https://
↪tacc.tapis.io/v3/systems/<parent-system-id>/unlinkChildren?all=true"
```

4.4 Minimal Definition and Restrictions

When creating a system the required attributes are: *id*, *systemType*, *host*, *defaultAuthnMethod* and *canExec*. Depending on the type of system and specific values for certain attributes there are other requirements. The restrictions are:

- If *systemType* is S3 then *bucketName* is required and *canExec* must be false.
- If *systemType* is LINUX or IRODS then *rootDir* is required and must begin with `/`.
- If *effectiveUserId* is `${apiUserId}` (i.e. it is not static) then *authnCredential* may not be specified.

- If *canExec* is true then *jobWorkingDir* is required and *jobRuntimes* must have at least one entry.
- If *canRunBatch* is true then *batchScheduler* must be specified.
- If *canRunBatch* is true then *batchLogicalQueues* must have at least one item.
  - If *batchLogicalQueues* has more than one item then *batchLogicalDefaultQueue* must be specified.
  - If *batchLogicalQueues* has exactly one item then *batchLogicalDefaultQueue* is set to that item.

4.5 Permissions

The permissions model allows for fine grained access control of Tapis systems.

At system creation time the owner is given full access to the system. Permissions for other users may be granted and revoked through the systems API. Please note that grants and revokes through this service only impact the default role for the user. A user may still have access through permissions in another role. So even after revoking permissions through this service, when permissions are retrieved the access may still be listed. This indicates access has been granted via another role.

Permissions are specified as either `*` for all permissions or some combination of the following specific permissions: `("READ","MODIFY","EXECUTE")`. Specifying permissions in all lower case is also allowed. Having `MODIFY` implies `READ`.

4.6 Sharing

In addition to fine grained permissions support, Tapis also supports a higher level approach to granting access. This approach is known simply as *sharing*. The sharing API allows you to share a system with a set of users as well as share publicly with all users in a tenant. Sharing provides `READ+EXECUTE` access. When the system has a dynamic *effectiveUserId*, sharing also allows for MODIFY access to all file paths for calls made through the Files service.

Note

Note that there is one other case when a system is treated as having a dynamic *effectiverUserId* in the context of sharing, even with a static *effectiverUserId*. This is when the system type is `IRODS` and the attribute *useProxy* is set to `true`. In this case the connection to the *IRODS* host is made using a special administrative account which then acts as the Tapis user. So please be aware that for this type of system sharing the system or a file path will allow for MODIFY access.

The most common use case for sharing a system is to publicly share the system with all users in the tenant. This would allow any user to use the system for execution or storage when running an application.

Note

If a system has a dynamic *effectiveUserId* and has been shared publicly or with specific users, then those users will have Tapis permissions to operate on on any files without explicitly sharing any paths through the Files service. This includes file listing, upload, download and delete.

If a system has a static *effectiveUserId*, then file paths will need to be explicitly shared using the Files service in order to allow users READ access. Having READ access allows users to list and download files.

These restrictions are in place in order to reduce the risks associated with sharing a system. With a dynamic *effectiveUserId* users are always logging in to the host as themselves. With a static *effectiveUserId* there is a privilege escalation security risk.

Warning

In the context of using a shared application to run a job, sharing a system (and hence all file paths on the system) will grant users READ and MODIFY access to the file paths, even for the case of a static effectiveUserId.

Note

Tapis permissions and sharing are independent of native permissions enforced by the underlying system host.

For more information on sharing please see *Resource Sharing*

4.7 Authentication Credentials

At system creation time the authentication credentials may be specified if the effective access user *effectiveUserId* is a specific user (such as a service account) and not a dynamic user (i.e. not equal to `${apiUserId}`).

If the effective access user is dynamic (i.e. equal to `${apiUserId}`) then authentication credentials for any user allowed to access the system must be registered in separate API calls. In this case the payload provided may contain the optional attribute *loginUser* which will be used to map the Tapis user to a username to be used when accessing the system. If the login user is not provided then there is no mapping and the Tapis user is always used when accessing the system.

Note that the Systems service does not store credentials. Credentials are persisted by the Security Kernel service and only specific Tapis services are authorized to retrieve credentials.

Also, note that there is support for only one set of credentials per user per system. Updating credentials overwrites previously registered data.

By default any credentials provided for LINUX and S3 type systems are verified. The query parameter *skipCredentialCheck=true* may be used to bypass the initial verification of credentials.

4.7.1 Use of PKI_KEYS as credentials

When using an ssh keypair as credentials there are several important points to keep in mind. As discussed above, the public key and private key must be encoded on a single line. This can sometimes be challenging. For example, copying and pasting may convert newline characters in a way that is not compatible with processing in Tapis. You may find the following linux command useful in converting a multi-line private key into a single line:

```
cat $privateKeyFile | awk -v ORS='\\n' '1'
```

When generating the keypair, do not use a passphrase. This can interfere with non-interactive use of the keypair.

Finally, please be aware that if the host has multi-factor authentication (MFA) enabled this may prevent Tapis from communicating with the host. Tapis does not currently support MFA.

If problems are encountered here are some suggestions on what to check:

- Public and private keys are each on one line in the json file. Newline characters in private key are properly encoded.
- Keypair does not have a passphrase
- Public key has been added to the authorized_keys file for the target user. File ~/.ssh/authorized_keys
- File ~/.ssh/authorized_keys has proper permissions.
- MFA is not enabled for the target host.

If problems persist you can also attempt to manually validate the keypair using a command similar to this:

```
ssh -i /tmp/my_private_key testuser@myhost.com
```

where /tmp/my_private_key contains the original multi-line private key. If everything is set up correctly and the keypair is valid you should be logged into the host without being prompted for a password.

4.7.2 Use of TMS_KEYS for credentials

Tapis supports the use of the Trust Manager System (TMS) for managing credentials. For more information on TMS refer to TMS_Documentation.

Please note that your Tapis site installation must have been configured by the site administrator to support TMS. See TMS_Config.

Also, any target hosts defined in Tapis systems must be configured to use the TMS KeyCmd program for ssh connections. Please refer to the TMS documentation for details.

The integration of Tapis with TMS allows users to have Tapis automatically create and use SSH keypairs rather than having to provide their own. In order to register TMS credentials for a system, begin by making sure the system is defined with *defaultAuthnMethod* set to TMS_KEYS. Then when creating credentials simply add the flag `createTmsKeys=true`.

For example, the CURL command to create TMS keys for a system might look as follows:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.
↪tapis.io/v3/systems/credential/tms-test/user/<userid>?createTmsKeys=true -d @cred_tmp.
↪json
```

Note that the request body may be empty.

Please note that the following restrictions apply:

- Tapis installation for your site must be configured to support the Trust Manager System (TMS).
- The host for the system must have the sshd configuration set up to use TMS.
- The *effectiveUserId* must be dynamic.
- Mapping of user using *loginUser* is not supported.

4.7.3 Registering Credentials for a Globus System

Registering credentials for a GLOBUS type system is a special case that involves multiple steps and is significantly different compared to registering other types of credentials. For a GLOBUS type system, the user will need to use the TOKEN authentication method and generate an `accessToken` and `refreshToken` using two special-purpose System service endpoints.

Please note that your Tapis site installation must have been configured by the site administrator to support Globus. See Globus_Config.

Obtain Globus Authorization Code

The first step in generating Globus credentials is for the user to call the systems *authUrl* credential endpoint to obtain a Globus authorization code.

Using CURL, the request would look something like this:

```
$curl -H "X-Tapis-Token: $JWT" https://dev.tapis.io/v3/systems/credential/<system>/
↪globus/authUrl
```

The response should look similar to the following. Note that for brevity and readability, only the result portion of the response is shown, the response has been split into multiple lines and various IDs are not filled in:

```
{
  "url": "https://auth.globus.org/v2/oauth2/authorize?client_id=<client_id>
      &redirect_uri=https%3A%2F%2Fauth.globus.org%2Fv2%2Fweb%2Fauth-code
      &scope=openid+profile+email+urn%3Aglobus%3Aauth%3Ascope%3Atransfer.api.globus.org
↪%3Aall
      &state=_default&response_type=code&code_challenge=<challenge_id>
      &code_challenge_method=S256&access_type=offline",
  "sessionId": "<session_id>"
}
```

The user should copy the url (as a single string, no line breaks) and make note of the session Id for later use. The user then visits the provided URL and is presented with a Globus logon page that will allow them to authenticate using one of thousands of supported identity providers, including through their existing organization using CILogon.

The user must use the following flow to obtain an authorization code:

1. Visit the provided URL and authenticate through Globus. After authentication, user is re-directed back to a Globus page showing the access being requested by Tapis.
2. Fill in a label for future reference and click *Allow* to authorize Tapis to access Globus on their behalf.
3. Copy the provided authorization code in preparation for the final step. Note that the code is valid for a short time (as of this writing it is valid for 10 minutes).

Exchange Authorization Code for Tokens

The final step is for the user to call the systems credential endpoint to exchange the authorization code and session ID for tokens which are stored by the Systems service in a credentials record.

Using CURL, the request would look something like this:

```
$curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT"
       https://dev.tapis.io/v3/systems/credential/<system>/user/<user>/globus/tokens/
↪<authCode>/<sessionId>
```

The response should look similar to the following:

```
{
  "result": null,
  "status": "success",
  "message": "SYSAPI_CRED_UPDATED Credential updated. ...",
  "version": "1.3.1",
  "commit": "619aa7ce",
  "build": "2023-04-02T19:06:38Z",
  "metadata": null
}
```

At this point the user will have registered credentials for a Tapis system that can be used as a source or destination for Globus operations.

4.7.4 Registering Credentials for an AWS S3 System

Registering credentials for a Tapis S3 type system referencing an AWS bucket is a special case that is significantly different compared to registering other types of credentials. For such a system, the user will need to first manage the S3 bucket and IAM access tokens within AWS, then register the AWS access token with Tapis as a key pair.

Creating the S3 Bucket within AWS

The first step in registering S3 as a storage solution with Tapis is creating the S3 bucket. Tapis does not require any special considerations to be taken when creating the bucket, but you will need to keep track of the bucket's name.

Obtaining an Access Key Pair from AWS

Although creating an IAM user is not strictly necessary, it is highly recommended. You can create an access token for the root user by going to 'my security credentials' in the IAM console in AWS but this is not recommended for security reasons. Instead, it is recommended to create an IAM user for Tapis access. Make sure that when you create the user, you explicitly give it permissions to access S3. Refer to the IAM documentation for instructions on assigning permissions.

Take the following steps to obtain an access token pair for an IAM user:

1. In the IAM console in AWS, select the user who will be registered in the Tapis system.
2. Navigate to the 'Security Credentials' tab, and select 'Create access key'.
3. Go through the creation wizard, making sure that you save both the access key and the secret access key on the last page. You will need both of these in the next step to register the credentials with Tapis.

Create the S3 System in Tapis

Now that you have everything you need from AWS, create the Tapis system. Refer to the Creating a System section for instructions, making sure to use S3 for systemType and the name of the bucket for host.

The system definition should look something like this:

```
{
  "id": "demo-aws-s3-bucket",
  "description": "Demo S3 AWS acct.",
  "host":"tapisdemo-bucket.s3.amazonaws.com",
  "systemType": "S3",
  "effectiveUserId": "tapisdemo",
  "defaultAuthnMethod": "ACCESS_KEY",
  "bucketName": "tapisdemo-bucket",
  "rootDir": "",
  "canExec": false
}
```

Registering AWS Credentials with Tapis

Now that we have a bucket, AWS tokens, and a Tapis system, the last step is to register the AWS credentials with Tapis.

using CURL, we can do something like:

```
$curl -X POST -H "content-type:applications/json" -H "X-Tapis-Token: $JWT"
  -d {
    "accessKey":"< AWS access key name >",
    "accessSecret":"< AWS access key secret >"
```

(continues on next page)

(continued from previous page)

```
  }
  https://dev.tapis.io/v3/systems/credential/<system>/user/<user>
```

Which should return a response similar to

```
{
"result": null,
"status": "success",
"message": "SYSAPI_CRED_UPDATED Credential updated. ...",
"version": "1.8.2",
"commit": "9e30ecbf",
"build": "2025-02-25T14:24:36Z",
"metadata": null
}
```

4.8 Runtime

Runtime environment supported by the system that may be used to run applications, such as docker, singularity or ZIP. Consists of the runtime type and version.

4.9 Logical Batch Queue

A queue that maps to a single HPC queue. Logical batch queues provide a uniform front end abstraction for an HPC queue. They also provide more features and flexibility than is typically provided by an HPC scheduler. Multiple logical queues may be defined for each HPC queue. If an HPC queue does not have a corresponding logical queue defined then a user will not be able use the Tapis system to directly submit a job via Tapis to that HPC queue.

4.10 Batch Scheduler Profile

The Systems service supports managing Tapis scheduler profiles. An HPC center often has certain conventions and restrictions around the use of batch schedulers. A scheduler profile resource can be defined to provide the Tapis Jobs service with additional site specific information to be used when executing applications using a scheduler. A scheduler profile contains information on options that should be hidden from the scheduler, the module load command to use and which modules should be loaded by default when running a job. Anyone in a tenant may create a scheduler profile for use by all users in the tenant. The owner of a profile or a tenant administrator may modify or delete a profile. A profile may be referenced in a system definition using the attribute *batchSchedulerProfile*. The profile to be used may also be set in the job submit request using the special scheduler option *--tapis-profile*. The value in the job submit request takes precedence over a value defined for the execution system.

For example, at TACC there is a certain module that must be loaded when running Slurm jobs via singularity. Also, use of the Slurm option *--mem* is prohibited. In support of this, most of the tenants at TACC make use of a profile similar to the following:

```
{
    "name": "tacc",
    "owner": "testuser1",
    "description": "Profile for TACC Slurm",
    "moduleLoads": [
        {
            "moduleLoadCommand": "module load",
```

(continues on next page)

(continued from previous page)

```
            "modulesToLoad": ["tacc-singularity"]
        }
    ],
    "hiddenOptions": ["MEM"]
}
```

The *moduleLoads* array contains one or more entries. Each entry contains a *moduleLoadCommand*, which specifies the local command used to load each of the modules (in order) in its *modulesToLoad* list.

The *hiddenOptions* array identifies scheduler options that the local implementation prohibits. Options specified here will have the corresponding Slurm option suppressed. Supported options are "MEM" for *--mem* and "PARTITION" for *--partition*. Including an option in the array indicates that the corresponding Slurm option should never be passed through to Slurm.

4.11 Deletion

A system may be deleted and undeleted. Deletion means the system is marked as deleted and is no longer available for use. By default deleted systems will not be included in searches and operations on deleted systems will not be allowed. When listing systems the query parameter *showDeleted* may be used in order to include deleted systems in the results.

4.12 System Attributes Table

Attribute	Type	Example	Notes
tenant	String	designsafe	• Name of the tenant for which the system is defined. • *tenant* + *id* must be unique. • Determined by the service at system creation time.
id	String	ds1.storage.default	• Identifier for the system. URI safe, see RFC 3986. • *tenant* + *id* must be unique. • Allowed characters: Alphanumeric [0-9a-zA-Z] and special characters [-._~].
description	String	Default storage	• Description

continues on next page

Table 1 – continued from previous page

Attribute	Type	Example	Notes
systemType	enum	LINUX	• Type of system. • Types: LINUX, S3, IRODS, GLOBUS
owner	String	jdoe	• username of *owner*. • Variable references: *${apiUserId}*. Resolved at create time. • By default this is the resolved value for *${apiUserId}*.
host	String	data.tacc.utexas.edu	• Host name, ip address, Globus endpoint ID or Globus collection ID.
enabled	boolean	FALSE	• Indicates if system currently enabled for use. • May be updated using the enable/disable endpoints. • By default this is *true*.
effectiveUserId	String	tg869834	• User to use when accessing the system. • May be a static string or a variable reference. • Variable references: *${apiUserId}*, *${owner}* • On output variable reference will be resolved.

continues on next page

Table 1 – continued from previous page

Attribute	Type	Example	Notes
defaultAuthnMethod	enum	PKI_KEYS	• How access authentication is handled by default. • Can be overridden as part of a request to get a system or credential. • Methods: PASSWORD, PKI_KEYS, ACCESS_KEY, TOKEN, TMS_KEYS • See table *Credential Attributes* below for more information.

continues on next page

Table 1 – continued from previous page

Attribute	Type	Example	Notes
authnCredential	Credential		• On input credentials to be stored in Security Kernel. • *effectiveUserId* must be static, either a string constant or ${owner}. • May not be specified if *effectiveUserId* is dynamic, i.e. *${apiUserId}*. • On output contains credential for *effectiveUserId* and requested *authnMethod*. • Returned credential contains relevant information based on *authnMethod*. • Credentials may be updated using the systems credentials endpoint. • By default for LINUX the credentials are verified during create or update. • Use query parameter skipCredentialCheck=true to bypass initial verification. • See table *Credential Attributes* below for more information.
bucketName	String	tapis-ds1-jdoe	• Name of bucket for an S3 system. • Required if *systemType* is S3. • Variable references: *${apiUserId}*, *${owner}*, *${tenant}*

continues on next page

Table 1 – continued from previous page

Attribute	Type	Example	Notes
rootDir	String	/home/${apiUserId}	• Required if *systemType* is LINUX or IRODS. • For LINUX or IRODS must begin with /. • Optional for S3 and GLOBUS. For S3 will typically not begin with /. • Variable references are resolved at create time. • Serves as effective root directory when listing or moving files. • May not be updated. Contact support to request a change. • Variable references: *${apiUserId}*, *${owner}*, *${tenant}*
port	int	22	• Port number used to access the system

continues on next page

Table 1 – continued from previous page

Attribute	Type	Example	Notes
useProxy	boolean	TRUE	• Indicates if system should be accessed through a proxy. • Currently only supported for IRODS type systems. • Indicates if an IRODS proxy administrative user should be used. • *effectiveUserId* is the IRODS proxy admin user. • Tapis user making the request is the IRODS user who will be impersonated. • For this case *effectiveUserId* is considered dynamic in context of sharing.
proxyHost	String		• Name of proxy host. • Not currently supported. Please contact support if needed.
proxyPort	int		• Port number for *proxyHost* • Not currently supported. Please contact support if needed.

continues on next page

Table 1 – continued from previous page

Attribute	Type	Example	Notes
dtnSystemId	String	default.corral.dtn	• A system that can be used as a Data Transfer Node (DTN). Use is optional. • This system and *dtnSystemId* must have the same *rootDir* and shared storage. • Used if job submission or application specify a DTN input or output directory.
canExec	boolean		• Indicates if system will be used to execute jobs.
canRunBatch	boolean		• Indicates if system supports running jobs using a batch scheduler. • By default this is *false*.
enableCmdPrefix	boolean		• Indicates if system allows a job submission request to specify a cmdPrefix. • By default this is *false*.
allowChildren	boolean		• Indicates if system supports creating child systems using this system as parent. • By default this is *false*.

continues on next page

Table 1 – continued from previous page

Attribute	Type	Example	Notes
jobRuntimes	[Runtime]		• List of runtime environments supported by the system. • At least one entry required if *canExec* is true. • Each Runtime specifies the Runtime type and version • Runtime type is required and must be one of: DOCKER, SINGULARITY, ZIP. • Runtime version is optional.
jobWorkingDir	String	workdir	• Parent directory from which a job is run. • Relative to the effective root directory *rootDir*. • Required if *canExec* is true. • Variable references: *${apiUserId}*, *${owner}*, *${tenant}*
jobEnvVariables	[KeyValuePair]		• Environment variables added to the shell environment in which the job is running. • Added to environment variables specified in job and application definitions. • Each entry has *key* (required) and *value* (optional) as well as other attributes. • See table *KeyValuePair Attributes* below for more information.

continues on next page

Table 1 – continued from previous page

Attribute	Type	Example	Notes
jobMaxJobs	int		• Max total number of jobs . • Set to -1 for unlimited.
jobMaxJobsPerUser	int		• Max total number of jobs associated with a specific user. • Set to -1 for unlimited.
batchScheduler	String	SLURM	• Type of scheduler used when running batch jobs. • Schedulers: SLURM
batchSchedulerProfile	String		• Default Tapis scheduler profile for batch jobs.
batchLogicalQueues	[LogicalQueue]		• List of logical queues available on the system. • Each logical queue maps to a single HPC queue. • Multiple logical queues may be defined for each HPC queue. • See table *LogicalQueue Attributes* below for more information.
batchDefaultLogical Queue	LogicalQueue		• Default logical batch queue for the system.
tags	[String]		• List of tags as simple strings.

continues on next page

Table 1 – continued from previous page

Attribute	Type	Example	Notes
notes	String	"{}"	• Simple metadata in the form of a Json object. • Not used by Tapis.
uuid	UUID		• Auto-generated by service.
deleted	boolean	FALSE	• Indicates if system has been deleted. • May be updated using the delete/undelete endpoints.
created	Timestamp	2020-06-19T15:10:43Z	• When the system was created. Maintained by service.
updated	Timestamp	2020-07-04T23:21:22Z	• When the system was last updated. Maintained by service.

4.13 Credential Attributes Table

Attribute	Type	Example	Notes
user	String	jsmith	• Username associated with the credential.
authnMethod	String	PKI_KEYS	• Indicates the authentication method associated with a retrieved credential. • When a credential is retrieved it is for a specific authentication method. • Methods: PASSWORD, PKI_KEYS, ACCESS_KEY, TOKEN, TMS_KEYS
loginUser	String		• Optional native username valid on the system. • May be used to map a Tapis user to a native login user.
password	String		• Password for when authnMethod is PASSWORD. For LINUX and IRODS systems.
privateKey	String		• Private key for when authnMethod is PKI_KEYS. For LINUX systems.
publicKey	String		• Public key for when authnMethod is PKI_KEYS. For LINUX systems.
accessKey	String		• Access key for when authnMethod is ACCESS_KEY. For S3 systems.
accessSecret	String		• Access secret for when authnMethod is ACCESS_KEY. For S3 systems.

4.14 KeyValuePair Attributes Table

Attribute	Type	Example	Notes
key	String	"INPUT_FILE"	• Environment variable name. Required.
value	String	"/tmp/file.input"	• Environment variable value
description	String		• Description
inputMode	enum	REQUIRED	• Indicates how argument is to be treated when processing individual job requests. • Modes: REQUIRED, FIXED, INCLUDE_ON_DEMAND, INCLUDE_BY_DEFAULT • Default is INCLUDE_BY_DEFAULT. • REQUIRED: Must be provided in a job request or application definition. • FIXED: Not overridable in application or job request. • INCLUDE_ON_DEMAND: Included if referenced in a job request. • INCLUDE_BY_DEFAULT: Included unless *include=false* in a job request.
notes	String	"{}"	• Simple metadata in the form of a Json object. • Not used by Tapis.

4.15 LogicalQueue Attributes Table

Attribute	Type	Example	Notes
name	String	tapisNormal	• Name for logical queue. Typically will match or be a variant of HPC queue name.
hpcQueueName	String	normal	• Name of the HPC queue for which this logical queue is a front end.
maxJobs	int		• Maximum total number of jobs that can be queued or running in this queue.
maxJobsPerUser	int		• Maximum number of jobs associated with a specific user that can be queued.
minNodeCount	int		• Minimum number of nodes that can be requested when submitting a job to the queue.
maxNodeCount	int		• Maximum number of nodes that can be requested when submitting a job to the queue.
minCoresPerNode	int		• Minimum number of cores per node that can be requested when submitting a job. • Default is 1
maxCoresPerNode	int		• Maximum number of cores per node that can be requested when submitting a job.
minMemoryMB	int		• Minimum memory in megabytes that can be requested when submitting a

4.16 Scheduler Profile Attributes Table

Attribute	Type	Example	Notes
name	String	tacc	• Name. Required. *tenant* + *name* uniquely identifies the profile.
owner	String	jdoe	• Tapis user that created the profile.
description	String		• Description
moduleLoads	Array		• Each entry contains a module command and a list of modules to load.
hiddenOptions	String[]	[“MEM”]	• List of locally prohibited scheduler options that should be filtered out. • Allowed values: MEM, PARTITION

4.17 Searching

The service provides a way for users to search for systems based on a list of search conditions provided either as query parameters for a GET call or a list of conditions in a request body for a POST call to a dedicated search endpoint.

4.17.1 Search using GET

To search when using a GET request to the `systems` endpoint a list of search conditions may be specified using a query parameter named `search`. Each search condition must be surrounded with parentheses, have three parts separated by the character `.` and be joined using the character `~`. All conditions are combined using logical AND. The general form for specifying the query parameter is as follows:

```
?search=(<attribute_1>.<op_1>.<value_1>)~(<attribute_2>.<op_2>.<value_2>)~ ... ~(
↪<attribute_N>.<op_N>.<value_N>)
```

Attribute names are given in the table above and may be specified using Camel Case or Snake Case.

Supported operators: `eq` `neq` `gt` `gte` `lt` `lte` `in` `nin` `like` `nlike` `between` `nbetween`

Example CURL command to search for systems that have `Test` in the id, are of type LINUX, are using a port less than `1024` and have a default authentication method of either `PKI_KEYS` or `PASSWORD`:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems?search="(id.like.
↪*Test*)~(system_type.eq.LINUX)~(port.lt.1024)~(DefaultAuthnMethod.in.PKI_KEYS,PASSWORD)
↪"
```

Notes:

- For the `like` and `nlike` operators the wildcard character `*` matches zero or more characters and `!` matches exactly one character.
- For the `between` and `nbetween` operators the value must be a two item comma separated list of unquoted values.
- If there is only one condition the surrounding parentheses are optional.
- In a shell environment the character `&` separating query parameters must be escaped with a backslash.
- In a shell environment the query value must be surrounded by double quotes and the following characters must be escaped with a backslash in order to be properly interpreted by the shell:
  - `" \ ``
- Attribute names may be specified using Camel Case or Snake Case.
- Following complex attributes not supported when searching:
  - `authnCredential` `jobRuntimes` `jobEnvVariables` `batchLogicalQueues` `notes`

4.17.2 Dedicated Search Endpoint

The service provides the dedicated search endpoint `systems/search/systems` for specifying complex queries. Using a GET request to this endpoint provides functionality similar to above but with a different syntax. For more complex queries a POST request may be used with a request body specifying the search conditions using an SQL-like syntax.

Search using GET on Dedicated Endpoint

Sending a GET request to the search endpoint provides functionality very similar to that provided for the endpoint `systems` described above. A list of search conditions may be specified using a series of query parameters, one for each attribute. All conditions are combined using logical AND. The general form for specifying the query parameters is as follows:

```
?<attribute_1>.<op_1>=<value_1>&<attribute_2>.<op_2>=<value_2>)& ... &<attribute_N>.<op_
↪N>=<value_N>
```

Attribute names are given in the table above and may be specified using Camel Case or Snake Case.

Supported operators: `eq` `neq` `gt` `gte` `lt` `lte` `in` `nin` `like` `nlike` `between` `nbetween`

Example CURL command to search for systems that have `Test` in the name, are of type `LINUX`, are using a port less than `1024` and have a default authentication method of either `PKI_KEYS` or `PASSWORD`:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems/search/systems?name.
↪like=*Test*\&enabled.eq=true\&system_type.eq=LINUX\&DefaultAuthnMethod.in=PKI_KEYS,
↪PASSWORD
```

Notes:

- For the `like` and `nlike` operators the wildcard character `*` matches zero or more characters and `!` matches exactly one character.
- For the `between` and `nbetween` operators the value must be a two item comma separated list of unquoted values.
- In a shell environment the character `&` separating query parameters must be escaped with a backslash.

- Attribute names may be specified using Camel Case or Snake Case.
- Following complex attributes not supported when searching:
  - `authnCredential` `jobRuntimes` `jobEnvVariables` `batchLogicalQueues` `tags` `notes`

Search using POST on Dedicated Endpoint

More complex search queries are supported when sending a POST request to the endpoint `systems/search/systems`. For these requests the request body must contain json with a top level property name of `search`. The `search` property must contain an array of strings specifying the search criteria in an SQL-like syntax. The array of strings are concatenated to form the full search query. The full query must be in the form of an SQL-like `WHERE` clause. Note that not all SQL features are supported.

For example, to search for systems that are owned by `jdoe` and of type `LINUX` or owned by `jsmith` and using a port less than `1024` create a local file named `system_search.json` with following json:

```
{
  "search":
    [
      "(owner = 'jdoe' AND system_type = 'LINUX') OR",
      "(owner = 'jsmith' AND port < 1024)"
    ]
}
```

To execute the search use a CURL command similar to the following:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.
↪tapis.io/v3/systems/search/systems -d @system_search.json
```

Notes:

- String values must be surrounded by single quotes.
- Values for BETWEEN must be surrounded by single quotes.
- Search query parameters as described above may not be used in conjunction with a POST request.
- SQL features not supported include:
  - `IS NULL` and `IS NOT NULL`
  - Arithmetic operations
  - Unary operators
  - Specifying escape character for `LIKE` operator

Map of SQL operators to Tapis operators

Sql Operator	Tapis Operator
=	eq
<>	neq
<	lt
<=	lte
>	gt
>=	gte
LIKE	like
NOT LIKE	nlike
BETWEEN	between
NOT BETWEEN	nbetween
IN	in
NOT IN	nin

4.18 Sort, Limit, Select and ListType

When a list of Systems is retrieved the service provides for sorting, filtering and limiting the results. By default, only resources owned by you will be included. The service provides a way for you to request that all resources accessible to you be included. This is determined by the query parameter *listType*.

When retrieving either a list of resources or a single resource the service also provides a way to *select* which fields (i.e. attributes) are included in the results. Sorting, limiting and attribute selection are supported using query parameters.

4.18.1 Selecting

When retrieving systems the fields (i.e. attributes) to be returned may be specified as a comma separated list using a query parameter named `select`. Attribute names may be given using Camel Case or Snake Case.

Notes:

- Special select keywords are supported: `allAttributes` and `summaryAttributes`
- Summary attributes include:
  - `id`, `systemType`, `owner`, `host`, `effectiveUserId`, `defaultAuthnMethod`, `canExec`
- By default all attributes are returned when retrieving a single resource via the endpoint *systems/<system_id>*.
- By default summary attributes are returned when retrieving a list of systems.
- Specifying nested attributes is not supported.
- The attribute `id` is always returned.

For example, to return only the attributes `host` and `effectiveUserId` the CURL command would look like this:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems?select=host,
↪effectiveUserId
```

The response should look similar to the following:

```
{
 "result": [
       {
```

(continues on next page)

(continued from previous page)


```
            "id": "CSys_CltSrchGet_011",
            "host": "hostCltSrchGet_011",
            "effectiveUserId": "effUserCltSrchGet_011"
        },
        {
            "id": "CSys_CltSrchGet_012",
            "host": "hostCltSrchGet_012",
            "effectiveUserId": "effUserCltSrchGet_012"
        },
        {
            "id": "CSys_CltSrchGet_013",
            "host": "hostCltSrchGet_013",
            "effectiveUserId": "effUserCltSrchGet_013"
        }
    ],
    "status": "success",
    "message": "TAPIS_FOUND Systems found: 12 systems",
    "version": "1.0.0",
    "metadata": {
        "recordCount": 3,
        "recordLimit": 100,
        "recordsSkipped": 0,
        "orderBy": null,
        "startAfter": null,
        "totalCount": -1
    }
}
```


4.18.2 Sorting

The query parameter for sorting is named `orderBy` and the value is the attribute name to sort on with an optional sort direction. The general format is `<attribute_name>(<dir>)`. The direction may be `asc` for ascending or `desc` for descending. The default direction is ascending.

Examples:

- orderBy=id
- orderBy=id(asc)
- orderBy=name(desc),created
- orderBy=id(asc),created(desc)

4.18.3 Limiting

Additional query parameters may be used in order to limit the number and starting point for results. This is useful for implementing paging. The query parameters are:

- `limit` - Limit number of items returned. For example limit=10.
  - Use 0 or less for unlimited.
  - Default is 100.
- `skip` - Number of items to skip. For example skip=10.
  - May not be used with startAfter.

  - Default is 0.

- `startAfter` - Where to start when sorting. For example limit=10&orderBy=id(asc),created(desc)&startAfter=101
  - May not be used with `skip`.
  - Must also specify `orderBy`.
  - The value of `startAfter` applies to the major `orderBy` field.
  - Condition is context dependent. For ascending the condition is value > `startAfter` and for descending the condition is value < `startAfter`.

When implementing paging it is recommend to always use `orderBy` and when possible use `limit+startAfter` rather than `limit+skip`. Sorting should always be included since returned results are not guaranteed to be in the same order for each call. The combination of `limit+startAfter` is preferred because `limit+skip` is more likely to result in inconsistent results as records are added and removed. Using `limit+startAfter` works best when the attribute has a natural sequential ordering such as when an attribute represents a timestamp or a sequential ID.

4.18.4 ListType

By default, you will only see the resources that you own. The query parameter *listType* allows you to see additional resources that are available to you.

Options:

OWNED
: Include only items owned by you (Default)

SHARED_PUBLIC
: Include only items shared publicly

ALL
: Include all items you are authorized to view.

4.19 Tapis Responses

For requests that return a list of resources the response result object will contain the list of resource records that match the user's query and the response metadata object will contain information related to sorting and limiting.

The metadata object will contain the following information:

- `recordCount` - Actual number of records returned.
- `recordLimit` - The limit query parameter specified in the request. -1 if query parameter was not specified.
- `recordsSkipped` - The skip query parameter specified in the request. -1 if query parameter was not specified.
- `orderBy` - The orderBy query parameter specified in the request. Empty string if query parameter was not specified.
- `startAfter` - The startAfter query parameter specified in the request. Empty string if query parameter was not specified.
- `totalCount` - Total number of records that would have been returned without a limit query parameter being imposed. -1 if total count was not computed.

For performance reasons computation of `totalCount` is only determined on demand. This is controlled by the boolean query parameter `computeTotal`. By default `computeTotal` is *false*.

Example query and response:

Query:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/systems?limit=2&orderBy=id(desc)
```

Response:


```
{
 "result": [
   {
     "id": "testMin0",
     "systemType": "S3",
     "owner": "testuser",
     "host": "my.example.host",
     "defaultAccessMethod": "ACCESS_KEY",
     "canExec": false
   },
   {
     "id": "MinSystem1c",
     "systemType": "LINUX",
     "owner": "testuser",
     "defaultAccessMethod": "PASSWORD",
     "host": "data.tacc.utexas.edu",
     "canExec": true
   }
 ],
 "status": "success",
 "message": "TAPIS_FOUND Systems found: 2 systems",
 "version": "1.0.0",
 "metadata": {
   "recordCount": 2,
   "recordLimit": 2,
   "recordsSkipped": 0,
   "orderBy": "id(desc)",
   "startAfter": null,
   "totalCount": -1
 }
```

CHAPTER

FIVE

FILES

The files service is the central point of interaction for all file operations in the Tapis ecosystem.

5.1 Overview

Through the Files service users can perform file listing, uploading, and various operations such as move, copy, mkdir and delete. The service also supports transferring files from one Tapis system to another.

Currently, the files service includes support for systems of type LINUX, S3, IRODS and GLOBUS.

Note that supported functionality varies by system type.

All file operations act upon Tapis *System* resources.[1] For more information on the Systems service please see Systems

5.2 Basic File Operations

5.2.1 Listing

Tapis supports listing files or objects on a Tapis system. The type for items listed will depend on system type. For example, for LINUX they will be posix files and for S3 they will be storage objects. See the next section below for additional considerations for S3 type systems. On S3 systems, for example, the recurse flag is ignored and all objects with keys matching the path as a prefix are always included.

For system types that support directory hierarchies the maximum recursion depth is 20.

To list the files in the effective *rootDir* directory of a Tapis system:

Using the official Tapis Python SDK:

```
t.files.listFiles(systemId="my-tapis-system", path="/")
```

Or using curl:

```
curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/files/ops/my-system/
```

And to list a sub-directory in the system, just add the path to the request:

Using CURL

```
curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/files/ops/aturing-storage/subDir1/
↪subDir2/subDir3/
```

Query Parameters

[1] With the exception of the *sourceURI* in a transfer request when the protocol is *http* or *https*.

limit
integer - Max number of results to return, default of 1000

offset
integer - Skip the first N listings

The JSON response of the API will look something like this:


```
{
  "status": "success",
  "message": "ok",
  "result": [
      {
        "mimeType": "text/plain",
        "type": "file",
        "owner": "1003",
        "group": "1003",
        "nativePermissions": "drwxrwxr-x",
        "url": "tapis://dev/aturing-storage/file1.txt",
        "lastModified": "2021-04-29T16:55:57Z",
        "name": "file1.txt",
        "path": "file1.txt",
        "size": 313
      },
      {
        "mimeType": "text/plain",
        "type": "file",
        "owner": "1003",
        "group": "1003",
        "nativePermissions": "-rw-rw-r--",
        "url": "tapis://dev/aturing-storage/file2.txt",
        "lastModified": "2020-12-17T22:46:29Z",
        "name": "file2.txt",
        "path": "file2.txt",
        "size": 21
      }
  ],
  "version": "1.1-84a31617",
  "metadata": {}
}
```


Listings and S3 Support

File listings on S3 type systems have some special considerations. Objects in an S3 bucket do not have a hierarchical structure. There are no directories. Everything is an object associated with a key.

One thing to note is that, as mentioned above, for S3 the recurse flag is ignored and all objects with keys matching the path as a prefix are always included.

Note that for S3 this means that when the path is an empty string all objects in the bucket with a prefix matching *rootDir* will be included. This is especially important to keep in mind when using the delete operation to remove objects matching a path.

The attribute *rootDir* is optional for S3 type systems. When defined it will be prepended to all paths and the resulting path will become the key.

Note

When *rootDir* is defined for an S3 system it typically should not begin with /. For S3 keys are typically created and manipulated using URLs and do not have a leading /.

Handling of symbolic links on Linux systems

If listing contains a symbolic link, it will show type of symbolic link:

```
{
    "group": "1002",
    "lastModified": "2023-05-09T19:53:53Z",
    "mimeType": null,
    "name": "x2",
    "nativePermissions": "rwxrwxrwx",
    "owner": "1002",
    "path": "x2",
    "size": 4,
    "type": "symbolic_link",
    "url": "tapis://mysystem/mySymLinkedFileOrDirectory"
},
```

If a listing for a path that is a symbolic link is requested, the symbolic link is followed, and the information is returned for path that the symbolic link points to. If the path doesn't exist, and error will be returned. If the link points to a file, the file's information will be returned (the type will be "file"). If the symbolic link points to a directory, the contents of that directory will be returned.

5.2.2 Move and Copy

To move or copy a file or directory using the files service, make a PUT request using the path to the current location of the file or folder.

For example, to copy a file located at */file1.txt* to */subdir/file1.txt*

```
curl -H "X-Tapis-Token: $JWT" -X PUT -d @body.json "https://tacc.tapis.io/v3/files/ops/
↪aturing-storage/file1.txt"
```

with a JSON body of

```
{
  "operation": "COPY",
  "newPath": "/subdir/file1.txt"
}
```

Handling of symbolic links on Linux systems

During a move or copy, if a symbolic link is encountered, it will be handled as shown in the tables below. The first and second columns indicate whether the link is the source or target and if it points to a file or directory.

Copy

Symbolic Link	Points To	Notes
source	file	If the destination path is to a file (this means that the path ends in a component that does exist, and it's a file or a component that does not exist): • a new symbolic link is created that points to the same place as the source. • if the destination path includes directories that do not exist, they will be created. If the destination path is to a directory (this means that the path ends in a component that does exist and it is a directory): • If the path given is to an existing directory, a new link with the same name will be created in that directory, and it will point to the same place as the source. Note that if the link is to a relative path, moving it could change where it actually points because the exact relative path will remain the same.
source	directory	If the destination path is to a file (this means that the path ends in a component that does exist, and it's a file or a component that does not exist): • a new symbolic link is created that points to the same place as the source. • if the destination path includes directories that do not exist, they will be created. If the destination path is to a directory (this means that the path ends in a component that does exist and it is a directory): • If the path given is to an existing directory, a new link with the same name will be created in that directory, and it will point to the same place as the source. Note that if the link is to a relative path, moving it could change where it actually points because the exact relative path will remain the same.
destination	file	The destination is replaced by the source. The source could be a file, directory, or link to a file or directory.
destination	directory	The new file, directory, or link is created inside of the existing directory.

Move

Symbolic Link	Points To	Notes
source	file	If the destination path is to a file (this means that the path ends in a component that does exist, and it's a file or a component that does not exist): • the symbolic link is renamed. • if the destination path includes directories that do not exist, they will be created, and the new link will be placed there. If the destination path is to a directory (this means that the path ends in a component that does exist and it is a directory): • If the path given is to an existing directory, the link is moved to that directory. Note that if the link is to a relative path, moving it could change where it actually points because the exact relative path will remain the same.
source	directory	If the destination path is to a file (this means that the path ends in a component that does exist, and it's a file or a component that does not exist): • the symbolic link is renamed. • if the destination path includes directories that do not exist, they will be created. If the destination path is to a directory (this means that the path ends in a component that does exist and it is a directory): • if the path given is to an existing directory, the souce link will be moved into that directory, and it will point to the same place as the source. Note that if the link is to a relative path, moving it could change where it actually points because the exact relative path will remain the same.
destination	file	The source link is renamed, to the destination path. The destination is replaced. The source could be a file, directory, or link to a file or directory.
destination	directory	The file, directory, or link is moved inside of the existing directory.

5.2.3 Making directories

To create a directory on a tapis system at the given path, issue a mkdir request. This is not supported for all system types. The mkdir operation is currently supported for LINUX, IRODS and GLOBUS type systems.

Using the Tapis Python SDK:

```
t.files.mkdir(systemId="my-system", path="/folderA/folderB/newDirectory")
```

Using CURL:

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json" -X POST https://tacc.
↪tapis.io/v3/files/ops/<systemId> -d '{"path":"<directory_path"}'
```

5.2.4 Uploading

To upload a file use a POST request. The file will be placed at the location specified in the *{path}* parameter in the request. Not all system types support this operation. For example, given the system *my-system*, to upload file *someFile.txt* to directory */folderA/folderB/folderC*:

Using the official Tapis Python SDK:

```
t.upload(source_file_path="experiment-results.hd5", system_id="my-system", dest_file_
↪path="/folderA/folderB/folderC/someFile.txt")
```

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json" -X POST -F
↪"file=@someFile.txt" https://tacc.tapis.io/v3/files/ops/my-system/folderA/folderB/
↪folderC/someFile.txt
```

For some system types (such as LINUX) any folders that do not exist in the specified path will automatically be created.

Note that for an S3 system an object will be created with a key of *rootDir*/{path}.

5.2.5 Deleting

To delete a file or folder, issue a DELETE request for the path to be removed.

```
curl -H "X-Tapis-Token: $JWT" -X DELETE "https://tacc.tapis.io/v3/files/ops/aturing-
↪storage/file1.txt"
```

The request above would delete `file1.txt`

For an S3 system, the path will represent either a single object or all objects in the bucket with a prefix matching the system *rootDir* if the path is the empty string.

Warning

For an S3 system if the path is the empty string, then all objects in the bucket with a key matching the prefix *rootDir* will be deleted. So if the *rootDir* is also the empty string, then all objects in the bucket will be removed.

Handling of symbolic links and special files on Linux Systems

If Tapis encounters a symbolic link during a delete operation, the link will be deleted. The file or directory that the link points to will be unaffected. If the delete encounters a special file (such as a device file or fifo, etc), it will not be deleted, and an error will be returned. If this is in the middle of a recursive delete operation, some files may have been already deleted.

5.2.6 Creating a directory

To create a directory, use POST and provide the path to the new directory in the request body. Not all system types support this operation.

```
$ curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json" -X POST -d @body.json␣
↪https://tacc.tapis.io/v3/files/ops/my-system
```

with a JSON body of

```
{
  "path": "path/to/new/directory/"
}
```

5.2.7 Getting Linux stat information

Get native stat information for a file or directory for a system of type LINUX.

For example, for */subdir/file1.txt*

```
curl -H "X-Tapis-Token: $JWT" "https://tacc.tapis.io/v3/files/utils/linux/aturing-
↪storage/subdir/file1.txt"
```

5.2.8 Running a Linux native operation

Run a native operation on a path. Operations are *chmod*, *chown* or *chgrp*. For a system of type LINUX.

For example, to change the owner of a file located at */file1.txt* to `aeinstein`

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json" -X POST -d @body.json
↪"https://tacc.tapis.io/v3/files/utils/linux/aturing-storage/file1.txt"
```

with a JSON body of

```
{
  "operation": "CHOWN",
  "argument": "aeinstein"
}
```

5.3 Content

Get file or directory contents as a stream of data. Not supported for all system types.

5.3.1 File Contents - Serving files

To return the actual contents (raw bytes) of a file:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/files/content/my-system/image.
↪jpg > image.jpg
```

Query Parameters

startByte
integer - Start at byte N of the file

count
integer - Return this number of bytes after startByte

zip
boolean - Zip the contents of a folder

Header Parameters

more
integer - Return 1 KB chunks of UTF-8 encoded text from a file starting after page *more*. This call can be used to page through a text based file. Note that if the contents of the file are not textual (such as an image file or other binary format), the output will be bizarre.

5.3.2 Download using ZIP

The query parameter *zip* may be used to request a stream compressed using the ZIP file format. This is not allowed if system *rootDir* plus *path* would result in all files on the host being included. Please download individual directories, files or objects.

For example, on a linux system a directory may be downloaded as a compressed archive using a command similar to the following:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/files/content/my-linux-system/
↪my_dir > my_dir.zip
```

The program *unzip* may then be used to extract the contents.

If the path being downloaded is a single file and the contents are placed in a file ending in the extension *.gz* then the *gunzip* utility may also be used to extract the contents.

5.4 Transfers

File transfers are used to move data between Tapis systems. They should be used for bulk data operations that are too large for the REST api to perform. Transfers occur *asynchronously*, and are executed concurrently where possible to increase performance. As such, the order in which the files are transferred is not deterministic.

When a transfer is initiated, a *bill of materials* is created that creates a record of all the files from the *sourceURI* that are to be transferred to the *destinationURI*. Unless otherwise specified, all files in the *bill of materials* must transfer successfully in order for the overall transfer to be considered successful. A transfer task has an attribute named *status* which is updated as the transfer progresses. The possible states for a transfer are:

ACCEPTED
The initial request has been processed and saved.

IN_PROGRESS
The bill of materials has been created and transfers are either in flight or waiting to begin.

FAILED
The transfer failed.

COMPLETED
The transfer completed successfully, all files have been transferred to the target system.

Unauthenticated HTTP endpoints are also possible to use as a source for transfers. This method can be utilized to include outputs from other APIs into Tapis jobs.

Note

For transfers involving Globus, both the source and destination system must be of type GLOBUS.

The number of files included in the *bill of materials* will depend on the system types and the *sourceURI* values provided in the transfer request. If the source system supports directories and *sourceURI* is a directory then the directory will be processed recursively and all files will be added to the *bill of materials*. If the source system is of type S3 then all objects matching the *sourceURI* path as a prefix will be included.

5.4.1 System types and supported functionality

As discussed above, the files included in a transfer will depend on the source system types and the *sourceURI* values provided in the transfer request. Here is a summary of the behavior:

LINUX/IRODS to LINUX/IRODS
When the *sourceURI* is a directory a recursive listing is made and the files and directory structure are replicated on the *destinationURI* system.

S3 to LINUX/IRODS
All objects matching the *sourceURI* path as a prefix will be created as files on the *destinationURI* system.

LINUX/IRODS to S3
When the *sourceURI* is a directory a recursive listing is made. For each entry in the listing the path relative to the source system rootDir is mapped to a key for the S3 destination system. In other words, a recursive listing is made for the directory on the *sourceURI* system and for each non-directory entry an object is created on the S3 *destinationURI* system.

S3 to S3
All objects matching the *sourceURI* path as a prefix will be re-created as objects on the *destinationURI* system.

HTTP/S to ANY
Transfer of a directory is not supported. Destination system may not be of type GLOBUS. The content of the object from the *sourceURI* URL is used to create a single file or object on the *destinationURI* system.

ANY to HTTP/S
Transfers not supported. Tapis does not support the use of protocol http/s for the *destinationURI*.

5.4.2 Creating Transfers

Lets say our user `aturing` needs to transfer data between two systems that are registered in tapis. The source system has an id of `aturing-storage` with the results of an experiment located in directory `/experiments/experiment-1/` that should be transferred to a system with id `aturing-compute`

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json" -X POST -d @body.json␣
↪https://tacc.tapis.io/v3/files/tranfers
```

```
{
  "tag": "An optional identifier",
  "elements": [
    {
      "sourceURI": "tapis://aturing-storage/experiments/experiment-1/",
      "destinationURI": "tapis://aturing-compute/"
    }
  ]
}
```

The request above will initiate a transfer that copies all files and folders in the `experiment-1` folder on the source system to the root directory of the destination system `aturing-compute`

HTTP Source

Unauthenticated HTTP/S endpoints can also be used as a source for a file transfer request. This can be useful, for instance, when the inputs for a job are from a separate web service, or perhaps stored in a public S3 bucket. Note that in this case the *sourceURI* does not refer to a Tapis system.

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json" -X POST -d @body.json␣
↪https://tacc.tapis.io/v3/files/tranfers
```

```
{
  "tag": "An optional identifier",
  "elements": [
    {
      "sourceURI": "https://some-web-application.io/calculations/12345/results.csv",
      "destinationURI": "tapis://aturing-compute/inputs.csv"
    }
  ]
}
```

The request above will place the output of the source URI into a file called `inputs.csv` in the `aturing-compute` system.

5.4.3 Getting transfer information

To retrieve information about a transfer including status and bytes transferred, simply make a GET request to the transfers API with the UUID of the transfer.

```
curl -H "X-Tapis-Token: $JWT"  https://tacc.tapis.io/v3/files/tranfers/{UUID}
```

The JSON response should look something like :

```
{
  "status": "success",
  "message": "ok",
  "result": {
    "id": 1,
    "username": "aturing",
    "tenantId": "tacc",
    "tag": "some tag",
    "uuid": "b2dcf71a-bb7b-409a-8c01-1bbs97e749fb",
    "status": "COMPLETED",
    "parentTasks": [
      {
        "id": 17,
        "tenantId": "tacc",
        "username": "aturing",
        "sourceURI": "tapis://sourceSystem/file1.txt",
        "destinationURI": "tapis://destSystem/folderA/",
        "totalBytes": 100000,
        "bytesTransferred": 100000,
        "taskId": 1,
```

(continues on next page)

(continued from previous page)


```
        "children": null,
        "errorMessage": null,
        "uuid": "8fdccda6-a504-4ddf-9464-7b22sa66bcc4",
        "status": "COMPLETED",
        "created": "2021-04-22T14:21:58.933851Z",
        "startTime": "2021-04-22T14:21:59.862356Z",
        "endTime": "2021-04-22T14:22:09.389847Z"
      }
    ],
    "estimatedTotalBytes": 100000,
    "totalBytesTransferred": 100000,
    "totalTransfers": 1,
    "completeTransfers": 1,
    "errorMessage": null,
    "created": "2021-04-22T14:21:58.933851Z",
    "startTime": "2021-04-22T14:21:59.838928Z",
    "endTime": "2021-04-22T14:22:09.376740Z"
  },
  "version": "1.1-094fd38d",
  "metadata": {}
}
```


5.4.4 Handling of symbolic links on Linux Systems

Transfer will always "follow links". If the source of the transfer is a symbolic link to a file or directory, it transfer the file or directory that is pointed to. If it doesn't exist, it will be an error (i.e. the link points to something that doesn't exist). In the case of a directory transfer where one of the entries in the directory encountered is a symbolic link it will be resolved in exactly the same way - the file is added to the archive to be downloaded, or the directory is walked, and it's content added to the archive. Symbolic links can create situations where infinite recursion can occur - for example, suppose you have a directory with a link that points to "../". That means that each time it's expanded the current directory will be added, and the link will be expanded again. Transfers (and really all file operations that involve recursing subdirectories) are limited by a recursion depth. The current maximum depth is 20.

5.5 Support for Globus

Please note that your Tapis site installation must have been configured by the site administrator to support Globus. Please see Globus_Config.

The integration of Globus and Tapis allows users to configure and use Globus endpoints and collections just as they would other types of storage systems defined in Tapis. As mentioned previously, not all operations are supported for all system types. For systems of type GLOBUS, the following operations are supported:

- listing
- mkdir
- move
- delete
- transfer between GLOBUS systems

For more information on setting up and registering credentials for a system of type GLOBUS, please see Systems_Globus.

5.6 File Permissions

The permissions model allows for fine grained access control of paths on a Tapis system. The system owner may grant READ and MODIFY permission to specific users. MODIFY implies READ.

Please note that Tapis permissions are independent of native permissions enforced by the underlying system host.

5.6.1 Getting permissions

Get the Tapis permissions for a user for the system and path. If no user specified then permissions are retrieved for the user making the request.

```
curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/files/perms/aturing-storage/
↪experiment1?username=aeinstein
```

5.6.2 Granting permissions

Lets say our user `aturing` has a system with ID `aturing-storage`. Alan wishes to allow his collaborator `aeinstein` to view the results of an experiment located at `/experiment1`

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json" -X POST -d @body.json␣
↪https://tacc.tapis.io/v3/files/perms/aturing-storage/experiment1/
```

with a JSON body with the following shape:

```
{
  "username": "aeinstein",
  "permission": "READ"
}
```

Other users can also be granted permission to write to the system by granting the `MODIFY` permission. The JSON body would then be:

```
{
  "username": "aeinstein",
  "permission": "MODIFY"
}
```

5.6.3 Revoking permissions

Our user `aturing` now wishes to revoke his former collaborators access to the folder above. He can issue a DELETE request on the path and specify the username in order to revoke access:

```
curl -H "X-Tapis-Token: $JWT" -X DELETE https://tacc.tapis.io/v3/files/perms/aturing-
↪storage/experiment1?username=aeinstein
```

5.7 File Path Sharing

In addition to fine grained permissions support, Tapis also supports a higher level approach to granting access. This approach is known simply as *sharing*. The sharing API allows you to share a file path with a set of users as well as share publicly with all users in a tenant. Sharing provides READ access. When the system has a dynamic *effectiveUserId*, sharing also allows for MODIFY access to all paths for calls made through the Files service.

Note

Note that there is one other case when a system is treated as having a dynamic *effectiverUserId* in the context of sharing, even with a static *effectiverUserId*. This is when the system type is `IRODS` and the attribute *useProxy* is set to `true`. In this case the connection to the *IRODS* host is made using a special administrative account which then acts as the Tapis user. So please be aware that for this type of system sharing a file path will allow for MODIFY access.

Warning

In the context of using a shared application to run a job, sharing a path will grant users READ and MODIFY access to the path, even for the case of a static effectiveUserId.

Note

Tapis permissions and sharing are independent of native permissions enforced by the underlying system host.

For more information on sharing please see *Resource Sharing*

5.7.1 Getting share information

Retrieve all sharing information for a path on a system. This includes all users with whom the path has been shared and whether or not the path has been made publicly available.

```
curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/files/share/aturing-storage/
↪experiment1
```

5.7.2 Sharing a path with users

Create or update sharing information for a path on a system. The path will be shared with the list of users provided in the request body. Requester must be owner of the system. For LINUX systems path sharing is hierarchical. Sharing a path grants users READ access to the path or, in the context of running a job, it grants users READ and MODIFY access to the path.

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json" -X POST -d @body.json␣
↪https://tacc.tapis.io/v3/files/share/aturing-storage/experiment1/
```

with a JSON body with the following shape:

```
{
  "users": [ "aeinstein", "rfeynman" ]
}
```

5.7.3 Sharing a path publicly

Share a path on a system with all users in the tenant. Requester must be owner of the system. Sharing a path grants users READ access to the path or, in the context of running a job, it grants users READ and MODIFY access to the path.

```
curl -H "X-Tapis-Token: $JWT" -X POST https://tacc.tapis.io/v3/files/share_public/
↪aturing-storage/experiment1/
```

5.7.4 Unsharing a path with users

Update sharing information for a path on a system. The path will be unshared with the list of users provided in the request body. Requester must be owner of the system.

```
curl -H "X-Tapis-Token: $JWT" -H "Content-Type:application/json" -X POST -d @body.json␣
↪https://tacc.tapis.io/v3/files/unshare/aturing-storage/experiment1/
```

with a JSON body with the following shape:

```
{
  "users": [ "rfeynman" ]
}
```

5.7.5 Unsharing a path publicly

Remove public sharing for a path on a system. Requester must be owner of the system.

```
curl -H "X-Tapis-Token: $JWT" -X POST https://tacc.tapis.io/v3/files/unshare_public/
↪aturing-storage/experiment1/
```

5.7.6 Removing all shares for a path

Remove all shares for a path on a system including public access. If the path is a directory this will also be done for all sub-paths.

```
curl -H "X-Tapis-Token: $JWT" -X POST https://tacc.tapis.io/v3/files/unshare_all/aturing-
↪storage/experiment1/
```

5.8 PostIts

The PostIts service is a URL service that allows you to create pre-authenticated, disposable URLs to files, directories, buckets, etc in the Tapis Platform's Files service. You have control over the lifetime and number of times the URL can be redeemed, and you can expire a PostIt at any time. The most common use of PostIts is to create URLs to files so that you can share with others without having to upload them to a third-party service.

5.8.1 Creating PostIts

To create a PostIt, send a POST request to the Files service's Create PostIt endpoint. The url will contain the systemId and the path that will be shared. The body of the post will contain a json document which describes how long the PostIt is valid, and how many times it can be redeemed. There are default values for each of these parameters if they are not included. If the number of times the PostIt may be redeemed (allowedUses) is set to -1, the PostIt may be redeemed an unlimited number of times. The expiration (validSeconds) must always contain a value. If one is not provided, there is a default that is used. The maximum value of 'validSeconds' is the maximum integer value in Java (Integer.MAX_VALUE = 2147483647).

Note

The maximum value would result in a PostIt that would be valid for nearly 70 years, however it is important to remember that the authentication and authorization are built into the PostIt redeem url. This means anyone who has the url could redeem it. For this reason, it's advisable to keep the uses and expiration times to the minimum required.

Default parameters:

- allowedUses: 1 (One use)
- validSeconds: 2592000 (30 days)

APPLICATION/JSON examples

Creating a postit with the default expiration and uses on a system called "tapisv3-storage" for the path "/myDirectory/myFile.txt"

Using curl

```
curl -X POST "https://tacc.tapis.io/v3/files/postits/tapisv3-storage/myDirectory/myFile.
↪txt" -H "X-Tapis-Token: $JWT" -H "Content-Type: application/json"
```

Using python

```
tapis.files.createPostIt(systemId="tapisv3-storage", path="myDirectory/myFile.txt")
```

Creating a postit supplying expiration (validSeconds 600) and uses (allowedUses 3) on a system called "tapisv3-storage" for the path "/myDirectory/myFile.txt"

Using curl

```
curl -X POST "https://tacc.tapis.io/v3/files/postits/tapisv3-storage/myDirectory/myFile.
↪txt" -H "X-Tapis-Token: $JWT" -H "Content-Type: application/json" -d '{"allowedUses":␣
↪3, "validSeconds": 600}'
```

Using python

```
tapis.files.createPostIt(systemId="tapisv3-storage", path="myDirectory/myFile.txt",␣
↪allowedUses=3, validSeconds=600)
```

Creating a postit supplying allowing unlimited uses (allowedUses -1) and the default value for expiration (default value for validSeconds is 30 days) on a system called "tapisv3-storage" for the path "/myDirectory/myFile.txt"

Using curl

```
curl -X POST "https://tacc.tapis.io/v3/files/postits/tapisv3-storage/myDirectory/myFile.
↪txt" -H "X-Tapis-Token: $JWT" -H "Content-Type: application/json" -d '{"allowedUses": -
↪1}'
```

Using python

```
tapis.files.createPostIt(systemId="tapisv3-storage", path="myDirectory/myFile.txt",␣
↪allowedUses=-1)
```

Example Postit Creation Response

```
{
  "status": "success",
  "message": "FAPI_POSTITS_OP_COMPLETE Operation completed. jwtTenant: tacc jwtUser:␣
↪example_user OboTenant: tacc OboUser: examaple_user Operation: createPostIt System:␣
↪tapisv3-storage Path: myDirectory/myFile.txt Id: e614ce8e-447c-4195-a3f7-55f5dec5d243-
↪010",
  "result": {
    "id": "e614ce8e-447c-4195-a3f7-55f5dec5d243-010",
    "systemId": "tapisv3-storage",
    "path": "myDirectory/myFile.txt",
    "allowedUses": 3,
    "timesUsed": 0,
    "jwtUser": "example_user",
    "jwtTenantId": "tacc",
    "owner": "example_user",
    "tenantId": "tacc",
    "redeemUrl": "https://tacc.tapis.io/v3/files/postits/redeem/e614ce8e-447c-4195-a3f7-
↪55f5dec5d243-010",
    "expiration": "2023-03-08T15:37:28.533641Z",
    "created": "2023-03-08T15:27:28.534250Z",
    "updated": "2023-03-08T15:27:28.534250Z"
  },
  "version": "1.3.1",
  "commit": "0c13ee3c",
  "build": "2023-03-07T21:56:41Z\n",
  "metadata": {}
}
```


Note

The PostIt returned by the create will contain the redeemUrl. This url may be used to download the content pointed to by the PostIt. No Authentication will be done during this call. The credentials used to access this content will be the credentials of the PostIt owner. If the owner's permissions change between creating the PostIt and redeeming the PostIt so that the owner is no longer allowed to read the content, redeeming the PostIt will fail.

Create PostIt parameters

Name	Type	Location	Default	Description
systemId	String	url	<none>	The systemId of the system containing the path to create the PostIt for.
path	String	url	<none>	The path to create the PostIt for.
allowedUses	integer	body	1	The number of times a postit can be redeemed. Valid values are 1 - 2147483647, or -1 for unlimited uses.
validSeconds	integer	body	214748	The number of seconds from creation that the PostIt will be redeemable. An expiration time is computed by adding this value to the current date and time.

5.8.2 Listing PostIts

PostIts can be listed by authenticated users. By default a listing of all PostIts owned by the authenticated user will be returned.

Using curl

```
curl "https://tacc.tapis.io/v3/files/postits" -H "X-Tapis-Token: $JWT"
```

Using python

```
tapis.files.listPostIts()
```

To list all PostIts that are visible to the authenticated user, supply the query parameter listType and set it's value to ALL. Typically users will only be able to see PostIts that they own, however tenant admins will be allowed to see all PostIts in their tenant.

Using curl

```
curl "https://tacc.tapis.io/v3/files/postits?listType=ALL" -H "X-Tapis-Token: $JWT"
```

Using python

```
tapis.files.listPostIts(listType="ALL")
```

Paging is handled by the query parameters limit, skip, and startAfter. By default 100 PostIts are returned, however this can be changed by setting the query parameter "limit". Skip is used to determine how many PostIts to skip. For example to get the second page of a list of PostIts containing 10 items per page, you would need to set the limit to 10 (10 items per page), and set skip to 10 (skip the first page). It's probably a good practice to set orderBy also, so that the list is ordered in the same way each time. You could for example set orderBy to id.

Using curl

```
curl "https://tacc.tapis.io/v3/files/postits?listType=ALL&limit=10&skip=10&orderBy=id" -
↪H "X-Tapis-Token: $JWT"
```

Using python

```
tapis.files.listPostIts(listType="ALL", limit=10, skip=10, orderBy="id")
```

Using startAfter is similar to using skip. When using startAfter, you must provide a PostIt id, and the list will start immediately after that PostIt. You **must** provide the orderBy parameter. You may not use skip and startAfter together.

To control which fields are returned, you can supply the select query parameter, and select only certain fields. Setting the select query parameter to id, redeemUrl and expiration would return only those fields (select=id,redeemUrl,expiration). Setting "select" to allAttributes will return all attributes, and setting "select" to summaryAttributes will only return a preset collection of attributes. The default is summaryAttributes. You can also set the value to summaryAttributes with additional attributes (select=summaryAttributes,updated).

Using curl

```
curl "https://tacc.tapis.io/v3/files/postits?listType=ALL&select=id,redeemUrl,expiration
↪" -H "X-Tapis-Token: $JWT"
```

Using python

```
tapis.files.listPostIts(listType="ALL", select="id,redeemUrl,expiration")
```

5.8.3 Retrieving a Single PostIt

Retrieving a single PostIt can be done by issuing a GET request containing the id of the PostIt. This is **not** the same as redeeming, and does not add to the redeem count. This will allow the owner of the PostIt or a tenant admin to view the PostIt. This could be used to see the number of times it's been retrieved, total number of uses allowed, expiration date, etc.

Using curl

```
curl "https://tacc.tapis.io/v3/files/postits/e614ce8e-447c-4195-a3f7-55f5dec5d243-010" -
↪H "X-Tapis-Token: $JWT"
```

Using python

```
tapis.files.getPostIt(postitId="e614ce8e-447c-4195-a3f7-55f5dec5d243-010")
```

For tenant admins, any PostIts in the tenant can be retreived in this way. For other users only PostIts that are owned by that user may be retreived, since access to the redeem url allows redemption of the PostIt.

5.8.4 Updating PostIts

The creator of a PostIt and tenant admins can update a PostIt. When updating a PostIt, the id of the posted is sent as part of the url, and a body containing allowedUses and/or validSeconds can be specified.

Note

The validSeconds parameter will add to the date and time as of the update request to compute the new expiration. It does **not** extend the current expiration by that many seconds.

If you need to update the url, you will need to delete or expire this PostIt and create a new one.

Update a PostIt to allow for 10 uses and to expire in 1 hour

Using curl / PATCH

```
curl -X PATCH "https://tacc.tapis.io/v3/files/postits/e614ce8e-447c-4195-a3f7-
↪55f5dec5d243-010" -H "Content-Type: application/json" -H "X-Tapis-Token: $JWT" -d '{
↪"allowedUses":10, "validSeconds":3600 }'
```

Using curl / POST

```
curl -X POST "https://tacc.tapis.io/v3/files/postits/e614ce8e-447c-4195-a3f7-
↪55f5dec5d243-010" -H "Content-Type: application/json" -H "X-Tapis-Token: $JWT" -d '{
↪"allowedUses":10, "validSeconds":3600 }'
```

Using python

```
tapis.files.updatePostIt(postitId="e614ce8e-447c-4195-a3f7-55f5dec5d243-010",␣
↪allowedUses=10, validSeconds=3600)
```

5.8.5 Redeeming PostIts

To redeem a PostIt, use the redeemUrl from the PostIt to make a non-authenticated HTTP GET request.

Using curl

```
curl -JO "https://tacc.tapis.io/v3/files/postits/redeem/e614ce8e-447c-4195-a3f7-
↪55f5dec5d243-010"
```

Note

The options -J and -O (specified above as -JO) tell curl to download the file content and use the filename in the content-disposition header. It's worth noting that using this filename is not entirely without risk as it could overwrite a file of the same name. If you would prefer to use a name that you specify, you could replace -JO with --output filename

Using python

```
tapis.files.redeemPostIt(postitId="e614ce8e-447c-4195-a3f7-55f5dec5d243-010")
```

By default if the PostIt you are redeeming points to a path that is a directory, you will get a zip file, and if it's a regular file, you will get an uncompressed file. If you want to force a file to be compressed, you can specify the query parameter zip and set it to true.

Using curl

```
curl -JO "https://tacc.tapis.io/v3/files/postits/redeem/e614ce8e-447c-4195-a3f7-
↪55f5dec5d243-010?zip=true"
```

Using python

```
tapis.files.redeemPostIt(postitId="e614ce8e-447c-4195-a3f7-55f5dec5d243-010", zip=True)
```

Note

If you specify zip=false for a PostIt that points to a directory, you will get an error. Directories can't be returned unless they are compressed.

The redeem URL can also be pasted into the address bar of your favorite browser, and it will download the file pointed to by the PostIt.

Handling of symbolic links on Linux Systems

Redeeming PostIts handles symbolic links exactly the same as *Transfers*

5.8.6 Expiring PostIts

There is no special endpoint for expiring a PostIt. To Expire a PostIt just update see *Updating PostIts* and set validSeconds to 0, or allowedUses to 0.

5.8.7 Deleting PostIts

PostIts can be deleted by specifying the PostIt id in the url. This is a hard delete, and cannot be undone. Only an owner or tenant admin can delete a PostIt.

Using curl

```
curl -X DELETE "https://tacc.tapis.io/v3/files/postits/e614ce8e-447c-4195-a3f7-
↪55f5dec5d243-010" -H "X-Tapis-Token: $JWT"
```

Using python

```
tapis.files.deletePostIt(postitId="e614ce8e-447c-4195-a3f7-55f5dec5d243-010")
```

CHAPTER

SIX

APPLICATIONS

In order to run a job on a system you will need to create or have access to a Tapis **application**.

6.1 Overview

A Tapis application represents all the information required to run a Tapis job on a Tapis system and produce useful results. Each application is versioned and is associated with a specific tenant and owned by a specific user who has special privileges for the application. In order to support this purpose an application definition includes information which allows the *Jobs* service to:

- Stage input prior to launching the application
- Launch the application
- Monitor the application during execution
- Archive output after application execution

6.2 Model

At a high level an application contains some information that is independent of the version and some information that varies by version.

6.2.1 Non-Versioned Attributes

id
: A short descriptive name for the application that is unique within the tenant.

owner
: A specific user set at application creation. Default is `${apiUserId}`, the user making the request to create the application.

enabled
: Indicates if application is currently considered active and available for use. Default is *true*.

deleted
: Indicates if application has been soft deleted.

created
: When the application was created. Maintained by service.

updated
: When the application was last updated. Maintained by service.

6.2.2 Versioned Attributes

version
Applications are expected to evolve over time. `Id` + `version` must be unique within a tenant.

description
An optional more verbose description for the application.

versionEnabled
Indicates if specific version is currently considered active and available for use. Default is *true*.

locked
Indicates if version is currently locked. When locked a version may not be modified. Default is *false*.

runtime
Runtime to be used when executing the application. DOCKER, SINGULARITY, ZIP. Default is DOCKER. For more information on how Tapis supports the various runtime types, please see JobsRuntimeSupport.

runtimeVersion
Runtime version to be used when executing the application.

runtimeOptions
Options that apply to the runtime. Currently only applicable for SINGULARITY runtime.

Warning

Please note that use of SINGULARITY_START has been deprecated. Support will be removed in a future release. If you have a need for this option please contact Tapis support (cicsupport@tacc.utexas.edu).

containerImage
Reference to be used when running the container image.

jobType
FORK or BATCH. Jobs submitted will be of this type by default. May be overridden in the job submit request. This allows an application designer to test an application run as a FORK job, for example, and then move on to running as a BATCH job which typically involves further design work. Default is FORK.

maxJobs
Maximum total number of jobs that can be queued or running for this application on a given execution system at a given time. Note that the execution system may also limit the number of jobs on the system which may further restrict the total number of jobs. Set to -1 for unlimited. Default is unlimited.

maxJobsPerUser
Maximum total number of jobs associated with a specific job owner that can be queued or running for this application on a given execution system at a given time. Note that the execution system may also limit the number of jobs on the system which may further restrict the total number of jobs. Set to -1 for unlimited. Default is unlimited.

strictFileInputs
Indicates if a job request is allowed to have unnamed file inputs. If set to true then a job request may only use the named file inputs defined in the application. See attribute *fileInputs* in the JobAttributes table. Default is *false*.

Job related attributes
Various attributes related to job execution such as *execSystemId*, *execSystemExecDir*, *execSystemInputDir*, *execSystemLogicalQueue archiveSystemId*, *fileInputs*, etc.

Note

Currently dynamic selection of an execution system is not supported. For this reason the job related attribute *dynamicExecSystem* should be set to *false* (the default) and *execSystemConstraints* should not be set.

6.2.3 Required Attributes

When creating a application the required attributes are: `id`, `version` and `containerImage`. Depending on the type of application and specific values for certain attributes there are other requirements.

The restrictions are:

- If `archiveSystemId` is specified then `archiveSystemDir` must be specified.

6.2.4 Attributes and the Job Execution Environment

The runtime environment of a Tapis job is determined by values in the job submit request, the application definition and the execution system definition. Generally speaking, for values that can be assigned in multiple places, the values in the job submit request override those in the application definition, which in turn override those in the system definition. There are special cases, however, where the values from different definitions are merged.

See the jobs/apps/systems parameter matrix for a detailed description of how each attribute is handled.

6.3 Getting Started

Before going into further details about applications, here we give some examples of how to create and view applications. In the examples below we assume you are using the TACC tenant with a base URL of `tacc.tapis.io` and that you have authenticated using PySDK or obtained an authorization token and stored it in the environment variable JWT, or perhaps both.

6.3.1 Creating an application

Create a local file named `app_sample.json` with json similar to the following:

```
{
  "id":"tacc-sample-app-<userid>",
  "version":"0.1",
  "description":"My sample application",
  "runtime":"DOCKER",
  "containerImage":"docker.io/hello-world:latest",
  "jobType":"FORK",
  "jobAttributes": {
    "description": "default job description",
    "execSystemId": "execsystem1"
  }
}
```

where <userid> is replaced with your user name.

Note

If specified, `execSystemId` must reference a system that exists and has `canExec` set to true. If `execSystemId` is not specified, then it must be provided as part of the job submit request.

Using PySDK:

```
import json
from tapipy.tapis import Tapis
t = Tapis(base_url='https://tacc.tapis.io', username='<userid>', password='************')
with open('app_sample.json', 'r') as openfile:
    my_app = json.load(openfile)
t.apps.createAppVersion(**my_app)
```

Using CURL:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.
↪tapis.io/v3/apps -d @app_sample.json
```

6.3.2 Viewing Applications

Retrieving details for an application

To retrieve details for a specific application, such as the one above:

Note

See the section below on *Selecting* to find out how to control the amount of information returned.

Using PySDK:

```
t.apps.getAppLatestVersion(appId='tacc-sample-app-<userid>')
```

Using CURL:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/apps/tacc-sample-app-<userid>
```

The response should look similar to the following:

```
{
   "result": {
       "tenant": "tacc",
       "id": "tacc-sample-app-<userid>",
       "version": "0.1",
       "description": "My sample application",
       "owner": "<userid>",
       "enabled": true,
       "versionEnabled": true,
       "locked": false,
       "runtime": "DOCKER",
       "runtimeVersion": null,
       "runtimeOptions": [],
       "containerImage": "docker.io/hello-world:latest",
       "jobType": "FORK",
       "maxJobs": 0,
       "maxJobsPerUser": 0,
       "strictFileInputs": false,
       "jobAttributes": {
           "description": "default job description",
```

(continues on next page)

(continued from previous page)


```
            "dynamicExecSystem": false,
            "execSystemConstraints": [],
            "execSystemId": "execsystem1",
            "execSystemExecDir": null,
            "execSystemInputDir": null,
            "execSystemOutputDir": null,
            "dtnSystemInputDir": null,
            "dtnSystemOutputDir": null,
            "execSystemLogicalQueue": null,
            "archiveSystemId": null,
            "archiveSystemDir": null,
            "archiveOnAppError": false,
            "isMpi": false,
            "mpiCmd": null,
            "cmdPrefix": null,
            "parameterSet": {
                "appArgs": [
                  {
                    "arg": "arg1",
                    "name": "appArg1Name",
                    "description": "description for app arg 1",
                    "inputMode": "FIXED",
                    "notes": {
                      "arg1note": "my first arg"
                    }
                  }
                ],
                "containerArgs": [],
                "schedulerOptions": [],
                "envVariables": [
                  {
                    "key": "ENV_1",
                    "value": "env_1_value",
                    "description": "my env var #1",
                    "inputMode": "INCLUDE_BY_DEFAULT",
                    "notes": {}
                  },
                  {
                    "key": "APP_ONLY_FIXED_SET",
                    "value": "app_only_fixed_set",
                    "description": "FIXED env var only in app definition",
                    "inputMode": "FIXED",
                    "notes": {
                      "app_only_fixed_notes": "testing_it"
                    }
                  },
                  {
                    "key": "APP_REQUIRED_SET",
                    "value": "app_required_set",
                    "description": "",
                    "inputMode": "REQUIRED",
                    "notes": {}
```


(continues on next page)

(continued from previous page)

```
                }
            ],
            "archiveFilter": {
                "includes": [],
                "excludes": [],
                "includeLaunchFiles": true
            }
        },
        "fileInputs": [
          {
            "name": "empty",
            "description": "An empty file",
            "inputMode": "OPTIONAL",
            "autoMountLocal": true,
            "notes": {},
            "sourceUrl": "tapis://test-storage-linux/data_input/empty.txt",
            "targetPath": "empty.txt"
          },
          {
            "name": "file1",
            "description": "A random text file",
            "inputMode": "REQUIRED",
            "autoMountLocal": true,
            "notes": {},
            "sourceUrl": "tapis://test-storage-linux/data_input/file1.txt",
            "targetPath": "file1.txt"
          },
          {
            "name": "s3_ceph_file",
            "description": "A file from an s3 ceph storage system.",
            "inputMode": "REQUIRED",
            "autoMountLocal": true,
            "notes": {},
            "sourceUrl": "tapis://test-storage-s3-ceph/object1",
            "targetPath": "s3_ceph_file.dat"
          },
          {
            "name": "s3_aws_test1",
            "description": "File from an s3 aws storage system.",
            "inputMode": "REQUIRED",
            "autoMountLocal": true,
            "notes": {},
            "sourceUrl": "tapis://test-s3-storage/object2",
            "targetPath": "s3_aws/test1.dat"
          }
        ],
        "fileInputArrays": [
          {
            "name": "fileInputArray1",
                  "description": "A list of files in a single directory",
            "sourceUrls": [
               "tapis://test-storage-linux/data_input/file1a.txt",
```

(continues on next page)

(continued from previous page)


```
                    "tapis://test-storage-linux/data_input/file2a.txt",
                    "tapis://test-storage-linux/data_input/file3a.txt"
                ],
                "targetDir": "myFileInputArrayDir/subdir1"
              }
            ],
            "nodeCount": 1,
            "coresPerNode": 1,
            "memoryMB": 100,
            "maxMinutes": 10,
            "subscriptions": [
              {
                "description": "Email on job new status",
                "enabled": true,
                "jobEventCategoryFilter": "JOB_NEW_STATUS",
                "deliveryTargets": [
                  {
                    "deliveryMethod": "EMAIL",
                    "deliveryAddress": "me@example.com"
                  }
                ],
                "ttlMinutes": 10080
              }
            ],
            "tags": []
        },
        "tags": [],
        "notes": {},
        "uuid": "40a60a11-41fe-45ea-8674-d2cfe04992f6",
        "deleted": false,
        "created": "2021-04-22T21:30:10.590999Z",
        "updated": "2021-04-22T21:30:10.590999Z"
    },
    "status": "success",
    "message": "TAPIS_FOUND App found: tacc-sample-app-<userid>",
    "version": "0.0.1-SNAPSHOT",
    "metadata": null
}
```


Retrieving details for all applications

To see the list of applications that you own:

Using PySDK:

```
t.apps.getApps()
```

Using CURL:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/apps?select=allAttributes
```

The response should contain a list of items similar to the single listing shown above.

Note

See the sections below on *Searching*, *Selecting*, *Sorting* and *Limiting* to find out how to control the amount of information returned.

6.4 Minimal Definition and Restrictions

When creating an application the required attributes are: *id*, *version* and *containerImage* Depending on the type of application and specific values for certain attributes there are other requirements. The restrictions are:

- If *archiveSystemId* is specified then *archiveSystemDir* is required.

6.5 Version

The versioning scheme is at the discretion of the application author. The combination of `tenant+id+version` uniquely identifies an application in the Tapis environment. It is recommended that a two or three level form of semantic versioning be used. The fully qualified application reference within a tenant is constructed by appending a hyphen to the name followed by the version string. For example, the first two versions of an application might be *myapp-0.0.1* and *myapp-0.0.2*. If a version is not specified when retrieving an application then by default the most recently created version of the application will be returned.

6.6 Containerized Application

An application that has been containerized is one that can be executed using a single container image. Tapis will use the appropriate container runtime command and provide support for making the input and output directories available to the container when running the container image.

Note

Currently only containerized applications are supported

6.7 Directory Semantics and Macros

At job submission time the Jobs service supports the use of macros based on template variables. These variables may be referenced when specifying directories in an application definition. For a full list of supported variables and more information please see the Jobs Service documentation. Here are some examples of variables that may be used when specifying directories for an application:

- *JobUUID* - The Id of the job determined at job submission.
- *JobOwner* - The owner of the job determined at job submission.
- *JobWorkingDir* - Default parent directory from which a job is run. This will be relative to the effective root directory *rootDir* on the execution system. *rootDir* and *jobWorkingDir* are attributes of the execution system.
- *HOST_EVAL($<ENV_VARIABLE>)* - The value of the environment variable *ENV_VARIABLE* when evaluated on the execution system host when logging in under the job's effective user ID. This is a dynamic value determined at job submission time. The function *HOST_EVAL()* extracts specific environment variable values for use during job setup. In particular, the TACC specific values of *$HOME*, *$WORK*, *$SCRATCH* and *$FLASH* can be referenced. The specified environment variable name is used **as-is**. It is **not** subject to macro substitution. However, the function call can have a path string appended to it, such as in *HOST_EVAL($SCRATCH)/tmp/${JobUUID}*, and macro substitution will be applied to the path string.

6.8 Data Transfer Node (DTN) Support

Tapis supports the use of a Data Transfer Node (DTN) when running an application. There are two fields in *jobAttributes* related to DTN support:

dtnSystemInputDir
: Directory relative to *rootDir* to which input files will be transferred prior to launching the application.

dtnSystemOutputDir
: Directory relative to *rootDir* from which output files will be transferred during the archiving phase.

By default, these are set to the special value *!tapis_not_set*. This value indicates that, by default, a DTN system will not be used for file inputs or archive outputs. In order to trigger the use of a DTN during either file input staging or archive output, these values must be set. For more information on using a DTN, please see DTN Configuration.

6.9 Permissions

The permissions model allows for fine grained access control of Tapis applications.

At application creation time the owner is given full access to the application. Permissions for other users may be granted and revoked through the applications API. Please note that grants and revokes through this service only impact the default role for the user. A user may still have access through permissions in another role. So even after revoking permissions through this service when permissions are retrieved the access may still be listed. This indicates access has been granted via another role.

Permissions are specified as either `*` for all permissions or some combination of the following specific permissions: `("READ","MODIFY","EXECUTE")`. Specifying permissions in all lower case is also allowed. Having `MODIFY` implies `READ`.

6.10 Sharing

In addition to fine grained permissions support, Tapis also supports a higher level approach to granting access. This approach is known simply as *sharing*. The sharing API allows you to share an application with a set of users as well as share publicly with all users in a tenant. Sharing grants `READ+EXECUTE` access and, more importantly, facilitates allowing others to run a job using the application.

Sharing an application gives a user certain implicit access to resources in the context of running a job. When a properly designed application is shared it may be used by many users to run jobs without the need to explicitly grant permissions to associated resources such as systems and file paths.

For more information on sharing please see *Resource Sharing*

6.11 Deletion

An application may be deleted and undeleted. Deletion means the application is marked as deleted and is no longer available for use. Note that although this is a soft delete the operation is intended for use when an application is to be permanently made unavailable for use. To temporarily make an application unavailable for use please use support for enabling and disabling an application.

By default deleted applications will not be included in searches and operations on deleted applications will not be allowed. When listing applications the query parameter *showDeleted* may be used in order to include deleted applications in the results. Note that deletion applies to all version of an application. It is not possible to delete a specific version.

6.12 Application Attributes Table

Attribute	Type	Example	Notes
tenant	String	designsafe	• Name of the tenant for which the application is defined. • *tenant* + $version* + *id* must be unique.
id	String	my-ds-app	• Name of the application. URI safe, see RFC 3986. • *tenant* + $version* + *id* must be unique. • Allowed characters: Alphanumeric [0-9a-zA-Z] and special characters [-._~]. • **Required** at creation time.
version	String	0.0.1	• Version of the application. URI safe, see RFC 3986. • *tenant* + $version* + *id* must be unique. • Allowed characters: Alphanumeric [0-9a-zA-Z] and special characters [-._~]. • **Required** at creation time.
description	String	A sample application	• Optional description
owner	String	jdoe	• User name of *owner*. • Variable references: *${apiUserId}* • Default is *${apiUserId}*
enabled	boolean	FALSE	• Indicates if application currently enabled for use. Default is TRUE.
versionEnabled	boolean	FALSE	• Indicates if specific version is currently enabled for use. Default is TRUE.

6.13 JobAttributes Table

Attribute	Type	Example	Notes
description	String		• Description to be filled in when this application is used to run a job. • Macros allow this to act as a template to be filled in at job runtime.
execSystemId	String		• Specific system on which the application is to be run.
execSystemExecDir	String		• Directory where application assets are staged. • Current working directory at application launch time. • Macro template variables such as ${JobWorkingDir} may be used. • Default is ${JobWorkingDir}/jobs/${JobUUID}
execSystemInputDir	String		• Directory where Tapis is to stage the inputs required by the application. • Macro template variables such as ${JobWorkingDir} may be used. • Default is ${JobWorkingDir}/jobs/${JobUUID}
execSystemOutputDir	String		• Directory where Tapis expects the application to store its final output results. • Files here are candidates for archiving. • Macro template variables such as ${JobWorkingDir} may be used. • Default is ${JobWorkingDir}/jobs/${JobUUID}/output

6.14 ParameterSet Attributes Table

Attribute	Type	Example	Notes
appArgs	[Arg]		• Command line arguments passed to the application. • See table below for more information on *Arg* type entries. • For more information on *appArgs* please see the chapter on *Jobs*
containerArgs	[Arg]		• Command line arguments passed to the container runtime. • See table below for more information on *Arg* type entries. • For more information on *containerArgs* please see the chapter on *Jobs*
schedulerOptions	[Arg]		• Scheduler options passed to the HPC batch scheduler. • See table below for more information on *Arg* type entries. • For more information on *schedulerOptions* please see the chapter on *Jobs*
envVariables	[KeyValuePair]		• Environment variables placed into the runtime environment. • Each entry has *key* (required) and *value* (optional) as well as other attributes. • See table *KeyValuePair Attributes* below for more information. • For more information on *envVariables* please see the chapter on *Jobs*

6.15 ArchiveFilter Attributes Table

Attribute	Type	Example	Notes
includes	[String]		• Files to include when archiving after execution of the application. • excludes list has precedence.
excludes	[String]		• Files to skip when archiving after execution of the application. • excludes list has precedence.
includeLaunchFiles	boolean		• Indicates if Tapis generated launch scripts are to be included when archiving. • The default is TRUE.

6.16 Arg Attributes Table

For more information on how the *inputMode* attribute is used during job processing, please see JobsArgProcessing.

Attribute	Type	Example	Notes
name	String		• Identifying label associated with the argument. • **Required** at creation time.
description	String		• Optional description of the argument which may include usage, purpose, etc.
inputMode	enum		• Indicates how argument is to be treated when processing individual job requests. • Modes: REQUIRED, FIXED, INCLUDE_ON_DEMAND, INCLUDE_BY_DEFAULT • Default is INCLUDE_ON_DEMAND. • REQUIRED: Must be provided in a job request. • FIXED: Completely defined in the application and not overridable in a job request. • INCLUDE_ON_DEMAND: Included if referenced in a job request. • INCLUDE_BY_DEFAULT: Included unless *include=false* in a job request.
arg	String		• Value for the argument • **Required** at creation time.
notes	String	{“fieldType”: “int”}	• Metadata in the form of a Json object, such as type, allowed values, etc. • Not used by Tapis.

6.17 KeyValuePair Attributes Table

Attribute	Type	Example	Notes
key	String	"INPUT_FILE"	• Environment variable name. Required.
value	String	"/tmp/file.input"	• Environment variable value
description	String		• Description
inputMode	enum	REQUIRED	• Indicates how argument is to be treated when processing individual job requests. • Modes: REQUIRED, FIXED, INCLUDE_ON_DEMAND, INCLUDE_BY_DEFAULT • Default is INCLUDE_BY_DEFAULT. • REQUIRED: Must be provided in a job request or application definition. • FIXED: Not overridable in application or job request. • INCLUDE_ON_DEMAND: Included if referenced in a job request. • INCLUDE_BY_DEFAULT: Included unless *include=false* in a job request.
notes	String	"{}"	• Simple metadata in the form of a Json object. • Not used by Tapis.

6.18 FileInput Attributes Table

Attribute	Type	Example	Notes
name	String		• Identifying label associated with the input. Typically used during a job request. • **Required** at creation time.
description	String		• Optional description.
inputMode	enum		• Indicates how input is to be treated when processing individual job requests. • REQUIRED, OPTIONAL, FIXED • Default is OPTIONAL.
autoMountLocal	boolean		• Indicates if Jobs service should automatically mount file paths into containers. • Note that not all container runtimes require this. • Setting to FALSE allows user complete control using *containerArg* parameters. • Default is TRUE.
sourceUrl	String		• Source used by Jobs service when staging file inputs.
targetPath	String		• Target path used by Jobs service when staging file inputs.

6.19 FileInputArray Attributes Table

Attribute	Type	Example	Notes
name	String		• Identifying label associated with the input. Typically used during a job request. • **Required** at creation time.
description	String		• Optional description.
inputMode	enum		• REQUIRED, OPTIONAL, FIXED • Default is OPTIONAL.
sourceUrls	[String]		• Array of sources used by Jobs service when staging file inputs.
targetDir	String		• Target directory used by Jobs service when staging file inputs.

6.20 Searching

The service provides a way for users to search for applications based on a list of search conditions provided either as query parameters for a GET call or a list of conditions in a request body for a POST call to a dedicated search endpoint.

6.20.1 Search using GET

To search when using a GET request to the `apps` endpoint a list of search conditions may be specified using a query parameter named `search`. Each search condition must be:

- surrounded with parentheses
- have three parts separated by the character `.`
- be joined using the character `~`.

All conditions are combined using logical AND. The general form for specifying the query parameter is as follows:

```
?search=(<attribute_1>.<op_1>.<value_1>)~(<attribute_2>.<op_2>.<value_2>)~ ... ~(
↪<attribute_N>.<op_N>.<value_N>)
```

Attribute names are given in the table above and may be specified using Camel Case or Snake Case.

Supported operators: `eq` `neq` `gt` `gte` `lt` `lte` `in` `nin` `like` `nlike` `between` `nbetween`

Example CURL command to search for applications that have `Test` in the id, are of type FORK and allow for *maxJobs* greater than `5`:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/apps?search="(id.like.*Test*)~
↪(job_type.eq.FORK)~(max_jobs.gt.5)"
```

Notes:

- For the `like` and `nlike` operators the wildcard character `*` matches zero or more characters and `!` matches exactly one character.
- For the `between` and `nbetween` operators the value must be a two item comma separated list of unquoted values.
- If there is only one condition the surrounding parentheses are optional.
- In a shell environment the character `&` separating query parameters must be escaped with a backslash.
- In a shell environment the query value must be surrounded by double quotes and the following characters must be escaped with a backslash in order to be properly interpreted by the shell:
  - `" \ ``
- Attribute names may be specified using Camel Case or Snake Case.
- Following complex attributes not supported when searching:
  - `jobAttributes` `notes`

6.20.2 Dedicated Search Endpoint

The service provides the dedicated search endpoint `apps/search/apps` for specifying complex queries. Using a GET request to this endpoint provides functionality similar to above but with a different syntax. For more complex queries a POST request may be used with a request body specifying the search conditions using an SQL-like syntax.

Search using GET on Dedicated Endpoint

Sending a GET request to the search endpoint provides functionality very similar to that provided for the endpoint `apps` described above. A list of search conditions may be specified using a series of query parameters, one for each attribute. All conditions are combined using logical AND. The general form for specifying the query parameters is as follows:

```
?<attribute_1>.<op_1>=<value_1>&<attribute_2>.<op_2>=<value_2>)& ... &<attribute_N>.<op_
↪N>=<value_N>
```

Attribute names are given in the table above and may be specified using Camel Case or Snake Case.

Supported operators: `eq` `neq` `gt` `gte` `lt` `lte` `in` `nin` `like` `nlike` `between` `nbetween`

Example CURL command to search for applications that have `Test` in the id, are of type FORK and allow for *maxJobs* greater than `5`:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/apps/search/apps?id.like=*Test*\
↪&job_type.eq=FORK\&max_jobs.gt=5
```

Notes:

- For the `like` and `nlike` operators the wildcard character `*` matches zero or more characters and `!` matches exactly one character.
- For the `between` and `nbetween` operators the value must be a two item comma separated list of unquoted values.

- In a shell environment the character & separating query parameters must be escaped with a backslash.
- Attribute names may be specified using Camel Case or Snake Case.
- Following complex attributes not supported when searching:
  - `jobAttributes` `tags` `notes`

Search using POST on Dedicated Endpoint

More complex search queries are supported when sending a POST request to the endpoint `apps/search/apps`. For these requests the request body must contain json with a top level property name of `search`. The `search` property must contain an array of strings specifying the search criteria in an SQL-like syntax. The array of strings are concatenated to form the full search query. The full query must be in the form of an SQL-like `WHERE` clause. Note that not all SQL features are supported.

For example, to search for apps that are owned by `jdoe` and of type `FORK` or owned by `jsmith` and allow for *maxJobs* less than `5` create a local file named `app_search.json` with following json:

```
{
  "search":
    [
      "(owner = 'jdoe' AND job_type = 'FORK') OR",
      "(owner = 'jsmith' AND max_jobs < 5)"
    ]
}
```

To execute the search use a CURL command similar to the following:

```
$ curl -X POST -H "content-type: application/json" -H "X-Tapis-Token: $JWT" https://tacc.
↪tapis.io/v3/apps/search/apps -d @app_search.json
```

Notes:

- String values must be surrounded by single quotes.
- Values for BETWEEN must be surrounded by single quotes.
- Search query parameters as described above may not be used in conjunction with a POST request.
- SQL features not supported include:
  - `IS NULL` and `IS NOT NULL`
  - Arithmetic operations
  - Unary operators
  - Specifying escape character for `LIKE` operator

Map of SQL operators to Tapis operators

Sql Operator	Tapis Operator
=	eq
<>	neq
<	lt
<=	lte
>	gt
>=	gte
LIKE	like
NOT LIKE	nlike
BETWEEN	between
NOT BETWEEN	nbetween
IN	in
NOT IN	nin

6.21 Sort, Limit, Select and ListType

When a list of Applications is retrieved the service provides for sorting, filtering and limiting the results. By default, only resources owned by you will be included. The service provides a way for you to request that all resources accessible to you be included. This is determined by the query parameter *listType*.

When retrieving either a list of resources or a single resource the service also provides a way to *select* which fields (i.e. attributes) are included in the results. Sorting, limiting and attribute selection are supported using query parameters.

6.21.1 Selecting

When retrieving applications the fields (i.e. attributes) to be returned may be specified as a comma separated list using a query parameter named `select`. Attribute names may be given using Camel Case or Snake Case.

Notes:

- Special select keywords are supported: `allAttributes` and `summaryAttributes`
- Summary attributes include:
  - `id`, `version`, `owner`
- By default all attributes are returned when retrieving a single resource via the endpoint apps/<app_id>.
- By default summary attributes are returned when retrieving a list of applications.
- Specifying nested attributes is not supported.
- The attribute `id` is always returned.

For example, to return only the attributes `version` and `containerImage` the CURL command would look like this:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/apps?select=version,
↪containerImage
```

The response should look similar to the following:

```
{
   "result": [
       {
```

(continues on next page)

(continued from previous page)


```
            "id": "TestApp1",
            "version": "0.0.1",
            "containerImage": "containterimage1"
        },
        {
            "id": "JobApp1",
            "version": "0.0.1",
            "containerImage": "containterimage1"
        },
        {
            "id": "JobAppWithInput",
            "version": "0.0.1",
            "containerImage": "containterimage1"
        },
        {
            "id": "SleepSeconds",
            "version": "0.0.1",
            "containerImage": "tapis/testapps:main"
        }
    ],
    "status": "success",
    "message": "TAPIS_FOUND Apps found: 11 applications",
    "version": "0.0.1-SNAPSHOT",
    "metadata": {
        "recordCount": 4,
        "recordLimit": 100,
        "recordsSkipped": 0,
        "orderBy": null,
        "startAfter": null,
        "totalCount": -1
    }
}
```


6.21.2 Sorting

The query parameter for sorting is named `orderBy` and the value is the attribute name to sort on with an optional sort direction. The general format is `<attribute_name>(<dir>)`. The direction may be `asc` for ascending or `desc` for descending. The default direction is ascending.

Examples:

- orderBy=id
- orderBy=id(asc)
- orderBy=name(desc),created
- orderBy=id(asc),created(desc)

6.21.3 Limiting

Additional query parameters may be used in order to limit the number and starting point for results. This is useful for implementing paging. The query parameters are:

- `limit` - Limit number of items returned. For example limit=10.

  - Use 0 or less for unlimited.
  - Default is 100.
- `skip` - Number of items to skip. For example skip=10.
  - May not be used with startAfter.
  - Default is 0.
- `startAfter` - Where to start when sorting. For example limit=10&orderBy=id(asc),created(desc)&startAfter=101
  - May not be used with `skip`.
  - Must also specify `orderBy`.
  - The value of `startAfter` applies to the major `orderBy` field.
  - Condition is context dependent. For ascending the condition is value > `startAfter` and for descending the condition is value < `startAfter`.

When implementing paging it is recommend to always use `orderBy` and when possible use `limit+startAfter` rather than `limit+skip`. Sorting should always be included since returned results are not guaranteed to be in the same order for each call. The combination of `limit+startAfter` is preferred because `limit+skip` is more likely to result in inconsistent results as records are added and removed. Using `limit+startAfter` works best when the attribute has a natural sequential ordering such as when an attribute represents a timestamp or a sequential ID.

6.21.4 ListType

By default, you will only see the resources that you own. The query parameter *listType* allows you to see additional resources that are available to you.

Options:

OWNED
: Include only items owned by you (Default)

SHARED_PUBLIC
: Include only items shared publicly

ALL
: Include all items you are authorized to view.

6.22 Tapis Responses

For requests that return a list of resources the response result object will contain the list of resource records that match the user's query and the response metadata object will contain information related to sorting and limiting.

The metadata object will contain the following information:

- `recordCount` - Actual number of records returned.
- `recordLimit` - The limit query parameter specified in the request. -1 if query parameter was not specified.
- `recordsSkipped` - The skip query parameter specified in the request. -1 if query parameter was not specified.
- `orderBy` - The orderBy query parameter specified in the request. Empty string if query parameter was not specified.
- `startAfter` - The startAfter query parameter specified in the request. Empty string if query parameter was not specified.
- `totalCount` - Total number of records that would have been returned without a limit query parameter being imposed. -1 if total count was not computed.

For performance reasons computation of `totalCount` is only determined on demand. This is controlled by the boolean query parameter `computeTotal`. By default `computeTotal` is *false*.

Example query and response:

Query:

```
$ curl -H "X-Tapis-Token: $JWT" https://tacc.tapis.io/v3/apps?limit=2&orderBy=id(desc)
```

Response:


```
{
   "result": [
       {
           "id": "TestApp1",
           "version": "0.0.1",
           "owner": "testuser2"
       },
       {
           "id": "tacc-sample-app",
           "version": "0.1",
           "owner": "testuser2"
       }
   ],
   "status": "success",
   "message": "TAPIS_FOUND Apps found: 2 applications",
   "version": "0.0.1-SNAPSHOT",
   "metadata": {
       "recordCount": 2,
       "recordLimit": 2,
       "recordsSkipped": 0,
       "orderBy": "id(desc)",
       "startAfter": null,
       "totalCount": -1
   }
}
```

CHAPTER

SEVEN

JOBS

7.1 Introduction to Jobs

The Tapis v3 Jobs service is specialized to run containerized applications on any host that supports container runtimes. Currently, Docker, Singularity and ZIP containers are supported. The Jobs service uses the Systems, Apps, Files and Security Kernel services to process jobs.

7.1.1 Implementation Status

The following table describes the current state of the Beta release of Jobs. All UrlPaths shown start with /v3/jobs. The unauthenticated health check, ready and hello APIs do not require a Tapis JWT in the request header.

Name	Method	UrlPath	Status
Submit	POST	/submit	Implemented
Resubmit	POST	/{JobUUID}/resubmit	Implemented
List	GET	/list	Implemented
Search	GET	/search	Implemented
Search	POST	/search	Implemented
Get	GET	/{JobUUID}	Implemented
Get Status	GET	/{JobUUID}/status	Implemented
Get History	GET	/{JobUUID}/history	Implemented
Get Output list	GET	/{JobUUID}/output/list/{outputPath}	Implemented
Download Output	GET	/{JobUUID}/output/download/{outputPath}	Implemented
Resubmit Request	GET	/{JobUUID}/resubmit_request	Implemented
Cancel	POST	/{JobUUID}/cancel	Implemented
Hide	POST	/{JobUUID}/hide	Implemented
Unhide	POST	/{JobUUID}/unhide	Implemented
SendEvent	POST	/{JobUUID}/sendEvent	Implemented
Post Share	POST	/{JobUUID}/share	Implemented
Get Share	GET	/{JobUUID}/share	Implemented
Delete Share	DELETE	/{JobUUID}/share/{user}	Implemented
Health Check	GET	/healthcheck	Implemented
Ready	GET	/ready	Implemented
Hello	GET	/hello	Implemented

7.1.2 Job Processing Overview

Before discussing the details of how to construct a job request, we take this opportunity to describe overall lifecycle of a job. When a job request is received as the payload of an POST call, the following steps are taken:

1. **Request authorization** - The tenant, owner, and user values from the request and Tapis JWT are used to authorize access to the application, execution system and, if specified, archive system.
2. **Request validation** - Request values are checked for missing, conflicting or improper values; all paths are assigned; required paths are created on the execution system; and macro substitution is performed to finalize all job parameters.
3. **Job creation** - A Tapis job object is written to the database.
4. **Job queuing** - The Tapis job is queue on an internal queue serviced by one or more Job Worker processes.
5. **Response** - The initial Job object is sent back to the caller in the response. This ends the synchronous portion of job submission.

Once a response to the submission request is sent to the caller, job processing proceeds asynchronously. Job worker processes read jobs from their work queues. The number of workers and queues is limited only by hardware resource constraints. Each job is assigned a worker thread. This thread shepards a job through its lifecycle until the job completes,

fails or becomes blocked due to a transient resource constraint. The job lifecycle is reflected in the *Job Status* and generally progresses as follows:

```
a) Stage inputs to execution system
b) Stage application artifacts to execution system
c) Queue or invoke job on execution system
d) Monitor job until it terminates
e) Collect job exit code
f) Archive job output
```

7.1.3 Simple Job Submission Example

The POST payload for the simplest job submission request looks like this:

```
{
 "name": "myJob"
 "appId": "myApp"
 "appVersion": "1.0"
}
```

In this example, all input and output directories are either specified in the *myApp* definition or are assigned their default values. Currently, the execution system on which an application runs must be specified in either the application definition or job request. Our example assumes that *myApp* assigns the execution system. Future versions of the Jobs service will support dynamic execution system selection.

An archive system can also be specified in the application or job request; the default is for it to be the same as the execution system.

7.2 The Job Submission Request

A job submission request must contain the name, appId and appVersion values as shown in the *Simple Job Submission Example*. Those values are marked *Required* in the list below, a list of all possible values allowed in a submission request. If a parameter has a default value, that value is also shown.

In addition, some parameters can inherit their values from the application or system definitions as discussed in *Parameter Precedence*. These parameters are marked *Inherit*. Parameters that merge inherited values (rather than override them) are marked *InheritMerge*.

Parameters that do not need to be set are marked *Not Required*. Finally, parameters that allow macro substitution are marked *MacroEnabled* (see *Macro Substitution* for details).

name
: The user chosen name of the job. *MacroEnabled*, *Required.*

appId
: The Tapis application to execute. *Required.*

appVersion
: The version of the application to execute. *Required.*

jobType
: A job's type can be either FORK or BATCH.

owner
: User ID under which the job runs. Administrators can designate a user other than themselves.

tenant
Tenant of job owner. Default is job owner's tenant.

description
Human readable job description. *MacroEnabled*, *Not Required*

archiveOnAppError
Whether archiving should proceed even when the application reports an error. Default is *true*.

dynamicExecSystem
Whether the best fit execution system should be chosen using *execSystemConstraints*. Default is *false*.

execSystemId
Tapis execution system ID. *Inherit*.

execSystemExecDir
Directory into which application assets are staged. *Inherit*, see *Directories* for default.

execSystemInputDir
Directory into which input files are staged. *Inherit*, see *Directories* for default.

execSystemOutputDir
Directory into which the application writes its output. *Inherit*, see *Directories* for default.

execSystemLogicalQueue
Tapis-defined queue that corresponds to a batch queue on the execution system. *Inherit* when applicable.

archiveSystemId
Tapis archive system ID. *Inherit*, defaults to *execSystemId*.

archiveSystemDir
Directory into which output files are archived after application execution. *Inherit*, see *Directories* for default.

nodeCount
Number of nodes required for application execution. *Inherit*, default is 1.

coresPerNode
Number of cores to use on each node. *Inherit*, default is 1.

memoryMB
Megabytes of memory to use on each node. *Inherit*, default is 100.

maxMinutes
Maximum number of minutes allowed for job execution. *Inherit*, default is 10.

fileInputs
Input files that need to be staged for the application. *InheritMerge*.

fileInputArrays
Arrays of input files that need to be staged for the application. *InheritMerge*.

parameterSet
Runtime parameters organized by category. *Inherit*.

execSystemConstraints
Constraints applied against execution system capabilities to validate application/system compatibility. *InheritMerge*.

subscriptions
Subscribe to the job's events. *InheritMerge*.

tags
An array of user-chosen strings that are associated with a job. *InheritMerge*.

notes
A JSON object containing any user-chosen data. *Inherit*.

isMpi
Indicates whether this job is an MPI job. *Inherit*, default is false.

mpiCmd
Specify the MPI launch command. Conflicts with cmdPrefix if isMpi is set. *Inherit*.

cmdPrefix
String prepended to the application invocation command. Conflicts with mpiCmd if isMpi is set. *Inherit*.

notes
Optional JSON object containing arbitrary user data, maximum length 65536 bytes. *Inherit*.

The following subsections discuss the meaning and usage of each of the parameters available in a job request. The schema and its referenced library comprise the actual JSON schema definition for job requests.

7.2.1 Parameter Precedence

The runtime environment of a Tapis job is determined by values in system definitions, the app definition and the job request, in low to high precedence order as listed. Generally speaking, for values that can be assigned in multiple definitions, the values in job requests override those in app definitions, which override those in system definitions. There are special cases, however, where the values from different definitions are merged.

See the jobs/apps/systems parameter matrix for a detailed description of how each parameter is handled.

7.2.2 Job Type

An execution system can run jobs using a batch scheduler (e.g., Slurm or Condor) or a native runtime (e.g., Docker or Singularity) or both. Users specify how to run a job using the *jobType* parameter, which is set to "BATCH" to use a batch scheduler or "FORK" to use a native runtime. The jobType can also be specified in application definitions. The final value assigned to the jobType of a job is calculated as follows:

```
1. If the user specifies jobType in the job request, use it.
2. Otherwise, if the app.jobType != null, use it.
3. Otherwise, query the execution system and set jobType=BATCH if execSys.
↪canRunBatch==true.
4. Otherwise, set jobType=FORK.
```

7.2.3 Directories

The execution and archive system directories are calculated before the submission response is sent. This calculation can include the use of macro definitions that get replaced by values at submission request time. The *Macro Substitution* section discusses what macro definitions are available and how substitution works. In this section, we document the default directory assignments which may include macro definitions.

Directory Definitions

The directories assigned when a system is defined:

```
rootDir - the effective root of the file system when accessed through this Tapis system.
jobWorkingDir - the default directory for files used or created during job execution.
```

The directories assigned in application definitions and/or in a job submission requests:

```
execSystemExecDir
execSystemInputDir
execSystemOutputDir
dtnSystemInputDir
dtnSystemOutputDir
archiveSystemDir
```

Directory Assignments

The rootDir and jobWorkingDir are always assigned upon system creation, so they are available for use as macros when assigning directories in applications or job submission requests.

When a job request is submitted, each of the job's four execution and archive system directories are assigned as follows:

1. If the job submission request assigns the directory, that value is used. Otherwise,
2. If the application definition assigns the directory, that value is used. Otherwise,
3. The default values shown below are assigned:

```
execSystemExecDir:    ${JobWorkingDir}/jobs/${JobUUID}
execSystemInputDir:   ${JobWorkingDir}/jobs/${JobUUID}
execSystemOutputDir:  ${JobWorkingDir}/jobs/${JobUUID}/output
archiveSystemDir:     /jobs/${JobUUID}/archive                (if archiveSystemId is␣
↪set)
```

7.2.4 FileInputs

The *fileInputs* in Applications definitions are merged with those in job submission requests to produce the complete list of inputs to be staged for a job. The following rules govern how job inputs are calculated.

1. The effective inputs to a job are the combined inputs from the application and job request.
2. Only named inputs are allowed in application definitions.
3. Application defined inputs are either REQUIRED, OPTIONAL or FIXED.
4. Applications can restrict the number and definitions of inputs (*strictFileInputs=true*).
5. Anonymous (unnamed) inputs can be specified in the job request unless prohibited by the application definition (*strictFileInputs=true*).
6. Job request inputs override values set in the application except for FIXED inputs.
7. The *tapislocal* URL scheme specifies in-place inputs for which transfers are not performed.

The fileInputs array in job requests contains elements that conform to the following JSON schema.

```
"JobFileInput": {
    "$comment": "Used to specify file inputs on Jobs submission requests",
    "type": "object",
        "properties": {
            "name": { "type": "string", "minLength": 1, "maxLength": 80 },
            "description": { "type": "string", "minLength": 1, "maxLength": 8096 },
            "envKey": {"type": "string", "minLength": 1},
            "autoMountLocal": { "type": "boolean"},
            "sourceUrl":  {"type": "string", "minLength": 1, "format": "uri"},
            "targetPath": {"type": "string", "minLength": 0},
```

(continues on next page)

(continued from previous page)

```
            "notes": {"type": "string", "minLength": 0}
        },
    "additionalProperties": false
}
```

JobFileInputs can be named or unnamed. When the *name* field is assigned, Jobs will look for an input with the same name in the application definition (all application inputs are named). When a match is found, values from the AppFileInput are merged into unassigned fields in the JobFileInput.

The *name* must start with an alphabetic character or an underscore (_) followed by zero or more alphanumberic or underscore characters. If the name does not match one of the input names defined in the application, then the application must have *strictFileInputs=false*. If the name matches an input name defined in the application, then the application's inputMode must be REQUIRED or OPTIONAL. An error occurs if the inputMode is FIXED and there is a name match–job inputs cannot override FIXED application inputs.

The *envKey* provides an easy way to insert the *targetPath* into the runtime environment of an application under a user-specified label. The *envKey* string is used as the name of an environment variable that Jobs makes accessible to executing applications. The environment variable's value is the *targetPath*. *envKey* can only contain alphanumerics and underscore (_) and it's first character cannot be a number.

The optional *notes* field can contain any valid user-specified JSON object.

Except for in-place inputs discussed below, the *sourceUrl* is the location from which data are copied to the *targetPath*. In Posix systems the sourceUrl can reference a file or a directory. When a directory is specified, the complete directory subtree is copied.

Any URL protocol accepted by the Tapis Files service can be used in a *sourceUrl*. The most common protocols used are tapis, http, and https. The standard tapis URL format is *tapis://<tapis-system>/<path>*; please see the Files service for the complete list of supported protocols.

The *targetPath* is the location to which data are copied from the *sourceUrl*. The target is rooted at the *execSystemInputDir* except, possibly, when HOST_EVAL() is used, in which case it is still relative to the execution system's rootDir.

A JobFileInput object is **complete** when its *sourceUrl* and *targetPath* are assigned; this provides the minimal information needed to effect a transfer. If only the *sourceUrl* is set, Jobs will use the simple directory or file name from the URL to automatically assign the *targetPath*. Specifying a *targetPath* as "*" results in the same automatic assignment. Whether assigned by the user or Jobs, all job inputs that are not in-place and do not use the HOST_EVAL() function are copied into the *execSystemInputDir* subtree.

After application inputs are added to or merged with job request inputs, all complete JobFileInput objects are designated for staging. Incomplete objects are ignored only if they were specified as OPTIONAL in the application definition. Otherwise, an incomplete input object causes the job request to be rejected.

In-Place Inputs (tapislocal)

Job inputs already present on an execution system do not need to be transferred, yet users may still want to declare them for documentation purposes or to control how they are mounted into containers. It's common, for example, for large data sets that cannot reasonably be copied to be mounted directly onto execution systems. The Jobs and Applications services provide custom syntax that allows such input to be declared, but instructs the Jobs service to **not** copy that input.

Tapis introduces a new URL scheme, *tapislocal*, that is only recognized by the Applications and Jobs services. Here are example URLs:

```
tapislocal://exec.tapis/home/bud/mymri.dcm
tapislocal://exec.tapis/corral/repl/shared
```

Like the *tapis* scheme and all common schemes (https, sftp, etc.), the first segment following the double slashes designates a host. For *tapislocal*, the host is always the literal **exec.tapis**, which serves as a placeholder for a job's execution system. The remainder of the URL is the path on the Tapis system. All paths on Tapis systems, including those using the HOST_EVAL() function and the tapislocal URL, are rooted at the Tapis system's rootDir.

A *tapislocal* URL can only appear in the sourceUrl field of AppFileInput and JobFileInput parameters.

The *tapislocal* scheme indicates to Jobs that a filepath already exists on the execution system and, therefore, does not require data transfer during job execution. If targetPath is "*", the Jobs service will assign the target path inside the container to be the last segment of the tapislocal URL path (/mymri.dcm and /shared in the examples above).

In container systems that require the explicit mounting of host filepaths, such as Docker, the Jobs service can mount the filepath into the container. Both application definitions and job requests support the *autoMountLocal* boolean parameter. This parameter is true by default, which causes Jobs to automatically mount the filepath into containers. Setting autoMountLocal to false allows the user complete control over mounting using a *containerArgs* parameter.

7.2.5 FileInputArrays

The *fileInputArrays* parameter provides an alternative syntax for specifying inputs in Applications and job requests. This syntax is convenient for specifying multiple inputs destined for the same target directory, an I/O pattern sometimes refered to as *scatter-gather*. Generally, input arrays support the same semantics as *FileInputs* with some restrictions.

The fileInputArrays parameter in job requests contains elements that conform to the following JSON schema.

```
"JobFileInputArray": {
     "type": "object",
     "additionalProperties": false,
     "properties": {
         "name": { "type": "string", "minLength": 1, "maxLength": 80 },
         "description": { "type": "string", "minLength": 1, "maxLength": 8096},
         "envKey": {"type": "string", "minLength": 1},
         "sourceUrls": { "type": ["array", "null"],
                         "items": { "type": "string", "format": "uri", "minLength": 1 } }
↪,
         "targetDir": { "type": "string", "minLength": 1 },
         "notes": {"type": "string", "minLength": 0}
     }
}
```

A fileInputArrays parameter is an array of JobFileInputArray objects, each of which contains an array of *sourceUrls* and a single *targetDir*. One restriction is that *tapislocal* URLs cannot appear in *sourceUrls* fields.

An application's fileInputArrays are added to or merged with those in a job request following the same rules established for fileInputs in the previous section. In particular, when names match, the *sourceUrls* defined in a job request override (i.e., completely replace) those defined in an application. After merging, each JobFileInputArray must have a non-empty *sourceUrls* array. See *FileInputs* and Applications for related information.

Each *sourceUrls* entry is a location from which data is copied to the *targetDir*. In Posix systems each URL can reference a file or a directory. In the latter case, the complete directory subtree is transferred. All URLs recognized by the Tapis Files service can be used (*tapislocal* is not recognized by Files).

The *targetDir* is the directory into which all *sourceUrls* are copied. The *targetDir* is always rooted at the *ExecSystemInputDir* and if *targetDir* is "*" or not specified, then it is assigned *ExecSystemInputDir*. The simple name of each *sourceUrls* entry is the destination name used in *targetDir*. Use different JobFileInputArrays with different targetDir's if name conflicts between *sourceUrls* entries exist.

The *envKey* provides an easy way to insert the *targetDir* into the runtime environment of an application under a user-specified label. The *envKey* string is used as the name of an environment variable that Jobs makes accessible to

executing applications. The environment variable's value is the *targetDir*. *envKey* can only contain alphanumerics and underscore (_) and it's first character cannot be a number.

The optional *notes* field can contain any valid user-specified JSON object.

7.2.6 ParameterSet

The job *parameterSet* argument is comprised of these objects:

Name	JSON Schema Type	Description
appArgs	*JobArgSpec* array	Arguments passed to user's application
containerArgs	*JobArgSpec* array	Arguments passed to container runtime
schedulerOptions	*JobArgSpec* array	Arguments passed to HPC batch scheduler
envVariables	*KeyValuePair* array	Environment variables injected into application container
archiveFilter	object	File archiving selector
logConfig	*LogConfig*	User-specified stdout and stderr redirection

Each of these objects can be specifed in Tapis application definitions and/or in job submission requests. In addition, the execution system can also specify environment variable settings.

appArgs

Specify one or more command line arguments for the user application using the *appArgs* parameter. Arguments specified in the application definition are appended to those in the submission request.

containerArgs

Specify one or more command line arguments for the container runtime using the *containerArgs* parameter. Arguments specified in the application definition are appended to those in the submission request.

schedulerOptions

Specify HPC batch scheduler arguments for the container runtime using the *schedulerOptions* parameter. Arguments specified in the application definition are appended to those in the submission request. The arguments for each scheduler are passed using that scheduler's conventions.

Tapis defines a special scheduler option, **--tapis-profile**, to support local scheduler conventions. Data centers sometimes customize their schedulers or restrict how those schedulers can be used. The Systems service manages *SchedulerProfile* resources that are separate from any system definition, but can be referenced from system definitions. The Jobs service uses directives contained in profiles to tailor application execution to local requirements.

As an example, below is the JSON input used to create the TACC scheduler profile.

The *moduleLoads* array contains one or more objects. Each object contains a *moduleLoadCommand*, which specifies the local command used to load each of the modules (in order) in its *modulesToLoad* list.

The *hiddenOptions* array identifies scheduler options that the local implementation prohibits. Options specified here will have the corresponding Slurm option suppressed. Supported options are "MEM" for *--mem* and "PARTITION" for *--partition*. Including an option in the array indicates that the corresponding Slurm option should never be passed through to Slurm.

```
{
    "name": "TACC",
    "owner": "user1",
    "description": "Test profile for TACC Slurm",
    "moduleLoads": [
```

(continues on next page)

(continued from previous page)

```
        {
            "moduleLoadCommand": "module load",
            "modulesToLoad": ["tacc-singularity"]
        }
    ],
    "hiddenOptions": ["MEM"]
}
```

Scheduler-Specific Processing

Jobs will perform *macro-substitution* on Slurm scheduler options *--job-name* or *-J*. This substitution allows Slurm job names to be dynamically generated before submitting them.

envVariables

Specify key/value pairs that will be injected as environment variables into the application's container when it's launched. Key/value pairs specified in the execution system definition, application definition, and job submission request are aggregated using precedence ordering (system < app < request) to resolve conflicts.

archiveFilter

The *archiveFilter* conforms to this JSON schema:

```
"archiveFilter": {
   "type": "object",
   "properties": {
      "includes": {"type": "array", "items": {"type": "string", "minLength": 1},
↪"uniqueItems": true},
      "excludes": {"type": "array", "items": {"type": "string", "minLength": 1},
↪"uniqueItems": true},
      "includeLaunchFiles": {"type": "boolean"}
   },
   "additionalProperties": false
}
```

An *archiveFilter* can be specified in the application definition and/or the job submission request. The *includes* and *excludes* arrays are merged by appending entries from the application definition to those in the submission request.

The *excludes* filter is applied first, so it takes precedence over *includes*. If *excludes* is empty, then no output file or directory will be explicitly excluded from archiving. If *includes* is empty, then all files in *execSystemOutputDir* will be archived unless explicitly excluded. If *includes* is not empty, then only files and directories that match an entry and not explicitly excluded will be archived.

Each *includes* and *excludes* entry is a string, a string with wildcards or a regular expression. Entries represent directories or files. The wildcard semantics are that of glob (*), which is commonly used on the command line. Tapis implements Java glob semantics. To filter using a regular expression, construct the pattern using Java regex semantics and then preface it with **REGEX:** (case sensitive). Here are examples of globs and regular expressions that could appear in a filter:

```
"myfile.*"
"*2021-*-events.log"
"REGEX:^[\\p{IsAlphabetic}\\p{IsDigit}_\\.\\-]+$"
"REGEX:\\s+"
```

When *includeLaunchFiles* is true (the default), then the script (*tapisjob.sh*) and environment (*tapisjob.env*) files that Tapis generates in the *execSystemExecDir* are also archived. These launch files provide valuable information about how a job was configured and launched, so archiving them can help with debugging and improve reproducibility. Since these files may contain application secrets, such database passwords or other credentials, care must be taken to not expose private data through archiving.

If no filtering is specified at all, then all files in *execSystemOutputDir* and the launch files are archived.

logConfig Spec

A *LogConfig* can be supplied in the job submission request and/or in the application definition, with the former overriding the latter when both are supplied. In supported runtimes (currently Singularity), the *logConfig* parameter can be used to redirect the application container's stdout and stderr to user-specified files.

7.2.7 MPI and Related Support

On many systems, running Message Passing Interface (MPI) jobs is simply a matter of launching programs that have been configured or compiled with the proper MPI libraries. Most of the work in employing MPI involves parallelizing program logic and specifying the correct libraries for the target execution system. Once that's done, a command such as *mpirun* (or on TACC systems, *ibrun*) is passed the program's pathname and arguments to kick off parallel execution.

Tapis's *mpiCmd* parameter lets users set the MPI launch command in a system definition, application definition and/or job submission request (lowest to highest priority). For example, if *mpiCmd=mpirun*, then the string "mpirun " will be prepended to the command normally used to execute the application. Some MPI launchers have their own parameters, for instance, *mpiCmd=ibrun -n 4* requests 4 MPI tasks.

The *isMpi* parameter is specified in an application definition and/or job request to toggle MPI launching on or off. This switch allows the same system to run both MPI and non-MPI jobs depending on the needs of particular jobs or applications. The *isMpi* default is false, so this switch must be explicitly turned on to run an MPI job. When turned on, *isMpi* requires *cmdMpi* be assigned in the system, application and/or job request.

The *cmdPrefix* parameter provides generalized support for launchers and is available in application definitions and job submission requests. Like *mpiCmd*, a *cmdPrefix* value is simply prepended to a program's pathname and arguments. Being more general, *cmdPrefix* could specify an MPI launcher, but it's not supported in system definitions and does not have a toggle to control usage.

mpiCmd and *cmdPrefix* are mutually exclusive; so if *isMpi* is true, then *cmdPrefix* must not be set.

7.2.8 ExecSystemConstraints

Future feature.

Specifying execution system constraints is not yet implemented. These will be part of the feature to allow an execution system to be selected dynamically.

7.2.9 Subscriptions

Users can subscribe to job execution events. Subscriptions specified in the application definition and those specified in the job request are merged to produce a job's initial subscription list. New subscriptions can be added while a job is running, but not after the job has terminated. A job's subscriptions can be listed and deleted. Only job owners or tenant administrators can subscribe to a job, see the subscription APIs for details.

When creating a subscription the *ttlminutes* parameter can specify up to 4 weeks. If the parameter is not specified or if it's set to 0, a default value of 1 week is used.

Subscribers are notified of job events by the Notifications service. Currently, only email and webhook delivery methods are supported. The event types to which users can subscribe are:

Event Type	Description
JOB_NEW_STATUS	When the job transitions to a new status
JOB_INPUT_TRANSACTION_ID	When an input file staging request is made
JOB_ARCHIVE_TRANSACTION_ID	When an archive file transfer request is made
JOB_SUBSCRIPTION	When a change to the job's subscriptions is made
JOB_SHARE_EVENT	When a job resource has been shared or unshared
JOB_ERROR_MESSAGE	When the job experienced an error
JOB_USER_EVENT	When a user sends the job a custom event
ALL	When any of the above occur

All event types other than JOB_USER_EVENT are generated by Tapis. See *Notification Messages* for a description of what Jobs returns for each of the Tapis-generated event.

A JOB_USER_EVENT contains a user-specified payload that targets an active job by using the job's UUID. The sender must be in the same tenant as the job to inject custom events into the job's event stream. The event payload must contain a JSON key named *eventData* with a string value of at least 1 character and no more than 16,384 characters. The string can be unstructured or structured (such as a JSON object) as determined by the sender. The payload can optionally contain an *eventDetail* key with a string value of no more than 64 characters. This key is used to further categorize events and, if not provided, will default to "DEFAULT". User events are always added to the job history and notifications are sent to subscribers interested in those events.

7.2.10 Request Validation

Jobs detects invalid request input early to streamline application debugging and to guard against improper execution. In particular, Tapis usually double quotes strings that contain *dangerous* characters when those strings will appear on a command line. The set of command line dangerous characters is { **&**, **>**, **<**, **|**, **`**, **;** }, all of which have special meaning when issued in a shell.

String validation on job submission requests is as follows:

- The request must conform to the job submission JSON schema.
- Strings **rejected** if they contain *ISO control characters* (0x00-0x1F and 0x7f-0x9f, which include tab, new line and carriage return):
  - job name
  - environment variables
  - system profile name
  - execution system constraints
  - scheduler options
  - container arguments
  - application arguments
  - path names
  - URLs
- Strings **rejected** if they contain *dangerous characters*:
  - scheduler option names
  - container argument names
  - application arguments

  - environment variable keys
  - mpi command
  - system ids
  - queue names
  - log file names
  - job owner
  - job tenant
- Strings **double quoted** if they contain *space characters* or *dangerous characters*:
  - path names
  - scheduler options
  - container arguments
  - environment variable values

Note

Application arguments can contain spaces. Tapis, however, does not automatically double quote these arguments since applications define their own command line conventions. *Therefore, it is the responsibility of the job submitter to use single quotes or escaped double quotes (\") if so desired.*

7.2.11 Shared Components

JobArgSpec

Simple argument strings can be specified in application definitions (AppArgSpec) and in job submission requests (JobArgSpec). These argument strings are passed to specific components in the runtime system, such as the batch scheduler (*schedulerOptions*), the container runtime (*containerArgs*) or the user's application (*appArgs*).

The following rules govern how job arguments are calculated.

1. All argument in application definitions must be named.
2. Application arguments are either REQUIRED, FIXED or one of two optional types.
3. Anonymous (unnamed) argument can be specified in job requests.
4. Job request argument override values set in the application except for FIXED arguments.
5. The final argument ordering is the same as the order specified in the definitions, with application arguments preceding those from the job request. Application arguments maintain their place even when overridden in the job request.
6. The notes field can be any JSON object, i.e., JSON that begins with a brace ("{").

We define a **complete** AppArgSpec as one that has a non-empty name and arg value. We define a **complete** JobArgSpec as one that has a non-empty arg value. A JobArgSpec with the same name as an AppArgSpec inherits from the application and may override the AppArgSpec values.

This is the JSON schema used to define runtime arguments in *ParameterSet*.

```
"JobArgSpec": {
    "$comment": "Used to specify parameters on Jobs submission requests",
    "type": "object",
```

(continues on next page)

(continued from previous page)

```
        "properties": {
            "name": { "type": "string", "minLength": 1, "maxLength": 80 },
            "description": { "type": "string", "minLength": 1, "maxLength": 8096 },
            "include": { "type": "boolean" },
            "arg":  {"type": "string", "minLength": 1},
            "notes": {"type": object}
        },
    "required": ["arg"],
    "additionalProperties": false
}
```

As mentioned, the JobArgSpec is used in conjunction with the AppArgSpec defined in Applications. Arguments in application definitions are merged into job request arguments using the same name matching alorithm as in *FileInputs*.

The *name* identifies the input argument. If present, the name must start with an alphabetic character or an underscore (_) followed by zero or more alphanumeric or underscore characters.

The *description* is used to convey usage information to job requester. If both application and request descriptions are provided, then the request description is appended as a separate paragraph to the application description.

The required *arg* value is an arbitrary string and is used as-is. If this argument's name matches that of an application argument, this *arg* value overrides the application's value except when *inputMode=FIXED* in the application.

The *include* field applies only on named arguments that are also defined in the application definition with *inputMode* INCLUDE_ON_DEMAND or INCLUDE_BY_DEFAULT; this parameter is ignored on all other inputModes. Argument inclusion is discussed in greater detail in following subsection.

Argument Processing

Applications use their AppArgSpecs to pass default values to job requests. The AppArgSpec's *inputMode* determines how to handle arguments during job processing. An *inputMode* field can have these values:

REQUIRED
: The argument must be provided for the job to run. If an arg value is not specified in the application definition, then it must be specified in the job request. When provided in both, the job request arg value overrides the one in application.

FIXED
: The argument is completely defined in the application and not overridable in a job request.

INCLUDE_ON_DEMAND
: The argument, if complete, will only be included in the final argument list constructed by Jobs if it's explicitly referenced and included in the Job request. This is the default value.

INCLUDE_BY_DEFAULT
: The argument, if complete, will automatically be included in the final argument list constructed by Jobs unless explicitly excluded in the Job request.

The truth table below defines how the AppArgSpec's *inputMode* and JobArgSpec's *include* settings interact to determine whether an argument is accepted or ignored during job processing.

AppArgSpec *inputMode*	JobArgSpec *include*	Meaning
INCLUDE_ON_DEMAND	True	include arg
INCLUDE_ON_DEMAND	False	exclude arg
INCLUDE_ON_DEMAND	undefined	include arg
INCLUDE_BY_DEFAULT	True	include arg
INCLUDE_BY_DEFAULT	False	exclude arg
INCLUDE_BY_DEFAULT	undefined	include arg

The JobArgSpec *include* value has no effect on REQUIRED or FIXED arguments. In the cases where the value does apply, not specifying *include* in a named JobArgSpec that matches an AppArgSpec is effectively the same as setting *include=True*. By setting *include=False*, a JobArgSpec can exclude any INCLUDE_ON_DEMAND or INCLUDE_BY_DEFAULT arguments.

KeyValuePair

The JSON schema for defining key/value pairs of strings in various *ParameterSet* components is below.

```
"KeyValuePair": {
    "$comment": "A simple key/value pair",
    "type": "object",
        "properties": {
           "key":   {"type": "string", "minLength": 1},
           "value": {"type": "string", "minLength": 0},
           "description": {"type": "string", "minLength": 1, "maxLength": 8096},
           "include": { "type": "boolean" },
           "notes": { "type": "object" }
        },
     "required": ["key", "value"],
     "additionalProperties": false
}
```

Both the *key* and *value* are required, though the *value* can be an empty string. Descriptions are optional but if present must contain 1 or more characters.

The *include* field applies only to attributes that are defined in application and system definitions with an *inputMode* of INCLUDE_ON_DEMAND or INCLUDE_BY_DEFAULT; this parameter is ignored for all other inputModes. May be used in a job submit request to exclude attributes that have *inputMode=INCLUDE_BY_DEFAULT*. The *notes* field can be any JSON object, i.e., JSON that begins with a brace ("{").

LogConfig

The JSON schema for used to redirect stdout and stderr to named file(s) in supported runtimes.

```
"logConfig": {
    "$comment": "Log file redirection and customization in supported runtimes",
    "type": "object",
    "required": [ "stdoutFilename", "stderrFilename" ],
    "additionalProperties": false,
        "properties": {
            "stdoutFilename": {"type": "string", "minLength": 1},
            "stderrFilename": {"type": "string", "minLength": 1}
        }
}
```

Currently, only the Singularity (Apptainer) and ZIP runtimes are supported. For the Docker runtime, applications can redirect their *stdout* and *stderr* streams to a file (or files) in the *execSystemOutputDir* using an environment variable listed in the next section. Jobs will treat this redirected output like all other application generated output for later retrieval.

When specified, both file name fields must be explicitly assigned, though they can be assigned to the same file. If a *logConfig* object is not specified, or in runtimes where it's not supported, then both stdout and stderr are directed to the default **tapisjob.out** file in the job's output directory. Output files, even when *logConfig* is used, are always relative to the ExecSystemOuputDir (see *Directory Definitions*).

7.3 Job Execution

7.3.1 Environment Variables

The following standard environment variables are passed into each application container run by Tapis as long as they have been assigned a value.

```
_tapisAppId - Tapis app ID
_tapisAppVersion - Tapis app version
_tapisArchiveOnAppError - true means archive even if the app returns a non-zero exit code
_tapisArchiveSystemDir - the archive system directory on which app output is archived
_tapisArchiveSystemId - Tapis system used for archiving app output
_tapisCoresPerNode - number of cores used per node by app
_tapisDtnSystemId - the Data Transfer Node system ID
_tapisDtnSystemInputDir -  the directory on the DTN to which input files will be␣
↪transferred during staging
_tapisDtnSystemOutputDir - the directory on the DTN from which output files will be␣
↪transferred during archiving
_tapisDynamicExecSystem - true if dynamic system selection was used
_tapisEffeciveUserId - the user ID under which the app runs
_tapisExecSystemExecDir - the exec system directory where app artifacts are staged
_tapisExecSystemHPCQueue - the actual batch queue name on an HPC host
_tapisExecSystemId - the Tapis system where the app runs
_tapisExecSystemInputDir - the exec system directory where input files are staged
_tapisExecSystemLogicalQueue - the Tapis queue definition that specifies an HPC queue
_tapisExecSystemOutputDir - the exec system directory where the app writes its output
_tapisStdoutFilename - Name of file to use for stdout.
_tapisStderrFilename - Name of file to use for stderr.
_tapisJobCreateDate - ISO 8601 date, example: 2021-04-26Z
_tapisJobCreateTime - ISO 8601 time, example: 18:44:55.544145884Z
_tapisJobCreateTimestamp - ISO 8601 timestamp, example: 2021-04-26T18:44:55.544145884Z
_tapisJobName - the user-chosen name of the Tapis job
_tapisJobOwner - the Tapis job's owner
_tapisJobUUID - the UUID of the Tapis job
_tapisJobWorkingDir - exec system directory that the app should use for temporary files
_tapisMaxMinutes - the maximum number of minutes allowed for the job to run
_tapisMemoryMB - the memory required per node by the app
_tapisNodes - the number of nodes on which the app runs
_tapisStderrFilename - the file into which the application's stderr is piped
_tapisStdoutFilename - the file into which the application's stdout is piped
_tapisSysBatchScheduler - the HPC scheduler on the execution system
_tapisSysBucketName - an object store bucket name
```

(continues on next page)

(continued from previous page)

```
_tapisSysHost - the IP address or DNS name of the exec system
_tapisSysRootDir - the root directory on the exec system
_tapisTenant - the tenant in which the job runs
```

7.3.2 Macro Substitution

Tapis defines macros or template variables that get replaced with actual values at well-defined points during job creation. The act of replacing a macro with a value is often called macro substitution or macro expansion. The complete list of Tapis macros can be found at JobTemplateVariables.

There is a close relationship between these macro definitions and the Tapis environment variables just discussed: Macros that have values assigned are passed as environment variables into application containers. This makes macros used during job creation available to applications at runtime.

Most macro definitions are *ground* definitions because their values do not depend on any other macros. On the other hand, *derived* macro definitions can include other macro definitions. For example, in *Directory Assignments* the default input file directory is constructed with two macro definitions:

```
execSystemInputDir = ${JobWorkingDir}/jobs/${JobUUID}
```

Macro values are referenced using the ${Macro-name} notation. Since derived macro definitions reference other macros, there is the possibility of circular references. Tapis detects these errors and aborts job creation.

Below is the complete, ordered list of derived macros. Each macro in the list can be defined using any ground macro and any macro that preceeds it in the list. Result are undefined if a derived macro references a macro that follows it in the derived list.

1. JobName
2. JobWorkingDir
3. ExecSystemInputDir
4. ExecSystemExecDir
5. ExecSystemOutputDir
6. ArchiveSystemDir
7. SchedulerOptions
8. AppArgs
9. ContainerArgs
10. LogConfig

Finally, macro substitution is applied to the job *description* field, whether the description is specified in an application or a submission request.

Macro Functions

Directory assignments in systems, applications and job requests can also use the **HOST_EVAL($var)** function at the beginning of their path assignments. This function dynamically extracts the named environment variable's value from an execution or archive host *at the time the job request is made*. Specifically, the environment variable's value is retrieved by logging into the host as the Job owner and issuing "echo $var". The example in *Data Transfer Nodes* uses this function.

To increase application portability, an optional default value can be passed into the **HOST_EVAL** function. The function's complete signature with the optional path parameter is:

HOST_EVAL($VAR, path)

If the environment variable VAR does not exist on the host, then the literal path parameter is returned by the function. This added flexibility allows applications to run in different environments, such as on TACC HPC systems that automatically expose certain environment variables and VMs that might not. If the environment variable does not exist and no optional path parameter is provided, the job fails due to invalid input.

7.3.3 Job Status

The list below contains all possible states of a Tapis job, which are indicated in the *status* field of a job record. The initial state is PENDING. Terminal states are FINISHED, CANCELLED and FAILED. The BLOCKED state indicates that the job is recovering from a resource constraint, network problem or other transient problem. When the problem clears, the job will restart from the state in which blocking occurred.

```
PENDING - Job processing beginning
PROCESSING_INPUTS - Identifying input files for staging
STAGING_INPUTS - Transferring job input data to execution system
STAGING_JOB - Staging runtime assets to execution system
SUBMITTING_JOB - Submitting job to execution system
QUEUED - Job queued to execution system queue
RUNNING - Job running on execution system
ARCHIVING - Transferring job output to archive system
BLOCKED - Job blocked
PAUSED - Job processing suspended
FINISHED - Job completed successfully
CANCELLED - Job execution intentionally stopped
FAILED - Job failed
```

Normal processing of a successfully executing job proceeds as follows:

```
PENDING->PROCESSING_INPUTS->STAGING_INPUTS->STAGING_JOB->SUBMITTING_JOB->
  QUEUED->RUNNING->ARCHIVING->FINISHED
```

7.3.4 Notification Messages

Notifications are the messages sent to subscribers who have registered interest in certain job events. In this section, we specify the messages sent to subscribers for event generated by the Jobs service. See *Subscriptions* for an introduction to the different event types, including user generated events (type JOB_USER_EVENT), which are not discussed here.

For events generated by the Jobs service, the *data* field in notification messages received by subscribers contains a JSON object that always include these fields:

- *jobName* - the user-specified job name
- *jobOwner* - the user who submitted the job
- *jobUuid* - the unique job ID
- *message* - a human readable message

Each of the Job event types also include additional fields as shown:

Job Event Type	Additional Fields
JOB_NEW_STATUS	newJobStatus, oldJobStatus
JOB_INPUT_TRANSACTION_ID	transferStatus, transactionId
JOB_ARCHIVE_TRANSACTION_ID	transferStatus, transactionId
JOB_SUBSCRIPTION	action, numSubscriptions
JOB_SHARE_EVENT	resourceType, shareType, grantee, grantor
JOB_ERROR_MESSAGE	jobStatus

Additionally, when either of these conditions hold:

1. JOB_NEW_STATUS messages indicate a **terminal** *newJobStatus*, or
2. JOB_ERROR_MESSAGE messages have *eventDetail* = "FINAL_MESSAGE",

then the following additional fields are included in the notification:

- *blockedCount* - the number of times the job blocked (JSON number)
- *condition* - a code that describes the job's final disposition (see *ConditionCodes*)
- *remoteJobId* - execution system job id (ex: pid, slurm id, docker hash, etc.)
- *remoteJobId2* - execution system auxilliary id associated with a job
- *remoteOutcome* - FINISHED, FAILED, FAILED_SKIP_ARCHIVE
- *remoteResultInfo* - application exit code
- *remoteQueue* - execution system scheduler queue
- *remoteSubmitted* - time job was submitted on remote system
- *remoteStarted* - time job started running on remote system
- *remoteEnded* - time job stopped running on remote system

Job terminal statuses are FINISHED, CANCELLED and FAILED.

7.3.5 Job Condition Codes on Termination

When a job terminates, in addition to being assigned a terminal status, a job is assigned one of the following condition codes.

```
CANCELLED_BY_USER - Job cancelled by user
JOB_ARCHIVING_FAILED - Job error while archiving outputs
JOB_DATABASE_ERROR - Jobs is unable to access its database
JOB_EXECUTION_MONITORING_ERROR - An error occurred during application monitoring
JOB_EXECUTION_MONITORING_TIMEOUT - Job time expired during execution monitoring
JOB_FILES_SERVICE_ERROR - An error involving the Files service occurred
JOB_INTERNAL_ERROR - Jobs service internal error
JOB_INVALID_DEFINITION - Invalid job record
JOB_LAUNCH_FAILURE - Tapis is unable to launch job
JOB_QUEUE_MONITORING_ERROR - An error occurred during application queue monitoring
JOB_RECOVERY_FAILURE - Tapis unable to recover job
JOB_RECOVERY_TIMEOUT - Tapis recovery time expired
JOB_REMOTE_ACCESS_ERROR - Jobs could not access a resource on a remote system
JOB_REMOTE_OUTCOME_ERROR - User application returned a non-zero exit code
JOB_UNABLE_TO_STAGE_INPUTS - Unable to stage application input files
JOB_UNABLE_TO_STAGE_JOB - Unable to stage application assets
```

(continues on next page)

(continued from previous page)

```
JOB_TRANSFER_FAILED_OR_CANCELLED - A file transfer failed or was cancelled
JOB_TRANSFER_MONITORING_TIMEOUT - Jobs transfer monitoring expired
NORMAL_COMPLETION - Job completed normally
SCHEDULER_CANCELLED - Batch scheduler cancelled job
SCHEDULER_DEADLINE - Batch scheduler deadline exceeded
SCHEDULER_OUT_OF_MEMORY - Batch scheduler out of memory error
SCHEDULER_STOPPED - Batch scheduler stopped job
SCHEDULER_TIMEOUT - Batch scheduler timed out job
SCHEDULER_TERMINATED - Batch scheduler terminated job
```

Codes that begin with "JOB" indicate Jobs service conditions. Those that begin with "SCHEDULER" indicate a condition reported by a batch scheduler. The CANCELLED_BY_USER condition results from direct user action. Jobs that successfully execute are assigned the code NORMAL_COMPLETION.

7.3.6 Dynamic Execution System Selection

Future feature.

Selecting an execution system dynamically is not yet implemented.

7.3.7 Data Transfer Nodes

Introduction

The Jobs, Applications, Files and Systems services cooperate to allow the configuration and execution of jobs. In this section, we discuss the use of a high throughput **Data Transfer Node** (DTN) to stage job inputs and archive outputs. DTNs are used to optimize transfer speeds and to allow nodes to be separately optimized for either storage or job execution. For example, users often stage input data to long-term storage (the DTN), such as *cloud.corral* at TACC, and then copy that data to a fast, temporary file system, such as a "scratch" file system, when running jobs. DTNs are also used when specialized transfer software, such as GLOBUS, is required on nodes to receive data, but those nodes are not appropriate to run jobs. In this case, the DTN receives the data from a remote location and then another transfer copies them from the DTN to an execution system.

The DTN usage pattern is effective when:

1. The DTN has high performance network and storage capabilities,
2. the target system for data transfers and the DTN system have shared storage, and
3. all file inputs or all file outputs are to be staged through the DTN.

In this situation, the data transfers performed by Jobs benefit from the DTN's high performance I/O capabilities, while applications can still access their input from systems optimized for execution.

Configuring Tapis for DTN Usage

In order to run a job and use a DTN for file input staging or output archiving, the execution system and the application must be properly configured. There are also a few rules to follow and requirements that must be satisfied.

Configuring the Execution and DTN Systems

The execution system must have the attribute *dtnSystemId* set to the Tapis system that can be used as a DTN. Note that use of the DTN is optional. Use of a DTN is triggered by the setting of *dtnSystemInputDir* or *dtnSystemOutputDir* in either the job submission request or the application definition.

As listed below under *Rules and Requirements*, the execution system and the DTN system must have the same *rootDir*, must have shared storage, and the *rootDir* must be part of the shared storage.

Configuring the Application

There are two relevant attributes in an Application definition, *dtnSystemInputDir* and *dtnSystemOutputDir*. By default, these are set to the special value *!tapis_not_set*. This value indicates that, by default, a DTN system will not be used for file inputs or archive outputs. In order to trigger the use of a DTN during either file input staging or archive output, these values must be set:

dtnSystemInputDir
: Directory relative to *rootDir* to which input files will be transferred prior to launching the application.

dtnSystemOutputDir
: Directory relative to *rootDir* from which output files will be transferred during the archiving phase.

Note that these attributes may be overriden by the job submission request. Please see the next section.

Constructing the Job Submission Request

In a job submission request, the values for *dtnSystemInputDir* or *dtnSystemOutputDir* can be set in order to override the values defined by the application. They can be set to *!tapis_not_set* to turn off DTN usage or set to an alternate path.

DTN Rules and Requirements

Certain rules and requirements need to be observed to correctly use a DTN.

- **Rule:** All file inputs or archive outputs will go through the DTN.
- **Rule:** Any HOST_EVAL expressions or macro substitutions (such as *effectiveUserId*) used in *dtnSystemInputDir* or *dtnSystemOutputDir* will be evaluated in the context of the execution system.
- **Rule:** Each job should use its own *dtnSystemInputDir* or *dtnSystemOutputDir* to avoid data from different jobs being mixed, overwritten or deleted before being used.
- **Requirement:** The execution system and the DTN system must have shared storage.
  - For example, there could be an NFS mount on the execution system pointing to the DTN host, or the DTN may be a Globus endpoint running on the execution host.
  - Note that for an HPC cluster it could be that the shared storage is set up on login nodes and not on compute nodes. This case is supported since the Tapis Jobs service typically uses a login node when executing an application.
- **Requirement:** Restrictions on *rootDir*
  - The **rootDir** for the execution system and the DTN system **must be same**. This is validated by the Systems service when the execution system is created or updated.
  - The **rootDir** must be part of the shared storage.
  - The absolute path to the shared storage must be the same on both the execution and DTN systems. This is because:
    - **Rule:** When the Tapis Jobs service moves data to or from the DTN system, it assumes no path translation is needed.
- **Requirement:** For any file inputs defined for the job, Tapis must support file transfers from the input source to the DTN system.
  - For example, if the source for the file input is a system of type GLOBUS, then the DTN must also be of type GLOBUS.
- **Requirement:** Credentials must be registered for the *effectiveUserId* for each system involved.

- **Requirement:** For each system involved, the *effectiveUserId* for the system must have appropriate Tapis and native file system access for the paths defined.
  - For the execution and DTN systems, the *effectiveUserId* must have READ/WRITE access for both *dtnSystemInputDir* and *dtnSystemOutputDir*.

Modified Jobs Service Behavior

When an application runs and the execution system specifies a DTN, the Jobs service will behave as follows:

During staging of input files

If the execution system specifies a DTN and *dtnSystemInputDir* is set:

- The source systems and paths defined in FileInputs are unchanged.
- The target system for the transfer request sent to the Files service will be the DTN as specified in the execution system definition.
- The target path will be *${DtnSystemInputDir}*.
- The staged input files will be *moved* from *${DtnSystemInputDir}* to *${ExecSystemInputDir}* on the execution system.
  - Note that the fully resolved absolute path to *${DtnSystemInputDir}* must be **the same** on both the execution system and the DTN system. If this is not true the operation may fail.
  - The *move* operation deletes the files from *${DtnSystemInputDir}*.

During archiving of output files

If the execution system specifies a DTN and *dtnSystemOutputDir* is set:

- The output files generated on the execution system are *moved* from *${ExecSystemOutputDir}* to *${DtnSystemOutputDir}*
  - Note that the fully resolved absolute path to *${DtnSystemOutputDir}* must be **the same** on both the execution system and the DTN system. If this is not true the operation may fail.
  - The *move* operation deletes the files from *${ExecSystemOutputDir}*.
- The source system for the transfer request sent to the Files service will be the DTN as specified in the execution system definition.
- The source path will be *${DtnSystemOutputDir}*.
- The target system and path as defined in the archive definition is unchanged.

Example DTN Configuration and Usage

We provide the examples below to illustrate DTN configuration and usage.

DTN system definition:

```
{
  "id":"example-dtn",
  "host":"dtn.host.example.com",
  "systemType":"LINUX",
  "effectiveUserId": "testuser1",
  "defaultAuthnMethod":"PKI_KEYS",
  "rootDir":"/",
```

(continues on next page)

(continued from previous page)

```
  "canExec": false
}
```

Execution system definition:

```
{
  "id":"example-exec-with-dtn",
  "host":"exec.host.example.com",
  "systemType":"LINUX",
  "effectiveUserId": "testuser1",
  "defaultAuthnMethod":"PKI_KEYS",
  "rootDir":"/",
  "canExec": true
  "canRunBatch": true,
  "jobRuntimes": [ { "runtimeType": "SINGULARITY" } ],
  "jobWorkingDir": "HOST_EVAL($SCRATCH)/dtn_test/jobs",
  "dtnSystemId": "example-dtn",
  "batchScheduler": "SLURM",
  "batchLogicalQueues": [
  {
    "name": "tapisNormal",
    "hpcQueueName": "skx-normal",
    "maxJobs": 50,
    "maxJobsPerUser": 10,
    "minNodeCount": 1,
    "maxNodeCount": 16,
    "minCoresPerNode": 1,
    "maxCoresPerNode": 68,
    "minMemoryMB": 1,
    "maxMemoryMB": 16384,
    "minMinutes": 1,
    "maxMinutes": 60
  }
],
"batchDefaultLogicalQueue": "tapisNormal",
"batchSchedulerProfile": "tacc"
}
```

Application definition:

```
{
  "id":"example-app-with-dtn",
  "version": "1",
  "containerImage": "example/example-app:1",
  "jobType": "BATCH",
  "runtime": "SINGULARITY",
  "runtimeOptions": ["SINGULARITY_RUN"],
  "jobAttributes": {
    "description": "Transfer file and verify transfer",
    "execSystemId": "example-exec-with-dtn",
    "execSystemExecDir": "${JobWorkingDir}/${JobUUID}",
    "execSystemInputDir": "{JobWorkingDir}/${JobUUID}/inputs",
```

(continues on next page)

(continued from previous page)

```
    "execSystemOutputDir": "${JobWorkingDir}/${JobUUID}/output",
    "dtnSystemInputDir": "/shared/storage/stage_inputs/${JobUUID}",
    "dtnSystemOutputDir": /shared/storage/stage_outputs/${JobUUID}",
    "archiveSystemId": "archive-storage-system",
    "archiveSystemDir": "/archive/storage/${JobUUID}",
    "fileInputs": [
      {
       "name": "file1",
       "description": "A text file",
       "inputMode": "REQUIRED",
       "sourceUrl": "tapis://example-data-repo/data/file1.txt",
       "targetPath": "file1.txt"
      }
    ]
  }
}
```

Job submission request:

```
{
  "name": "Tapis V3 example job using a DTN",
  "appId": "example-app-with-dtn",
  "appVersion": "0.0.1",
  "parameterSet": {
    "schedulerOptions": [ { "arg": "-A EXAMPLE-ALLOCATION" } ]
}
```

Note that the execution system specifies the DTN system and the application specifies the DTN directories:

dtnSystemId
: The Tapis system to use as a DTN.

dtnSystemInputDir
: Directory relative to *rootDir* to which input files will be transferred prior to launching the application.

dtnSystemOutputDir
: Directory relative to *rootDir* from which output files will be transferred during the archiving phase.

7.4 Container Runtimes

The Tapis v3 Jobs service supports Docker, Singularity and ZIP containers run natively (i.e., not run using a batch scheduler like Slurm). In general, Jobs launches an application's container on a remote system, monitors the container's execution, and captures the application's exit code after it terminates. Jobs uses SSH to connect to the execution system to issue Docker, Singularity or native operating system commands.

To launch a job, the Jobs service creates a bash script, **tapisjob.sh**, with the runtime-specific commands needed to execute the container. This script references **tapisjob.env**, a file Jobs creates to pass environment variables to application containers. Both files are staged in the job's execSystemExecDir and, by default, are archived with job output on the archive system. See *archiveFilter* to override this default behavior, especially if archives will be shared and the scripts pass sensitive information into containers.

7.4.1 Docker

To launch a Docker container, the Jobs service will SSH to the target host and issue a command using this template:

```
docker run [docker options] image[:tag|@digest] [application args]
```

1. docker options: (optional) user-specified arguments passed to docker
2. image: (required) user-specified docker application image
3. application arguments: (optional) user-specified command line arguments passed to the application

The docker run-command options *--cidfile*, *-d*, *-e*, *--env*, *--name*, *--rm*, and *--user* are reserved for use by Tapis. Most other Docker options are available to the user. The Jobs service implements these calling conventions:

1. The container name is set to the job UUID.
2. The container's user is set to the user ID used to establish the SSH session.
3. The container ID file is specified as *<JobUUID>.cid* in the execSystemExecDir, i.e., the directory from which the container is launched.
4. The container is removed after execution using the *-rm* option or by calling *docker rm*.

Logging

Logging should be considered up front when defining Tapis applications to run under Docker. Since Jobs removes Docker containers after they execute, the container's log is lost under the default Docker logging configuration. Typically, Docker pipes *stdout* and *stderr* to the container's log, which requires the application to take deliberate steps to preserve these outputs.

An application can maintain control over its log output by logging to a file outside of the container. The application can do this by redirecting *stdout* and *stderr* or by explicitly writing to a file. As discussed in *dir-definitions*, the application always has read/write access to the host's *execSystemOutputDir*, which is mounted at /TapisOutput in the container (see next section).

On the other hand, applications can run on machines where the default Docker log driver is configured to write to files or services outside of containers. In addition, Tapis passes any user-specified *log-driver* and *log-opts* options to *docker run*, so all customizations supported by Docker are possible.

Volume Mounts

In addition to the above conventions, bind mounts are used to mount the execution system's standard Tapis directories at the same locations in every application container.

```
execSystemExecDir   on host is mounted at /TapisExec in the container.
execSystemInputDir  on host is mounted at /TapisInput in the container.
execSystemOutputDir on host is mounted at /TapisOutput in the container.
```

7.4.2 Singularity

Tapis provides two distinct ways to launch a Singularity containers, using *singluarity instance start* or *singularity run*.

Warning

Please note that use of the SINGULARITY_START runtime option has been deprecated. Support for *singluarity instance start* will be removed in a future release. If you have a need for this option please contact Tapis support (cicsupport@tacc.utexas.edu).

Singularity Start (DEPRECATED)

Singularity's support for detached processes and services is implemented natively by its instance start, stop and list commands. To launch a container, the Jobs service will SSH to the target host and issue a command using this template:

```
singularity instance start [singularity options] <image id> [application arguments] <job␣
↪uuid>
```

where:

1. singularity options: (optional) user-specified argument passed to singularity start
2. image id: (required) user-specified singularity application image
3. application arguments: (optional) user-specified command line arguments passed to the application
4. job uuid: the job uuid used to name the instance (always set by Jobs)

The singularity options *--pidfile*, *--env* and *--name* are reserved for use by Tapis. Users specify the environment variables to be injected into their application containers via the *envVariables* parameter. Most other singularity options are available to users.

Jobs will then issue *singularity instance list* to obtain the container's process id (PID). Jobs determines that the application has terminated when the PID is no longer in use by the operating system.

By convention, Jobs will look for a **tapisjob.exitcode** file in the Job's output directory after containers terminate. If found, the file should contain only the integer code the application reported when it exited. If not found, Jobs assumes the application exited normally with a zero exit code.

Finally, Jobs issues a *singularity instance stop <job uuid>* to clean up the singularity runtime environment and terminate all processes associated with the container.

Singularity Run

Jobs also supports a more do-it-yourself approach to running containers on remote system using singularity run. To launch a container, the Jobs service will SSH to the target host and issue a command using this template:

```
nohup singularity run [singularity options] <image id> [application arguments] >␣
↪tapisjob.out 2>&1 &
```

where:

1. nohup: allows the background process to continue running even if the SSH session ends.
2. singularity options: (optional) user-specified arguments passed to singularity run.
3. image id: (required) user-specified singularity application image.
4. application arguments: (optional) user-specified command line arguments passed to the application.
5. redirection: stdout and stderr are redirected to **tapisjob.out** in the job's output directory.

The singularity *--env* option is reserved for use by Tapis. Users specify the environment variables to be injected into their application containers via the *envVariables* parameter. Most other singularity options are available to users.

Jobs will use the PID returned when issuing the background command to monitor the container's execution. Jobs determines that the application has terminated when the PID is no longer in use by the operating system.

Jobs uses the same **TapisJob.exitcode** file convention introduced above to attain the application's exit code (if the file exists).

Required Scripts

The Singularity Start and Singularity Run approaches boath allow SSH sessions between Jobs and execution hosts to end without interrupting container execution. Each approach, however, requires that the application image be appropriately constructed. Specifically,

```
Singularity start requires the startscript to be defined in the image.
Singularity run requires the runscript to be defined in the image.
```

Required Termination Order

Since Jobs monitors container execution by querying the operating system using the PID obtained at launch time, the initially launched program should be the last part of the application to terminate. The program specified in the image script can spawn any number of processes (and threads), but it should not exit before those processes complete.

Optional Exit Code Convention

Applications are not required to support the **TapisJob.exitcode** file convention as described above, but it is the only way in which Jobs can report the application specified exit status to the user.

7.4.3 ZIP

Standard archive files, such as zip and compressed tar, can be treated as a type of custom *application image* and launched in a runtime environment defined by Tapis. This *ZIP runtime environment* maximizes application flexibility while retaining much of the reproducibility and automation benefits of Tapis.

To define a ZIP application, specify "ZIP" as the *runtime* parameter in an application definition. The ZIP runtime works with either BATCH or FORK job types. For the ZIP runtime, *containerImage* must be an absolute path or a URL in a format supported by the Files service, such as URLs using the http, https or tapis protocols. The Applications service validates the *containerImage* attribute when an application is created or updated.

An application's archive file can contain scripts, binary executables, libraries and any other data the application developer needs—Tapis does not restrict content. The archive must be in zip format or any format readable by tar. In order to run correctly, the application must:

1. Have a designated executable program for Tapis to launch,
2. guarantee that the executable code is compatible with the target system's runtime,
3. guarantee that for FORK jobs the launched program continues to run until the application completes,
4. include in the archive all application dependencies not present on the target system.

There are also a number of conventions that ZIP applications should observe:

1. FORK jobs should write their exit code to *${execSystemOutputDir}/tapisjob.exitcode*.
2. Except during testing, allow Tapis to remove downloaded archive files from the execution system after job completion.

The following sections describe the ZIP runtime processing during each phase of a job's lifecycle.

Staging Inputs

Inputs are staged in exactly the same way as in other runtime environments.

Staging Application Assets

ZIP jobs package executable code and data in an archive file that either already exists on the execution system or gets downloaded as part of job execution. The archive file is unpacked into the *execSystemExecDir*.

The archive file's location is specified in the *containerImage* attribute of the application. The location may either be (1) an absolute path on the execution system or (2) a URL. When an absolute path is used, the application archive already resides on the execution system. When a URLs is used, the archive is downloaded using one of following protocols: *http://*, *https://* and *tapis://*. The destination is always *execSystemExecDir* when URLs are used.

Application archives are always unpacked into *execSystemExecDir* whether the archive already existed on the system or was downloaded to the system. Jobs uses either *unzip* or *tar* as shown below.

```
unzip <pathToArchiveFile>
tar -xf <pathToArchiveFile>
```

The <*pathToArchiveFile*> is the absolute path to the archive file, such as */path/to/archive/app.zip* or */path/to/archive/app.tgz*. The <*pathToArchiveFile*> must be present and accessible on the execution system. When an application archive is already present on the system (the non-URL case), the archive is not be copied, but its contents are extracted directly into the *execSystemExecDir*.

Note that the version of tar distributed with typical Linux distributions can unpack a number of compression formats, including gzip, bzip2 and xz, but **not** zip. When an archive file name uses the *.zip* suffix, Tapis assumes *zip* formatting is being used. Tapis checks that the *unzip* command is available on the execution system and, if not, the job aborts.

Launching the Application

Once an application archive is unpacked, Jobs creates its own launch script, *tapisjob.sh*, and places it in the *execSystemExecDir*. From that directory, *tapisjob.sh* calls the user-provided executable. This executable is either **./tapisjob_app.sh**, if it exists, or an executable named in the **./tapisjob.manifest** file. The manifest file has these characteristics:

- The manifest file is optional.
- If *tapisjob_app.sh* does not exist then *tapisjob.manifest* must exist at the top-level of the archive and must specify the key/value pair: *tapisjob_executable*=<*relative path to executable file*>.
  - The *tapisjob_executable* value is a path relative to *execSystemExecDir*.
  - The *tapisjob_executable* value cannot contain a double dot ".." or semicolon ";".
- If both *tapisjob_app.sh* and *tapisjob.manifest* exist, then *tapisjob_app.sh* will be used.
- If Jobs is not able to determine an executable to run then the job will fail.
- The application can put other information in *tapisjob.manifest*, such as build number, application version, etc.
  - The format of all entries in the manifest is: <key>=<value>, each on its own line.
  - Tapis ignores additional entries in the manifest.

Other than *tapisjob.manifest* and *tapisjob_app.sh*, no user-defined files in the top level archive directory should have a name that starts with "tapisjob". All such file names are reserved by Tapis and may be overwritten by the Jobs service. Specifically, the Jobs service creates files named *tapisjob.sh*, *tapisjob.env* and several temporary files that start with "tapisjob".

Environment and Executable Output

ZIP applications can expect to run in a environment that closely matches other Tapis runtimes. In particular, Jobs exports all user-specified and Tapis-generated environment variables in the SSH terminal from which it launches the application's executable. These are the same variables available to applications in other runtime environments. Similarly, *appArgs*, *containerArgs* and *schedulerOptions* semantics are unchanged.

Jobs also sets up the redirection of stdout and stderr using the *LogConfig* parameter as it does in other environments. Both streams are directed to *execSystemOutputDir/tapisjob.out* by default.

Monitoring the Application

For ZIP BATCH jobs, Tapis monitors the slurm job just as it does for other runtime types.

For ZIP FORK jobs, Tapis monitors the PID of the process launched by *tapisjob.sh*. When that PID goes away, Jobs assumes application processing has completed.

Note

It is the application developer's responsibility to terminate the executable launched by Tapis ONLY AFTER all other processes spawned by the application have terminated.

After a ZIP FORK job completes, Jobs looks for a file named **tapisjob.exitcode** in the *execSystemOutputDir*. If present, it is assumed to contain a single line with the application's exit code. A non-zero code is interpreted as a failure. If the file is not found, Jobs assumes success and reports an exit code of zero.

As with other runtimes, files created by Tapis during job execution are archived if *includeLaunchFiles* is true (the default). These files include *tapisjob.sh* and *tapisjob.env*.

Archiving Application Output

ZIP job outputs are archived using the same source and target specifications as other runtime types.

For ZIP jobs that specify their *containerImage* with an absolute path, the application archive file is never removed.

For ZIP jobs that specify their *containerImage* with a URL, Jobs removes the downloaded archive file by default to conserve storage. To prevent this automatic clean up, a new *containerArgs* flag, **--tapis-zip-save**, is defined. The flag takes no value and can be specified in the *containerArgs* list in the application definition or in the job submission request. If this argument is present, then the application's archive file will be left in the *execSystemExecDir*.

Usage Notes

To minimize ZIP archive size, jobs that invoke singularity containers may want to pre-position any large SIF files in shared directories on execution systems.

If *isMPI=true*, then Jobs will insert the MPI run command on the command line as usual. This is true for both BATCH and FORK job types. If the user-designated executable program launches MPI jobs itself, setting *isMPI=false* (the default) prevents Jobs from making conflicting MPI calls.

7.5 Querying Jobs

7.5.1 Get Jobs list

With PySDK:

```
$ t.jobs.getJobList(limit=2, orderBy='lastUpdated(desc),name(asc)', computeTotal=True)
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/list?limit=2&orderBy=lastUpdated(desc),
↪name(asc)&computeTotal=true
```

The response will look something like the following:


```
{
"result": [
    {
        "uuid": "731b65f4-43e9-4a7a-b3a0-68644b53c1cb-007",
        "name": "SyRunSleepSecondsNoIPFiles-2",
        "owner": "testuser2",
        "appId": "SyRunSleepSecondsNoIPFiles-2",
        "created": "2021-07-21T19:56:02.163984Z",
        "status": "FINISHED",
        "remoteStarted": "2021-07-21T19:56:18.628448Z",
        "ended": "2021-07-21T19:56:52.637554Z",
        "tenant": "dev",
        "execSystemId": "tapisv3-exec2",
        "archiveSystemId": "tapisv3-exec2",
        "appVersion": "0.0.1",
        "lastUpdated": "2021-07-21T19:56:52.637554Z"
    },
    {
        "uuid": "79dfaba5-bfb4-4c6d-a198-643bda211dbf-007",
        "name": "SlurmSleepSeconds",
        "owner": "testuser2",
        "appId": "SlurmSleepSecondsVM",
        "created": "2021-07-21T19:16:02.019916Z",
        "status": "FINISHED",
        "remoteStarted": "2021-07-21T19:16:35.102868Z",
        "ended": "2021-07-21T19:16:57.909940Z",
        "tenant": "dev",
        "execSystemId": "tapisv3-exec2-slurm",
        "archiveSystemId": "tapisv3-exec2-slurm",
        "appVersion": "0.0.1",
        "lastUpdated": "2021-07-21T19:16:57.909940Z"
    }
],
"status": "success",
"message": "JOBS_LIST_RETRIVED Jobs list for the user testuser2 in the tenant dev␣
↪retrived.",
"version": "1.0.0-rc1",
"metadata": {
    "recordCount": 2,
    "recordLimit": 2,
    "recordsSkipped": 0,
    "orderBy": "lastUpdated(desc),name(asc)",
    "startAfter": null,
    "totalCount": 1799
}
}
```

7.5.2 Get Job Details

With PySDK:

```
$ t.jobs.getJob(jobUuid='ba34f946-8a18-44c4-9b25-19e21dfadf69-007')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007
```

The response will look something like the following:


```
  {
 "result": {
    id: 9594
    name: SleepSeconds
    owner: testuser2
    tenant: dev
    description: Transfer files and sleep for a specified amount of time
    status: FINISHED
    condition: NORMAL_COMPLETION
    lastMessage: Setting job status to FINISHED.
    created: 2024-02-21T17:18:01.774368Z
    ended: 2024-02-21T17:18:49.182976Z
    lastUpdated: 2024-02-21T17:18:49.182976Z
    uuid: aa83fce9-074c-47fb-a8a0-a9ad216093bb-007
    appId: SleepSeconds
    appVersion: 0.0.1
    archiveOnAppError: true
    dynamicExecSystem: false
    execSystemId: tapisv3-exec2
    execSystemExecDir: /workdir/jobs/aa83fce9-074c-47fb-a8a0-a9ad216093bb-007
    execSystemInputDir: /workdir/jobs/aa83fce9-074c-47fb-a8a0-a9ad216093bb-007
    execSystemOutputDir: /workdir/jobs/aa83fce9-074c-47fb-a8a0-a9ad216093bb-007/output
    execSystemLogicalQueue: null
    archiveSystemId: tapisv3-exec
    archiveSystemDir: /jobs/aa83fce9-074c-47fb-a8a0-a9ad216093bb-007/archive
    dtnSystemId: null
    dtnSystemInputDir: null
    dtnSystemOutputDir: null
    nodeCount: 1
    coresPerNode: 1
    memoryMB: 100
    maxMinutes: 240
    fileInputs: [{"name": "empty", "notes": "{}", "envKey": null, "optional": true,
↪"sourceUrl": "tapis://tapisv3-exec/jobs/input/empty.txt", "targetPath": "empty.txt",
↪"description": "An empty file", "autoMountLocal": true, "srcSharedAppCtx": "",
↪"destSharedAppCtx": ""}, {"name": "file1", "notes": "{}", "envKey": null, "optional":␣
↪true, "sourceUrl": "tapis://tapisv3-exec/jobs/input/file1.txt", "targetPath": "file1.
↪txt", "description": "A random text file", "autoMountLocal": true, "srcSharedAppCtx": "
↪", "destSharedAppCtx": ""}, {"name": "file2", "notes": "{}", "envKey": null, "optional
↪": true, "sourceUrl": "tapis://tapisv3-exec/jobs/input/file2.txt", "targetPath":
↪"file2.txt", "description": "Another random text file", "autoMountLocal": true,
↪"srcSharedAppCtx": "", "destSharedAppCtx": ""}]
```


(continues on next page)

(continued from previous page)

```
    parameterSet: {"appArgs": [], "logConfig": {"stderrFilename": "tapisjob.out",
↪"stdoutFilename": "tapisjob.out"}, "envVariables": [{"key": "_tapisAppId", "notes":␣
↪null, "value": "SleepSeconds", "include": null, "description": null}, {"key": "_
↪tapisAppVersion", "notes": null, "value": "0.0.1", "include": null, "description":␣
↪null}, {"key": "_tapisArchiveOnAppError", "notes": null, "value": "true", "include":␣
↪null, "description": null}, {"key": "_tapisArchiveSystemDir", "notes": null, "value":
↪"/jobs/aa83fce9-074c-47fb-a8a0-a9ad216093bb-007/archive", "include": null, "description
↪": null}, {"key": "_tapisArchiveSystemId", "notes": null, "value": "tapisv3-exec",
↪"include": null, "description": null}, {"key": "_tapisCoresPerNode", "notes": null,
↪"value": "1", "include": null, "description": null}, {"key": "_tapisDynamicExecSystem",
↪ "notes": null, "value": "false", "include": null, "description": null}, {"key": "_
↪tapisEffectiveUserId", "notes": null, "value": "testuser2", "include": null,
↪"description": null}, {"key": "_tapisExecSystemExecDir", "notes": null, "value": "/
↪workdir/jobs/aa83fce9-074c-47fb-a8a0-a9ad216093bb-007", "include": null, "description
↪": null}, {"key": "_tapisExecSystemId", "notes": null, "value": "tapisv3-exec2",
↪"include": null, "description": null}, {"key": "_tapisExecSystemInputDir", "notes":␣
↪null, "value": "/workdir/jobs/aa83fce9-074c-47fb-a8a0-a9ad216093bb-007", "include":␣
↪null, "description": null}, {"key": "_tapisExecSystemOutputDir", "notes": null, "value
↪": "/workdir/jobs/aa83fce9-074c-47fb-a8a0-a9ad216093bb-007/output", "include": null,
↪"description": null}, {"key": "_tapisJobCreateDate", "notes": null, "value": "2024-02-
↪21Z", "include": null, "description": null}, {"key": "_tapisJobCreateTime", "notes":␣
↪null, "value": "17:18:01.774367559Z", "include": null, "description": null}, {"key": "_
↪tapisJobCreateTimestamp", "notes": null, "value": "2024-02-21T17:18:01.774367559Z",
↪"include": null, "description": null}, {"key": "_tapisJobName", "notes": null, "value
↪": "SleepSeconds", "include": null, "description": null}, {"key": "_tapisJobOwner",
↪"notes": null, "value": "testuser2", "include": null, "description": null}, {"key": "_
↪tapisJobUUID", "notes": null, "value": "aa83fce9-074c-47fb-a8a0-a9ad216093bb-007",
↪"include": null, "description": null}, {"key": "_tapisJobWorkingDir", "notes": null,
↪"value": "/workdir", "include": null, "description": null}, {"key": "_tapisMaxMinutes",
↪ "notes": null, "value": "240", "include": null, "description": null}, {"key": "_
↪tapisMemoryMB", "notes": null, "value": "100", "include": null, "description": null}, {
↪"key": "_tapisNodes", "notes": null, "value": "1", "include": null, "description":␣
↪null}, {"key": "_tapisStderrFilename", "notes": null, "value": "tapisjob.out", "include
↪": null, "description": null}, {"key": "_tapisStdoutFilename", "notes": null, "value":
↪"tapisjob.out", "include": null, "description": null}, {"key": "_tapisSysHost", "notes
↪": null, "value": "129.114.35.53", "include": null, "description": null}, {"key": "_
↪tapisSysRootDir", "notes": null, "value": "/home/testuser2", "include": null,
↪"description": null}, {"key": "_tapisTenant", "notes": null, "value": "dev", "include
↪": null, "description": null}, {"key": "JOBS_PARMS", "notes": "{}", "value": "15",
↪"include": true, "description": "nothing important"}, {"key": "MAIN_CLASS", "notes": "
↪{}", "value": "edu.utexas.tacc.testapps.tapis.SleepSeconds", "include": null,
↪"description": ""}], "archiveFilter": {"excludes": [], "includes": ["Sleep*"],
↪"includeLaunchFiles": true}, "containerArgs": [], "schedulerOptions": []}
    execSystemConstraints: null
    subscriptions: []
    blockedCount: 0
    remoteJobId: 1e35edb11ee05bacf3da7cf0fedc55cfb6e616e15982df5b6d5ba69a44029351
    remoteJobId2: null
    remoteOutcome: FINISHED
    remoteResultInfo: 0
    remoteQueue: null
    remoteSubmitted: null
```

(continues on next page)

(continued from previous page)

```
    remoteStarted: 2024-02-21T17:18:21.478768Z
    remoteEnded: 2024-02-21T17:18:43.465360Z
    remoteSubmitRetries: 0
    remoteChecksSuccess: 3
    remoteChecksFailed: 0
    remoteLastStatusCheck: 2024-02-21T17:18:43.463098Z
    inputTransactionId: 4e47d395-50c5-4715-808e-3317f555e85f
    inputCorrelationId: c62e5e7f-1142-47d6-bd85-cabfb9c8dc6b
    archiveTransactionId: d20479da-50da-4ebd-be28-2680722a87d1
    archiveCorrelationId: 2a70d8b9-fa5a-4cfd-a425-d095f69d8dfd
    stageAppTransactionId: null
    stageAppCorrelationId: null
    dtnInputTransactionId: null
    dtnInputCorrelationId: null
    dtnOutputTransactionId: null
    dtnOutputCorrelationId: null
    tapisQueue: tapis.jobq.submit.DefaultQueue
    visible: true
    createdby: testuser2
    createdbyTenant: dev
    tags: [sleep, test]
    jobType: FORK
    mpiCmd: null
    cmdPrefix: null
    sharedAppCtx:
    sharedAppCtxAttribs: []
    notes: {"obj": {"f1": "v1", "f2": 44.0}, "array": ["x", "y", "z"], "count": 27.0,
↪"fname": "bud", "happy": true, "lname": "jones"}
    mpi: null
  "status": "success",
  "message": "JOBS_RETRIEVED Job ba34f946-8a18-44c4-9b25-19e21dfadf69-007 retrieved.",
  "version": "1.0.0-rc1",
  "metadata": null
}
```

Understanding Job Timestamps

Timestamps are assigned as a job progresses, so they won't always have a value. All job timestamps are in Coordinated Universal Time (UTC) and have the following meanings:

1. *created*: When the job request has been accepted by the Jobs front-end.
2. *ended*: When the job reached a terminal state (FINISHED, FAILED or CANCELLED).
3. *lastUpdated*: The last time any part of the job record was updated.
4. *remoteSubmitted*: When the job was submitted to a scheduler on the execution system.
5. *remoteStarted*: When Jobs detected that the job began running on the execution system.
6. *remoteEnded*: When Job detected that the job stopped running on the executions system.

The *remoteSubmitted* timestamp indicates when batch jobs are submitted to the batch scheduler. Forked jobs, however, are not submitted to a scheduler, so in these cases *remoteSubmitted* indicates when application invocation began. Job monitoring typically involves polling an execution system to determine a job's state. The precision of the *remoteStarted*

and *remoteEnded* timestamps is a function of the polling frequency, which changes based on Job service configuration and policies.

7.5.3 Get Job Status

With PySDK:

```
$ t.jobs.getJobStatus(jobUuid='ba34f946-8a18-44c4-9b25-19e21dfadf69-007')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-
↪007/status
```

The response will look something like the following:

```
{
  "result": {
    "status": "FINISHED",
    "condition": "NORMAL_COMPLETION"
    },
    "status": "success",
    "message": "JOBS_STATUS_RETRIEVED Status of the Job ba34f946-8a18-44c4-9b25-
↪19e21dfadf69-007 retrieved.",
    "version": "1.0.0-rc1",
    "metadata": null
    }
```

7.5.4 Get Job History

With PySDK:

```
$ t.jobs.getJobHistory(jobUuid='ba34f946-8a18-44c4-9b25-19e21dfadf69-007')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-
↪007/history
```

The response will look something like the following:

```
{
"result": [
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:02.365996Z",
        "jobStatus": "PENDING",
        "description": "The job has transitioned to a new status: PENDING.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:02.799166Z",
```

(continues on next page)

(continued from previous page)


```
        "jobStatus": "PROCESSING_INPUTS",
        "description": "The job has transitioned to a new status: PROCESSING_INPUTS. The␣
↪previous job status was PENDING.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:10.203007Z",
        "jobStatus": "STAGING_INPUTS",
        "description": "The job has transitioned to a new status: STAGING_INPUTS. The␣
↪previous job status was PROCESSING_INPUTS.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:10.226013Z",
        "jobStatus": "STAGING_JOB",
        "description": "The job has transitioned to a new status: STAGING_JOB. The␣
↪previous job status was STAGING_INPUTS.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:20.720637Z",
        "jobStatus": "SUBMITTING_JOB",
        "description": "The job has transitioned to a new status: SUBMITTING_JOB. The␣
↪previous job status was STAGING_JOB.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:20.888569Z",
        "jobStatus": "QUEUED",
        "description": "The job has transitioned to a new status: QUEUED. The previous␣
↪job status was SUBMITTING_JOB.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:20.902511Z",
        "jobStatus": "RUNNING",
        "description": "The job has transitioned to a new status: RUNNING. The previous␣
↪job status was QUEUED.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
```


(continues on next page)

(continued from previous page)


```
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:42.427492Z",
        "jobStatus": "ARCHIVING",
        "description": "The job has transitioned to a new status: ARCHIVING. The␣
↪previous job status was RUNNING.",
        "transferTaskUuid": null,
        "transferSummary": {}
    },
    {
        "event": "JOB_NEW_STATUS",
        "created": "2021-07-12T23:56:55.966883Z",
        "jobStatus": "FINISHED",
        "description": "The job has transitioned to a new status: FINISHED. The previous␣
↪job status was ARCHIVING.",
        "transferTaskUuid": null,
        "transferSummary": {}
    }
],
"status": "success",
"message": "JOBS_HISTORY_RETRIEVED Job ba34f946-8a18-44c4-9b25-19e21dfadf69-007 history␣
↪retrieved for user testuser2 tenant dev",
"version": "1.0.0-rc1",
"metadata": {
    "recordCount": 9,
    "recordLimit": 100,
    "recordsSkipped": 0,
    "orderBy": null,
    "startAfter": null,
    "totalCount": -1
}
}
```


7.5.5 Get Job Output Listing

With PySDK:

```
$ t.jobs.getJobOutputList(jobUuid='ba34f946-8a18-44c4-9b25-19e21dfadf69-007', outputPath=
↪'/')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-
↪007/output/list/
```

The response will look something like the following:


```
{
"result": [
    {
        "mimeType": null,
        "type": "file",
        "owner": "1003",
```


(continues on next page)

(continued from previous page)


```
        "group": "1003",
        "nativePermissions": "rw-rw-r--",
        "uri": "tapis://dev/tapisv3-exec/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007/
↪archive/SleepSeconds.out",
        "lastModified": "2021-07-12T23:56:54Z",
        "name": "SleepSeconds.out",
        "path": "/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007/archive/SleepSeconds.out
↪",
        "size": 3538
    },
    {
        "mimeType": null,
        "type": "file",
        "owner": "1003",
        "group": "1003",
        "nativePermissions": "rw-rw-r--",
        "uri": "tapis://dev/tapisv3-exec/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007/
↪archive/tapisjob.env",
        "lastModified": "2021-07-12T23:56:53Z",
        "name": "tapisjob.env",
        "path": "/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007/archive/tapisjob.env",
        "size": 1051
    },
    {
        "mimeType": null,
        "type": "file",
        "owner": "1003",
        "group": "1003",
        "nativePermissions": "rw-rw-r--",
        "uri": "tapis://dev/tapisv3-exec/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007/
↪archive/tapisjob.exitcode",
        "lastModified": "2021-07-12T23:56:54Z",
        "name": "tapisjob.exitcode",
        "path": "/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007/archive/tapisjob.exitcode
↪",
        "size": 1
    },
    {
        "mimeType": null,
        "type": "file",
        "owner": "1003",
        "group": "1003",
        "nativePermissions": "rw-rw-r--",
        "uri": "tapis://dev/tapisv3-exec/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007/
↪archive/tapisjob.out",
        "lastModified": "2021-07-12T23:56:54Z",
        "name": "tapisjob.out",
        "path": "/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007/archive/tapisjob.out",
        "size": 3566
    },
    {
        "mimeType": "application/x-shar",
```


(continues on next page)

(continued from previous page)


```
        "type": "file",
        "owner": "1003",
        "group": "1003",
        "nativePermissions": "rw-rw-r--",
        "uri": "tapis://dev/tapisv3-exec/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007/
↪archive/tapisjob.sh",
        "lastModified": "2021-07-12T23:56:54Z",
        "name": "tapisjob.sh",
        "path": "/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-007/archive/tapisjob.sh",
        "size": 979
    }
],
"status": "success",
"message": "JOBS_OUTPUT_FILES_LIST_RETRIEVED Job ba34f946-8a18-44c4-9b25-19e21dfadf69-
↪007 output files list retrieved for the user testuser2 in the tenant dev.",
"version": "1.0.0-rc1",
"metadata": {
    "recordCount": 5,
    "recordLimit": 100,
    "recordsSkipped": 0,
    "orderBy": null,
    "startAfter": null,
    "totalCount": 0
}
}
```


The Job output list API retrieves job's output files list for a previously submitted job by its UUID. By default, the job must be in a terminal state (FINISHED or FAILED or CANCELLED) for the API to list the job's output files . There is a query parameter allowIfRunning set to false by default. If allowIfRunning=true, the API returns the job output files list even if the job is not in the terminal state. Note that if a file is being written, still the file is listed.

7.5.6 Get Job Output Download

With PySDK:

```
$ t.jobs.getJobOutputDownload(jobUuid='ba34f946-8a18-44c4-9b25-19e21dfadf69-007',␣
↪outputPath='/')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/ba34f946-8a18-44c4-9b25-19e21dfadf69-
↪007/output/download/ --output joboutput.zip
```

All the files in the the requested outputPath get downloaded in a zip file.

The Jobs output download API retrieves the job's output files for a previously submitted job by its UUID. By default, the job must be in a terminal state (FINISHED or FAILED or CANCELLED) for the API to download the job's output files. There is a query parameter allowIfRunning set to false by default. If allowIfRunning=true, the API allows downloading the job output files even if the job is not in the terminal state. Note that if a file is being written at the time of the request, the file is still downloaded with the current content.

7.6 Dedicated Search Endpoint

The jobs service provides dedicated search end-points to query jobs based on different conditions. The GET end-point allows to specify the query in the query parameters while the POST end-point allows complex queries in the request body using SQL-like syntax.

7.6.1 Search using GET on Dedicated Endpoint

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/search?limit=2&status.eq=FINISHED&
↪created.between=2021-07-01,2021-07-21&orderBy=lastUpdated(desc),name(asc)&
↪computeTotal=True
```

The response will look something like the following:


```
{
 "result": [
      {
          "uuid": "79234b2a-0995-4632-956e-b940d10607ba-007",
          "name": "SyRunSleepSecondsNoIPFiles-2",
          "owner": "testuser2",
          "appId": "SyRunSleepSecondsNoIPFiles-2",
          "created": "2021-07-20T23:56:02.616Z",
          "status": "FINISHED",
          "remoteStarted": "2021-07-20T23:56:20.368Z",
          "ended": "2021-07-20T23:56:54.409Z",
          "tenant": "dev",
          "execSystemId": "tapisv3-exec2",
          "archiveSystemId": "tapisv3-exec",
          "appVersion": "0.0.1",
          "lastUpdated": "2021-07-20T23:56:54.409Z"
      },
      {
          "uuid": "432f7018-070d-41c3-ba0e-a685f7f11e5c-007",
          "name": "SlurmSleepSeconds",
          "owner": "testuser2",
          "appId": "SlurmSleepSecondsVM",
          "created": "2021-07-20T23:16:01.629Z",
          "status": "FINISHED",
          "remoteStarted": "2021-07-20T23:16:24.781Z",
          "ended": "2021-07-20T23:16:58.745Z",
          "tenant": "dev",
          "execSystemId": "tapisv3-exec2-slurm",
          "archiveSystemId": "tapisv3-exec",
          "appVersion": "0.0.1",
          "lastUpdated": "2021-07-20T23:16:58.745Z"
      }
  ],
  "status": "success",
  "message": "JOBS_SEARCH_RESULT_LIST_RETRIEVED Jobs search list for the user testuser2␣
↪in the tenant dev retrieved.",
  "version": "1.0.0-rc1",
  "metadata": {
```


(continues on next page)

(continued from previous page)

```
      "recordCount": 2,
      "recordLimit": 2,
      "recordsSkipped": 0,
      "orderBy": "lastUpdated(desc),name(asc)",
      "startAfter": null,
      "totalCount": 246
  }
  }
```

7.6.2 Search using POST on Dedicated Endpoint

A user can make complex queries to Jobs service by specifying SQL-like syntax in the request body to the end-point /v3/jobs/search. An example request body in json format is shown below:

```
{
  "search":
    [
      "(status = 'FINISHED' AND name = 'SleepSeconds') ",
      " OR (tags IN ('test'))"
    ]
}
```

With cURL:

```
curl --location '$BASE_URL/v3/jobs/search?listType=ALL_JOBS&limit=2&computeTotal=True&
↪select=name%2Ctags%2Cstatus%2CappId' \
--header 'X-Tapis-Token: $jwt' \
--header 'Content-Type: application/json' \
--data '{
  "search":
  [
     "(status = '\''FINISHED'\'' AND name = '\''SleepSeconds'\'') ",
     " OR (tags IN ('\''test'\''))"
     ]
   }'
```

The response looks like this:

```
{
"result": [
    {
        "name": "SleepSecondsLoadTest",
        "status": "FINISHED",
        "appId": "SleepSeconds-Load",
        "tags": [
            "sleep",
            "test"
        ],
        "uuid": "e17edea6-33f8-441c-867c-d9d23509dd55-007"
    },
    {
        "name": "SleepSecondsLoadTest",
```

(continues on next page)

(continued from previous page)


```
        "status": "FINISHED",
        "appId": "SleepSeconds-Load",
        "tags": [
            "sleep",
            "test"
        ],
        "uuid": "a3a539a9-c0e5-4a02-82c3-0dfbcada47f9-007"
    }
],
"status": "success",
"message": "JOBS_SEARCH_RESULT_LIST_RETRIEVED Jobs search list for the user testuser2 in␣
↪the tenant dev retrieved. ",
"version": "1.3.0",
"commit": "ee1b3342",
"build": "2023-03-01T15:42:55Z",
"metadata": {
    "recordCount": 2,
    "recordLimit": 2,
    "recordsSkipped": 0,
    "orderBy": null,
    "startAfter": null,
    "totalCount": 27345
}}
```


7.7 Job Actions

7.7.1 Job Cancel

A previously submitted job not in terminal state can be cancelled by its UUID.

With PySDK:

```
$ t.jobs.cancelJob(jobUuid='19b06299-4e7c-4b27-ae77-2258e9dc4734-007')
```

With CURL:

```
$ curl -X POST -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/19b06299-4e7c-4b27-ae77-
↪2258e9dc4734-007/cancel
```

The response will look something like the following:


```
 {
 "result": {
     "message": "JOBS_JOB_CANCEL_ACCEPTED Request to cancel job 19b06299-4e7c-4b27-ae77-
↪2258e9dc4734-007 has been accepted. "
 },
 "status": "success",
 "message": "JOBS_JOB_CANCEL_ACCEPTED_DETAILS Request to cancel job 19b06299-4e7c-4b27-
↪ae77-2258e9dc4734-007 has been accepted. If the job is in a terminal state, the␣
↪request will have no effect. If the job is transitioning between active and blocked␣
↪states, another cancel request may need to be sent.",
```


(continues on next page)

(continued from previous page)

```
 "version": "1.2.1",
 "metadata": null
}
```

7.7.2 Hide Job

With PySDK:

```
$ t.jobs.hideJob(jobUuid='19b06299-4e7c-4b27-ae77-2258e9dc4734-007')
```

With CURL:

```
$ curl -X POST -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/19b06299-4e7c-4b27-ae77-
↪2258e9dc4734-007/hide
```

The response will look something like the following:

```
 {
 "result": {
     "message": "JOBS_JOB_CHANGED_VISIBILITY Job 19b06299-4e7c-4b27-ae77-2258e9dc4734-
↪007 has been changed to hidden."
 },
 "status": "success",
 "message": "JOBS_JOB_CHANGED_VISIBILITY Job 19b06299-4e7c-4b27-ae77-2258e9dc4734-007␣
↪has been changed to hidden.",
 "version": "1.2.1",
 "metadata": null
}
```

7.7.3 Unhide Job

With PySDK:

```
$ t.jobs.unhideJob(jobUuid='19b06299-4e7c-4b27-ae77-2258e9dc4734-007')
```

With CURL:

```
$ curl -X POST -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/19b06299-4e7c-4b27-ae77-
↪2258e9dc4734-007/unhide
```

The response will look something like the following:

```
{
"result": {
    "message": "JOBS_JOB_CHANGED_VISIBILITY Job 19b06299-4e7c-4b27-ae77-2258e9dc4734-007␣
↪has been changed to unhidden."
},
"status": "success",
"message": "JOBS_JOB_CHANGED_VISIBILITY Job 19b06299-4e7c-4b27-ae77-2258e9dc4734-007 has␣
↪been changed to unhidden.",
"version": "1.2.1",
"metadata": null
}
```

7.8 Job Sharing

7.8.1 Share a Job

A previously submitted job can be shared with a user in the same tenant. Job resources that can shared are: JOB_HISTORY, JOB_RESUBMIT_REQUEST, JOB_OUTPUT. Currently only READ permission on the resources are allowed.

With PySDK:

```
$ t.jobs.shareJob(jobUuid='ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007',
     jobResource=["JOB_HISTORY","JOB_RESUBMIT_REQUEST","JOB_OUTPUT"],
     jobPermission='READ',
     grantee='testuser6')
```

With CURL:

```
$ curl -X POST -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/ccec730b-22ad-4088-a87e-
↪bb8cfb2ab2e6-007/share -d '{
 "jobResource": ["JOB_HISTORY", "JOB_RESUBMIT_REQUEST", "JOB_OUTPUT"], "jobPermission":
↪"READ", "grantee": "testuser6"}'
```

The response will look something like the following:

```
 {
 "result": {
     "message": "JOBS_JOB_SHARED The job ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007␣
↪resource is shared by testuser2 to testuser6 in tenant dev"
 },
 "status": "success",
 "message": "JOBS_JOB_SHARED The job ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007 resource␣
↪is shared by testuser2 to testuser6 in tenant dev",
 "version": "1.2.1",
 "metadata": null
}
```

7.8.2 Get Job Share Information

With PySDK:

```
$ t.jobs.getJobShare(jobUuid='ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-
↪007/share
```

The response will look something like the following:

```
{
"result": [
    {
```

(continues on next page)

(continued from previous page)


```
        "tenant": "dev",
        "createdby": "testuser2",
        "jobUuid": "ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007",
        "grantee": "testuser5",
        "jobResource": "JOB_HISTORY",
        "jobPermission": "READ",
        "created": "2022-06-16T14:53:31.899199Z"
    },
    {
        "tenant": "dev",
        "createdby": "testuser2",
        "jobUuid": "ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007",
        "grantee": "testuser5",
        "jobResource": "JOB_OUTPUT",
        "jobPermission": "READ",
        "created": "2022-06-16T14:53:32.004831Z"
    },
    {
        "tenant": "dev",
        "createdby": "testuser2",
        "jobUuid": "ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007",
        "grantee": "testuser6",
        "jobResource": "JOB_HISTORY",
        "jobPermission": "READ",
        "created": "2022-06-16T17:17:50.981844Z"
    },
    {
        "tenant": "dev",
        "createdby": "testuser2",
        "jobUuid": "ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007",
        "grantee": "testuser6",
        "jobResource": "JOB_RESUBMIT_REQUEST",
        "jobPermission": "READ",
        "created": "2022-06-16T17:17:51.059726Z"
    },
    {
        "tenant": "dev",
        "createdby": "testuser2",
        "jobUuid": "ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007",
        "grantee": "testuser6",
        "jobResource": "JOB_OUTPUT",
        "jobPermission": "READ",
        "created": "2022-07-14T19:57:15.838019Z"
    }
],
"status": "success",
"message": "JOBS_JOB_SHARE_INFO_RETRIEVED Share information retrieved for the job␣
↪ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007 for user testuser2 in tenant dev.",
"version": "1.2.1",
"metadata": {
    "recordCount": 5,
    "recordLimit": 100,
```


(continues on next page)

(continued from previous page)

```
    "recordsSkipped": 0,
    "orderBy": null,
    "startAfter": null,
    "totalCount": 5
}
}
```

7.8.3 Unshare Job

With PySDK:

```
$ t.jobs.deleteJobShare(jobUuid='ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007', user=
↪'testuser6')
```

With CURL:

```
$ curl -X DELETE -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/ccec730b-22ad-4088-a87e-
↪bb8cfb2ab2e6-007/share/testuser6
```

The response will look something like this:

```
{
"result": {
    "message": "JOBS_JOB_UNSHARED The job ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007␣
↪resource is unshared by testuser2 to testuser6 in tenant dev"
},
"status": "success",
"message": "JOBS_JOB_UNSHARED The job ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007 resource␣
↪is unshared by testuser2 to testuser6 in tenant dev",
"version": "1.2.1",
"metadata": null
}
```

7.8.4 List Shared Jobs

The query parameter listType=SHARED_JOBS in the jobs list end-point allows to list all shared jobs for a user, say testuser6, using testuser6's JWT. Note testuser6 is not the owner of the jobs listed. Default value for listType is MY_JOBS and to list all jobs including both shared and testuser6 owned job, listType=ALL_JOBS.

With PySDK:

```
$ t.jobs.getJobList(listType='SHARED_JOBS')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/jobs/list?listType=SHARED_JOBS
```

The response will look something like this:

```
{
 "result": [
  {
      "uuid": "ccec730b-22ad-4088-a87e-bb8cfb2ab2e6-007",
```

(continues on next page)

(continued from previous page)

```
      "name": "SleepSeconds",
      "owner": "testuser2",
      "appId": "SleepSeconds",
      "created": "2021-04-09T02:47:57.760Z",
      "status": "FINISHED",
      "remoteStarted": "2021-04-09T02:48:08.946Z",
      "ended": "2021-04-09T02:48:16.669Z",
      "tenant": "dev",
      "execSystemId": "tapisv3-exec2",
      "archiveSystemId": "tapisv3-exec",
      "appVersion": "0.0.1",
      "lastUpdated": "2021-04-09T02:48:16.669Z"
  }
  ],
  "status": "success",
  "message": "JOBS_LIST_RETRIVED Jobs list for the user testuser6 in the tenant dev␣
↪retrived.",
  "version": "1.2.1",
  "metadata": {
  "recordCount": 1,
  "recordLimit": 100,
  "recordsSkipped": 0,
  "orderBy": null,
  "startAfter": null,
  "totalCount": -1
  }
  }
```

7.8.5 Job Search on Shared Job

Job search on a list of jobs can be performed using the query parameter listType=SHARED_JOBS in the search endpoint.

In the following example, we use testuser5 JWT to do job search on list of jobs shared with testuser5.

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" '$BASE_URL/v3/jobs/search?listType=SHARED_JOBS&name.
↪eq=SleepSeconds&created.between=2022-07-05,2022-07-06'
```

The response will look something like this:

```
{
"result": [
    {
        "uuid": "3b9cb514-4962-44e8-a851-55e933e558c0-007",
        "name": "SleepSeconds",
        "owner": "testuser2",
        "appId": "SleepSeconds",
        "created": "2022-07-05T21:34:11.355Z",
        "status": "FINISHED",
        "remoteStarted": "2022-07-05T21:34:48.776Z",
        "ended": "2022-07-05T21:35:31.808Z",
```

(continues on next page)

(continued from previous page)


```
        "tenant": "dev",
        "execSystemId": "tapisv3-exec2",
        "archiveSystemId": "tapisv3-exec",
        "appVersion": "0.0.1",
        "lastUpdated": "2022-07-05T21:35:31.808Z"
    },
    {
        "uuid": "eacde4c4-1d93-4393-b220-63aea509b32c-007",
        "name": "SleepSeconds",
        "owner": "testuser2",
        "appId": "SleepSeconds",
        "created": "2022-07-05T21:43:05.371Z",
        "status": "FINISHED",
        "remoteStarted": "2022-07-05T21:43:44.899Z",
        "ended": "2022-07-05T21:44:27.509Z",
        "tenant": "dev",
        "execSystemId": "tapisv3-exec2",
        "archiveSystemId": "tapisv3-exec",
        "appVersion": "0.0.1",
        "lastUpdated": "2022-07-05T21:44:27.509Z"
    }
],
"status": "success",
"message": "JOBS_SEARCH_RESULT_LIST_RETRIEVED Jobs search list for the user testuser5 in␣
↪the tenant dev retrieved.",
"version": "1.2.1",
"metadata": {
    "recordCount": 2,
    "recordLimit": 100,
    "recordsSkipped": 0,
    "orderBy": null,
    "startAfter": null,
    "totalCount": -1
}
}
```


7.8.6 Share Job Output

As shown in the previous example of share a job, job output resource can be shared with a user, say testuser5, in the same tenant. This includes testuser5 can do job output listing and job output download with its own JWT even though its not the owner of the job.

7.8.7 Share Job History

As shown in the previous example of share a job, job history can be shared with a user, say testuser5, in the same tenant. This includes testuser5 can get job's history, status and job's detail information for jobs that are shared with it using its own JWT.

CHAPTER

EIGHT

META

Meta V3 is a REST API Microservice for MongoDB which provides server-side Data, Identity and Access Management for Web and Mobile applications.

8.1 Why Meta V3

Meta V2 functionality was built on MongoDB version 1.0 technology. Although limited, it brought the basic functions of a document store to Agave/Tapis platform. Some projects decided to use basic key/value storage to add metadata to Jobs, Apps, Systems and Files. Other projects stored complex document data to associate Jobs and other entities to add a richer information layer to their portals. The limited search functionality and imposed document structure created impediments to using MongoDB as projects had envisioned. Meta V3 removes these impediments.

Meta V3 is built on top of MongoDB version 4.2 technology. The REST API opens up the full functionality of MongoDB as a document store. and delivers MongoDB as a service so that projects are free to create metadata and documents in a fashion that fits their needs. Some of the Meta V3 advantages over Meta V2 include:

- Any valid MongoDB document structure can be used.
- If a search runs in MongoDB CLI, it should run from the API.
- Aggregations are available.
- Database, collection and document creation can be managed by tenant administrator.
- Performance is many times faster.

8.2 Migration from Meta V2 to V3

Migration can simply be accomplished by creating a new database with one or more collections for your project. The tenant administrator can request the initial permissions setup. Once your collection(s) are in place move the result and associatedIds into your new document model.

8.3 Overview

Meta V3 is:

A Stateless Microservice. With Meta V3 projects can focus on building Angular or other frontend applications, because most of the server-side logic necessary to manage database operations, authentication / authorization and related APIs is automatically handled, without the need to write any server-side code except for the UX/UI.

For example, to insert data into MongoDB a developer has to just create client-side JSON documents and then execute POST operations via HTTP to Meta V3. Other functions of a modern MongoDB installation like flexible schema, geoJson and aggregation pipelines ease the development process.

Every tenant will have access to at least one database where they can store and manage json documents. Documents are the trailing end of a nested hierarchy of data that begins with a database that houses one or more collections. The collections house json documents the structure of which is left up to the administrators of the tenant database.

Permissions for access to databases, collections and documents must be predefined before accessing those resources. The definitions for access are defined within the Security Kernel API of Tapis V3 and must be added by a tenant or service administrator. See the Permissions section below for some examples of permissions definitions and access to resources in the Meta V3 API.

8.4 Getting Started

8.4.1 Create a document

We have a database named MyTstDB and a collection name MyCollection. To add a json document to MyCollection, we can do the following:

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" --data '{
↪"name": "test document slt 7.21.2020-14:27","jimmyList": ["1","3"],"description": "new␣
↪whatever",}'  $BASE_URL/v3/meta/MyTstDB/MyCollection?basic=true
```

The response will have an empty response body with a status code of 201 “Created” unless the “basic” url query parameter is set to true. Setting the “basic” parameter to true will give a Tapis Basic response along with the “_id” of the newly created document. A more detailed discussion of autogenerated ids and specified ids can be found in the “Create Document” section of “Document Resources”.


```
{
  "result": {
    "_id": "5f189316e37f7b5a692285f3"
  },
  "status": "201",
  "message": "Created",
  "version": "0.0.1"
}
```


8.4.2 List documents

Using our MyTstDb/MyCollection resources we can ask for a default list of documents in MongoDB default sorted order. The document we created earlier should be listed with a new “_id” field that was autogenerated by MongoDB.

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" $BASE_URL/
↪v3/meta/MyTstDB/MyCollection
```

The response will be an array of json documents from MyCollection :


```
[
  {
    "_id": {
      "$oid": "5f189316e37f7b5a692285f3"
    },
    "name": "test document slt 7.21.2020-14:27",
    "jimmyList": [
```


(continues on next page)

(continued from previous page)


```
      "1",
      "3"
    ],
    "description": "new whatever",
    "_etag": {
      "$oid": "5f189316296c81742a6a3e4c"
    }
  },
  {
    "_id": {
      "$oid": "5f1892ece37f7b5a692285e9"
    },
    "name": "test document slt 7.21.2020-14:25",
    "jimmyList": [
      "1",
      "3"
    ],
    "description": "new whatever",
    "_etag": {
      "$oid": "5f1892ec296c81742a6a3e4b"
    }
  }
]
```


8.4.3 Get a document

If we know the "_id" of a created document, we can ask for it directly.

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" $BASE_URL/
↪v3/meta/MyTstDB/MyCollection/5f1892ece37f7b5a692285e9
```

The response will be a json document from MyCollection with the "_id" of 5f1892ece37f7b5a692285e9 :


```
{
  "_id": {
    "$oid": "5f1892ece37f7b5a692285e9"
  },
  "name": "test document slt 7.21.2020-14:25",
  "jimmyList": [
    "1",
    "3"
  ],
  "description": "new whatever",
  "_etag": {
    "$oid": "5f1892ec296c81742a6a3e4b"
  }
}
```

8.4.4 Find a document

We can pass a query parameter named "filter" and set the value to a json MongoDB query document. Let's find a document by a specific "name".

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" --data-
↪urlencode filter='{"name": "test document slt 7.21.2020-14:25"}' $BASE_URL/v3/meta/
↪MyTstDB/MyCollection
```

The response will be an array of json documents from MyCollection :


```
[
  {
    "_id": {
      "$oid": "5f1892ece37f7b5a692285e9"
    },
    "name": "test document slt 7.21.2020-14:25",
    "jimmyList": [
      "1",
      "3"
    ],
    "description": "new whatever",
    "_etag": {
      "$oid": "5f1892ec296c81742a6a3e4b"
    }
  }
]
```


8.5 Resources

8.5.1 General resources

An unauthenticated Health check is in included in the Meta V3 API to let any user know the current condition of the service.

Health Check

An unauthenticated request for the health status of Meta V3 API.

With pySDK operation:

```
$ t.meta.healthCheck()
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json" $BASE_URL/v3/meta/
```

The response will be a Basic Tapis response on health:


```
{
  "result": "",
  "status": "200",
  "message": "OK",
```


(continues on next page)

(continued from previous page)

```
  "version": "0.0.1"
}
```

8.5.2 Root resources

The Root resource space represents the root namespace for databases on the MongoDB host. All databases are located here. Requests to this space are limited to READ only for tenant administrators.

List DB Names

A request to the Root resource will list Database names found on the server. This request has been limited to those users with tenant administrative roles.

With pySDK operation:

```
$ t.meta.listDBNames()
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" $BASE_URL/
↪v3/meta/
```

The response will a json list of database names:

```
[
    "StreamsDevDB",
    "v1airr"
]
```

8.5.3 Database resources

The Database resource is the top level for many tenant projects. The resource maps directly to a MongoDB named database in the database server. Case matters for matching the name of the database and must be specified when making requests for collections or documents. Currently

List Collection Names

This request will return a list of collection names from the specified database {db}. The permissions for access to the database are set prior to access.

With pySDK operation:

```
$ t.meta.listCollectionNames(db='')
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d '' $BASE_
↪URL/v3/meta/{db}
```

Here is an example response:

```
[
  "streams_alerts_metadata",
  "streams_channel_metadata",
  "streams_instrument_index",
```

(continues on next page)

(continued from previous page)

```
  "streams_project_metadata",
  "streams_templates_metadata",
  "tapisKapa-local"
]
```

Get DB Metadata

This request will return the metadata properties associated with the database. The core server generates an etag in the _properties collection for a database that is necessary for future deletion.

With pySDK operation:

```
$ t.meta.getDBMetadata(db='')
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d '' $BASE_
↪URL/v3/meta/{db}/_meta
```

Here is an example response:

```
{
   "_id": "_meta",
   "_etag": { "$oid": "5ef6232b296c81742a6a3e02" }
}
```

Create DB

TODO: this implementation is not exposed. Creation of a database by tenant administrators is scheduled for inclusion in an administrative interface API in a future release.

This request will create a new named database in the MongoDB root space by a tenant or service administrator.

With pySDK operation:

```
$ t.meta.createDB(db='')
```

With CURL:

```
$ curl -v -X PUT -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d '' $BASE_
↪URL/v3/meta/{db}
```

Here is an example response:

```
Status: 200 ok
```

Delete a DB TODO: this implementation is not exposed. Deletion of a database by tenant administrators is scheduled for inclusion in an administrative interface API in a future release.

This request will delete a named database in the MongoDB root space by a tenant or service administrator.

With pySDK operation:

```
$ t.meta.deleteDB(db='')
```

With CURL:

```
$ curl -v -X DELETE -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d ''
↪$BASE_URL/v3/meta/{db}
```

Here is an example response:

```
Status: 204 Deleted DB
```

8.5.4 Collection Resources

The Collection resource allows requests for managing and querying json documents within a MongoDB collection.

Create a Collection

You can create a new collection of documents by specifying a collection name under a specific database. /v3/meta/{db}/{collection}

With pySDK operation:

```
$ t.meta.createCollection(db='',collection='')
```

With CURL:

```
$ curl -v -X PUT -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" $BASE_URL/
↪v3/meta/{db}/{collection}
```

Here is an example response:

```
Status: 201 Collection created. An Etag header value is returned for collection␣
↪identification
```

List Documents

A default number of documents found in the collection are returned in an array of documents.

With pySDK operation:

```
$ t.meta.listDocuments(db='',collection='',filter='')
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d '' $BASE_
↪URL/v3/meta/{db}/{collection}
```

The response will look like the following:

```
[
  {
    "_id": {
      "$oid": "5f1892ece37f7b5a692285e9"
    },
    "name": "test document slt 7.21.2020-14:25",
    "description": "new whatever",
    "_etag": {
      "$oid": "5f1892ec296c81742a6a3e4b"
    }
  },
```

(continues on next page)

(continued from previous page)


```
  {
    "_id": {
      "$oid": "5f1892ece37f7b5a69228533"
    },
    "name": "test document slt 7.21.2020-14:25",
    "description": "new whatever",
    "_etag": {
      "$oid": "5f1892ec296c81742a6a3e444"
    }
  }
]
```


List Documents Large Query

A default number of documents found in the collection are returned in an array of documents.

With pySDK operation:

```
$ t.meta.submitLargeQuery(db='',collection='',page='',pagesize='',sort='',keys='',
↪fileinput)
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d␣
↪@FILENAME '' $BASE_URL/v3/meta/{db}/{collection}/_filter
```

The response will look like the following:


```
[
  {
    "_id": {
      "$oid": "5f1892ece37f7b5a692285e9"
    },
    "name": "test document slt 7.21.2020-14:25",
    "description": "new whatever",
    "_etag": {
      "$oid": "5f1892ec296c81742a6a3e4b"
    }
  },
  {
    "_id": {
      "$oid": "5f1892ece37f7b5a69228533"
    },
    "name": "test document slt 7.21.2020-14:25",
    "description": "new whatever",
    "_etag": {
      "$oid": "5f1892ec296c81742a6a3e444"
    }
  }
]
```


Delete a Collection

This administrative method is only available to tenant or meta administrators and requires an If-Match header parameter of the Etag for the collection. The Etag value, if not already known, can be retrieved from the "_meta" call for a

collection.

With pySDK operation:

```
$ t.meta.deleteCollection(db='',collection='')
```

With CURL:

```
$ curl -v -X DELETE -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" $BASE_
↪URL/v3/meta/{db}/{collection}
```

Here is an example response:

```
Status: 204 Deleted named collection from the database
```

Get Collection Size

You can find the given size or number of documents in a given collection by calling "_size" on a collection.

With pySDK operation:

```
$ t.meta.getCollectionSize(db='',collection=' ')
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" $BASE_URL/
↪v3/meta/{db}/{collection}/_size
```

Here is an example response:

```
Status: 200 ok

{
  "_size": 1
}
```

Get Collection Metadata

You can find the metadata properties of a given collection by calling "_meta" on a collection. This would include the Etag value for a collection that is needed for deletion.

With pySDK operation:

```
$ t.meta.getCollectionMetadata(db='',collection=' ')
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" $BASE_URL/
↪v3/meta/{db}/{collection}/_meta
```

Here is an example response:

```
{
  "_id": "_meta",
  "_etag": {
    "$oid": "5f2b2b7a204ce7637579c85f"
  }
}
```

8.5.5 Document Resources

Document resources are json documents found in a collection. Reading, creating, deleting and updating documents along with batch processing make up the operations that can be applied to documents in a collection. There various ways to retrieve one or more documents from a collection, including using a filter query parameter and value in the form of a MongoDB query document. Batch addition of documents, as well as, batch updates based on queries is also allowed.

Create a Document

Creating a new document within a collection. Submitting a json document within the request body of a POST request will create a new document within the specified collection with a MongoDB autogenerated "_id". Batch document addition is possible by POSTing an array of new documents with a request body for the specified collection. The rules for "_id" creation operates the same way on multiple documents as they do with a single document.

The default representation returned is an empty response body along with a 201 Http status code "Created". However if an additional query parameter named "basic" is added with the value of "true" a basic Tapis response is returned along with the newly created "_id" of the document.

With pySDK operation:

```
$ t.meta.createDocument(db='',collection=' ',basic=' ',request_body=' ')
```

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d '{
↪"docName":"test doc"}' $BASE_URL/v3/meta/{db}/{collection}
```

Here is an example response:

```
Status: 201 Document created
```

Multiple documents can be added to a collection by POSTing a json array of documents. The batch addition of documents only supports the default response.

With pySDK operation:

```
$ t.meta.createDocument(db='',collection=' ',basic=' ', request_body=' ')
```

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d '[{
↪"docName":"test doc1"},{"docName":"test doc2"}]' $BASE_URL/v3/meta/{db}/{collection}
```

The response body will be empty:

```
Status: 201 Document created
```

Get a Document

Get a specific document by its "_id".

With pySDK operation:

```
$ t.meta.getDocument(db='',collection=' ',docId=' ')
```

With CURL:

```
$ curl -v -X GET -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt"  $BASE_URL/
↪v3/meta/{db}/{collection}/{document_id}
```

The response will be the standard json response:

```
Status: 200 Document returned
{
    "_id"}
```

Replace a Document

This call replaces an existing document identified by document id ("_id"), with the json supplied in the request body.

With pySDK operation:

```
$ t.meta.replaceDocument(db='',collection=' ',docId=' ')
```

With CURL:

```
$ curl -v -X PUT -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d '{
↪"docName":"test doc another one"}' $BASE_URL/v3/meta/{db}/{collection}/{document_id}
```

Here is an example response:

```
Status: 201 document upserted
```

Modify a Document

This call will replace a portion of a document identified by document id ("_id") with the supplied json.

With pySDK operation:

```
$ t.meta.modifyDocument(db='',collection=' ',docId=' ')
```

With CURL:

```
$ curl -v -X PATCH -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d '{
↪"docName":"test changed"}' $BASE_URL/v3/meta/{db}/{collection}/{document_id}
```

Here is an example response:

```
Status: 201 document modified
```

Delete Document

Deleting a document with a specific document id ("_id"), removes it from the collection.

With pySDK operation:

```
$ t.meta.deleteDocument(db='',collection=' ',docId=' ')
```

With CURL:

```
$ curl -v -X DELETE -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d ''
↪$BASE_URL/v3/meta/{db}/{collection}/{document_id}
```

Here is an example response:

```
Status: 204 Document deleted
```

8.5.6 Index Resources

Indexes can help speed up queries of your collection and the API gives you the ability to define and manage your indexes. You can create an index for a collection, list indexes for a collection and delete an index. Indexes can't be updated they must be deleted and recreated.

List Indexes

List the indexes defined for a collection.

With pySDK operation:

```
$ t.meta.listIndexes(db='database_name',collection='collection_name')
```

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt"  $BASE_URL/
↪v3/meta/{db}/{collection}/_indexes
```

Here is an example response:

```
[
  {
    "v":2,
    "key":{
      "id":1
    }
    "_id":"_id_"
  }
]
```

Create Index

Create a new Index with a new name. To create an index you have to specify the keys and the index options. Let's create an unique, sparse index on property qty and name our index "qtyIndex".

PUT /v3/meta/{db}/{collection}/_indexes/qtyIndex

```
{"keys": {"qty": 1},"ops": {"unique": true, "sparse": true }}
```

With pySDK operation:

```
$ t.meta.createIndex(db='jupyterhub_v3_metadata', collection='jupyterhub_dev', request_
↪body={request_body}, indexName='test'
```

With CURL:

```
$ curl -v -X PUT -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d '{ "keys
↪":  <keys>, "ops": <options> }' $BASE_URL/v3/meta/{db}/{collection}/_indexes/
↪{indexName}
```

Here is an example response:

```
Status: 201 Empty Response
```

Delete Index

Remove a named Index from the index list.

With pySDK operation:

```
$ t.meta.deleteIndex(db='jupyterhub_v3_metadata', collection='jupyterhub_dev', indexName=
↪'test1')
```

With CURL:

```
$ curl -v -X DELETE -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" $BASE_
↪URL/v3/meta/{db}/{collection}/_indexes/{indexName}
```

Here is an example response:

```
Status: 204 Deleted index from collection
```

8.5.7 Aggregation Resources

Aggregations operations process data records and return computed results. Aggregation operations group values from multiple documents together, and can perform a variety of operations on the grouped data to return a single result. Aggregations in the API are predefined and added to a collections properties. They may also be parameterized for use with multiple sets of inputs.

Create an Aggregation

Create an aggregation pipeline by adding the aggregation to the collection for future execution. The aggregation may have variables that are defined so that a future request may pass variable values for aggregation execution. See "Execute an Aggregation".

```
{ "aggrs" : [
      { "stages" : [ { "$match" : { "name" : { "$var" : "n" } } },
            { "$group" : { "_id" : "$name",
                  "avg_age" : { "$avg" : "$age" }
                } }
          ],
        "type" : "pipeline",
        "uri" : "example-pipeline"
      }
    ]
}
```

Property	Mandatory	Description
type	yes	• for aggregation pipeline operations is “pipeline”
uri	yes	• specifies the URI when the operation is bound under the path /<db>/<collection>/_aggrs.
stages	yes	• the MongoDB aggregation pipeline stages.

For more information refer to https://docs.mongodb.org/manual/core/aggregation-pipeline/

With pySDK operation:

```
$ t.meta  To Appear in Future Release
```

With CURL:

```
$ curl -v -X PUT -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt"
     -d '{ "aggrs" : [{ "stages" : [ { "$match" : { "name" : { "$var" : "n" } } },{ "
↪$group" : { "_id" : "$name","avg_age" : { "$avg" : "$age" }} } ],
        "type" : "pipeline","uri" : "example-pipeline"}]}' $BASE_URL/v3/meta/{db}/
↪{collection}
```

Here is an example response:

```
Status: 201 empty response body on created
```

Execute an Aggregation

TODO

With pySDK operation:

```
$ t.meta  To Appear in Future Release
```

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d ''
↪$BASE_URL/v3/meta/
```

Here is an example response:

```
TODO
```

Delete an Aggregation

TODO

With pySDK operation:

```
$ t.meta To Appear in Future Release
```

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d ''
↪$BASE_URL/v3/meta/
```

Here is an example response:

```
Status: 204 Empty response body on delete
```

CHAPTER

NINE

PODS

9.1 Introduction to the Pods Service

The Pods Service is a Tapis (TACC API) service which manages Kubernetes pods for a user. The service implements a message broker and processor model that requests pods, alongside a health module to poll for pod data, including logs, status, and health. The service manages networking, certificates, authentication, and routing. The primary use of this service is to have quick-to-deploy long-lived services based on Docker images that are exposed via HTTP or TCP endpoints listed by the API all running in Kubernetes.

The Pods service provides endpoints for managing pods and their constituant components:

- **Pods** - The base unit of the service, a pod is a Docker container which a user can manage and interact with.
- **Templates** - A pod template is a pre-configured pod definition that can be recursively referenced by the definition of a pod.
- **Images** - Endpoints for managing the allowlist of Docker images that can be used in pods.
- **Volumes** - Volume endpoints are used to manage storage via networked storage solutions.
- **Snapshots** - Snapshots are used to save the state of a volume at a given time.
- **Cluster** - *WIP*

9.1.1 Use Cases

The Pods service is designed to be a flexible and powerful tool for deploying and managing containerized services. Pods are meant to be easy to use with the ability to reference pre-configured templates alongside a UI component in *TapisUI <https://tacc.tapis.io>*. Some common use cases for the Pods service include:

- **Web Services** - Quickly deploy a web service from a Docker image.
- **Databases** - Postgres, Neo4j, MongoDB, MariaDB*.
- **Machine Learning** - Deploy inference interfaces like Ollama and OpenWebUI with GPUs.
- **Development** - Develop and test applications in a containerized environment.
- **Open-Source Tools** - Jupyter, VSCode, Bookstack, Uptime Kuma, and more.
- **Authenticated Apps** - Tapis OAuth2/OIDC auth can be configure to secure your pods.

The flexible nature of pods means that most containerized applications can be deployed and managed using the Pods service. The exception to this are generally either applications with unique networking requirements or truly resource intensive applications (Multi-GPU + High RAM) due to the shared nature of our Kubernetes cluster.

9.1.2 Feedback and Support

Please create issues on our github repo and report problems to the Tapis team. The service is available to researchers and students. To learn more about the the system, including getting access, follow the instructions in *Getting Started*.

9.2 Getting Started

This Getting Started guide will walk you through the initial steps of setting up the necessary accounts and installing the required software before moving to the Pods Quickstart, where the guide will walk through creating a first pod. If you are already using Docker Hub and Tapis, feel free to jump right to the *Pods Quickstart* or check out the Pods Live Docs for an OpenAPI V3 specification of the API.

9.2.1 Account Creation and Software Installation

Create a TACC account

The main instance of the Pods platform is hosted at the Texas Advanced Computing Center (TACC). TACC designs and deploys some of the world's most powerful advanced computing technologies and innovative software solutions to enable researchers to answer complex questions. To use the TACC-hosted Pods service, please create a TACC account.

Create a Docker account

Docker is an open-source container runtime providing operating-system-level virtualization. The Pods service pulls images for its pods from the public Docker Hub. To register pods you will need to publish images on Docker Hub, which requires a Docker account .

Install Tapipy or use cURL

To interact with the TACC-hosted Abaco platform in Python, we will leverage the Tapis Python SDK, tapipy. To install it, simply run:

```
$ pip3 install tapipy
```

Tapipy provides a simple interface to interact with Tapis services. It provides tab-completion and support for all Tapis services. Read more about Tapipy *here* (see *Working with TACC OAuth*).

This documentation will also walk you through using the Pods service with the cURL command line tool. If you will use cURL then make use of this guide *Tapis Quickstart's cURL example* and continue to the *Pods Quickstart*.

9.2.2 Working with TACC OAuth

Authentication and authorization to the Tapis APIs uses OAuth2, a widely-adopted web standard. Our implementation of OAuth2 is designed to give you the flexibility you need to script and automate use of Tapis while keeping your access credentials and digital assets secure. This is covered in great detail in our Tenancy and Authentication section, but some key concepts will be highlighted here, interleaved with Python code.

Create an Tapis Client Object

The first step in using the Tapis Python SDK, tapipy, is to create a Tapis Client object. First, import the `Tapis` class and create python object called `t` that points to the Tapis server using your TACC username and password. Do so by typing the following in a Python shell:

Important

Support for Pods service in Tapipy was added in version 1.2.3.

```
# Import the Tapis object
from tapipy.tapis import Tapis

# Log into you the Tapis service by providing user/pass and url.
t = Tapis(base_url='https://tacc.tapis.io',
          username='your username',
          password='your password')
```

Generate a Token

With the `t` object instantiated, we can exchange our credentials for an access token. In Tapis, you never send your username and password directly to the services; instead, you pass an access token which is cryptographically signed by the OAuth server and includes information about your identity. The Tapis services use this token to determine who you are and what you can do.

```
# Get tokens that will be used for authenticated function calls
t.get_tokens()
print(t.access_token.access_token)

Out[1]: eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9...
```

Note that the tapipy `t` object will store and pass your access token for you, so you don't have to manually provide the token when using the tapipy operations. You are now ready to check your access to the Tapis APIs. It will expire though, after 4 hours, at which time you will need to generate a new token. If you are interested, you can create an OAuth client (a one-time setup step, like creating a TACC account) that can be used to generate access and refresh tokens. For simplicity, we are skipping that but if you are interested, check out the Tenancy and Authentication section.

Check Access to the Tapis APIs

The tapipy `t` object should now be configured to talk to all Tapis APIs on your behalf. We can check that the client is configured properly by making any API call. For example, we can use the authenticator service to retrieve the full TACC profile of our user. To do so, use the `get_profile()` function associated with the `authenticator` object on the `t` object, passing the username of the profile to retrieve, as follows.

```
t.authenticator.get_profile(username='apitest')

Out[1]:
create_time: None
dn: cn=apitest,ou=People,dc=tacc,dc=utexas,dc=edu
email: aci-cic@tacc.utexas.edu
username: apitest
```

9.3 Pods Quickstart

9.3.1 Registering a Pod via Template

To get started we're going to create a Pod via a Template. You can use `tapipy` via Python or `cURL` via bash to make requests to the Pods service. It's good to mention that the service is a RESTful API, so you can use any tool that can make HTTP requests.

Authentication is done via an `X-Tapis-Token` header, which is an OAuth2 access token, shown in the cURL flow and done in the background by Tapipy. Authentication is the first step in using Tapis services and is shown in this step.

Once a user has authenticated, they can create a pod. The Pods service uses a templating system to allow users to create pods from pre-configured, versioned, templates. Users can view templates via TapisUI or the API. The following example shows how to create a pod using the *neo4j* template. Note that fields that a user specifies will override any existing fields in the template.

You should receive a response that looks like this.

```
creation_ts: None
data_attached: []
data_requests: []
description: My example pod!
environment_variables:

pod_id: test
template: template/neo4j
status: REQUESTED
status_container:

status_requested: ON
update_ts: None
url: docpod.pods.tacc.develop.tapis.io
```

Notes:

- The *pod_id* given will be the id used by you to access the pod at all times. It must be lowercase and alphanumeric. It also must be unique within the tenant.
- Pods returned a status of `REQUESTED` for the pod; behind the scenes, the Pods service has sent a message requesting the pod described to our backend *spawner* infrastructure. The pod's image must be pulled, a pod service must be created (for networking), and the networking changes must propagate to the Pod's proxy before the Pod is ready for use.
- When the pod itself has began running, the status will change to `AVAILABLE`. Networking takes time to propagate (expect <1 minute).
- An `AVAILABLE` pod only means the pod itself has started, check pod logs to see what your container is writing to stdout.

At any point we can check the details of our pods, including its status, with the following:

The response format is identical to that returned from the `t.pods.create_pod()` method.

9.3.2 Accessing a Pod

Once your pod is in the `AVAILABLE` state your pod's specified networking ports should be routed to port 443 at specified urls. Read more at **:ref:`Pod Networking`_**.

A pod's access urls specified in the pod's *networking* attribute. A pod can have multiple urls, each with different protocols and ports.

9.3.3 Retrieving the Logs

The Pods service collects the latest 10 MB of logs from the pod when running and makes them available via the `logs` endpoint. Let's retrieve the logs from the pod we just made. We use the `get_pod_logs()` method, passing in `pod_id`:

```
t.pods.get_pod_logs(pod_id='docpod')
```

The response should be similar to the following:

```
logs:
Fetching versions.json for Plugin 'apoc' from https://neo4j-contrib.github.io/neo4j-apoc-
↪procedures/versions.json
Installing Plugin 'apoc' from https://github.com/neo4j-contrib/neo4j-apoc-procedures/
↪releases/download/4.4.0.6/apoc-4.4.0.6-all.jar to /var/lib/neo4j/plugins/apoc.jar
Applying default values for plugin apoc to neo4j.conf
Fetching versions.json for Plugin 'n10s' from https://neo4j-labs.github.io/neosemantics/
↪versions.json
Installing Plugin 'n10s' from https://github.com/neo4j-labs/neosemantics/releases/
↪download/4.4.0.1/neosemantics-4.4.0.1.jar to /var/lib/neo4j/plugins/n10s.jar
Applying default values for plugin n10s to neo4j.conf
2022-06-16 00:36:14.423+0000 INFO  Starting...
2022-06-16 00:36:15.602+0000 INFO  This instance is ServerId{eba2fb15} (eba2fb15-713d-
↪47ba-92a5-0a688696264d)
2022-06-16 00:36:17.468+0000 INFO  ======== Neo4j 4.4.8 ========
2022-06-16 00:36:21.713+0000 INFO  [system/00000000] successfully initialized: CREATE␣
↪USER podsservice SET PLAINTEXT PASSWORD 'servicepass' SET PASSWORD CHANGE NOT REQUIRED
2022-06-16 00:36:21.734+0000 INFO  [system/00000000] successfully initialized: CREATE␣
↪USER test SET PLAINTEXT PASSWORD 'userpass' SET PASSWORD CHANGE NOT REQUIRED
2022-06-16 00:36:30.268+0000 INFO  Upgrading security graph to latest version
2022-06-16 00:36:30.268+0000 INFO  Setting version for 'security-users' to 2
2022-06-16 00:36:30.270+0000 INFO  Upgrading 'security-users' version property from 2 to␣
↪3
2022-06-16 00:36:30.556+0000 INFO  Called db.clearQueryCaches(): Query caches␣
↪successfully cleared of 1 queries.
2022-06-16 00:36:30.667+0000 INFO  Bolt enabled on [0:0:0:0:0:0:0:0%0]:7687.
2022-06-16 00:36:31.745+0000 INFO  Remote interface available at http://pods-tacc-tacc-
↪docpod:7474/
2022-06-16 00:36:31.750+0000 INFO  id:␣
↪B1F0F170083249DAAF9127203310961EF79B262C90EA04D9F08EB7F077DF19E7
2022-06-16 00:36:31.750+0000 INFO  name: system
2022-06-16 00:36:31.751+0000 INFO  creationDate: 2022-06-16T00:36:19.073Z
2022-06-16 00:36:31.751+0000 INFO  Started.
```

We can see logs from our Neo4j image during the initialization process.

9.3.4 Conclusion

Congratulations! You've now created, registered, and accessed your first pod. There is a lot more you can do with the Pods service though. To learn more about the additional capabilities, please continue on to the Technical Guide.

9.4 Templates

Tapis Pods supports templating, which allows users to create reusable pod definitions. Users make use of templates and their underlying template tags to define a pod.

9.4.1 Templates

Templates are essentially groups of template tags that are shareable, providing users with access to any associated tag within the template. Templates facilitate the organization and sharing of pod configurations, allowing users to leverage predefined setups. The only way to currently share pod configurations is through templates, making them a crucial component for collaborative and consistent deployment practices.

Field	Description
template_id	**Required**. Name of the template. Type: string.
description	Description of the template. Type: string. Default: “”.
metatags	Metadata tags for additional search/listing functionality for the template. Type: array of strings. Default: [].
archive_message	If set, metadata message to give users of this template. Type: string. Default: “”.
creation_ts	Time (UTC) that this template was created. Type: string <date-time>.
update_ts	Time (UTC) that this template was updated. Type: string <date-time>.
tenant_id	Tapis tenant used during creation of this template. Type: string. Default: “”.
site_id	Tapis site used during creation of this template. Type: string. Default: “”.
permissions	Template permissions in user:level format. Type: array of strings. Default: [].

9.4.2 Template Tags

Template tags are the underlying definitions within a template, each marked with a `tag_timestamp` and immutable once created. These tags specify the pod configurations and can be overridden with user-defined values at runtime. Template tags can also reference other template tags, enabling a recursive definition of pod configurations. This flexibility allows users to save and share complex pod setups efficiently.

Each template must have at least one tag, which specifies the pod configuration. If no tags are specified, `latest` is used by default. When a template tag is created a `tag_timestamp` field is used to specifically version the tag.

Template tags resolve in the following ways when referenced:

- `template:tag@timestamp` - for a specific version of a template tag.
- `template:tag` - for a template with the newest version of a tag.
- `template` - for a template with the newest version of the `latest` tag.

Field	Description
template_id	**Required**. Name of the template. Type: string.
pod_definition	**Required**. Pod definition fields. Type: object. Can be overridden.
commit_message	**Required** Commit message for the template tag. Type: string. Default: “”.
tag	Tag name for the template tag. Type: string. Default: “latest”.

The following fields are set on creation and are useful to know:

- **tag_timestamp**: Tag timestamp for this object and used for referencing this tag.
- **added_by**: User who added this template tag.
- **creation_ts**: Time (UTC) that this template tag was created.

This example shows how to create a template tag for the `mongo` template:

9.4.3 Using a Template Tag to Create a Pod

To create a pod from a specific template tag and timestamp, use:

Ensure that the template name and tag match exactly as shown in the Tapis UI or the `list_templates()` output.

9.5 Neo4j

Assuming the user has created a Neo4j pod and retrieved credentials (user/pass), the user can now connect to the DB with the Neo4j browser interface or Python Neo4j driver.

9.6 Postgres

Assuming the user has created a Postgres pod and retrieved credentials (user/pass), the user can now connect to the DB with the Postgres' PgAdmin interface or Python Postgres drivers.

9.7 Future work. Only quickstart is currently complete.

Please view our API Reference to see what additional functionality is currently available.

9.8 API Reference

The following link is to our live-documentation that takes our OpenAPI v3 specification that is automatically generated and gives users the public endpoints available within the Pods API along with request body expected and descriptions for each field.

https://tapis-project.github.io/live-docs/?service=Pods

CHAPTER

TEN

ACTORS

10.1 Introduction to Abaco

10.1.1 What is Abaco

Abaco is an NSF-funded web service and distributed computing platform providing functions-as-a-service (FaaS) to the research computing community. Abaco implements functions using the Actor Model of concurrent computation. In Abaco, each actor is associated with a Docker image, and actor containers are executed in response to messages posted to their inbox which itself is given by a URI exposed over HTTP.

Abaco will ultimately offer three primary higher-level capabilities on top of the underlying Actor model:

- *Reactors* for event-driven programming
- *Asynchronous Executors* for scaling out function calls within running applications, and
- *Data Adapters* for creating rationalized microservices from disparate and heterogeneous sources of data.

Reactors and Asynchronous Executors are available today while Data Adapters are still under active development.

10.1.2 Using Abaco

Abaco is in production and has been adopted by several projects. Abaco is available to researchers and students. To learn more about the the system, including getting access, follow the instructions in *Getting Started*.

10.2 Getting Started

This Getting Started guide will walk you through the initial steps of setting up the necessary accounts and installing the required software before moving to the Abaco Quickstart, where you will create and execute your first Abaco actor. If you are already using Docker Hub and the TACC Cloud APIs, feel free to jump right to the *Abaco Quickstart* or check out the Abaco Live Docs site.

10.2.1 Account Creation and Software Installation

Create a TACC account

The main instance of the Abaco platform is hosted at the Texas Advanced Computing Center (TACC). TACC designs and deploys some of the world's most powerful advanced computing technologies and innovative software solutions to enable researchers to answer complex questions. To use the TACC-hosted Abaco service, please create a TACC account .

Create a Docker account

Docker is an open-source container runtime providing operating-system-level virtualization. Abaco pulls images for its actors from the public Docker Hub. To register actors you will need to publish images on Docker Hub, which requires a Docker account .

Install the Tapis Python SDK

To interact with the TACC-hosted Abaco platform in Python, we will leverage the Tapis Python SDK, tapipy. To install it, simply run:

```
$ pip3 install tapipy
```

Attention

`tapipy` works with Python 3.

10.2.2 Working with TACC OAuth

Authentication and authorization to the Tapis APIs uses OAuth2, a widely-adopted web standard. Our implementation of OAuth2 is designed to give you the flexibility you need to script and automate use of Tapis while keeping your access credentials and digital assets secure. This is covered in great detail in our Tenancy and Authentication section, but some key concepts will be highlighted here, interleaved with Python code.

Create an Tapis Client Object

The first step in using the Tapis Python SDK, tapipy, is to create a Tapis Client object. First, import the `Tapis` class and create python object called `t` that points to the Tapis server using your TACC username and password. Do so by typing the following in a Python shell:

```
# Import the Tapis object
from tapipy.tapis import Tapis

# Log into you the Tapis service by providing user/pass and url.
t = Tapis(base_url='https://tacc.tapis.io',
          username='your username',
          password='your password')
```

Generate a Token

With the `t` object instantiated, we can exchange our credentials for an access token. In Tapis, you never send your username and password directly to the services; instead, you pass an access token which is cryptographically signed by the OAuth server and includes information about your identity. The Tapis services use this token to determine who you are and what you can do.

```
# Get tokens that will be used for authenticated function calls
t.get_tokens()
print(t.access_token.access_token)

Out[1]: eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9...
```

Note that the tapipy `t` object will store and pass your access token for you, so you don't have to manually provide the token when using the tapipy operations. You are now ready to check your access to the Tapis APIs. It will expire though, after 4 hours, at which time you will need to generate a new token. If you are interested, you can create an

OAuth client (a one-time setup step, like creating a TACC account) that can be used to generate access and refresh tokens. For simplicity, we are skipping that but if you are interested, check out the Tenancy and Authentication section.

Check Access to the Tapis APIs

The tapipy `t` object should now be configured to talk to all Tapis APIs on your behalf. We can check that the client is configured properly by making any API call. For example, we can use the authenticator service to retrieve the full TACC profile of our user. To do so, use the `get_profile()` function associated with the `authenticator` object on the `t` object, passing the username of the profile to retrieve, as follows.

```
t.authenticator.get_profile(username='apitest')

Out[1]:
create_time: None
dn: cn=apitest,ou=People,dc=tacc,dc=utexas,dc=edu
email: aci-cic@tacc.utexas.edu
username: apitest
```

10.3 Abaco Quickstart

In this Quickstart, we will create an Abaco actor from a basic Python function. Then we will execute our actor on the Abaco cloud and get the execution results.

10.3.1 A Basic Python Function

Suppose we want to write a Python function that counts words in a string. We might write something like this:

```
def string_count(message):
    words = message.split(' ')
    word_count = len(words)
    print('Number of words is: ' + str(word_count))
```

In order to process a message sent to an actor, we use the `raw_message` attribute of the `context` dictionary. We can access it by using the `get_context` method from the `actors` module in `tapipy`.

For this example, create a new local directory to hold your work. Then, create a new file in this directory called `example.py`. Add the following to this file:

```
# example.py

from tapipy.actors import get_context

def string_count(message):
    words = message.split(' ')
    word_count = len(words)
    print('Number of words is: ' + str(word_count))

context = get_context()
message = context['raw_message']
string_count(message)
```

10.3.2 Building Images From a Dockerfile

To register this function as an Abaco actor, we create a docker image that contains the Python function and execute it as part of the default command.

We can build a Docker image from a text file called a Dockerfile. You can think of a Dockerfile as a recipe for creating images. The instructions within a Dockerfile either add files/folders to the image, add metadata to the image, or both.

The FROM Instruction

Create a new file called `Dockerfile` in the same directory as your `example.py` file.

We can use the `FROM` instruction to start our new image from a known image. This should be the first line of our Dockerfile. We will start an official Python image:

```
FROM python:3.6
```

The RUN, ADD and CMD Instructions

We can run arbitrary Linux commands to add files to our image. We'll run the `pip` command to install the `tapipy` library in our image:

```
RUN pip install --no-cache-dir tapipy
```

(Note: there is a `abacosample` image that contains Python and the tapipy library; see the Samples section for more details, coming soon.)

We can also add local files to our image using the `ADD` instruction. To add the `example.py` file from our local directory, we use the following instruction:

```
ADD example.py /example.py
```

The last step is to write the command from running the application, which is simply `python /example.py`. We use the `CMD` instruction to do that:

```
CMD ["python", "/example.py"]
```

With that, our `Dockerfile` is now ready. This is what is looks like:

```
FROM python:3.6

RUN pip install --no-cache-dir tapipy
ADD example.py /example.py

CMD ["python", "/example.py"]
```

Now that we have our `Dockerfile`, we can build our image and push it to Docker Hub. To do so, we use the `docker build` and `docker push` commands [note: user is your user on Docker, you must also $ docker login] :

```
$ docker build -t user/my_actor .
$ docker push user/my_actor
```

10.3.3 Registering an Actor

Now we are going to register the Docker image we just built as an Abaco actor. To do this, we will use the `Tapis` client object we created above (see *Working with TACC OAuth*).

To register an actor using the tapipy library, we use the `actors.add()` method and pass the arguments describing the actor we want to register through the `body` parameter. For example:

```
my_actor = {"image": "user/my_actor", "name": "word_counter", "description": "Actor that␣
↪counts words."}
t.actors.create_actor(**my_actor)
```

You should see a response like this:

```
_links:
executions: https://tacc.tapis.io/actors/v3/JWpkNmBwKewYo/executions
owner: https://tacc.tapis.io/profiles/v3/jstubbs
createTime: 2020-10-21T17:20:20.718177
default_environment:
description: Actor that counts words.
hints: []
id: JWpkNmBwKewYo
image: abacosamples/wc
last_update_time: 2020-10-21T17:20:20.718177
link:
mounts: [
container_path: /home/tapis/runtime_files/_abaco_data1
host_path: /home/apim/staging/runtime_files/data1
mode: ro,
container_path: /home/tapis/runtime_files/_abaco_data2
host_path: /home/apim/staging/runtime_files/data2/master/abaco
mode: rw]
owner: abaco
privileged: False
queue: default
state:
stateless: True
status: SUBMITTED
status_message:
token: false
type: none
use_container_uid: False
webhook:
```

Notes:

- Abaco assigned an id to the actor (in this case `JWpkNmBwKewYo`) and associated it with the image (in this case, `abacosamples/wc`) which it began pulling from the public Docker Hub.
- Abaco returned a status of `SUBMITTED` for the actor; behind the scenes, Abaco is starting a worker container to handle messages passed to this actor. The worker must initialize itself (download the image, etc) before the actor is ready.
- When the actor's worker is initialized, the status will change to `READY`.

At any point we can check the details of our actor, including its status, with the following:

```
t.actors.get_actor(actor_id='JWpkNmBwKewYo')
```

The response format is identical to that returned from the `.add()` method.

10.3.4 Executing an Actor

We are now ready to execute our actor by sending it a message. We built our actor to process a raw message string, so that is what we will send, but there other options, including JSON and binary data. For more details, see the *Messages* section.

We send our actor a message using the `send_message()` method:

```
t.actors.send_message(actor_id='JWpkNmBwKewYo',
                      request_body={'message': 'Actor, please count these words.'})
```

Abaco queues up an execution for our actor and then responds with JSON, including an id for the execution contained in the `execution_id`:

```
_links:
messages: https://tacc.tapis.io/actors/v3/JWpkNmBwKewYo/messages
owner: https://tacc.tapis.io/profiles/v3/jstubbs
execution_id: kA1P1m8NkkolK
msg: Actor, please count these words.
```

In general, an execution does not start immediately but is instead queued until a future time when a worker for the actor can take the message and start an actor container with the message. We can retrieve the details about an execution, including its status, using the `get_execution()` method:

```
>>> t.actors.get_execution(actor_id='JWpkNmBwKewYo', execution_id='kA1P1m8NkkolK')
```

The response will be similar to the following:

```
_links:
logs: https://tacc.tapis.io/actors/v3/JWpkNmBwKewYo/executions/kA1P1m8NkkolK/logs
owner: https://tacc.tapis.io/profiles/v3/jstubbs
actor_id: JWpkNmBwKewYo
api_server: https://tacc.tapis.io
cpu: 9678006850
executor: jstubbs
exitCode: 1
final_state:
Dead: False
Error:
ExitCode: 1
FinishedAt: 2020-10-21T17:26:49.77Z0
OOMKilled: False
Paused: False
Pid: 0
Restarting: False
Running: False
StartedAt: 2020-10-21T17:26:45.24Z0
Status: exited
finish_time: 2020-10-21T17:26:49.77Z0
id: kA1P1m8NkkolK
io: 152287298
message_received_time: 2020-10-21T17:26:44.367Z
runtime: 5
start_time: 2020-10-21T17:26:44.841Z
status: COMPLETE
```

(continues on next page)

(continued from previous page)

```
worker_id: QBmoQx4pOx1oA
```

Note that a status of `COMPLETE` indicates that the execution has finished and we are ready to retrieve our results.

10.3.5 Retrieving the Logs

The Abaco system collects all standard out from an actor execution and makes it available via the `logs` endpoint. Let's retrieve the logs from the execution we just made. We use the `get_execution_logs()` method, passing out `actor_id` and our `execution_id`:

```
t.actors.get_execution_logs(actor_id='JWpkNmBwKewYo', execution_id='kA1P1m8NkkolK')
```

The response should be similar to the following:

```
_links:
execution: https://tacc.tapis.io/actors/v3/JWpkNmBwKewYo/executions/kA1P1m8NkkolK
owner: https://tacc.tapis.io/profiles/v3/jstubbs
logs: Number of words is: 5\n
```

We see our actor output *Number of words is: 5*, which is the expected result!

10.3.6 Conclusion

Congratulations! At this point you have created, registered and executed your first actor, but there is a lot more you can do with the Abaco system. To learn more about the additional capabilities, please continue on to the Technical Guide.

10.4 Actor Registration

When registering an actor, the only required field is a reference to an image on the public Docker Hub. However, there are several other properties that can be set. The following table provides a list of the configurable properties available to all users and their descriptions.

Property Name	Description
image	The Docker image to associate with the actor. This should be a fully qualified image available on the public Docker Hub. We encourage users to use to image tags to version control their actors.
name	A user defined name for the actor.
description	A user defined description for the actor.
default_er	The default environment is a set of key/value pairs to be injected into every execution of the actor. The values can also be overridden when passing a message to the reactor in the query parameters (see *Messages*).
hints	A list of strings representing user-defined "tags" or metadata about the actor. "Official" Abaco hints can be applied to control configurable aspects of the actor runtime, such as the algorithm used (see *Autoscaling*).
link	Actor identifier (id or alias) of an actor to "link" this actor's events to. Requires execute permissions on the linked actor, and cycles are not permitted. (see *Actor Links, Events, and Webhooks*).
privileged	(True/False) - Whether the actor runs in privileged mode and has access to the Docker daemon. *Note*: Setting this parameter to True requires elevated permissions.
stateless	(True/False) - Whether the actor stores private state as part of its execution. If True, the state API will not be available, but in a future release, the Abaco service will be able to automatically scale reactor processes to execute messages in parallel. The default value is False.
token	(True/False) - Whether to generate an OAuth access token for every execution of this actor. Generating an OAuth token add about 500 ms of time to the execution start up time. *Note: the default value for the ``token`` attribute varies from tenant to tenant. Always explicitly set the token attribute when registering new actors to ensure the proper behavior.*
use_cor	Run the actor using the UID/GID set in the Docker image. *Note*: Setting this parameter to True requires elevated permissions.
run_as_	Run the actor using the UID/GID of the executor rather than the owner *Note*: this parameter is only available to certain tenants *Note*: that this cannot be on while the use_container_uid is also on
webhook	URL to publish this actor's events to. (see *Actor Links, Events, and Webhooks*)
cron_sc	String of 'yyyy-mm-dd hh + <increment>' format. Turns on cron feature for actor so that at the specified times the actor with execute with a generic static message. Increment looks like "5 days" or "1 week" (hours, days, weeks, months). There's currently no support for increments shorter that an hour. (see *Actor Cron*)
cron_or	(True/False) - Whether to turn on or off the cron schedule feature for the actor (see *Actor Cron*)

10.4.1 Notes

- The `default_environment` can be used to provide sensitive information to the actor that cannot be put in the image.
- In order to execute privileged actors or to override the UID/GID used when executing an actor container, talk to the Abaco development team about your use case.
- Abaco supports running specific actors within a given tenant on dedicated and/or specialized hardware for performance reasons. It accomplishes this through the use of actor `queues`. If you need to run actors on dedicated resources, talk to the Abaco development team about your use case.

10.4.2 Examples

curl

Here is an example using curl; note that to set the default environment, we *must* pass content type `application/json` and be sure to pass properly formatted JSON in the payload.

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-H "Content-Type: application/json" \
-d '{"image": "abacosamples/test", "name": "test", "description": "My test actor using␣
↪the abacosamples image.", "default_environment":{"key1": "value1", "key2": "value2"} }
↪' \
https://tacc.tapis.io/v3/actors
```

Python

To register the same actor using the tapipy library, we use the `actors.create_actor()` method and pass the same arguments through the *request_body* parameter. In this case, the `default_environment` is just a standard Python dictionary where the keys and values are `str` type. For example,

```
from tapipy.tapis import Tapis
t = Tapis(api_server='https://tacc.tapis.io', username='<username>', password='<password>
↪')
t.get_tokens()
actor = {"image": "abacosamples/test",
         "name": "test",
         "description": "My test actor using the abacosamples image registered using␣
↪tapipy.",
         "default_environment":{"key1": "value1", "key2": "value2"} }
t.actors.create_actor(**actor)
```

10.5 Abaco Context & Container Runtime

In this section we describe the environment that Abaco actor containers can utilize during their execution.

10.5.1 Context

When an actor container is launched, Abaco injects information about the execution into a number of environment variables. This information is collectively referred to as the `context`. The following table provides a complete list of variable names and their description:

Variable Name	Description
_abaco_actor_id	The id of the actor.
_abaco_actor_dbid	The Abaco internal id of the actor.
_abaco_container_repo	The Docker image used to launch this actor container.
_abaco_worker_id	The id of the worker for the actor overseeing this execution.
_abaco_execution_id	The id of the current execution.
_abaco_access_token	An OAuth2 access token representing the user who registered the actor.
_abaco_api_server	The OAuth2 API server associated with the actor.
_abaco_actor_state	The value of the actor's state at the start of the execution.
_abaco_Content-Type	The data type of the message (either 'str' or 'application/json').
_abaco_username	The username of the "executor", i.e., the user who sent the message.
_abaco_api_server	The base URL for the Abaco API service.
MSG	The message sent to the actor, as a raw string.

Notes

- The `_abaco_actor_dbid` is unique to each actor. Using this id, an actor can distinguish itself from other actors registered with the same function providing for SPMD techniques.
- The `_abaco_access_token` is a valid OAuth token that actors can use to make authenticated requests to other TACC Cloud APIs during their execution.
- The actor can update its state during the course of its execution; see the section *Actor State* for more details.
- The "executor" of the actor may be different from the owner; see *Sharing* for more details.

Access from Python

The `tapipy.actors` module provides access to the above data in native Python objects. Currently, the actors module provides the following utilities:

- **`get_context()` - returns a Python dictionary with the following fields:**
  - `raw_message` - the original message, either string or JSON depending on the Contetnt-Type.
  - `content_type` - derived from the original message request.
  - `message_dict` - A Python dictionary representing the message (for Content-Type: application/json)
  - `execution_id` - the ID of this execution.
  - `username` - the username of the user that requested the execution.
  - `state` - (for stateful actors) state value at the start of the execution.
  - `actor_id` - the actor's id.
- `get_client()` - returns a pre-authenticated `tapipy.Tapis` object.
- `update_state(val)` - Atomically, update the actor's state to the value `val`.

10.5.2 Runtime Environment

The environment in which an Abaco actor container runs has been built to accommodate a number of typical use cases encountered in research computing in a secure manner.

Container UID and GID

When Abaco launches an actor container, it instructs Docker to execute the process using the UID and GID associated with the TACC account of the owner of the actor. This practice guarantees that an Abaco actor will have exactly the same accesses as the original author of the actor (for instance, access to files or directories on shared storage) and that files created or updated by the actor process will be owned by the underlying API user. Abaco API users that have elevated privilleges within the platform can override the UID and GID used to run the actor when registering the actor (see *Registration*).

POSIX Interface to the TACC WORK File System

When Abaco launches an actor container, it mounts the actor owner's TACC WORK file system into the running container. The owner's work file system is made available at `/work` with the container. This gives the actor a POSIX interface to the work file system.

10.6 Messages, Executions, and Logs

Once you have an Abaco actor created the next logical step is to send this actor some type of job or message detailing what the actor should do. The act of sending an actor information to execute a job is called sending a message. This sent message can be raw string data, JSON data, or a binary message.

Once a message is sent to an Abaco actor, the actor will create an execution with a unique `execution_id` tied to it that will show results, time running, and other stats which will be listed below. Executions also have logs, and when the log are called for, you'll receive the command line logs of your running execution. Akin to what you'd see if you and outputted a script to the command line. Details on messages, executions, and logs are below.

Note: Due to each message being tied to a specific execution, each execution will have exactly one message that can be processed.

10.6.1 Messages

A message is simply the message given to an actor with data that can be used to run the actor. This data can be in the form of a raw message string, JSON, or binary. Once this message is sent, the messaged Abaco actor will queue an execution of the actor's specified image.

Once off the queue, if your specified image has inputs for the messaged data, then that messaged data will be visible to your program. Allowing you to set custom parameters or inputs for your executions.

Sending a message

cURL

To send a message to the `messages` endpoint with cURL, you would do the following:

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-d "message=<your content here>" \
https://tacc.tapis.io/v3/actors/<actor_id>/messages
```

Python

To send a message to the `messages` endpoint with `tapipy` and Python, you would do the following:

```
t.actors.send_message(actor_id='<actor_id>',
                      request_body={'message':'<your content here>'})
```

Results

These calls result in a list similar to the following:

```
_links:
messages: https://tacc.tapis.io/actors/v3/NPpjZkmZ4elY8/messages
owner: https://tacc.tapis.io/profiles/v3/jstubbs
execution_id: WrMk5EPmwYoL6
msg: <your content here>
```

Get message count

It is possible to retrieve the current number of messages an actor has with the `messages` end point.

cURL

The following retrieves the current number of messages an actor has:

```
$ curl -H "X-Tapis-Token: $TOKEN" \
https://tacc.tapis.io/v3/actors/<actor_id>/messages
```

Python

To retrieve the current number of messages with `tapipy` the following is done:

```
t.actors.get_messages(actor_id='<actor_id>')
```

Results

The result of getting the `messages` endpoint should be similar to:

```
_links:
owner: https://tacc.tapis.io/profiles/v3/jstubbs
messages: 12
```

Binary Messages

An additional feature of the Abaco message system is the ability to post binary data. This data, unlike raw string data, is sent through a Unix Named Pipe (FIFO), stored at /_abaco_binary_data, and can be retrieved from within the execution using a FIFO message reading function. The ability to read binary data like this allows our end users to do numerous tasks such as reading in photos, reading in code to be ran, and much more.

The following is an example of sending a JPEG as a binary message in order to be read in by a TensorFlow image classifier and being returned predicted image labels. For example, sending a photo of a golden retriever might yield, 80% golden retriever, 12% labrador, and 8% clock.

This example uses Python and `tapipy` in order to keep code in one script.

Python with Tapipy

Setting up an `Tapis` object with token and API address information:

```
from tapipy.tapis import Tapis
t = Tapis(api_server='https://tacc.tapis.io', username='<username>', password='<password>
↪')
t.get_tokens()
```

Creating actor with the TensorFlow image classifier docker image:

```
my_actor = {'image': 'abacosamples/binary_message_classifier',
            'name': 'JPEG_classifier',
            'description': 'Labels a read in binary image'}
actor_data = t.actors.create_actor(**my_actor)
```

The following creates a binary message from a JPEG image file:

```
with open('<path to jpeg image here>', 'rb') as file:
    binary_image = file.read()
```

Sending binary JPEG file to actor as message with the sendBinaryMessage function (You can also just set the headers with `Content-Type: application/octet-stream`):

```
result = t.actors.send_binary_message(actor_id = actor_data.id,
                                      request_body = binary_image)
```

The following returns information pertaining to the execution:

```
execution = t.actors.get_execution(actor_id = actor_data.id,
                                   execution_id = result.execution_id)
```

Once the execution has complete, the logs can be called with the following:

```
exec_logs = t.actors.get_execution_logs(actor_id = actor_data.id,
                                        execution_id = result.execution_id)
```

Sending binary from execution

Another useful feature of Abaco is the ability to write to a socket connected to an Abaco endpoint from within an execution. This Unix Domain (Datagram) socker is mounted in the actor container at /_abaco_results.sock.

In order to write binary data this socket you can use `tapipy` functions, in particular the `send_bytes_result()` function that sends bytes as single result to the socket. Another useful function is the `send_python_result()` function that allows you to send any Python object that can be pickled with `cloudpickle`.

In order to retrieve these results from Abaco you can get the `/actors/<actor_id>/executions/<execution_id>/results` endpoint. Each get of the endpoint will result in exactly one result being popped and retrieved. An empty result with be returned if the results queue is empty.

As a socket, the maximum size of a result is 131072 bytes. An execution can send multiple results to the socket and said results will be added to a queue. It is recommended to to return a reference to a file or object store.

As well, results are sent to the socket and available immediately, an execution does not have to complete to pop a result. Results are given an expiry time of 60 minutes from creation.

cURL

To retrieve a result with cURL you would do the following:

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-d "message=<your content here>" \
https://tacc.tapis.io/v3/actors/<actor_id>/executions/<execution_id>/results
```

Synchronous Messaging

Important

Support for Synchronous Messaging was added in version 1.1.0.

Starting with *1.1.0*, Abaco provides support for sending a synchronous message to an actor; that is, the client sends the actor a message and the request blocks until the execution completes. The result of the execution is returned as an HTTP response to the original message request.

Synchronous messaging prevents the client from needing to poll the executions endpoint to determine when an execution completes. By eliminating this polling and returning the response as soon as it is ready, the overall latency is minimized.

While synchronous messaging can simplify client code and improve performance, it introduces some additional challenges. Primarily, if the execution cannot be completed within the HTTP request/response window, the request will time out. This window is usually about 30 seconds.

Warning

Abaco strictly adheres to message ordering and, in particular, synchronous messages do not skip to the front of the actor's message queue. Therefore, a synchronous message *and all queued messages* must be processed within the HTTP timeout window. To avoid excessive synchronous message requests, Abaco will return a 400 level request if the actor already has more than 3 queued messages at the time of the synchronous message request.

To send a synchronous message, the client appends *_abaco_synchronous=true* query parameter to the request; the rest of the messaging semantics follows the rules and conventions of asynchronous messages.

cURL

The following example uses the curl command line client to send a synchronous message:

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-d "message=<your content here>" \
https://tacc.tapis.io/v3/actors/<actor_id>/messages?_abaco_synchronous=true
```

As stated above, the request blocks until the execution (and all previous executions queued for the actor) completes. To make the response to a synchronous message request, Abaco uses the following rules:

1. If a (binary) result is registered by the actor for the execution, that result is returned with along with a content-type *application/octet-stream*.
2. If no result is available when the execution completes, the logs associated with the execution are returned with content-type *text/html* (charset utf8 is assumed).

10.6.2 Executions

Once you send a message to an actor, that actor will create an execution for the actor with the inputted data. This execution will be queued waiting for a worker to spool up or waiting for a worker to be freed. When the execution is initially created it is given an execution_id so that you can access information about it using the execution_id endpoint.

Access execution data

cURL

You can access the `execution_id` endpoint using cURL with the following:

```
$ curl -H "X-Tapis-Token: $TOKEN" \
https://tacc.tapis.io/v3/actors/<actor_id>/executions/<execution_id>
```

Python

You can access the `execution_id` endpoint using `tapipy` and Python with the following:

```
t.actors.get_execution(actor_id='<actor_id>',
                       execution_id='<execution_id>')
```

Results

Access the `execution_id` endpoint will result in something similar to the following:

```
_links:
logs: https://tacc.tapis.io/actors/v3/JWpkNmBwKewYo/executions/kA1P1m8NkkolK/logs
owner: https://tacc.tapis.io/profiles/v3/jstubbs
actor_id: JWpkNmBwKewYo
api_server: https://tacc.tapis.io
cpu: 9678006850
executor: jstubbs
exitCode: 1
final_state:
Dead: False
Error:
ExitCode: 1
FinishedAt: 2020-10-21T17:26:49.77Z0
OOMKilled: False
Paused: False
Pid: 0
Restarting: False
Running: False
StartedAt: 2020-10-21T17:26:45.24Z0
Status: exited
finish_time: 2020-10-21T17:26:49.77Z0
id: kA1P1m8NkkolK
io: 152287298
message_received_time: 2020-10-21T17:26:44.367Z
runtime: 5
start_time: 2020-10-21T17:26:44.841Z
status: COMPLETE
worker_id: QBmoQx4pOx1oA
```

List executions

Abaco allows users to retrieve all executions tied to an actor with the `executions` endpoint.

cURL

List executions with cURL by getting the `executions endpoint`

```
$ curl -H "X-Tapis-Token: $TOKEN" \
https://tacc.tapis.io/v3/actors/<actor_id>/executions
```

Python

To list executions with `tapipy` the following is done:

```
t.actors.list_executions(actor_id='<actor_id>')
```

Results

Calling the list of executions should result in something similar to:

```
_links:
owner: https://master.staging.tapis.io/profiles/v3/abaco
actor_id: WP7vMmRvrDXxN
api_server: https://master.staging.tapis.io
executions: [
finish_time: Wed, 21 Oct 2020 17:48:33 GMT
id: QBmoQx4pOx1oA
message_received_time: Wed, 21 Oct 2020 17:48:20 GMT
start_time: Wed, 21 Oct 2020 17:48:20 GMT
status: COMPLETE,
finish_time: None
id: QZY8W1Z30Zmbq
message_received_time: Wed, 21 Oct 2020 17:49:56 GMT
start_time: None
status: SUBMITTED]
owner: abaco
totalCpu: 61248097463
totalExecutions: 2
totalIo: 752526010
totalRuntime: 13
```

Reading message in execution

One of the most important parts of using data in an execution is reading said data. Retrieving sent data depends on the data type sent.

Python - Reading in raw string data or JSON

To retrieve JSON or raw data from inside of an execution using Python and `tapipy`, you would get the message context from within the actor and then get it's `raw_message` field.

```
from tapipy.actors import get_context

context = get_context()
message = context['raw_message']
```

Python - Reading in binary

Binary data is transmitted to an execution through a FIFO pipe located at /_abaco_binary_data. Reading from a pipe is similar to reading from a regular file, however `tapipy` comes with an easy to use `get_binary_message()` function to retrieve the binary data.

Note: Each Abaco execution processes one message, binary or not. This means that reading from the FIFO pipe will result with exactly the entire sent message.

```
from tapipy.actors import get_binary_message

bin_message = get_binary_message()
```

10.6.3 Logs

At any point of an execution you are also able to access the execution logs using the `logs` endpoint. This returns information about the log along with the log itself. If the execution is still in the submitted phase, then the log will be an empty string, but once the execution is in the completed phase the log would contain all outputted command line data.

Retrieving an executions logs

cURL

To call the `log` endpoint using cURL, do the following:

```
$ curl -H "X-Tapis-Token: $TOKEN" \
https://tacc.tapis.io/v3/actors/<actor_id>/executions/<execution_id>/logs
```

Python

To call the `log` endpoint using `tapipy` and Python, do the following:

```
t.actors.get_execution_logs(actor_id='<actor_id>',
                            execution_id='<execution_id>')
```

Results

This would result in data similar to the following:

```
_links:
execution: https://tacc.tapis.io/actors/v3/JWpkNmBwKewYo/executions/kA1P1m8NkkolK
owner: https://tacc.tapis.io/profiles/v3/jstubbs
logs: <command line output here>
```

10.7 Database Search

With the introduction of Abaco 1.6.0 database searching has been introduced using the Mongo aggregation system, full-text searching, and indexing. Searching can be done on actor, worker, execution, and log information. This feature allows for users to search based on any information across all objects that they have permission to view. For example, search would allow checking of all viewable executions for ERRORS in one easy call. The search currently makes use of logical operators and datetime to allow for easy searching of any object based on any specific field.

Attention

Search in Abaco was implemented in version 1.6.0.

Search is available on the actors, workers, executions, and logs databases. Search has been implemented on a new `{base}/actors/search/{database}` endpoint alongside being implemented on the `{base}/actors`, `{base}/actors/{actor_id}/workers`, `{base}/actors/{actor_id}/executions`, and `{base}/actors/{actor_id}/executions/{execution_id}/logs` endpoints.

To use search on the `{base}/actors/search/{database}` endpoint the database to be searched must be specified as either `actors`, `workers`, `executions`, or `logs` in the URL. With no query arguments Abaco will return all entries in the database that you have permission to view. To specify query arguments the user can add a `?` to the end of their url and specify the parameters they are looking to implement.

A table of search parameters, their function, and examples are below.

Parameter	Function	Examples
search	Completes a fuzzy full-text search based on inputs. Returns results by best accuracy/score.	?search=stringToSearchFor
exactsearch	Completes a full-text search and looks for exact matches with inputs.	?exactsearch=stringToMatchExactly
eq	Checks if given value is equal to db value matching given key.	?id.eq=AKY5o4Z847lB3
neq	Checks if given value is not equal to db value matching given key.	?id.neq=AKY5o4Z847lB3
gt	Checks if given value is greater than db value matching given key.	?start_time.gt=2020-04-29+06:00
gte	Checks if given value is greater than or equal to db value matching given key.	?runtime.gte=423
lt	Checks if given value is less than db value matching given key.	?message_received_time.lt=2020
lte	Checks if given value is less than or equal to db value matching given key.	?final_state.FinishedAt.lte=2020-04-29
in	Checks if db value matching given key match any values in the given list of values.	?status.in=[“BUSY”,”REQUESTED”,”READY”]
nin	Checks if db value matching given key does not match any values in the given list of values.	?status.nin=[“COMPLETED”, “READY”]
like	Checks if given value in (through regex) db value matching given key.	?image.like=abaco_docker_username
nlike	Checks if given value not in (through regex) db value matching given key.	?image.nlike=abaco_test
between	Checks if db value matching given key is greater than or equal to first given value, and less than or equal to second given value.	?start_time.between=2020-04-29T20:15:52:246Z,2021-06-24-05:00
limit	Sets a limit on total amount of results returned. Defaults to 10 results.	?limit=20
skip	Skips a specified amount of results when returning.	?skip=4

You may use as many parameters as you want in one query sans `limit` and `skip`, where each may only be used once.

10.7.1 Metadata

Abaco search slightly alters the expected result of a request in the fact that the returned result from a search now returns two objects, the expected result, `search`, and `_metadata`.

This new `_metadata` object returns pertinent information about the amount of records returned, the amount of records the return is limited to, the amount of records skipped (specified in query), and the total amount of records that match the query searched for. This is useful to implement paging or to only receive a set amount of records.

Important

A new `_metadata` object is now returned alongside the usual result in `result.`

Example of result with new `result`

```
{'message': 'Executions search completed successfully.',
 'result': {'_metadata': {'count_returned': 1,
                          'record_limit': 10,
                          'records_skipped': 0,
                          'total_count': 1},
            'search': [{'_links': {'logs': 'https://dev.tenants.aloedev.tacc.cloud/v3/
↪actors/joBjeDkWyBwLx/logs',
                                   'owner': 'https://dev.tenants.aloedev.tacc.cloud/
↪profiles/v3/testuser',
                                   'self': 'https://dev.tenants.aloedev.tacc.cloud/v3/
↪actors/joBjeDkWyBwLx/executions/1JKkQwX75vE56'},
                        'actor_id': 'joBjeDkWyBwLx',
                        'cpu': 444097006,
                        'executor': 'testuser',
                        'exitCode': 0,
                        'final_state': {'Dead': False,
                                        'Error': '',
                                        'ExitCode': 0,
                                        'FinishedAt': '2020-04-29T21:47:21.385Z',
                                        'OOMKilled': False,
                                        'Paused': False,
                                        'Pid': 0,
                                        'Restarting': False,
                                        'Running': False,
                                        'StartedAt': '2020-04-29T21:47:19.382Z',
                                        'Status': 'exited'},
                        'id': '1JKkQwX75vE56',
                        'io': 716,
                        'message_received_time': '2020-04-29T21:47:18.7Z00',
                        'runtime': 2,
                        'start_time': '2020-04-29T21:47:18.954Z',
                        'status': 'COMPLETE',
                        'worker_id': '7kvAAKYKB6Qk6'}]},
 'status': 'success',
 'version': ':dev'}
```

10.7.2 Inputs

All inputs are given to the search function as query parameters and thus are converted to strings. It is then up to Abaco's side to convert these inputs back to the intended formats. Strings are left untouched. Booleans are expected to be "False" or "false" and "True" or "true" to be converted. Numbers are converted all to floats, these are still comparable to database instances of int, so there should be no issue. Lists are parsed with `json.loads` and will accept either `["test"]` or `['test']` with post-processing on Abaco's end to convert to lists.

The last consumed input type is datetime objects. Abaco accepts a broad range of ISO 8601 like strings. An example of the most detailed string accepted is `2020-04-29T20:15:52:246252-06:00`. `2020-04-29T20:15:52:246Z`, `2020-04-29T20:15:52-06:00`, `2020-04-29T20:15-06:00`, `2020-04-29T20-06:00`, `2020-04-29-06:00`, `2020-04Z`, and `2020` are also acceptable.

Attention

Abaco stores all times in UTC, so addition of your timezone or conversion to UTC is important. If no timezone information is given (`-06:00` or `Z` (to signal UTC)) the datetime is assumed to be in UTC.

Important

Comparison with datetime rounds to the minimum time possible. For instance if you want to see if 2020-12-30 is greater than 2020, you would receive True as 2020 is rounded to *2020-01-01T00:00:00Z*. This holds true until you reach millisecond accurate time.

Creating ISO 8601 formatted strings

Python - String with Timezone

The following gets the current time as an ISO 8601 formatted string with timezone:

```
import datetime
import pytz

austin_time_zone = pytz.timezone("America/Chicago")
isoString = datetime.datetime.now(tz=austin_time_zone).isoformat()
print(isoString)
```

This prints `2020-04-29T16:21:34.602078-05:00`.

Python - UTC String

The following gets the current UTC time as an ISO 8601 formatted string:

```
import datetime

isoString = datetime.datetime.utcnow().isoformat()
print(isoString)
```

This prints `2020-04-29T21:21:34.602078`. Feel free to add the Z or leave it absent.

10.7.3 Searching

Like mentioned above, a search may contain as many parameters as a user wants sans for `limit` and `skip`, where each may only be used once. Search on the new `{base}/actors/search/{database}` always takes place and when given no parameters returns any information the user has access to. To activate on search on the other endpoints, at least one query parameter must be declared.

Important

`x-nonce` queries will still work as expected and do not need any modification.

Performing searches on different endpoints

{base}/actors/search/actors

You can use `actors`, `workers`, `executions`, or `logs` as database inputs for the endpoints. Each queries the specified database.

cURL

```
$ curl -H "X-Tapis-Token: $TOKEN" \
https://tacc.tapis.io/v3/actors/search/actors?image=abacosamples/test&create_time.
↪gt=2020-04-29&status.in=["READY", "BUSY"]
```

Result

```
{'message': 'Search completed successfully.',
'result': {'_metadata': {'count_returned': 1,
                         'record_limit': 10,
                         'records_skipped': 0,
                         'total_count': 1},
           'search': [{'_links': {'executions': 'https://dev.tenants.aloedev.tacc.cloud/
↪v3/actors/joBjeDkWyBwLx/executions',
                                  'owner': 'https://dev.tenants.aloedev.tacc.cloud/
↪profiles/v3/testuser',
                                  'self': 'https://dev.tenants.aloedev.tacc.cloud/v3/
↪actors/joBjeDkWyBwLx'},
                       'createTime': '2020-04-29T21:46:53.393Z',
                       'default_environment': {'default_env_key1': 'default_env_value1',
                                               'default_env_key2': 'default_env_value2'},
                       'description': '',
                       'gid': None,
                       'hints': [],
                       'id': 'joBjeDkWyBwLx',
                       'image': 'abacosamples/test',
                       'last_update_time': '2020-04-29T21:46:53.393Z',
                       'link': '',
                       'max_cpus': None,
                       'max_workers': None,
                       'mem_limit': None,
                       'mounts': [{'container_path': '/_abaco_data1',
                                   'host_path': '/data1',
```

(continues on next page)

(continued from previous page)

```
                                    'mode': 'ro'}],
                          'name': 'abaco_test_suite_default_env',
                          'owner': 'testuser',
                          'privileged': False,
                          'queue': 'default',
                          'state': {},
                          'stateless': True,
                          'status': 'READY',
                          'status_message': ' ',
                          'tasdir': None,
                          'token': 'false',
                          'type': 'none',
                          'uid': None,
                          'use_container_uid': False,
                          'webhook': ''}]},
'status': 'success',
'version': ':dev'}
```

{base}/actors/joBjeDkWyBwLx/executions

For a search from an endpoint like this the actor_id will already be in the query, so for this example you would only receive executions with the actor_id of `joBjeDkWyBwLx`. `{base}/actors/joBjeDkWyBwLx/workers` would result in the same behaviour, but for workers. This usage means that performing a search on `{base}/actors/joBjeDkWyBwLx/executions/1JKkQwX75vE56/logs` would always result in one result. Only search on the `{base}/actors` thus is the only full search available that does not use the `{base}/actors/search/{database}` endpoint.

Attention

Use the `{base}/actors/search/{database}` endpoint for a full search of the specified database.

cURL

```
$ curl -H "X-Tapis-Token: $TOKEN" \
https://tacc.tapis.io/v3/actors/search/actors/joBjeDkWyBwLx/executions?status=COMPLETE&
↪start_time.gt=2019
```

Result

```
{'message': 'Executions search completed successfully.',
 'result': {'_metadata': {'count_returned': 1,
                          'record_limit': 10,
                          'records_skipped': 0,
                          'total_count': 1},
            'search': [{'_links': {'logs': 'https://dev.tenants.aloedev.tacc.cloud/v3/
↪actors/joBjeDkWyBwLx/logs',
                                   'owner': 'https://dev.tenants.aloedev.tacc.cloud/
↪profiles/v3/testuser',
                                   'self': 'https://dev.tenants.aloedev.tacc.cloud/v3/
```

(continues on next page)

(continued from previous page)

```
↪actors/joBjeDkWyBwLx/executions/1JKkQwX75vE56'},
                        'actor_id': 'joBjeDkWyBwLx',
                        'cpu': 444097006,
                        'executor': 'testuser',
                        'exitCode': 0,
                        'final_state': {'Dead': False,
                                        'Error': '',
                                        'ExitCode': 0,
                                        'FinishedAt': '2020-04-29T21:47:21.385Z',
                                        'OOMKilled': False,
                                        'Paused': False,
                                        'Pid': 0,
                                        'Restarting': False,
                                        'Running': False,
                                        'StartedAt': '2020-04-29T21:47:19.382Z',
                                        'Status': 'exited'},
                        'id': '1JKkQwX75vE56',
                        'io': 716,
                        'message_received_time': '2020-04-29T21:47:18.7Z00',
                        'runtime': 2,
                        'start_time': '2020-04-29T21:47:18.954Z',
                        'status': 'COMPLETE',
                        'worker_id': '7kvAAKYKB6Qk6'}]},
 'status': 'success',
 'version': ':dev'}
```

10.8 Cron Schedule

Note

The Abaco Cron Schedule feature was implemented in version 1.7.0.

Abaco's cron schedule is a tool to automatically execute your actor based on a schedule. Each actor has two user-defined parameters associated with the cron execution: `cron_schedule` and `cron_on`. The scheduler has another variable, `cron_next_ex`, which holds the next execution time of the actor. This is an internal variable and cannot be edited by users. To create a schedule, set the `cron_schedule` parameter when registering a new actor or updating an existing actor. The value of `cron_schedule` should be a string in the following format:

```
yyyy-mm-dd hh + <number> <unit of time>
```

where the first part is the datetime when the first execution will happen, and the second part is the time increment for each subsequent execution. Note that the spaces, plus characters (+) and dash characters (-) in the template above are meaningful and are a required part of the format. Abaco's cron schedule also has an alias called `now`, which lets you execute the actor at the current UTC time. For example, if an actor was registered with this schedule

```
"cron_schedule": "now + 1 hour"
```

the actor would execute at the current time, and then again at the top of the hour every hour.

Note

The highest granularity is the hour, and the units of time that can be used are hour, day, week, and month.

To create an actor with a schedule, make a request like the following:

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-H "Content-Type: application/json" \
-d '{"image": "abacosamples/test", "cron_schedule": "2020-09-28 16 + 1 hour"}' \
https://tacc.tapis.io/v3/actors
```

To update the schedule, make a request like the following:

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-X PUT \
-H "Content-Type: application/json" \
-d '{"image": "abacosamples/test", "cron_schedule": "2020-12-11 16 + 3 days"}' \
https://tacc.tapis.io/v3/actors/<actor_id>
```

This last request above will update the cron schedule for the actor with id <actor_id> as follows: the actor will be scheduled to automatically execute on December 11th, 2020 at 4 pm, UTC timezone. That actor will be executed again 3 days later on the 14th, at 4pm, and then 3 days after that, again at 4pm. This execution will recur every 3 days until the user changes the cron schedule, turns off the cron schedule, or deletes the actor.

Note

The cron schedule runs on the UTC timezone.

Note

When making requests to set the cron_schedule, be sure to pass "application/json" content to avoid issues requiring escaping characters inside the schedule value.

To turn off the schedule, use the *cron_on* switch like so:

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-X PUT \
-H "Content-Type: application/json" \
-d '{"image": "abacosamples/test", "cron_on": "False"}' \
https://tacc.tapis.io/v3/actors/<actor_id>
```

By turning off the schedule, the actor will no longer execute itself at each increment. You can turn it on again at any time, and the actor will resume incrementing as before. For example, if an actor is set to execute every hour, and then the cron switch is turned off, the actor will stop executing itself. After a week, the switch can be turned back on, and the actor will resume executing on the hour.

10.8.1 Cron Schedule - Error Messages

If users supply a value for *cron_schedule* in an incorrect format, they will receive an error letting them know to check the format. The API also checks that the schedule sent in has not already past. For example, if you pass in the year 1955, you will get an error message saying the cron schedule has already passed. The error message will also tell you the current UTC time for reference.

10.8.2 Cron Message and Execution

When it is time to execute an actor configured with a `cron_schedule`, Abaco's internal cron agent simply queues a message on the actor's internal message queue, just as if a client had sent a message to the actor using the `/messages` API. If the actor already has (unprocessed) messages in its queue, these messages will be processed first before the cron message. This means that there could be some delay between the time Abaco internally queues the message and the actor starts executing it.

Currently, the cron message sent to the actor is the static string

```
This is your cron execution
```

Accordingly, the `_abaco_Content_Type` context variable is set to `str`. The rest of the context variables are set normally, as described in context.

10.9 Actor State

In this section we describe the state that can persist through Abaco actor container executions.

10.9.1 State

When an actor is registered, its `stateless` property is automatically set to `true`. An actor must be registered with `stateless=false` to be stateful (maintain state ass executions).

Once an actor is executed, the associated worker `GETs` data from the `/v3/actors/{actor_id}/state` endpoint and injects it into the actor's `_abaco_actor_state` environment variable. While an actor is executing, the actor can update its state by `POSTing` to the aforementioned endpoint.

Notes

- The worker only `GETs` data from the state endpoint one time as the actor is initiated. If the actor updates its state endpoint during execution, the worker does not inject the new state until a new execution.
- Stateful actors may only have one associated worker in order to avoid race conditions. Thus generally, stateless actors will execute quicker as they can operate in parallel.
- Issuing a state to a stateless actor will return a `actor is stateless.` error.
- The `state` variable must be JSON-serializable. An example of passing JSON-serializable data can be found under *Examples <target state examples>* below.

10.9.2 Utilizing State in Actors to Accomplish Something

WIP

10.9.3 Examples

curl

Here are some examples interacting with state using curl.

Registering an actor specifying statefulness: `stateless=false`.

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-d "image=abacosamples/test&stateless=false" \
https://tacc.tapis.io/v3/actors
```

POSTing a state to a particular actor; keep in mind we must indicate in the header that we are passing content type `application/json`.

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-H "Content-Type: application/json" \
-d '{"some variable": "value", "another variable": "value2"}' \
https://tacc.tapis.io/v3/actors/<actor_id>/state
```

GETting information about a particular actor's state.

```
$ curl -H "X-Tapis-Token: $TOKEN" \
https://tacc.tapis.io/v3/actors/<actor_id>/state
```

Python

Here are some examples interacting with state using Python. The `tapipy.actors` module provides access to an actor's environment data in native Python objects.

Registering an actor specifying statefulness: `stateless=false`.

```
from tapipy.tapis import Tapis
t = Tapis(api_server='https://tacc.tapis.io', username='<username>', password='<password>
↪')
t.get_tokens()
actor = {"image": "abacosamples/test",
         "stateless": "False"}
actor_res = t.actors.create_actor(**actor)
```

POSTing a state to a particular actor; again keep in mind we must pass in JSON serializable data.

```
state = {"some variable": "value", "another variable": "value2"}
t.actors.update_state(actor_id = actor_res.id,
                      request_body = state)
```

GETting information about a particular actor's state. This function returns a Python dictionary with many fields one of which is state.

```
from tapipy.actors import get_context
get_context()
{'raw_message': '<text>', 'content_type': '<text>', 'execution_id': '<text>', 'username
↪': '<text>', 'state': 'some_state', 'actor_dbid': '<text>', 'actor_id': '<text>', 'raw_
↪message_parse_log': '<text>', 'message_dict': {}}
```

10.10 Actor Sharing and Nonces

Abaco provides a basic permissions system for securing actors. An actor registered with Abaco starts out as private and only accessible to the API user who registered it. This API user is referred to as the "owner" of the actor. By making a POST request to the permissions endpoint for an actor, a user can manage the list of API users who have access to the actor.

10.10.1 Permission Levels

Abaco supports sharing an actor at three different permission levels; in increasing order, they are: *READ*, *EXECUTE* and *UPDATE*. Higher permission imply lower permissions, so a user with *EXECUTE* also has *READ* while a user with *UPDATE* has *EXECUTE* and *READ*. The permission levels provide the followig accesses:

- *READ* - ability to list the actor to see it's details, list executions and retrieve execution logs.
- *EXECUTE* - ability to send an actor a message.
- *UPDATE* - ability to change the actor's definition.

cURL

To share an actor with another API user, make a POST request to the */permissions* endpoint; the following example uses curl to grant READ permission to API user *jdoe*.

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-d "user=jdoe&level=READ" \
https://tacc.tapis.io/v3/actors/<actor_id>/permissions
```

Example response:

```
{
  "message": "Permission added successfully.",
  "result": {
    "jdoe": "READ",
    "testuser": "UPDATE"
  },
  "status": "success",
  "version": "1.0.0"
}
```

We can list all permissions associated with an actor at any time using a GET request:

```
$ curl -H "X-Tapis-Token: $TOKEN" \
https://tacc.tapis.io/v3/actors/<actor_id>/permissions
```

Example response:

```
{
  "message": "Permissions retrieved successfully.",
  "result": {
    "jdoe": "READ",
    "jsmith": "EXECUTE",
    "testuser": "UPDATE"
  },
  "status": "success",
  "version": "1.0.0"
}
```

Note

To remove a user's permission, POST to the permission endpoint and set *level=NONE*

10.10.2 Public Actors

At times, it can be useful to grant **all** API users access to an actor. To enable this, Abaco recognizes the special ABACO_WORLD user. Granting a permission to the ABACO_WORLD user will effectively grant the permission to all API users.

10.10.3 cURL

The following grants *READ* permission to all API users:

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-d "user=ABACO_WORLD&level=READ" \
https://tacc.tapis.io/v3/actors/<actor_id>/permissions
```

10.10.4 Nonces

Abaco provides a capability referred to as actor *nonces* to ease integration with third-party systems leveraging different authentication mechanisms. An actor *nonce* can be used in place of the typical TACC API access token (bearer token). However, unlike an access token which can be used for any actor the user has access, a nonce can only be used for a specific actor.

Creating Nonces

API users create nonces using the nonces endpoint associated with an actor. Nonces can be limited to a specific permission level (e.g., *READ* only), and can have a finite number of uses or an unlimited number.

The following example uses curl to create a nonce with *READ* level permission and with 5 uses.

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-d "maxUses=5&level=READ" \
https://tacc.tapis.io/v3/actors/<actor_id>/nonces
```

A typical response:

```
{
  "message": "Actor nonce created successfully.",
  "result": {
    "_links": {
      "actor": "https://tacc.tapis.io/v3/actors/rNjQG5BBJoxO1",
      "owner": "https://tacc.tapis.io/profiles/v3/testuser",
      "self": "https://tacc.tapis.io/v3/actors/rNjQG5BBJoxO1/nonces/DEV_qBMrvO6Zy0yQz"
    },
    "actor_id": "rNjQG5BBJoxO1",
    "api_server": "http://172.17.0.1:8000",
    "createTime": "2019-06-18 12:20:53.087704",
    "currentUses": 0,
    "description": "",
    "id": "TACC_qBMrvO6Zy0yQz",
    "lastUseTime": "None",
    "level": "READ",
    "maxUses": 5,
    "owner": "testuser",
    "remainingUses": 5,
  },
  "status": "success",
```

(continues on next page)

(continued from previous page)

```
  "version": "1.0.0"
}
```

The *id* of the nonce (in the above example, *TACC_qBMrvO6Zy0yQz*) can be used to access the actor in place of the access token.

Note

Roles are used throughout the TACC API's to grant users with specific privileges (e.g., administrative access to certain APIs). The roles of the API user generating the nonce are captured at the time the nonce is created; when using a nonce, a request will have permissions granted via those roles. Most users will not need to worry about TACC API roles.

To create a nonce with unlimited uses, set *maxUses=-1*.

Redeeming Nonces

To use a nonce in place of an access token, simply form the request as normal and add the query paramter *x-nonce=<nonce_id>*.

For example

```
$ curl -X POST -d "message=<your content here>" \
https://tacc.tapis.io/v3/actors/<actor_id>/messages?x-nonce=TACC_vr9rMO6Zy0yHz
```

The response will be exactly the same as if issuing the request with an access token.

10.11 Networks of Actors

Working with individual, isolated actors can augment an existing application with a lot of additional functionality, but the full power of Abaco's actor-based system is realized when many actors coordinate together to solve a common problem. Actor coordination introduces new challenges that the system designer must address, and Abaco provides features specifically designed to address these challenges.

10.11.1 Actor Aliases

An *alias* is a user-defined name for an actor that is managed independently of the actor itself. Put simply, an alias maps a name to an actor id, and Abaco will replace a reference to an alias in any request with the actor id defined by the alias at the time. Aliases are useful for insulating an actor from changes to another actor to which it will send messages.

For example, if actor A sends messages to actor B, the user can create an alias for actor B and configure A to send messages to that alias. In the future, if changes need to be made to actor B or if messages from actor A need to be routed to a different actor, the alias value can be updated without any code changes needed on the part of actor A.

Creating and managing aliases is done via the `/aliases` collection.

cURL

To create an alias, make a POST request passing the alias and actor id. For example, suppose we have an actor that counts the words sent in a message. We might create an alias for it with the following:

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-d "alias=counter&actor_id=6PlMbDLa4zlON" \
https://tacc.tapis.io/v3/actors/aliases
```

Example response:


```
{
  "message": "Actor alias created successfully.",
  "result": {
    "_links": {
      "actor": "https://tacc.tapis.io/v3/actors/6PlMbDLa4zlON",
      "owner": "https://tacc.tapis.io/profiles/v3/jstubbs",
      "self": "https://tacc.tapis.io/v3/actors/aliases/counter"
    },
    "actor_id": "6PlMbDLa4zlON",
    "alias": "counter",
    "owner": "apitest"
  },
  "status": "success",
  "version": "1.1.0"
}
```


With the alias `counter` created, we can now use it in place of the actor id in any Abaco request. For example, we can get the actor's details:

```
$ curl -H "X-Tapis-Token: $TOKEN" \
https://tacc.tapis.io/v3/actors/counter
```

The response returned is identical to that returned when the actor id is used.

10.11.2 Nonces Attached to Aliases

Important

Support for Nonces attached to aliases was added in version 1.1.0.

Important

The nonces attached to aliases feature was updated in version 1.5.0, so that 1) `UPDATE` permission on the underlying actor id is required and 2) It is no longer possible to create an alias nonce for permission level `UPDATE`.

Nonces can be created for aliases in much the same way as creating nonces for a specific actor id - instead of using the `/nonces` endpoint associated with the actor id, use the `/nonces` endpoint associated with the alias instead. The POST message payload is the same. For example:

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-d "maxUses=5&level=READ" \
https://tacc.tapis.io/v3/actors/aliases/counter/nonces
```

will create a nonce associated with the `counter` alias.

Note

Listing, creating and deleting nonces associated with an alias requires the analagous permission for both the alias **and** the associated actor.

10.11.3 Actor Events, Links and WebHooks

Important

Support for Actor events, links and webhooks was added in version 1.2.0.

Abaco captures certain events pertaining to the evolution of the system runtime and provides mechanisms for users to consume these events in actors as well as in external systems.

First, Abaco provides a facility to automatically send a message to a specified actor whenever certain events occur. This mechanism is called an actor *link*: if actor A is registered with a `link` property specifying actor B, then Abaco will automatically send actor B a message whenever any of the recognized events occurs.

Second, an actor can be registered with a `webhook` property: a single string representing a URL to send an HTTP POST request to. The Abaco events subsystem will send a POST request **exactly once** to the specified URL whenever a recognized event occurs.

Webhooks and event messages are guaranteed to be delivered in order relative to the order the events occurred for the specific actor. Since there is no total ordering on events across different actors, there is no analagous order guarantee.

Links or Webhooks - Which to use?

In both cases, the details of the event are described in a JSON message (sent to an actor in the case of a link, and sent in the POST payload in the case of a webhook).

However, the actor link is far more general and flexible since the user can define arbitrary logic to handle the event. Even when the ultimate goal is a webhook, the user may opt for defining a link to an actor that performs the webhook. This approach enables users to customtize the webhook processing in various ways, including retry logic, authentication, etc. In fact, the `abacosamples/webhook` image provides a webhook dispatcher built to parse the Abaco events message with many configurable options.

Use of an actor's `webhook` property is really intended for simple use cases or situations missed or dropped events will not cause a major issue.

Adding a Link

Registering an actor with a link (or updating an exisitng actor to add a link property) follows the same semantics as defined in the *Registration* section; simply add the `link` attribute in the payload. For example, the following request creates an actor with a link to actor id `6PlMbDLa4zlON`.

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-H "Content-Type: application/json" \
-d '{"image": "abacosamples/test", "name": "test", "link": "6PlMbDLa4zlON", "description
↪": "My test actor using the abacosamples image.", "default_environment":{"key1":
↪"value1", "key2": "value2"} }' \
https://tacc.tapis.io/v3/actors
```

It is also possible to link an actor to an alias: just pass `link=<the_alias>` in the registration payload.

Note

Setting a link attribute requires `EXECUTE` permission for the associated actor.

Note

Defining a link property that would result in a cycle of linked actors is not permitted, as this would result in infinite messages. In particular, an actor cannot link to itself.

Adding a WebHook

Registering an actor with a webhook is accomplished similarly by setting the `webhook` property in the actor registration (POST) or update (PUT) payload. For example, the following request creates an actor with a webhook set to the requestbin at `https://eniih104j4tan.x.pipedream.net`.

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-H "Content-Type: application/json" \
-d '{"image": "abacosamples/test", "name": "test", "webhook": "https://eniih104j4tan.x.
↪pipedream.net", }' \
https://tacc.tapis.io/v3/actors
```

Events and Event Message Format

Whenever a supported event occurs, Abaco sends a JSON message to the linked actor or webhook with data about the event. The included data depends on the event type, as documented below.

In the case of a linked actor, all the typical context variables, as documented in *Context*, will be injected as usual, excepted where noted below. In this case, note that there are details about two actors: the actor for which the event occurred and the linked actor itself (which are always different, as self-links are not permitted). The former is described in the message itself with variables such as `actor_id`, `tenant_id`, etc., while the latter is described using the special reserved Abaco variables, e.g., `_abaco_actor_id`, etc.

Variable Name	Description	Event Type
actor_id	The id of the actor for which the event occurred.	all types
tenant_id	The id of the tenant of the actor for which the event occurred.	all types
actor_dbid	The internal id of the actor for which the event occurred.	all types
event_type	The event type associated with the event. (see table below)	all types
event_time_utc	The time of the event, in UTC, as a float.	all types
event_time_display	The time of the event, as a string, formatted for display.	all types
_abaco_link	The actor id of the linked actor (the actor receiving the event message)	all types
_abaco_username	'Abaco Event'	all types
status_message	A message indicating details about the error status.	ACTOR_ERROR
execution_id	The id of the completed execution.	EXECUTION_COMPLETE
exit_code	The exit code of the completed execution.	EXECUTION_COMPLETE
status	The final status of the completed execution.	EXECUTION_COMPLETE

The following table lists all events by their `event_type` value and a brief description. Additional event types may be

added in subsequent releases.

Event type	Description
ACTOR_READY	The actor is ready to accept messages.
ACTOR_ERROR	The actor is in error status and requires manual intervention.
EXECUTION_COMPLETE	An actor execution has just completed.

10.12 Autoscaling Actors

The Abaco platform has an optional autoscaler subsystem for automatically managing the pool of workers associated with the registered actors. In general, the autoscaler ignores actors that are registered with `stateless: False`, as it assumes these actors must process their message queues synchronously. For *stateless* actors without custom configurations, the austocaling algorithm is as follows:

1. Every 5 seconds, check the length of the actor's message queue.
2. If the queue length is greater than 0, and the actor's worker pool is less than the maximum workers per actor, start a new worker.
3. If the queue length is 0, reduce the actor's worker pool until: a) the worker pool size becomes 0 or b) the actor receives a message.

In particular, the worker pool associated with an actor with 0 messages in its message queue will be reduced to 0 to free up resources on the Abaco compute cluster.

10.12.1 Official "sync" Hint

Important

Support for actor hints and the official "sync" hint was added in version 1.4.0.

For some use cases, reducing an actor's worker pool to 0 as soon as its message queue is empty is not desirable. Starting up a worker takes significant time, typically on the order of 10 seconds or more, depending on configuration options for the actor, and adding this overhead to actors that have low latency requirements can be a serious issue. In particular, actors that will respond to "synchronous messages" (i.e., `_abaco_synchronous=true`) have low latency requirements to respond within the HTTP timeout window.

For this reason, starting in version 1.4.0, Abaco recognizes an "official" actor hint, `sync`. When registered with the `sync` hint, the Abaco autoscaler will leave at least one worker in the actor's worker pool up to a configurable period of idle time (specific to the Abaco tenant). For the Abaco public tenant, this period is 60 minutes.

The `hints` attribute for an actor is saved at registration time. In the following example, we register an actor with the `sync` hint using curl:

```
$ curl -H "X-Tapis-Token: $TOKEN" \
-H "Content-type: application/json" \
-d '{"image": "abacosamples/wc", "hints": ["sync"]}' \
https://tacc.tapis.io/v3/actors
```

10.13 API Reference

The following table lists the public endpoints within the Abaco API.

GET	POST	PUT	DELETE	Endpoint	Description
X				/v3/actors/utilization	Get high-level usage stats.
X	X			/v3/actors	List/create actors.
X	X			/v3/actors/aliases	List/create aliases.
X			X	/v3/actors/aliases/{alias}	List/delete an alias.
X		X	X	/v3/actors/{actor_id}	List/update/delete an actor.
X	X			/v3/actors/{actor_id}/messages	Get number messages/send message
X	X			/v3/actors/{actor_id}/nonces	List/create actor nonces.
X			X	/v3/actors/{actor_id}/nonces/{nonce_id}	Get nonce details/delete nonce.
X	X			/v3/actors/{actor_id}/state	Retrieve/update actor state.
X	X			/v3/actors/{actor_id}/workers	List/create actor workers.
X			X	/v3/actors/{actor_id}/workers/{worker_id}	Get worker details/delete worker
X	X			/v3/actors/{actor_id}/permissions	List/update actor permissions.
X				/v3/actors/{actor_id}/executions	Retrieve execution details.
			X	/v3/actors/{actor_id}/executions/{eid}	Halt running execution.
X				/v3/actors/{actor_id}/executions/{eid}/logs	Retrieve execution logs.
X				/v3/actors/{actor_id}/executions/{eid}/resu	Retrieve execution results.
X				/v3/actors/search/{database}	Searches specified database
X				/metrics	

CHAPTER

ELEVEN

SECURITY

11.1 Introduction to the Security Kernel

The Security Kernel (SK) microservice provides role-based authorization and secrets management for Tapis. Authentication is based on JSON Web Tokens (JWTs) managed by the Authentication subsystem.

SK uses a PostgreSQL database to store its authorization data and the open source version of HashiCorp Vault as its secrets backend. In the sections that follow, we discuss SK's authorization and secrets model, interfaces and capabilities. The actual SK REST APIs can be found here.

11.2 Authorization

SK authorization is based on an extended version of the Apache Shiro authorization model, which defines both roles and permissions.

11.2.1 Roles

In a tenant, each role has a unique name and is assigned an owner. SK provides a set of *role endpoints* to create, update, query and delete roles. It also provides *user endpoints* to grant users roles, revoke roles from users, and query the roles assigned to a user. With these primitives, Tapis implements fairly typical role-base access control using a distributed architecture. Among the most called endpoints is /user/hasRole, which checks whether a user has a certain role.

All role-based authorizations go through SK, which provides site-wide, network access to authorization checking. A possible downside of this approach is the extra network cost incurred on Tapis calls that authorize users (many service to service calls avoid this overhead). So far this overhead has had minimal impact.

Built-In Roles

Each tenant has at least one tenant administrator, which is nothing more than a user assigned a distinguished, tightly controlled role (*$!tenant_admin*). The initial tenant administrator is assigned during tenant creation; see Tenants for details. There are special endpoints for granting and revoking the tenant administrator role, and for validating and listing administrators in a tenant. Only a tenant administrator can grant or revoke the administrator role to another user.

Each user is implicitly given a default role. These roles have names that begin with "$$" and end with the user's ID. Default roles are most commonly used in conjunction permissions, as discussed below.

Hierarchical Roles

One feature that distinguishes SK's implementation of role-based authorization is that roles can be arranged in directed acyclic graphs (DAGs) based on parent/child relationships. We say a parent role *contains* a child role, and a child can have zero or more parents. The set of roles defined in a tenant can be thought of as a forest of DAGs.

This contains relation allows SK users to define roles with fine granularity and then compose them in flexible ways. For example, let DirA be the root of a directory subtree that is shared among users. We could define *DirA_Owner* as a parent role with *DirA_Reader* and *DirA_Writer* as child roles (assume the typical semantics implied by their names). Users assigned *DirA_Owner* would implicitly also be assigned *DirA_Reader* and *DirA_Writer*. A user only assigned *DirA_Reader* would not have write privileges.

Taking the example one step further, assume for DirB we define *DirB_Owner* as a parent role with *DirB_Reader* and *DirB_Writer* children. We could then define another role, *AllDir_Reader*, with *DirA_Reader* and *DirB_Reader* children. Users assigned *AllDir_Reader* would have read access to both DirA and DirB.

11.2.2 Permissions

In addition to checking whether a user has been granted a certain role, SK authorization can also be based on permissions. SK roles can contain zero or more permission strings. The syntax and semantics of these permissions are explained in the Shiro documentation. The /usr/isPermitted and related SK endpoints are called to determine if a user has a permission matching a required permission string.

Permissions only exist inside roles. For convenience, SK implements permission endpoints that automatically apply to a user's default role. See the /user/grantUserPermission endpoint for details.

Extended Permissions

SK implements the full Shiro permission model and extends it to accommodate hierarchical resources such as file systems. For certain registered permission schemas, the last component of a specification can be treated as *extended path attribute*. Extended path attributes enhance the standard Shiro matching algorithm with one that treats designated components as hierarchical names, such as Posix file or directory path names. Consider, for example, permissions that conform to the registered *files* schema:

```
SCHEMA:    files:<tenant>:<operation>:<systemId>:<path>
Examples:  files:tacc:read:mysystem:/home/bud/data
           files:mytenant:read,write:mysystem:/home/mary/images
```

When a user is assigned a role that contains the first example permission string, then that user is authorized to read files in the */home/bud/data* directory subtree. A user assigned the second permission is authorized to read and write files in the */home/mary/images* directory subtree.

SK's extended attribute permissions are used to maintain authorization to hierarchical resources *outside of those resources*. The need for externalized authorization control arises, for instance, when a single service account is used to access data on a system for multiple actual users. In this case, the host system is always accessed using the same account, but authorization needs to be carried out for different actual Tapis users.

11.3 Secrets

SK uses HashiCorp Vault as is backend database for storing and managing secrets. There is no direct access to Vault for users or services–all access comes through SK. SK allows secrets to be created, read, versioned, deleted and destroyed by reflecting in its API the capabilities of Vault's version 2 Key/Value secrets engine.

SK overlays Vault's native capabilities with its own *typed secrets model*. The basic idea is that SK requires users to provide a *secretType* and *secretName* on most of its calls. Using this information, SK calculates the virtual paths (i.e., locations) in Vault being referenced. Users do not need to understand Vault's naming scheme and SK has complete control of where secrets reside inside of Vault. The following table lists the secret types supported by SK.

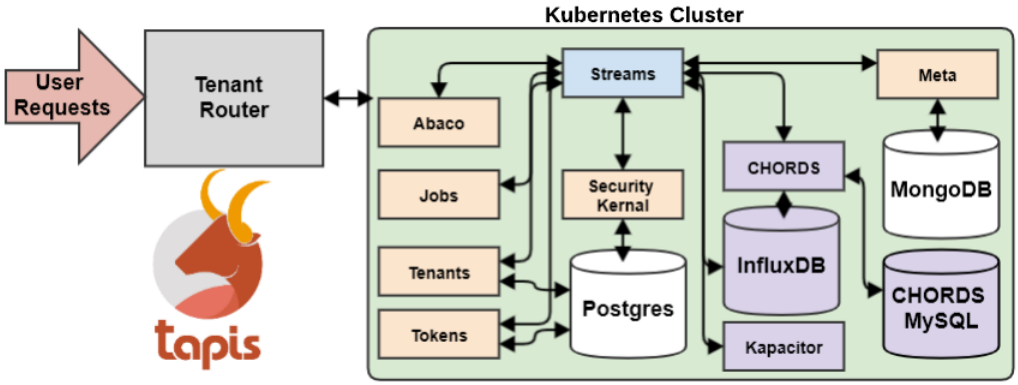
Secret Type	Description
Service Password	Password used by services to acquire their JWTs
JWT Signing Key	Tenant-specific JWT signing key used by Tokens service
DB Credentials	Credentials used by services to access their databases
System Credentials	Credentials for accessing Tapis systems
User	User secrets

Only the User secret type can be used by Tapis users; the rest are reserved for Tapis services only. Currently, SK only allows a single secret to be referenced by each secretType/secretName combination. Otherwise, the full capabilities of the underlying Vault secrets engine is reflected in the SK secrets API.

CHAPTER

TWELVE

STREAMS



12.1 Projects

Projects are defined at a top level in the hierarchy of Streams resources. A user registers a project by providing metadata information such as the principal Investigator, project URL, funding resource, etc. A list of authorized users can be added to various project roles to have a controlled access over the project resources. When a project is first registered, a collection is created in the back-end MongoDB. User permissions to access this collection are then set up in the security kernel. Every request to access the project resource or documents within (i.e sites, instruments, variables) goes through a security kernel check and only the authorized user requests are allowed to be processed.

12.1.1 Create Project

With PySDK:

```
$ t.streams.create_project(project_name='tapis_demo_project_testuser6',description='test␣
↪project', owner='testuser6', pi='testuser6', funding_resource='tapis', project_url=
↪'test.tacc.utexas.edu', project_id='tapis_demo_project_testuser6',active=True)
```

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d '{
↪"project_name": "tapis_demo_project_testuser6",
                                                    "project_id":"tapis_demo_project_
↪testuser6",
```

(continues on next page)

(continued from previous page)

```
                                                                "owner": "testuser6",
                                                                "pi": "testuser6",
                                                                "description": "test project",
                                                                "funding_resource": "tapis",
                                                                "project_url": "test.tacc.utexas.
↪edu",
                                                                "active": "True"}' $BASE_URL/v3/
↪streams/projects
```

The response will look something like the following:

```
active: True
description: test project
funding_resource: tapis
owner: testuser6
permissions:
users: ['testuser6']
pi: testuser6
project_id: tapis_demo_project_testuser6
project_name: tapis_demo_project_testuser6
project_url: test.tacc.utexas.edu
```

12.1.2 List Projects

With PySDK:

```
$ t.streams.list_projects()
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/streams/projects
```

The response will look something like the following:

```
[
 active: True
 description: project for early adopters demo
 funding_resource: tapis
 owner: testuser6
 permissions:
 users: ['testuser6']
 pi: ajamthe
 project_id: wq_demo_project12
 project_name: wq_demo_project12
 project_url: test.tacc.utexas.edu,

 active: True
 description: test project
 funding_resource: tapis
```

(continues on next page)

(continued from previous page)

```
 owner: testuser6
 permissions:
 users: ['testuser6']
 pi: testuser6
 project_id: tapis_demo_project_testuser6
 project_name: tapis_demo_project_testuser6
 project_url: test.tacc.utexas.edu,
]
```

12.1.3 Get Project Details

With PySDK:

```
$ t.streams.get_project(project_id='tapis_demo_project_testuser6')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" $BASE_URL/v3/streams/projects/tapis_demo_project_testuser6
```

The response will look something like the following:

```
active: True
description: project for early demo
funding_resource: tapis
owner: testuser6
permissions:
users: ['testuser6']
pi: testuser6
project_id: tapis_demo_project_testuser6
project_name: tapis_demo_project_testuser6
project_url: test.tacc.utexas.edu
```

12.1.4 Update Project

With PySDK:

```
$ t.streams.update_project(project_id='tapis_demo_project_testuser6', project_name=
↪'tapis_demo_project_testuser6', pi='testuser6', owner='testuser6', description=
↪'changed description',project_url='tapis_demo_project.tacc.utexas.edu')
```

With CURL:

```
$ curl -v -X PUT -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d '{
↪"project_name": "tapis_demo_project_testuser6",
                                                    "project_id":"tapis_demo_project_
```

(continues on next page)

(continued from previous page)

```
↪testuser6",
                                                    "owner": "testuser6",
                                                    "pi": "testuser6",
                                                    "description": "changed␣
↪description",
                                                    "funding_resource": "tapis",
                                                    "project_url": "tapis_demo_
↪project.tacc.utexas.edu",
                                                    "active": "True"}' $BASE_URL/v3/
↪streams/projects/tapis_demo_project_testuser6
```

The response will look something like the following:

```
active: True
description: changed description
funding_resource: tapis
last_updated: 2020-07-20 17:34:58.848079
owner: testuser6
permissions:
users: ['testuser6']
pi: testuser6
project_id: tapis_demo_project_testuser6
project_name: tapis_demo_project_testuser6
project_url: tapis_demo_project.tacc.utexas.edu
```

12.1.5 Delete Project

With PySDK:

```
$ t.streams.delete_project(project_id='tapis_demo_project_testuser6')
```

With CURL:

```
$ curl -X DELETE -H "X-tapis-token:$jwt" $BASE_URL/v3/streams/projects/tapis_demo_
↪project_testuser6
```

The response will look something like the following:

```
active: True
description: project for early adopters demo
funding_resource: tapis
last_updated: 2020-12-04 15:06:41.460343
owner: testuser6
permissions:
users: ['testuser6']
pi: testuser6
project_id: tapis_demo_project_testuser6
project_name: tapis_demo_project_testuser6
project_url: test.tacc.utexas.edu
tapis_deleted: True
```

12.2 Sites

Site is a geographical location that may hold one or more instruments. Sites are next in the streams hierarchy and they inherit permissions from the projects. Project owners can create sites by providing the geographical information such as latitude, longitude and elevation of the site or GeoJSON encoded spatial information. This spatial information is useful when searching sites or data based on location. In the back-end database a site is represented as a JSON document within the project collection. Site permissions are inherited from the project.

12.2.1 Create Site

With PySDK:

```
$ t.streams.create_site(project_id='tapis_demo_project_testuser6',request_body[{"site_
↪name":"tapis_demo_site", "site_id":"tapis_demo_site", "latitude":50,"longitude":10,
↪"elevation":2,"description":"test_site"}])
```

With CURL:

```
$  curl -X POST -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" --data '[{
↪"site_name":"tapis_demo_site","latitude":50,"longitude":10,"elevation":2,"site_id":
↪"tapis_demo_site", "description":"test_site"}]' $BASE_URL/v3/streams/projects/tapis_
↪demo_project_testuser6/sites
```

The response will look something like the following:

```
chords_id: 27
created_at: 2020-06-08 18:27:12.416134
description: test_site
elevation: 2
latitude: 50
location:
coordinates: [10.0, 50.0]
type: Point
longitude: 10
site_id: tapis_demo_site
site_name: tapis_demo_site
```

12.2.2 List Sites

With PySDK:

```
$ t.streams.list_sites(project_id='tapis_demo_project_testuser6')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt"  $BASE_URL/v3/streams/projects/tapis_demo_project_
↪testuser6/sites
```

The response will look something like the following:

```
[
 chords_id: 13
 created_at: 2020-07-20 19:00:55.220397
 description: demo site
 elevation: 1
 latitude: 1.0
 location:
 coordinates: [2.0, 1.0]
 type: Point
 longitude: 2
 site_id: demo_site
 site_name: demo_site,

 chords_id: 12
 created_at: 2020-07-20 18:15:25.404740
 description: test_site
 elevation: 2
 latitude: 50
 location:
 coordinates: [10.0, 50.0]
 type: Point
 longitude: 10
 site_id: tapis_demo_site
 site_name: tapis_demo_site]
```

12.2.3 Get Site Details

With PySDK:

```
$ t.streams.get_site(project_id='tapis_demo_project_testuser6', site_id='tapis_demo_site1
↪')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt"  $BASE_URL/v3/streams/projects/tapis_demo_project_
↪testuser6/sites/tapis_demo_site
```

The response will look something like the following:

$ t.streams.get_site(project_id='tapis_demo_project_testuser6', site_id='tapis_demo_site')

```
chords_id: 12
created_at: 2020-07-20 18:15:25.404740
description: test_site
elevation: 2
latitude: 50
location:
coordinates: [10.0, 50.0]
type: Point
longitude: 10
```

(continues on next page)

(continued from previous page)

```
site_id: tapis_demo_site
site_name: tapis_demo_site
```

12.2.4 Update Site

With CURL:

```
$ curl -X PUT -H "Content-Type:application/json"  -H "X-Tapis-Token:$jwt" -d '{"project_
↪id": "tapis_demo_project_testuser6","site_name":"tapis_demo_site","latitude":10,
↪"longitude":80, "elevation":2, "description":"test site changed"}' $BASE_URL/v3/
↪streams/projects/tapis_demo_project_testuser6/sites/tapis_demo_site
```

With PySDK

```
$ t.streams.update_site(project_id='tapis_demo_project_testuser6',site_name='tapis_demo_
↪site', site_id='tapis_demo_site', latitude=10, longitude = 80, elevation=2,description=
↪'test_site changed')
```

The response will look something like the following:

```
chords_id: 4
created_at: 2020-08-10 19:36:48.649316
description: test_site changed
elevation: 2
last_updated: 2020-08-10 19:37:20.115021
latitude: 10
location:
coordinates: [80.0, 10.0]
type: Point
longitude: 80
site_id: tapis_demo_site
site_name: tapis_demo_site
```

12.2.5 Delete Site

With CURL:

```
$ curl -X DELETE -H "X-Tapis-Token:$jwt"  $BASE_URL/v3/streams/projects/tapis_demo_
↪project_testuser6/sites/tapis_demo_site
```

With PySDK

```
$ t.streams.delete_site(project_id='tapis_demo_project_testuser6', site_id='tapis_demo_
↪site')
```

12.3 Instruments

Instruments are physical entities that may have one or more embedded sensors to sense various parameters such as temperature, relative humidity, specific conductivity, etc. These sensors referred to as variables in Streams API generate measurements, which are stored in the influxDB along with a ISO8601 timestamp. Instruments are associated with specific sites and projects. Information about the instruments such as site and project ids, name and description of the instrument, etc. are stored in the mongoDB sites JSON document.

12.3.1 Create Instrument

With PySDK

```
$ t.streams.create_instrument(project_id='tapis_demo_project_testuser6',site_id='tapis_
↪demo_site',request_body=[{"topic_category_id":"2",  "inst_name":"tapis_demo_instrument
↪","inst_description":"demo instrument", "inst_id":"tapis_demo_instrument"}])
```

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" --data '[{
↪"topic_category_id":"2","inst_name":"tapis_demo_instrument","inst_description":"demo␣
↪instrument", "inst_id":"tapis_demo_instrument"}]'  $BASE_URL/v3/streams/projects/tapis_
↪demo_project_testuser6/sites/tapis_demo_site/instruments
```

The response will look something like the following:

```
chords_id: 10
created_at: 2020-07-20 20:09:11.990814
inst_description: demo instrument
inst_id: tapis_demo_instrument
inst_name: tapis_demo_instrument
topic_category_id: 2
```

12.3.2 List Instruments

With PySDK

```
$ t.streams.list_instruments(project_id='tapis_demo_project_testuser6', site_id='tapis_
↪demo_site')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt"  $BASE_URL/v3/streams/projects/tapis_demo_project_
↪testuser6/sites/tapis_demo_site/instruments
```

The response will look something like the following:

```
[
 chords_id: 10
 created_at: 2020-07-20 20:09:11.990814
 inst_description: demo instrument
```

(continues on next page)

(continued from previous page)

```
inst_id: tapis_demo_instrument
inst_name: tapis_demo_instrument
topic_category_id: 2,

chords_id: 11
created_at: 2020-07-20 20:14:20.512383
inst_description: demo instrument
inst_id: tapis_demo_instrument
inst_name: tapis_demo_instrument1
project_id: tapis_demo_project_testuser6
site_id: tapis_demo_site
topic_category_id: 2,

chords_id: 12
created_at: 2020-07-20 20:20:45.171473
inst_description: demo instrument
inst_id: demo_instrument
inst_name: demo_instrument
topic_category_id: 2,

chords_id: 13
created_at: 2020-07-20 20:21:52.842495
inst_description: demo instrument
inst_id: demo_instrument_aj
inst_name: demo_instrument_aj
topic_category_id: 2]
```

12.3.3 Get instrument Details

With PySDK

```
$ t.streams.list_instruments(project_id='tapis_demo_project_testuser6', site_id='tapis_
↪demo_site',inst_id='demo_instrument')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt"  $BASE_URL/v3/streams/projects/tapis_demo_project_
↪testuser6/sites/tapis_demo_site/instruments/demo_instrument
```

The response will look something like the following:

```
chords_id: 12
created_at: 2020-07-20 20:20:45.171473
inst_description: demo instrument
inst_id: demo_instrument
inst_name: demo_instrument
topic_category_id: 2
```

12.3.4 Update Instrument

With PySDK

```
$ t.streams.update_instrument(inst_id= 'Ohio_River_Robert_C_Byrd_Locks', project_id='wq_
↪demo_tapis_streams_proj2020-08-26T08:41:11.813391', site_id='wq_demo_site', inst_name=
↪'test', inst_description='test')
```

With CURL:

```
$ curl -X PUT -H "X-Tapis-token:$jwt" -H "Content-Type:application/json" --data '{"inst_
↪id": "Ohio_River_Robert_C_Byrd_Locks",
"site_id": "wq_demo_site", "inst_name": "UpdatedNAME","inst_description": "updated␣
↪descript"}'
$BASE_URL/v3/streams/projects/wq_demo_tapis_streams_proj2020-08-26T08:41:11.813391/sites/
↪wq_demo_site/instruments/Ohio_River_Robert_C_Byrd_Locks'
```

The response will look something like the following:

```
chords_id: 6
inst_description: test
inst_id: Ohio_River_Robert_C_Byrd_Locks
inst_name: test
site_chords_id: 7
updated_at: 2020-08-26 18:40:07.534077
variables: [
chords_id: 21
shortname: temp
updated_at: 2020-08-26 16:15:49.835211
var_id: temp
var_name: temperature,
chords_id: 22
shortname: bat
updated_at: 2020-08-26 16:15:50.349601
var_id: batv
var_name: battery,
chords_id: 23
shortname: spc
updated_at: 2020-08-26 16:15:50.749192
var_id: spc
var_name: specific_conductivity,
chords_id: 24
shortname: turb
updated_at: 2020-08-26 16:15:51.158687
var_id: turb
var_name: turbidity,
chords_id: 25
shortname: ph
updated_at: 2020-08-26 16:15:51.588573
var_id: ph
var_name: ph_level]
```

12.3.5 Delete Instrument

With PySDK

```
$ t.streams.delete_instrument(inst_id= 'tapis_demo_instrument', project_id='tapis_demo_
↪project_testuser6_3', site_id='tapis_demo_site')
```

With CURL:

```
$ curl -X DELETE -H "X-Tapis-token:$jwt" $BASE_URL/v3/streams/projects/tapis_demo_
↪project_testuser6_3/sites/tapis_demo_site/instruments/tapis_demo_instrument
```

12.4 Variables

12.4.1 Create Variables

With PySDK

```
$ t.streams.create_variable(project_id='tapis_demo_project_testuser6', inst_id='demo_
↪instrument', site_id='tapis_demo_site', request_body=[{"topic_category_id":"2", "var_
↪name":"battery", "shortname":"bat", "var_id":"batv"}])
```

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" --data '{
↪"project_id":"tapis_demo_project_testuser6", "topic_category_id":"2","site_id":"tapis_
↪demo_site", "inst_id":"demo_instrument", "var_name":"battery", "shortname":"bat", "var_
↪id":"batv"}'  $BASE_URL/v3/streams/projects/tapis_demo_project_testuser6/sites/tapis_
↪demo_site/instruments/demo_instrument/variables
```

The response will look something like the following:

```
chords_id: 39
shortname: bat
updated_at: 2020-07-20 21:51:38.712035
var_id: batv
var_name: battery
```

12.4.2 List Variables

With PySDK

```
$ t.streams.list_variables(project_id='tapis_demo_project_testuser6',site_id='tapis_demo_
↪site', inst_id='demo_instrument')
```

With CURL:

```
$ curl -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt"  $BASE_URL/v3/streams/
↪projects/tapis_demo_project_testuser6/sites/tapis_demo_site/instruments/demo_
↪instrument/variables
```

The response will look something like the following:

```
[
 chords_id: 38
 shortname: bat
 updated_at: 2020-07-20 21:50:46.382558
 var_id: batv
 var_name: battery,

 chords_id: 39
 shortname: bat
 updated_at: 2020-07-20 21:51:38.712035
 var_id: batv
 var_name: battery,

 chords_id: 40
 inst_id: demo_instrument_1
 project_id: tapis_demo_project_testuser6
 shortname: bat
 site_id: tapis_demo_site
 topic_category_id: 2
 updated_at: 2020-07-20 21:56:45.555381
 var_id: batv
 var_name: battery]
```

12.4.3 Get Variable Details

With PySDK

```
$ t.streams.get_variable(project_id='tapis_demo_project_testuser6_1', site_id='tapis_
↪site_final', inst_id='tapis_inst_final', var_id='batv')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt"  $BASE_URL/v3/streams/projects/tapis_demo_project_
↪testuser6_1/sites/tapis_site_final/instruments/tapis_inst_final/variables/batv
```

The response will look something like the following:

```
[
chords_id: 21
shortname: bat
updated_at: 2020-08-18 20:46:11.673033
var_id: batv
var_name: battery]
```

12.4.4 Update Variable

With PySDK

```
$ t.streams.update_variable(var_name='"updated_temp', var_id='temp', shortname='temp_
↪updated', project_id='wq_demo_tapis_streams_proj2020-08-25T16:21:30.113392', site_id=
↪'wq_demo_site',inst_id='Ohio_River_Robert_C_Byrd_Locks')
```

With CURL:

```
$ curl -X PUT -H "X-Tapis-token:$jwt" -H "Content-type:application/json"  --data '{ "var_
↪name": "updated_temp","var_id": "temp","shortname":"temp_updated"}' $BASE_URL/v3/
↪streams/projects/wq_demo_tapis_streams_proj2020-08-25T16:21:30.113392/sites/wq_demo_
↪site/instruments/Ohio_River_Robert_C_Byrd_Locks/variables/temp
```

The response will look something like the following:

```
chords_id: 16
inst_chords_id: 5
shortname: temp_updated
site_chords_id: 6
updated_at: 2020-08-27 14:36:04.271154
var_id: temp
var_name: "updated_temp
```

12.4.5 Delete Variable

With PySDK

```
$ t.streams.delete_variable( var_id='139', project_id='tapis_demo_instrument', site_id=
↪'tapis_demo_site',inst_id='tapis_demo_instrument')
```

With CURL:

```
$ curl -v -X DELETE  -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt"  $BASE_
↪URL/v3/streams/projects/tapis_demo_project_testuser6_3/sites/tapis_demo_site/
↪instruments/tapis_demo_instrument/variables/batv
```

The response will look something like the following:

```
inst_chords_id: 24
updated_at: 2020-12-03 02:52:27.437378
var_id: 139
```

12.5 Measurements

12.5.1 Create Measurements

With PySDK

```
$ t.streams.create_measurement(inst_id='demo_instrument',vars=[{"batv":10, "temp":90,
↪"datetime":"2020-07-20T22:19:25Z"}])
```

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" --data '{
↪"inst_id":"demo_instrument", "vars":[{"datetime":"2020-07-20T23:19:25Z", "batv":10,
↪"temp":90}]}'  $BASE_URL/v3/streams/measurements
```

The response will look something like the following:


```
{
 "message": "Measurements Saved",
 "result": {
   "batv": {
     "2020-07-20T23:19:25Z": 10
   },
   "temp": {
     "2020-07-20T23:19:25Z": 90
   }
 },
 "status": "success",
 "version": "dev"
}
```


12.5.2 List Measurements

With PySDK

```
$ t.streams.list_measurements(inst_id='demo_instrument',start_date='2020-05-08T00:00:00Z
↪',end_date='2020-07-21T22:19:25Z', format='csv',project_id='tapis_demo_project_
↪testuser6',site_id='tapis_demo_site')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt"  $BASE_URL/v3/streams/measurements/demo_instrument
```

The response will look something like the following:


```
b'time,batv\n2020-07-20T22:19:25Z,10.0\n2020-07-20T23:19:25Z,10.0\n'
```

12.6 Channels

12.6.1 Channel Alert Types

12.6.2 Channel Actions

The type of actions that a channel should be performed can be defined under the action parameter within a triggers_with_actions object.

12.6.3 Create Channels

With PySDK

```
$ t.streams.create_channels(channel_id="demo.tapis.channel",
                            channel_name='demo.tapis.channel',
                            template_id="demo_channel_template",
                            triggers_with_actions=[{
                                "inst_ids":["demo_instrument"],
                                "condition":{
                                        "key":"demo_instrument.batv",
                                        "operator":">",
                                        "val":20 },
                                "action":{
                                        "method":"ACTOR",
                                        "actor_id" :"XXXX",
                                        "message":"Instrument: demo_instrument exceeded␣
↪threshold", "abaco_base_url":"https://api.tacc.utexas.edu","nonces":"XXXX-YYYY-ZZZZ"
                        }}])
```

With CURL:

```
$ curl -v -X POST -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" --data '{
↪"channel_id":"demo.tapis.channel","channel_name":"demo.tapis.channel_1","template_id":
↪"demo_channel_template","triggers_with_actions":[{"inst_ids":["demo_instrument"],
↪"condition":{"key":"demo_instrument.batv","operator":">", "val":"20"}, "action":{
↪"method":"ACTOR","actor_id" :"XXXX","message":"Instrument: demo_instrument batv␣
↪exceeded threshold", "abaco_base_url":"https://api.tacc.utexas.edu","nonces":"XXXX-
↪YYYY-ZZZZ"}}]}'  $BASE_URL/v3/streams/channels
```

The response will look something like the following:

```
channel_id: demo.tapis.channel
channel_name: demo.tapis.channel
create_time: 2020-07-21 03:02:51.755215
last_updated: 2020-07-21 03:02:51.755227
permissions:
users: ['testuser6']
status: ACTIVE
template_id: demo_channel_template
triggers_with_actions: [
action:
abaco_base_url: https://api.tacc.utexas.edu
actor_id: XXXX
message: Instrument: demo_instrument exceeded threshold
```

(continues on next page)

(continued from previous page)

```
method: ACTOR
nonces: XXXX-YYYY-ZZZZ
condition:
key: demo_instrument.batv
operator: >
val: 20
inst_ids: ['demo_instrument']]
```

12.6.4 List Channels

With PySDK

```
$ t.streams.list_channels()
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt"  $BASE_URL/v3/streams/channels
```

The response will look something like the following:

```
{
  "message": "Channels found",
  "result": [],
  "status": "success",
  "version": "dev"
}
```

12.6.5 Get Channel Details

With PySDK

```
$ t.streams.get_channel(channel_id='demo.tapis.channel')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt"  $BASE_URL/v3/streams/channels/demo.tapis.channel
```

The response will look something like the following:

```
channel_id: demo.tapis.channel
channel_name: demo.tapis.channel
create_time: 2020-07-21 03:02:51.755215
last_updated: 2020-07-21 03:02:51.755227
permissions:
users: ['testuser6']
status: ACTIVE
template_id: demo_channel_template
triggers_with_actions: [
action:
```

(continues on next page)

(continued from previous page)

```
abaco_base_url: https://api.tacc.utexas.edu
actor_id: XXXX
message: Instrument: demo_instrument exceeded threshold
method: ACTOR
nonces: XXXX-YYYY-ZZZZ
condition:
    key: demo_instrument.batv
    operator: >
    val: 20
    inst_ids: ['demo_instrument']]
```

12.6.6 Update Channels:

With PySDK

```
$ t.streams.update_channel(channel_id="test1", channel_name='demo.wq.channel', template_
↪id="demo_channel_template",triggers_with_actions=[{"inst_ids":[
"Ohio_River_Robert_C_Byrd_Locks"],"condition":{"key":"Ohio_River_Robert_C_Byrd_Locks.temp
↪","operator":">", "val":30},
"action":{"method":"ACTOR","actor_id" :"XXXX","message":"Instrument: Ohio_River_Robert_C_
↪Byrd_Locks  exceeded threshold",
"abaco_base_url":"https://api.tacc.utexas.edu","nonces":"XXXX-YYYY-ZZZZ" }}])
```

With CURL:

```
$ curl -X PUT -H "X-Tapis-Token:$jwt" -H "Content-Type:application/json" $BASE_URL/v3/
↪streams/channels/test1 -d '{"channel_id": "test1","channel_name":"demo.wq.channel",
↪"template_id": "demo_channel_template",
"triggers_with_actions": [{"inst_ids": ["Ohio_River_Robert_C_Byrd_Locks" ],
"condition": {"key": "Ohio_River_Robert_C_Byrd_Locks.temp","operator": ">","val": "40" }␣
↪}]}'
```

The response will look something like the following:

```
channel_id: test1
channel_name: demo.wq.channel
create_time: 2020-08-18 20:51:41.350377
last_updated: 2020-08-18 21:57:42.174860
permissions:
users: ['testuser2']
status: ACTIVE
template_id: demo_channel_template
triggers_with_actions: [
action:
abaco_base_url: https://api.tacc.utexas.edu
actor_id: XXXX
message: Instrument: Ohio_River_Robert_C_Byrd_Locks  exceeded threshold
method: ACTOR
```

(continues on next page)

(continued from previous page)

```
nonces: XXXX-YYYY-ZZZZ
condition:
key: Ohio_River_Robert_C_Byrd_Locks.temp
operator: >
val: 30
inst_ids: ['Ohio_River_Robert_C_Byrd_Locks']]
```

12.6.7 Update Channels Status

With PySDK

```
$ t.streams.update_status(channel_id='demo.tapis.channel', status='INACTIVE')
```

With CURL:

```
$ curl -X POST -H "Content-Type:application/json" -H "X-Tapis-Token:$jwt" -d '{"status":
↪"INACTIVE"}' $BASE_URL/v3/streams/channels/demo.tapis.channel
```

The response will look something like the following:

```
channel_id: demo.tapis.channel
channel_name: demo.tapis.channel
create_time: 2020-07-21 03:02:51.755215
last_updated: 2020-07-22 18:09:19.940080
permissions:
users: ['testuser6']
status: INACTIVE
template_id: demo_channel_template
triggers_with_actions: [
action:
abaco_base_url: https://api.tacc.utexas.edu
actor_id: XXXX
message: Instrument: demo_instrument exceeded threshold
method: ACTOR
nonces: XXXX-YYYY-ZZZZ
condition:
key: demo_instrument.batv
operator: >
val: 90
inst_ids: ['demo_instrument']]
```

12.7 Templates

12.7.1 Create Template

With PySDK

```
$ t.streams.create_template(template_id='test_template_for_tutorial', type='stream',
       script=' var crit lambda \n var channel_id string\n stream\n    |from()\n       ␣
↪.measurement(\'tsdata\')\n        '
              ' .groupBy(\'var\')\n   |alert()\n       '
              ' .id(channel_id +  \' {{ .Name }}/{{ .Group }}/{{.TaskName}}/{{index .
↪Tags \"var\" }}\')\n         .crit(crit)\n    .noRecoveries()\n      '
              '  .message(\'{{.ID}} is {{ .Level}} at time: {{.Time}} as value: {{␣
↪index .Fields \"value\" }} exceeded the threshold\')\n       '
              ' .details(\'\')\n         .post()\n         .endpoint(\'api-alert\')\n  ␣
↪  .captureResponse()\n    |httpOut(\'msg\')', _tapis_debug=True)
```

The response will look something like the following:

```
create_time: 2020-07-22 15:30:58.244391
last_updated: 2020-07-22 15:30:58.244407
permissions:
users: ['testuser6']
script:  var crit lambda
 var channel_id string
 stream
    |from()
        .measurement('tsdata')
         .groupBy('var')
   |alert()
        .id(channel_id +  ' {{ .Name }}/{{ .Group }}/{{.TaskName}}/{{index .Tags "var" }}
↪')
         .crit(crit)
    .noRecoveries()
        .message('{{.ID}} is {{ .Level}} at time: {{.Time}} as value: {{ index .Fields
↪"value" }} exceeded the threshold')
        .details('')
         .post()
         .endpoint('api-alert')
     .captureResponse()
    |httpOut('msg')
template_id: test_template_for_tutorial
type: stream
```

12.7.2 List Templates

With PySDK

```
$ t.streams.list_templates()
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt"  $BASE_URL/v3/streams/templates
```

The response will look something like the following:


```
 {
  "message": "Templates found",
  "result": [],
  "status": "success",
  "version": "dev"
}
```


12.7.3 Get Template Details

With PySDK

```
$ t.streams.get_template(template_id='test_template_for_tutorial')
```

With CURL:

```
$ curl  -H "X-Tapis-Token:$jwt" $BASE_URL/v3/streams/templates/test_template_for_tutorial
```

The response will look something like the following:

```
create_time: 2020-07-22 15:30:58.244391
last_updated: 2020-07-22 15:30:58.244407
permissions:
users: ['testuser6']
script:  var crit lambda
 var channel_id string
 stream
    |from()
        .measurement('tsdata')
         .groupBy('var')
   |alert()
        .id(channel_id +  ' {{ .Name }}/{{ .Group }}/{{.TaskName}}/{{index .Tags "var" }}
↪')
         .crit(crit)
    .noRecoveries()
        .message('{{.ID}} is {{ .Level}} at time: {{.Time}} as value: {{ index .Fields
↪"value" }} exceeded the threshold')
        .details('')
         .post()
         .endpoint('api-alert')
     .captureResponse()
    |httpOut('msg')
template_id: test_template_for_tutorial
type: stream
```

12.7.4 Update Template

With PySDK

```
t.streams.update_template(template_id='test_template_for_tutorial', type='stream',
        script=' var period=5s\n var every=0s\n var crit lambda \n var channel_id string\
↪n stream\n    |from()\n        .measurement(\'tsdata\')\n        '
               ' .groupBy(\'var\')\n   |alert()\n       '
               ' .id(channel_id +  \' {{ .Name }}/{{ .Group }}/{{.TaskName}}/{{index .
↪Tags \"var\" }}\')\n         .crit(crit)\n    .noRecoveries()\n      '
               '  .message(\'{{.ID}} is {{ .Level}} at time: {{.Time}} as value: {{␣
↪index .Fields \"value\" }} exceeded the threshold\')\n       '
               ' .details(\'\')\n         .post()\n         .endpoint(\'api-alert\')\n  ␣
↪  .captureResponse()\n    |httpOut(\'msg\')', _tapis_debug=True)
```

The response will look something like the following:

```
create_time: 2020-08-19 19:48:59.177935
last_updated: 2020-08-19 19:50:00.102827
permissions:
users: ['testuser2']
script:  var period=5s
 var every=0s
 var crit lambda
 var channel_id string
 stream
    |from()
        .measurement('tsdata')
         .groupBy('var')
   |alert()
        .id(channel_id +  ' {{ .Name }}/{{ .Group }}/{{.TaskName}}/{{index .Tags "var" }}
↪')
         .crit(crit)
    .noRecoveries()
        .message('{{.ID}} is {{ .Level}} at time: {{.Time}} as value: {{ index .Fields
↪"value" }} exceeded the threshold')
        .details('')
         .post()
         .endpoint('api-alert')
     .captureResponse()
    |httpOut('msg')
template_id: test_template_update
type: stream
```

12.8 Alerts

12.8.1 List Alerts

With PySDK

```
$ t.streams.list_alerts(channel_id='demo_wq_channel2020-06-19T17_34_46.425419')
```

With CURL:

```
$ curl  -H "X-Tapis-Token:$jwt" $BASE_URL/v3/streams/channels/demo_wq_channel2020-06-
↪19T17_34_46.425419/alerts
```

The response will look something like the following:

```
alerts: [
    actor_id: XXXX
    alert_id: 70fa63b4-c6b1-45a4-91a8-f4e9803ec898
    channel_id: demo_wq_channel2020-06-19T17_34_46.425419
    channel_name: demo.wq.channel
    create_time: 2020-06-19 20:51:44.390887
    execution_id: 7mBGaJbD4q0M1
    message: demo_wq_channel2020-06-19T17_34_46.425419 tsdata/var=11/demo_wq_channel2020-
↪06-19T17_34_46.425419/11 is CRITICAL at time: 2020-06-19 20:51:43.229988 +0000 UTC as␣
↪value: 150 exceeded the threshold,
    actor_id: XXXX
    alert_id: c16ab843-8417-4af0-a06c-ce1e4e7e4816
    channel_id: demo_wq_channel2020-06-19T17_34_46.425419
    channel_name: demo.wq.channel
    create_time: 2020-06-19 20:51:21.138143
    execution_id: ByOkp5W8Jxkqj
    message: demo_wq_channel2020-06-19T17_34_46.425419 tsdata/var=11/demo_wq_channel2020-
↪06-19T17_34_46.425419/11 is CRITICAL at time: 2020-06-19 20:51:20.114319 +0000 UTC as␣
↪value: 150 exceeded the threshold,
    actor_id: XXXX
    alert_id: 4c4b7e70-a034-419b-be8c-2c337803e5d4
    channel_id: demo_wq_channel2020-06-19T17_34_46.425419
    channel_name: demo.wq.channel
    create_time: 2020-06-19 20:51:10.454269
    execution_id: jboJWNqRKAA6V
    message: demo_wq_channel2020-06-19T17_34_46.425419 tsdata/var=11/demo_wq_channel2020-
↪06-19T17_34_46.425419/11 is CRITICAL at time: 2020-06-19 20:51:09.862752 +0000 UTC as␣
↪value: 150 exceeded the threshold]
    num_of_alerts: 3
]
```

12.9 Roles

Streams service uses **roles** to manage permissions on the streams resources. CRUD operations on Streams resources such as Sites, Instruments and Variables can be performed by authorized users having a specific role on the Project. Similarly CRUD operations on Channels and Templates can be done by authorized users having specific roles. Streams service supports three types of roles: *admin*, *manager* and *user*.

Admin has elevated privileges. An *admin* can create, update, or delete any of the Streams resources.

Manager can perform all read and write operations on Streams resources, with an exception of deleting them.

User can only perform read operations on the resources and are not authorized to write or delete the resources.

Table 1 below summarizes the authorized actions with respect to user roles.

Role	Request permitted
admin	GET, PUT, POST, DELETE
manager	GET, PUT, POST
user	GET

When a user creates project, channel or template, an admin role of the form: **streams_projects_$project-oid_admin**, **streams_channels_$channel-oid_admin** or **streams_templates_$template-oid_admin**, respectively is created in the Security Kernel and is assigned to the requesting (JWT) user. Oid is the unique object id generated by the backend MongoDB for each of the Project, Channel or Template.

Admins can further grant roles such as **admin**, **manager** or **user** for other users listed on the project. To perform CRUD operations on Projects, Sites, Instruments and Variables, users must have appropriate role on the Project. To perform CRUD operation on either Channels and Templates, users must have role associated with each of the resources.

12.9.1 List Roles

To get the list of user roles on a project, channel or template, the requesting(JWT) user must provide following three parameters:

1) **resource_type** : project/channel/template
2) **resource_id**: project_id/channel_id/template_id
3) **user**: username for whom roles are to be checked

In order to list the user roles on a resource (Project, Channel, Template) the requesting(JWT) user must have one of the three roles (admin, manager, user) on it.

With PySDK

```
$ t.streams.list_roles(resource_id=<resource_id>, user=<username>,resource_type='project
↪')
$ t.streams.list_roles(resource_id=<resource_id>, user=<username>,resource_type='channel
↪')
$ t.streams.list_roles(resource_id=<resource_id>, user=<username>,resource_type='template
↪')
```

With CURL:

```
$ curl -H "X-Tapis-Token:$jwt" {BASE_URL}/v3/streams/roles?user={userid}&resource_type=
↪{project/channel/template}&resource_id={project_id/channel_id/template_id}
```

The response will look like the following with the Python Client:

```
result: ['admin']
```

There are three possible responses depending on if the requesting(JWT) user and user specified in query parameters are same or different.

Case I: When requesting(JWT) user and user specified in the query parameters are same and both have role on the project/channel/template The result will include all the roles for the user in query parameters for the given resource_id


```
{
 "message": "Roles found",
 "result": [
    "admin"
],
"status": "success",
"version": "dev"
}
```


Case II: When requesting(JWT) user and user specified in the query parameters are different and JWT user does not have any role on the project/channel/template


```
{
   "message": "User not authorized to access roles",
   "result": "",
   "status": "success",
   "version": "dev"
}
```


Case III: When requesting(JWT) user and user specified in the query parameters are different. JWT user has role on the project and user in query parameter does not have role on the project/channel/template


```
{
   "message": "Roles not found",
   "result": "",
   "status": "success",
   "version": "dev"
}
```


12.9.2 Grant Roles

Roles can be granted by Project/Channel/Template *admins* or *managers* so that users can perform CRUD operations on the Streams resources.

Table 2 below shows that *admin* can grant any of the three roles to other users. Same or lower level permissions can be granted by *admins* and *managers*. Self role granting is not permitted.

Managers can only grant *manager* and *user* to other users.

Users do **not** have privileges to grant roles.

- Roles of the requesting(JWT) user are first checked by querying SK.
- If the username provided in the request body is the same as the JWT user, then self role granting is not permitted.
- If the JWT user and user provided in the request body are different, then existing roles for the username provided in the request body are retrieved and if the user already has the role, JWT user user is asking for no action is taken.
- If the role does not exist then JWT user roles are retrieved and compared with the rolename provided in the request body. Role is granted only if the JWT user has **same** or **higher** roles than the role name specified in the request body (*admin* role has highest rank, followed by *manager* and then *user*). Otherwise an error message saying, *User not authorized to grant role* is given in the response.

Role	Grant
admin	admin, manager, user
manager	manager, user
user	cannot grant roles

With PySDK

```
$ t.streams.grant_role(resource_id=<resource_id>, user=<user>,resource_type='project/
↪channel/template',role_name='admin/manager/user')
```

With CURL:

```
$ curl -X POST -H "X-Tapis-Token:$jwt" {BASE_URL}/v3/streams/roles

Request body: { "user":"user_id",
                "resource_type":"project/channel/template",
                "resource_id":"project_uuid/channel_id/template_id",
                "role_name": "admin/manager/user"
               }
```

The response will vary based on following cases:

Case I: If the username provided in the request body is the same as the JWT user, then self role granting is not permitted.

With PySDK

```
$ t.streams.grant_role(resource_id='test_proj', user='testuser2',resource_type='project',
↪role_name='manager')
```

```
{
  "message": "Cannot grant role for self",
  "metadata": {},
  "result": "",
  "status": "error",
  "version": "dev"}
```

Case II: If the JWT user and username provided in the request body are different, then existing roles for the username provided in the request body are retrieved and if the user already has the role JWT user user is asking for, no action is taken.

With PySDK

```
$ t.streams.grant_role(resource_id='test_proj', user='testuser6',resource_type='project',
↪role_name='manager')
```

```
{
    "message": "Role already exists",
    "metadata": {},
    "result": [
        "manager"
    ],
    "status": "success",
    "version": "dev"
}
```

Case III: If the role does not exist then JWT user roles are retrieved and compared with the rolename provided in the request body. Role is granted only if the JWT user has **same** or **higher*** roles than the role name specified in the request body. Otherwise an error message saying, "User not authorized to grant role" is given in the response.

For example testuser4 has ***manager** role on the project and the request is to grant testuser5 **admin*** role, then the request will not be fulfilled.

```
$ t.streams.grant_role(resource_id='test_proj', user='testuser5',resource_type='project',
↪role_name='admin')
```


```
{
   "message": "Role admin cannot be granted",
   "result": "",
   "status": "error",
   "version": "dev"
}
```


If the requesting (JWT) user only has a **user*** role, then no role can be granted to other users, and the response will be following


```
{
   "message": "Role manager cannot be granted",
   "result": "",
   "status": "error",
   "version": "dev"
}
```


Case IV: If the requesting (JWT) user has no role on the project/channel/template, then the user is not authorized to grant any roles


```
{
   "message": "User not authorized to grant role",
   "result": "",
   "status": "error",
   "version": "dev"
}
```


12.9.3 Revoke Roles

Users in **admin** role are capable of revoking any of the three roles: **admin**, **manager** and **user** for other users. Self role revoking is not permitted. Users in *manager* and *user* role are not capable of revoking roles.

Role	Revoke
admin	admin, manager, user
manager	Cannot revoke role
user	Cannot revoke role

With PySDK

```
$ permitted_client.streams.revoke_role(resource_id='test_proj', user='testuser6',
↪resource_type='project',role_name='manager')
```

With CURL:

```
$ curl -X POST -H "X-Tapis-Token:$jwt" {BASE_URL}/v3/streams/revokeRole

Request body: { "user":"user_id",
                "resource_type":"project/channel/template",
                "resource_id":"project_uuid/channel_id/template_id",
                "role_name": "admin/manager/user"
               }
```

The response will be following:

```
 {
  "message": "Role manager successfully deleted for user testuser6",
  "metadata": [],
  "result": "",
  "status": "success",
  "version": "dev"
}
```

Responses will vary based on following cases:

Case I: If the requesting (JWT) user has a **manager** or **user** role

```
{
   "message": "User not authorized to revoke role",
   "result": "",
   "status": "error",
   "version": "dev"
}
```

Case II: If the JWT user is trying to revoke self role

```
{
   "message": "Cannot delete role for self",
   "result": "",
   "status": "error",
   "version": "dev"
}
```

CHAPTER

THIRTEEN

WORKFLOWS

The Workflows Service is an API and Workflow Engine with which Tapis users can create and execute their research computing workflows.

13.1 Overview

Before getting into the details about how to create and run a workflow, here are some important concepts you must understand in order to properly use *Tapis Workflows*

13.1.1 Important Concepts

- **Group**: A *group* is a collection of Tapis users that own or have access to workflow resources. In order to create your first workflow, you must first belong to, or create your own *group*.
- **Pipeline**: A *pipeline* is a collection of tasks and a set of rules governing how those tasks are to be executed.
- **Archive**: An *archive* is the storage medium for the results created during a pipeline run. By default, the results produced by each task are deleted at the end of a pipeline run.
- **Task**: *Tasks* are discrete units of work performed during the execution of a workflow. They can be represented as nodes on a directed acyclic graph (DAG), with the order of their execution determined by their dependencies, and where all tasks without dependencies are executed first. There are different types of *tasks* that users can leverage to perform diffent types of work. These are called task primitives and they will be discussed in detail later:

Type	Example	Supported
image_build	Builds Docker and Singularity images from recipe files and pushes the to repositories	yes
request	Sends requests using various protocols to resources external to the workflow (Only HTTP protocol and GET currently fully supported)	partial
tapis_job	Submits a *Tapis job*	yes
tapis_acto	Executes an *Tapis actor*	no
container_run	Runs a container based on the provided image and tag	no
function	Runs user-defined code in the language and runtime of their choice	yes

Note

Security Note

In order to create and run workflows that are automated and reproducible, the Workflow Executor must sometimes be furnished with secrets(passwords, access keys, access secrets, ssh keys, etc) that enable it access restricted resources.

To ensure the safe storage and retrieval of this sensitive data, the Workflows service integrates with *Tapis SK*, a built-in secrets management service backed by *Hashicorp Vault*.

It is also important to note that, when a user creates a task that accesses some restricted resource, the Workflow Executor will execute that task on behalf of that user with the credentials that they provided for every run of the pipeline. If those credentials expire, or the user has their access revoked for those resources, the pipeline run will fail on that task.

13.2 Quick start

Warning

This Quick Start is deprecated. There is a new one comming soon!

We will be creating a pipeline that

- pulls code from a private Github repository
- builds an image from a Dockerfile located in that source code
- then pushes the resultant image to a Dockerhub image registry

In the examples below we assume you are using the TACC tenant with a base URL of `tacc.tapis.io` and that you have authenticated using PySDK or obtained an authorization token and stored it in the environment variable JWT, or perhaps both.

13.2.1 Summary of Steps

1. Create a *Group*
2. Create an *Archive* to which the results of the pipeline run will be persisted
3. Create the *Pipeline* and its *Tasks* which act as instructions to the workflow executor

13.2.2 Creating a Group

Create a local file named `group.json` with json similar to the following:

```
{
  "id": "<group_id>",
  "users": [
      {
        "username":"<user_id>",
        "is_admin": true
      }
```

(continues on next page)

(continued from previous page)

```
  ]
}
```

Note

You do not need to add your own Tapis id to the users list. The owner of the Group is added by default.

Replace <*group_id*> with your desired group id and <*user_id*> in the user objects with the tapis user ids of the other users that you want to grant access to this group's workflow resources.

Submit the definition.

13.2.3 Creating Identities

We will be creating 2 identity mappings. One for Github and one for Dockerhub. After creating the identities, we will need to retrieve the UUIDs of the newly created identities. You can do this in a separate call, or simple grab the UUID from the url in the result after each operation.

Warning

Do **NOT** commit these files to source control!

Create the first file named `github-identity.json` with the following json:

```
{
  "type": "github",
  "name": "my-github-identity",
  "description": "My github identity",
  "credentials": {
    "username": "<github_username>",
    "personal_access_token": "<github_personal_access_token>"
  }
}
```

Then submit the definition

Create the second file named `dockerhub-identity.json` with the following json

```
{
    "type": "dockerhub",
    "name": "my-dockerhub-identity",
    "description": "My Dockerhub identity",
    "credentials": {
      "username": "<docerkhub_username>",
      "token": "<dockerhub_access_token>"
    }
}
```

Then submit the definition

13.2.4 Creating an Archive

In this step, we create the Archive. The results of the pipeline run will be persisted to the archive.

> **Note**
>
> This step requires that you have "**MODIFY**" permissions on some Tapis System. If you do not have access to one, you can create it following the instruction in the "Systems" section.

Create a local file named `archive.json` with json similar to the following:

```
{
  "id": "my-sample-archive",
  "type": "system",
  "system_id": "<your-tapis-system-id>",
  "archive_dir": "/workflows/archive/"
}
```

> **Note**
>
> The archive_dir is relative to your system's rootDir. You can change this value to whatever you like.

13.2.5 Creating a Pipeline

In this step, we define the pipeline. There are many more properties that can be defined at both the pipeline and task level, but for simplicity, we will be leaving them out.

Create a local file named `pipeline.json` with json similar to the following:

```
{
  "id": "my-sample-workflow",
  "archives": [ "<archive_id>" ]
  "tasks": [
    {
      "id": "my-image-build",
      "type": "image_build",
      "builder": "kaniko",
      "context": {
          "branch": "main",
          "build_file_path": "<path/to>/Dockerfile",
          "sub_path": null,
          "type": "github",
          "url": "<account>/<repo>",
          "visibility": "private",
          "identity_uuid": "<github_identity_uuid>"
      },
      "destination": {
          "tag": "<some_image_tag>",
          "type": "dockerhub",
          "url": "<account>/<registry>",
```

(continues on next page)

(continued from previous page)

```
            "identity_uuid": "<dockerhub_identity_uuid>"
        }
      }
    ]
}
```

Go through the definition above and replace all of the placeholders with the correct values.

Now it's time to run the pipeline.

13.2.6 Run a Pipeline (runPipeline)

After the pipeline has finished running, take a look in your Dockerhub image repository and you will find your newly pushed image.

If you SSH into the *Tapis System* that you selected as your archive, you will also find that you have some new directories and files in your **rootDir**;

`/workflows/archive/<UUID of the pipeline run>/my-image-build/output/.stdout`.

If you want to find the output for any task for a given pipeline run, simply navigate to the archive directory, `cd` into directory with the pipline run UUID, then `cd` into the directory with that task's name. Inside the `output/` directory, you will find all of the data created by that task.

13.3 Groups

A *group* is a collection of Tapis users that own or have access to workflow resources. In order to create your first workflow, you must first belong to, or create a *group*. A group and all of its resources are tied to a particular tenant which is resolved from the url of the request to create the group.

Groups have an *id* which must be unique within the tenant to which it belongs. Groups also have *users*. This is a simple list of user objects where `user.username` is the Tapis username/id and `is_admin` is a Boolean. Users with **admin** permissions i.e. `is_admin == true` are able to add and remove users from a group. Only the **owner** of a group may add or delete other admin users.

13.3.1 Group Attributes Table

Attribute	Type	Example	Notes
id	String	my.group	• Must be unique within the tenant.
owner	String	someuser	• Cannot be removed from group unless ownership is transferred.
tenant_id	String	tacc	• Automatically set at create-time. Determined from the url of the request. Does not need to be specified.
uuid	String(UUIDv4)	e48ada7a-56b4-4d48-974c-7574d51a8789	• Automatically set at create-time
users	Array(Users)		

GroupUser Attributes Table

Attribute	Type	Example	Notes
username	String	jsmith	• Must be unique within the group.
is_admin	Boolean	True	• Must be an admin or group owner to add a user as admin
uuid	String(UUIDv4)	e48ada7a-56b4-4d48-974c-7574d51a8789	• Automatically set at create-time
group	String(UUIDv4)	e48ada7a-56b4-4d48-974c-7574d51a8789	• Automatically set at create-time

13.3.2 Retrieval (getGroup)

Retrieve details for a specific group

```
{
  "success": true,
```

(continues on next page)

(continued from previous page)


```
  "status": 200,
  "message": "Success",
  "result": {
    "id": "my.group",
    "owner": "someuser",
    "tenant_id": "tacc",
    "uuid": "f60fdf8a-4ceb-4273-b49f-4c0dd94111c3",
    "users": [
      {
        "group": "f60fdf8a-4ceb-4273-b49f-4c0dd94111c3",
        "username": "someuser",
        "is_admin": true,
        "uuid": "c6b7acfd-da4b-4a1d-acbd-adbfa6aa4057"
      },
      {
        "group": "f60fdf8a-4ceb-4273-b49f-4c0dd94111c3",
        "username": "anotheruser",
        "is_admin": false,
        "uuid": "d6ca476a-2c19-4168-8054-264bcaaa70e7"
      }
    ]
  }
}
```


13.3.3 Deletion

Groups may only be deleted by the group **owner**. Upon deletion of a group, every workflow object owned by the group will also be deleted; pipelines, tasks, archives, etc.

13.4 Pipelines

A *pipeline* is a collection of tasks and a set of rules governing how those tasks are to be executed.

13.4.1 Pipeline Attributes Table

Attribute	Type	Example	Notes
id	String	my.pipeline	• Must be unique within the group
uuid	String(UUIDv4)	e48ada7a-56b4-4d48-974c-7574d51a8789	• Globally unique identifier for the pipeline
owner	String	jsmith	• The only user that can delete the pipeline
group	String(UUIDv4)	e48ada7a-56b4-4d48-974c-7574d51a8789	• The uuid of the group that owns this pipeline
last_run	String(UUIDv4)	e48ada7a-56b4-4d48-974c-7574d51a8789	• The UUID of the previous pipeline run
current_run	String(UUIDv4)	e48ada7a-56b4-4d48-974c-7574d51a8789	• The UUID of the current running pipeline
tasks	Array[Task]	**See the Task section for the Task object**	
execution_profile	Object	**See table below**	
env	Object	**See Pipeline Envrionment section below**	• Environment variables to be used by tasks in a pipeline run

Execution Profile Attributes Table

Overrides the default behavior of the Workflow Executor regarding task retries, backoff algorithm, max lifetime of the pipeline, etc. All Execution Profile properties of the pipeline are inherited by the tasks that belong to the pipeline unless otherwise specified in the task definition.

Attribute	Type	Example	Notes
max_retries	Int	0, 3, 10	• The number of times that the Workflow Executor will try to rerun the task if it fails. Defualts to **0**
invocation_mode	Enum	async, sync	• Default is "async". When "async" is selected, all tasks will be executed concurrently. Currently, async is the only support option
retry_policy	Enum	exponential_backoff	• Dictates which policy to employ when restarting tasks. Default(and only supported option) is exponential_backoff
max_exec_time	Int	60, 3600, 10800	• The maximum amount of time in seconds that a pipeline(or task) is permitted to run. As soon as the sum of all task runs equals this limit, the pipeline(or task) is terminated. Defaults to 3600 seconds(1 hour)

Pipeline Envrionment

The Pipeline Envrionment (the *env* property of a pipeline definition) is a mechanism for exposing static global data to task inputs. The Pipeline Envrionment is an object in which the keys are the name of the variables and the value is either a scalar data (string, number, etc) or an object with a *value_from* property which references data from a source external to the the workflow (ex. Tapis Security Kernel).

```
{
  "env": {
    "TAPIS_SYSTEM_ID": "tapisv3-exec",
    "MANIFEST_FILES_DIR": "/path/to/manifest/files",
    "TAPIS_USERNAME": "someusername",
    "TAPIS_PASSWORD": {
```

(continues on next page)

(continued from previous page)

```
      "value_from": {
        "tapis-security-kernel": "some+sk+id"
      }
    }
  }
}
```

13.4.2 Retrieval

Retrieve details for a specific pipeline

The response should look similar to the following:

```
{
   "success": true,
   "status": 200,
   "message": "Success",
   "result": {
     "id": "some_pipeline_id",
     "group": "c487c25f-6c6e-457d-a781-85120df9f10b",
     "invocation_mode": "async",
     "max_exec_time": 10800,
     "max_retries": 0,
     "owner": "testuser2",
     "retry_policy": "exponential_backoff",
     "uuid": "e48ada7a-56b4-4d48-974c-7574d51a8789",
     "current_run": null,
     "last_run": null,
     "tasks": [
       {
         "id": "build",
         "cache": false,
         "depends_on": [],
         "description": "Build an image from a repository and push it to an image␣
↪registry",
         "input": null,
         "invocation_mode": "async",
         "max_exec_time": 3600,
         "max_retries": 0,
         "output": null,
         "pipeline": "e48ada7a-56b4-4d48-974c-7574d51a8789",
         "poll": null,
         "retry_policy": "exponential_backoff",
         "type": "image_build",
         "uuid": "e442b5df-8a9e-4d55-b4da-c51b7241a79f",
         "builder": "singularity",
         "context": "5bd771ab-8df5-43cd-a059-fbaa2323841b",
         "destination": "b34d1439-d2c9-4238-ab74-13b5fd7f3b1f",
         "auth": null,
         "data": null,
         "headers": null,
         "http_method": null,
```

(continues on next page)

(continued from previous page)

```
        "protocol": null,
        "query_params": null,
        "url": null,
        "image": null,
        "tapis_job_def": null,
        "tapis_actor_id": null
      }
    ]
  }
}
```

13.4.3 Deletion

Deleting a Pipeline will delete all of it's tasks. This operation can only be performed the owner of the pipeline.

13.5 Tasks

Tasks are discrete units of work performed during the execution of a workflow. They can be represented as nodes on a directed acyclic graph (DAG), with the order of their execution determined by their dependencies, and where all tasks without dependencies are executed first.

Tasks can be defined when creating pipeline, or after the pipelines creation. Every task must have an `id` that is unique within the pipeline.

Task may also specify their dependencies in a number of ways. The first way is by declaring the dependency explicity in the `depends_on` property. This is an *Array* of **TaskDependency** objects which only have 2 attributes. The `id`, which is the id of the task that it depends on, and the `can_fail` attribute(Boolean) which specifies whether the dependent task is allowed to run if that *TaskDependency* fails.

13.5.1 Task Attributes Table

This table contains all of the properties that are shared by all tasks. Different types of tasks will have other unique properties in addition to all of the properties in the table below.

Attribute	Type	Example	Notes
id	String	my-task, my.task, my_task	• Must be unique within the pipeline that it belongs to
type	Enum	image_build, tapis_job, tapis_actor, request, container_run, function	• Only *image_build* is fully supported. Partial support for the *request* type exists; HTTP GET requests only
depends_on	Array[TaskDependency]	**see table below**	• Explicitly declares this task's dependencies. Task with the specified `id` must exist or the pipeline will not run.
execution_profile	Object	**see execution profile table in the pipeline section**	• Inherits the `execution_profile` set in the pipeline definition.
description	String	My task description	
input	Object	**see input table below**	
output	Object		
pipeline	String(UUIDv4)	5bd771ab-8df5-43cd-a059-fbaa2323841b	• UUID of the pipeline that this task is a part of
uuid	String(UUIDv4)	5bd771ab-8df5-43cd-a059-fbaa2323841b	• A globally unique identifier for this task

13.5.2 Input Object

Input is an object in which the keys are the id of the input(String) and the value is an object that defines the type of data, and either the literal value of the data via the *data* property or a reference to where that data can be found via the *data_from*.

Key-Val	Type	Example	Notes
key	String	TAPIS_USERNAME, PATH_TO_FILES, _SOME_INPUT, etc.	• Must be an alpha numeric string that conforms to the following regex pattern: `^[_]?[a-zA-Z]+[a-zA-Z0-9_]*`
value	InputValueObject	**see table below**	

13.5.3 InputValue Object

Attributes	Type	Example	Notes
type	Enum	string, number, tapis_file_input, tapis_file_input_array	
value	String, Int, Float	123, "hello world", 1.23	
value_from	ValueFrom Object	**see table below**	• Used to reference data from a workflow definition (env, params, task_output)

13.5.4 ValueFrom Object

The Value From object is a key value pair in which the *key* is an enum denoting the source object of the value(pipeline envrionment, workflow submission parmeteters, and task outputs) and the *value* is the key on that source where the value data can be found

Key-Val	Type	Example	Notes
key	Enum	env, params, task_output	
value	String or TaskOutput Object(**see table below**)	TAPIS_USERNAME, PATH_TO_FILES	

13.5.5 TaskOutput Object

Attributes	Type	Example	Notes
task_id	String	task_1, my-task	• The id of a previously run task
output_id	String	123, "hello world", 1.23	• The id of the output for the specified task

13.5.6 Example Task Definition with Inputs

Note

The example below is in yaml format for readability, but the Tapis Workflows API only accepts JSON

```yaml
tasks:
- id: generate-manifests-and-process-next
  type: function
  description: |
    Python script that creates manifest files and outputs a
    tapis file inputs array to stdout
  runtime: python3.9
  installer: pip # poetry
  command: ""
  packages:
    - tapipy==1.2.20
  code: "<base64 encoded user-defined code here>"
  input:
    TAPIS_USERNAME:
      type: string
      value_from:
        env: TAPIS_USERNAME
    TAPIS_PASSWORD:
      type: string
      value_from:
        env: TAPIS_PASSWORD
    TAPIS_SYSTEM_ID:
      type: string
      value: "some-system-id"
    TARGET_DIR:
      type: string
      value: target/dir/for/file-input-array
    INBOX:
      type: string
      value: "/scratch/08294/jplneid/tacc_ep/INBOX"
    MANIFESTS:
      type: string
      value: "/scratch/08294/jplneid/tacc_ep/MANIFESTS"
    TAPIS_BASE_URL:
      type: string
      value_from:
        params: base_url
  output:
    fileInputArrays:
      type: "tapis_file_input_arrays"
    manifestFilePath:
      type: "string"
```

13.5.7 Task Types

There are different types of *tasks* types users can leverage to perform diffent types of work. These are called task *types* or *primitives*. Task types include the **image_build** type, the **request** type, the **tapis_job** type, the **tapis_actor** type, the **container_run** type, and the **function** task.

When defining tasks on a pipeline, the **type** must be present in the task definition along with all other attributes specific to the task type.

Back to tasks

13.5.8 Retrieval

Retrieve details for a specific task in a pipeline

The response should look similar to the following:


```
{
   "success": true,
   "status": 200,
   "message": "Success",
   "result": {
     "id": "build",
     "cache": false,
     "depends_on": [],
     "description": "Build an image from a repository and push it to an image registry",
     "input": null,
     "invocation_mode": "async",
     "max_exec_time": 3600,
     "max_retries": 0,
     "output": null,
     "pipeline": "ececc546-3ee0-437e-ae50-5882ec03356a",
     "poll": null,
     "retry_policy": "exponential_backoff",
     "type": "image_build",
     "uuid": "01eac121-19bf-4d8e-957e-faa27bdaa1f8",
     "builder": "singularity",
     "context": "ea58c3ef-7175-41b0-9671-e50700a33c77",
     "destination": "6eac73da-5799-4e74-957c-03b5cee97149",
     "auth": null,
     "data": null,
     "headers": null,
     "http_method": null,
     "protocol": null,
     "query_params": null,
     "url": null,
     "image": null,
     "tapis_job_def": null,
     "tapis_actor_id": null
   }
 }
```

13.5.9 Deletion

Deleting a task can only be done by a pipeline administrator. If any tasks depend on the deleted task, the pipeline will fail when run

---

13.6 Archives

Archives are the storage mechanisms for pipeline results. Without an archive, the results produced by each task will be permanently deleted at the end of each pipeline run. By default, archiving occurs at the end of a pipeline run when all tasks have reached a terminal state.

13.6.1 Archive Attributes Table

This table contains all of the properties that are shared by all archives. Different types of archives will have other unique properties in addition to all of the properties in the table below.

Attribute	Type	Example	Notes
id	String	my-task, my.task, my_task	• Must be unique within the group that it belongs to
archive_dir	String	path/to/archive/dir	• Relative to either the "root directory" of the archive's file system
type	Enum	system, S3	

13.6.2 Archive Types

Back to archives

CHAPTER

FOURTEEN

NOTIFICATIONS

The Notifications service supports the publish-subscribe model for delivering information to intereseted parties. Currently only a Tapis service may directly create subscriptions and post events through this service. Users may subscribe to job related events by creating subscriptions through the Jobs service. For more information on creating subscriptions through the Jobs service, please see Job Subscriptions

Warning

Currently subscriptions for notifications may only be managed through the Tapis Jobs service.

14.1 Overview

In Tapis, a notification *event* represents an occurrence that may be of interest to other parties. The events are delivered asynchronously using a publish-subscribe model. Once an interested party has created a *subscription*, matching *events* are delivered to the interested party as part of a *notification* object. Deliveries are made via webhook or email.

Currently only services may directly create subscriptions and publish events.

The model for a notification event is based on the CloudEvent specification version 1.0. For more information about CloudEvents and the specification, please see https://cloudevents.io and https://github.com/cloudevents/spec.

Please note that although currently the Notifications service is only accessed by users through the Jobs service, this document discusses the general design and details of the service in order to provide information for future planned development.

14.2 Event Model

An event contains the attributes listed below. The attributes *source*, *type* and *timestamp* are required. Note that some attributes are maintained by Tapis, they are present when the event is part of a delivered notification. The attributes maintained by Tapis may be present when publishing an event, but they will be ignored.

Event attributes:

source
: Context in which the event happened. For example, for a job related event originating from Tapis at the primary TACC site, the source would be `https://tapis.io/jobs`.

type
: Type of event. Used for routing notifications. A series of 3 fields separated by the dot character. The first field is the service name, the second field is the category and the third field is the detail. For example, when a job transitions to the FINISHED state, the type is `jobs.JOB_NEW_STATUS.FINISHED`.

subject
Subject of event in context of service. Examples: job Id, system Id, file path, role name, etc.

timestamp
When the event happened.

data
Optional. Additional information associated with the event. For example, data specific to the service associated with the event.

seriesId
Optional. Id to be used for grouping events from the same tenant, source and subject. In a series, event order is preserved when sending out notifications. For example, the Jobs service (the source) sends out events with the job UUID as the subject and sets the seriesId to the job UUID. That way, a subscription can be created to follow (in order) all events of various types related to the job.

deleteSubscriptionsMatchingSubject
Boolean indicating that all subscriptions whose *subjectFilter* matches the *subject* of the event should be deleted once all notifications are delivered.

tenant
Tapis tenant associated with the event.

uuid
Tapis generated unique identifier for the event.

user
Tapis user associated with the event.

received
Tapis generated timestamp for when the event was received by the Notifications service.

seriesSeqCount
Tapis generated counter for seriesId. Can be used by notification receivers to track expected order. Notifications for events will be sent in order but may not be received in order.

Note that events are not persisted by the front end api service process. When events are received they are sent to a message broker for asynchronous processing. The back end process will persist events temporarily in order to support recovery.

14.2.1 Event Type

An event type represents a channel through which users and services receive events. Services and users create subscriptions with an event type filter in order to select the events delivered to them. The identifier for each event type must have three parts in the format **<service>.<category>.<detail>**. For example *jobs.JOB_NEW_STATUS.PENDING*, *apps.APP.UPDATE* or *files.OBJECT.DELETE*.

14.3 Subscription

A service can create a subscription for an event type in order to allow users and services to receive events. Two delivery methods are supported, WEBHOOK and EMAIL.

At a high level, a subscription contains the following information:

name
Optional short descriptive name. *owner+name* must be unique. Composed of alphanumeric characters and the following special characters: `-._~`. If not provided, the service will create one.

owner
A specific user set at create time. Default is *${apiUserId}*.

description
An optional more verbose description.

typeFilter
Filter to use when matching events. Matches against event type. Has three parts for matching <*service*>.<*category*>.<*detail*>. Each field may be a specific type or the wildcard character *.

subjectFilter
Filter to use when matching events. Matches against event subject. This may be a specific subject such as a job Id or the wildcard character, *.

deliveryTargets
List of targets to be used when delivering an event. Each target includes delivery method (EMAIL or WEBHOOK) and delivery address.

ttlMinutes
Time to live in minutes. Specified when subscription created. Default is one week from creation. A TTL of 0 or less indicates no expiration. May be updated through an API call. Attribute *expiry* is recomputed when this attribute is updated.

expiry
Time at which the subscription expires. Maintained by the service. Computed at create time and recomputed when attribute *ttlMinutes* is updated.

The attributes *typeFilter*, *subjectFilter* and *deliveryTargets* are required.

14.3.1 Subscription Name

For each owner the name must be unique and can be composed of alphanumeric characters and the following special characters: `-._~`. If the attribute *name* is not provided, then the service will generate one using the template:

```
<jwtUser>~<owner>~<oboTenant>~<subjectFilter>~<random4>
```

For example:

```
jobs~testuser1~dev~jobs.JOB_NEW_STATUS.ALL~m4Kx
```

Note that when constructing the name:

- *subjectFilter* will be truncated to 40 characters
- If *subjectFilter* is the wildcard character *, it is replaced with the string `ALL` when constructing the name.
- The last 4 characters are generated at random from the set of alphanumeric characters including upper case, lower case and digits.

14.3.2 Delivery Target

Each subscription will contain a list of delivery targets for use in delivering events. The list must contain at least one item. WEBHOOK and EMAIL deliveries are supported.

A delivery target contains the following information:

- *deliveryMethod* - The type of delivery method: WEBHOOK, EMAIL
- *deliveryAddress* - URL for WEBHOOK or email address for EMAIL

14.4 Notification Model

A notification is an object encapsulating the information sent to a delivery target. It contains the following:

- *uuid* - Unique identifier for the notification.
- *event* - All information contained in the event. See above under the section *Event Model*.
- *eventUuid* - Unique identifier for the event.
- *tenant* - tenant associated with the notification.
- *subscriptionName* - Name of subscription associated with the notification.
- *deliveryTarget* - the delivery target
- *created* Timestamp for when the notification was created.

Example of a notification sent to a webhook:


```
{
  "uuid": "30d70395-d5e9-43a4-ae90-2306b6bb00d6",
  "tenant": "admin",
  "subscriptionName": "4d0abbce-5cec-4d6e-8065-cdc5b2777389",
  "eventUuid": "50cfb971-c4b3-4d33-89c3-2b0f56f16e19",
  "event": {
    "source": "notifications",
    "type": "notifications.test.begin",
    "subject": "4d0abbce-5cec-4d6e-8065-cdc5b2777389",
    "data": null,
    "seriesId": "4d0abbce-5cec-4d6e-8065-cdc5b2777389",
    "timestamp": "2023-09-15T14:47:50.287792699Z",
    "deleteSubscriptionsMatchingSubject": false,
    "tenant": "admin",
    "user": "notifications",
    "received": "2023-09-15T14:47:51.000",
    "uuid": "50cfb971-c4b3-4d33-89c3-2b0f56f16e19",
    "seriesSeqCount": 4
  },
  "deliveryTarget": {
    "deliveryMethod": "WEBHOOK",
    "deliveryAddress": "https://admin.develop.tapis.io/v3/notifications/test/callback/
↪4d0abbce-5cec-4d6e-8065-cdc5b2777389/"
  },
  "created": "2023-09-15T14:47:50.315188203Z"
}
```


Example of a notification sent to an email address:


```
{
  "uuid": "befe2475-58ad-4a5c-bcf2-593f04e49a20",
  "tenant": "dev",
  "subscriptionName": "jobs~testuser2~dev~ef9004c3-09d5-41d5-acd3-be7c9fd3daf6-007~cxh2",
  "eventUuid": "1d16202d-2248-4690-bcc9-a0134a4089cd",
  "event": {
    "source": "https://tapis.io/jobs",
    "type": "jobs.JOB_NEW_STATUS.FINISHED",
```


(continues on next page)

(continued from previous page)


```
    "subject": "ef9004c3-09d5-41d5-acd3-be7c9fd3daf6-007",
    "data": "{\"newJobStatus\":\"FINISHED\",\"oldJobStatus\":\"ARCHIVING\",\
↪"blockedCount\":0,\"remoteJobId\":\
↪"35299a7d78f1591e395fdcec9dc6b1f3606be9f56f38453129b6ccc383ed9759\",\"remoteJobId2\
↪":null,\"remoteOutcome\":\"FINISHED\",\"remoteResultInfo\":\"0\",\"remoteQueue\":null,\
↪"remoteSubmitted\":\"2023-09-15T15:11:18.354731067Z\",\"remoteStarted\":null,\
↪"remoteEnded\":null,\"jobName\":\"Tapis V3 smoketest job\",\"jobUuid\":\"ef9004c3-09d5-
↪41d5-acd3-be7c9fd3daf6-007\",\"jobOwner\":\"testuser2\",\"message\":\"The job has␣
↪transitioned to a new status: FINISHED. The previous job status was ARCHIVING.\"}",
    "seriesId": "ef9004c3-09d5-41d5-acd3-be7c9fd3daf6-007",
    "timestamp": "2023-09-15T15:11:23.947827477Z",
    "deleteSubscriptionsMatchingSubject": true,
    "tenant": "dev",
    "user": "jobs",
    "received": "2023-09-15T14:47:51.000",
    "uuid": "1d16202d-2248-4690-bcc9-a0134a4089cd",
    "seriesSeqCount": -1
  },
  "deliveryTarget": {
    "deliveryMethod": "EMAIL",
    "deliveryAddress": "me@example.com"
  },
  "created": "2023-09-15T15:11:23.965413696Z"
}
```


14.5 Notification Delivery

14.5.1 Background

When events are published to the Notifications front end api service, they are initially placed on a message broker queue to be picked up asynchronously by a back end worker process known as the dispatch service. Currently RabbitMQ is used as the message broker.

The dispatch service reads events from the queue and assigns them to workers known as *delivery bucket managers*. Delivery bucket managers are threads that receive their assigned events from in-memory queues. The dispatch service assigns events to a bucket manager by taking a hash of the event *tenant*, *source*, *subject* and *seriesId*. The hash allows for distributing work among the bucket managers while ensuring that for a given *seriesId* the same bucket manager will process that series of events. This is how the service ensures that notifications for events in a series are sent out in order.

When a bucket manager worker receives an event to process, it first finds all matching subscriptions by querying a database. As discussed above, the matching is based on the *typeFilter* and *subjectFilter* defined in a subscription.

For each delivery target in each matching subscription, the worker creates a Notification object and persists it to a database. By persisting to a database we are able to support recovery and retries. The worker then begins the process of delivering the notifications.

14.5.2 Configuring for EMAIL Delivery

Supporting delivery by EMAIL involves configuring the Tapis Notifications service to use an SMTP relay. This must be done by the Tapis systems administrator. Parameters for the relay are set as environment variables to be picked up by the dispatcher service when it is started during a deployment. For more information on deployer configuration please see Notifications_Email_Config.

Please note that deployer currently only supports template variables for TAPIS_MAIL_PROVIDER, TAPIS_SMTP_HOST and TAPIS_SMTP_PORT. Other environment variables must be set manually in the deployment.

The environment variables used to configure email delivery are:

TAPIS_MAIL_PROVIDER
: Optional. Supported values: SMTP, LOG, NONE. Default is LOG. This should typically be set to SMTP. Setting to LOG results in the dispatcher generating a log message showing the email information. Setting to NONE results in delivery being a NO-OP.

TAPIS_SMTP_HOST
: Required if provider is SMTP. Host to use as relay when sending email via SMTP.

TAPIS_SMTP_PORT
: Optional. Port used when sending email using SMTP. Default is 25.

TAPIS_SMTP_FROM_NAME
: Optional. Name for the email *From:* field. Default value is *Tapis Notifications Service*.

TAPIS_SMTP_FROM_ADDRESS
: Optional. Address for the email *From:* field. Default value is *no-reply@nowhere.com*.

TAPIS_SMTP_AUTH
: Optional. Boolean indicating if SMTP server requires a username and password. Default is *false*.

TAPIS_SMTP_USER
: Required if TAPIS_SMTP_AUTH is *true*.

TAPIS_SMTP_PASSWORD
: Required if TAPIS_SMTP_AUTH is *true*.

14.5.3 EMAIL Delivery

When the notification delivery method is of type EMAIL, the dispatch worker will send an email using SMTP.

The `To:` field for the email will be the notification delivery address.

The `From:` field for the email will depend on the configuration parameters, as discussed above in the section *Configuring for EMAIL Delivery*. By default this will be:

```
Tapis Notifications Service <no-reply@nowhere.com>
```

The `Subject:` of the email will have the following format:

```
Tapis v3 notification. Event type: <event_type> subject: <subject>
```

If the event has no *subject* then the email subject will not have the subject portion.

An example email subject for the case where the event contains a *subject* attribute:

```
Tapis v3 notification. Event type: jobs.JOB_NEW_STATUS.FINISHED subject: 1451b0ef-c057-
↪4177-acd5-51a4901acb07-007
```

The body of the email will contain the notification data as json. An example may be found above under the section *Notification Model*.

14.5.4 WEBHOOK Delivery

When the notification delivery method is of type WEBHOOK, the dispatch worker will deliver the notification using an HTTP POST request. The media type for the request will be *application/json* and the following header will be added: `User-Agent: Tapis/v3`.

The request body will be a json structure with the notification information. An example may be found above under the section *Notification Model*.

14.6 Tables

14.6.1 Subscription Attributes

Attribute	Type	Example	Notes
tenant	String	designsafe	• Name of the tenant associated with the subscription. • *tenant* + *owner* + *name* must be unique. • Determined by the service at creation time.
name	String	my-email-ntf-1	• Optional short descriptive name. • *tenant* + *owner* + *name* must be unique. • Allowed characters: Alphanumeric [0-9a-zA-Z] and `-._~`. • If not provided the service will create one.
owner	String	jdoe	• username of *owner*. • Variable references: *${apiUserId}*. Resolved at create time. • By default this is the resolved value for *${apiUserId}*.
description	String	My email	• Optional more verbose description. Maximum length of 2048 characters.
enabled	boolean	FALSE	• Indicates if subscription is active. • May be updated using the enable/disable endpoints. • By default this is *true*.
typeFilter	String	apps.APP.DELETE	• Filter to use when matching events. • Matches against event type. • Has three dot sep-

14.6.2 Event Attributes

Attribute	Type	Example	Notes
source	String	https://tapis.io/jobs	• Context in which event happened.
type	String	jobs.JOB_NEW_STATUS.(	• Type of event. Used for routing notifications. • Pattern is *<service>.<category>.<detail>*
subject	String	<job-id>	• Subject of event in the context of the service. • Examples: job Id, app Id, file path, role name, etc.
timestamp	String	2020-06-19T15:10:43Z	• When the event happened.
data	String		• Optional. Other data associated with the event. • For example, service specific information.
deleteSubscriptionsMatchingSubject	boolean		• Delete subscriptions where subjectFilter matches subject
endSeries	boolean		• Delete tracking data for the series.
seriesId	String	<job-id>	• Optional. Group events based on tenant,source,subject. • Preserves event order during notification delivery.
tenant	String	tacc	• Tapis tenant associated with the event.
uuid	String		• Tapis generated unique identifier.
user	String		• Tapis user associated with the event.

14.6.3 Notification Attributes

Attribute	Type	Notes
uuid	String	Unique identifier for the notification.
tenant	String	Tenant associated with the notification.
subscriptionName	String	Name of subscription associated with the notification.
eventUuid	String	Unique identifier for the event contained in the notification.
event	Object	Event that triggered the notification.
deliveryTarget	Object	The delivery target for the notification.
created	String	When the notification was created.

14.6.4 Delivery Target Attributes

Attribute	Type	Notes
deliveryMethod	enum	WEBHOOK or EMAIL
deliveryAddress	String	URL for WEBHOOK or email address for EMAIL

CHAPTER

FIFTEEN

RESOURCE SHARING

15.1 Introduction to Tapis Access Controls

Tapis users define resources such as systems, applications, files, streams, functions and workflows. These resources can be used to generate other resources, such as job, workflow and function outputs. The ability to share resources greatly extends their utility and, in general, the usefulness of Tapis. On the other hand, users need to safeguard their data by controlling access to resources. To facilitate both sharing and access control, Tapis provides two built-in authorization mechanisms.

The first facility is implemented using *roles and permissions*. These controls operate at a low level of abstraction and apply to the individual Tapis resources to which they are associated. The second facility, *Tapis shared resources* (or simply *shared resources*), operate at a higher level of abstraction by authorizing complex actions in Tapis. Such actions include running jobs, which authorizes access to an application, execution and archive system. In this case, the granting of a single share affects multiple Tapis resources. We explore both roles/permissions and shared resources in the following sections.

15.2 Roles and Permissions

The Security Kernel (SK) implements a distributed, role-based access control (RBAC) facility in which users are assigned roles that limit or allow access to resources. At its most basic, a role is simply a named entity with an owner. Tenant administrators can manage roles in their tenant and Tapis services can manage roles in any tenant. Typically, services only create roles in the administrative tenant at a site (each site defines a restricted administrative tenant).

Access to specific resources is controlled by assigning roles to users. Tapis supports user-based APIs that check if a user has a certain role. Services and other software that perform these checks are free to determine what membership in a role means.

To make managing user authorizations more flexible and convenient, a role can contain zero or more other roles. A contained role is the *child* of the containing role. A child role can have any number of *parents*, that is, be contained in any number of other roles. Roles form a forest of directed acyclic graphs.

When checking whether a user has been assigned a particular role, SK APIs check whether the user has been assigned that role or any of its parent roles. When a user is assigned a role, the user is also implicitly assigned all the children of that role.

In addition, roles can contain *permissions*, which are case-sensitive strings that follow the format defined by Apache Shiro. The permission-creation API adds a permission to a role. The permission-checking API takes a required permission and determines if a user has a matching permission in any of its roles.

Below are examples of permissions enforced by the Tapis Systems service. The first permission allows read/write access to *system1* in the *MyTenant* tenant. A user assigned a role that contains this permission would have access to *system1*. Similarly, the second permission allows its assignees to create, read, write and delete any system in the *MyTenant* tenant.

```
system:MyTenant:read,write:system1
system:MyTenant:create,read,write,delete:*
```

For convenience, each user is automatically assigned a default role that is implicitly created by Tapis. Assigning a permission to a user really means adding the permission to the user's default role.

15.2.1 Implementations of Roles and Permissions

Below are links to the roles and permissions APIs for each service. Also see the roles and permissions discussions in each service's documentation.

- Systems-rbac
- Applications-rbac
- Files-rbac
- Streams-rbac
- Actors-rbac

15.3 Tapis Shared Resources

The roles and permissions discussed in the last section allow fine-grained authorization to Tapis resources, such as systems or applications. Although the semantics of a role or permission can authorize access to multiple resources of the same type, they cannot easily authorize access to all the resources encountered in a complex workflow, such as job execution. To get a deeper understanding of the challenge, consider the authorizations needed to run a job:

1. Read access to the application definition.
2. Read access to the execution system definition.
3. Read access to each job input's storage system.
4. Read access to each job input's file path or object ID.
5. Read/write access to the execution system's input, output and application staging directories.
6. Read access to the job's archive system definition.
7. Write access to the job's archive system's archive path.

If a user simply wants to share an application with another user so that the latter can execute it, many individual role or permission grants would have to be put in place across multiple services.

To solve this problem, *Tapis sharing* grants some implicit access to all resources needed to perform an *action*. Tapis sharing defines a runtime *context* in which limited Tapis authorization is implicitly granted to users performing an action. This context can span multiple services. For example, if a file path is shared with a user, then access to the Tapis system on which the file resides is implicitly granted when the user attempts access through the Files service.

An important aspect of sharing is that implicit access applies within a certain context and does not apply outside of that context. In the case of a shared file path, for instance, read access to the required system definition is only valid when accessing that file path through the Files API. If the user tries to access the system definition directly through the Systems API, the request will be rejected as unauthorized (assuming no other authorizations apply).

Another characteristic of sharing is that implicit authorizations apply only to Tapis resources. In the file path sharing scenario, the system definition that is part of the shared context is an artifact defined within and under the control of Tapis. Tapis is the controlling agent. Access to the path itself, however, is ultimately under the control of the file system on which the path resides, such as a Posix file system. Tapis sharing has no effect on authorizations enforced by external systems. For example, a user could share a path in their home directory, but unless the grantee already has Posix access to that path, requests to access it will be denied by the operating system.

In addition to sharing resources with individual users, Tapis sharing also supports granting public access to resources. The *public-grantee* access designation allows all users in a tenant access to the shared resource. Where supported, the *public-grantee-no-authn* access designation grants access to all users, even those that have not authenticated with Tapis. See individual service documentation for details on public access support.

15.3.1 Shared Application Contexts (SACs)

The concept of a *Shared Application Context (SAC)* recognizes that applications run in the context of a Tapis job. This context is leveraged by multiple, cooperating services to allow limited implicit access to all the resources needed to run a job. In this case, the term *limited implicit access* means that for certain resources, the user running the job will have the application owner's authorizations in addition to their own. Specifically, users are able to access systems and file paths which they cannot normally access but the application owner can access.

When a job runs in a SAC, services grant this *limited implicit access* for **resources explicitly specified in the application definition**. Important characteristics of a SAC are:

1. **The SAC-aware services are Systems, Applications, Jobs and Files.**
    a) These services know when they are running in a SAC and how to alter their behavior.
2. **SAC-aware services grant implicit access only during Job execution of a shared application.**
    a) Users are not conferred any special privileges on application-specified resources outside of job execution.
    b) Relaxed authorization checking applies only to systems and file paths referenced in the application definition.
3. **SSH authentication to a host is not affected by SAC processing.**
    a) The Tapis system definition still determines the credentials used to login to a host.
    b) The host operating system still authorizes access to host resources.
4. **File system and object store authorization is not affected by SAC processing.**
    a) The authenticated user must still be authorized by the persistent storage systems.

In summary, a user can share an application with another user and the Tapis file and system resources referenced in the application definition are also implicitly shared. This implicit sharing is implemented by supplementing the requesting user's authorizations with the application owner's authorizations for these resources (and only these resources). The underlying operating systems' and persistent storage systems' authentication and authorization mechanisms are unchanged, so users have no more low-level access than they would otherwise. Tapis simply relaxes its access constraints *during job execution*, but all host authorizations are still enforced.

SAC-Eligible Attributes

The following attributes of application definitions are SAC-eligible, meaning that implicit access to the resources they designate can be granted to jobs running in a SAC.

1. execSystemId
2. execSystemExecDir
3. execSystemInputDir
4. execSystemOutputDir
5. archiveSystemId
6. archiveSystemDir
7. dtnSystemId

8. dtnSystemInputDir
9. dtnSystemOutputDir
10. fileInputs sourceUrl
11. fileInputs targetPath

If an execution system, for instance, is specified in a shared application definition, *and that system is not overridden in the job submission request*, then jobs running in a SAC will be granted implicit access to the system's definition. The same is true for the other SAC-eligible attributes: If their values are specified in the application and those values are not overridden when a job is submitted, Tapis implicitly grants access to the designated Tapis resource.

In particular, Data Transfer Nodes (DTNs) provide a staging area for job input and output files. These nodes usually support high I/O bandwidth or specialized software stacks. When a DTN is specified by a system, that DTN and its input and output directories are shared using the same SAC rules described above for execution systems.

Implementations of Tapis Sharing

Below are links to the sharing APIs for each service. Also see the sharing discussions in each serivce's documentation.

- Systems-Sharing
- Applications-Sharing
- Files-Sharing
- Jobs-Sharing

CHAPTER

SIXTEEN

TAPIPY AND TAPISSERVICE

Tapipy and TapisService are powerful Python libraries for interacting with Tapis.

16.1 Tapipy User Guide

16.1.1 What is Tapipy

Tapipy is a Python library for interacting with an instance of the Tapis API framework. The package is hosted on PyPi - tapipy. You can install it with:

```
pip install tapipy
```

The library is automatically generated by referencing the OpenAPI spec files which a `Tapis` client built from the OpenAPI spec files from TACC's Tapis services. With this functionality a user is able to authorize itself with the `Tapis` client and have a 'live' library in order to interact with all Tapis services. Usage of `tapipy` allows for automatic token refreshing, verbose error messages regarding endpoint inputs, and up to date syntax for all services.

Links

PyPi Package: https://pypi.org/project/tapipy
Github Repo: https://github.com/tapis-project/tapipy

16.1.2 Initializing Tapipy

To initialize a `Tapis` client object, `t`, you will replace `your_tacc_username` and `your_tacc_password` with your TACC username and password, preserving the quotation marks shown in the command below. This command will authenticate you with Tapis for your respective `base_url` and store your retrieved token for later `tapipy` calls.

```
from tapipy.tapis import Tapis

# Create python Tapis client for user
t = Tapis(base_url= "https://tacc.tapis.io",
          username="your_tacc_username",
          password="your_tacc_password")

# Get tokens now that you're initialized
t.get_tokens()
```

These calls do not produce output. However you can use the following code to see the access token you just generated.

```
t.access_token
```

The output should look similar to the following; it describes the access token that was just created.

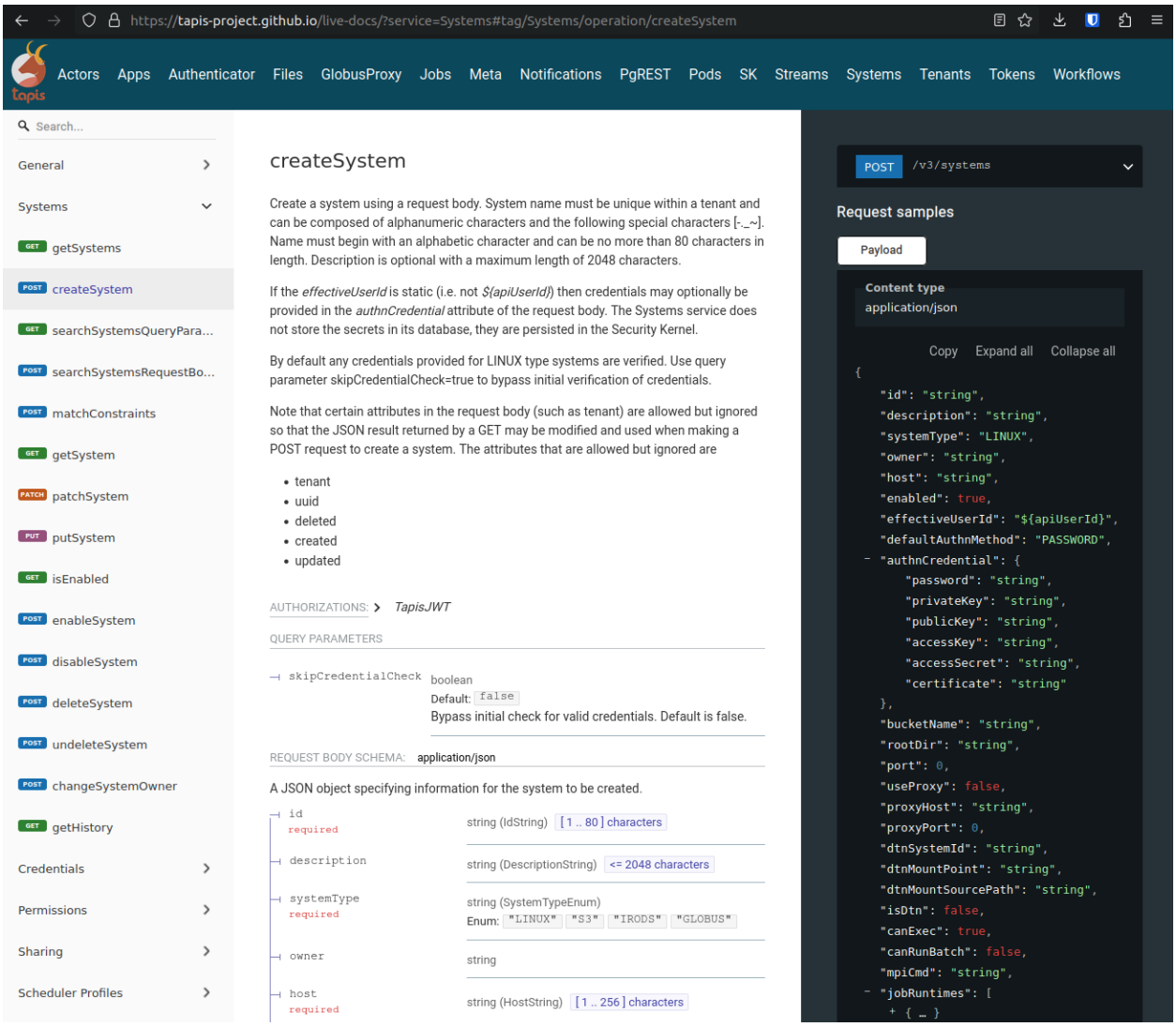
```
access_token: *very long string of alphanumeric characters*
claims: {
    'jti': '007fa9e6-f044-4817-a812-12292b2bdbe3',
    'iss': 'https://tacc.tapis.io/v3/tokens',
    'sub': 'your_tacc_username',
    'tapis/tenant_id': 'tacc',
    'tapis/token_type': 'access',
    'tapis/delegation': False,
    'tapis/delegation_sub': None,
    'tapis/username': 'your_tacc_username',
    'tapis/account_type': 'user', 'exp': 1657686889,
    'tapis/client_id': None,
    'tapis/grant_type': 'password'
}
expires_at: 2022-07-13 04:34:49+00:00
expires_in: <function Tapis.add_claims_to_token.<locals>._expires_in at 0x10a070280>
jti: 007fa9e6-f044-4817-a812-12292b2bdbe3
original_ttl: 14400
```

Where you will have your own access token and the placeholder `your_tacc_username` will be replaced with the username you used.

16.1.3 Using Tapipy

Now in order to use the `Tapis` client you can reference the Tapis Live Docs to browse all service specifications. Tapipy is generated off of these specific specifications, so this is your most up-to-date reference on `tapipy` functionality (if `tapipy` is fully updated).

For example, if you wanted to create a system using the Systems service you would navigate to the proper service in the top bar, browse the sidebar for functions and view the specification in the main pane. An example is displayed below:



For this example, creating a system, I can see I need to run the following:

```
t.systems.createSystem(id="id", host="host", systemType="LINUX")
```

Important

Tapipy 1.3.2 introduces native support for `t.files.insert()`. Previous a workaround, `t.upload(filePath, systemId, path)` was required. Read more *here*.

You can specify any parameter identified by the spec. Tapipy will detail missing required parameters if they're missing.

The live-docs are a helpful GUI on top of OpenAPI v3 specifications that each service writes to detail endpoints, attributes, proper content-types, return objects, etc. Depending on the verbosity of the specifications the site could detail what each function and attribute is, along with defaults and type restrictions.

16.1.4 Special User Query Parameters and Headers

For the most part, arguments that can or should be passed to a Tapis endpoint are described in the OpenAPI definition files and recognized automatically by `tapipy`. However, due to limitations in what can be expressed in OpenAPI, there are some paramaters that are not defined in the definition files; for example, the search parameters for various endpoints.

To accommodate these cases, `tapipy` recognizes two special keyword arguments to all of its methods that correspond to Tapis API calls (i.e., all of its "operations"). They are:

- `_tapis_headers` - dictionary-like object of header names (keys) and vales.
- `_tapis_query_parameters` - dictionary-like object of query parameter names (keys) and values.

Use the above two special arguments for passing headers (respectively, query parameters) that are not specified in the OpenAPI definition of an endpoint.

For example, I can issue a search using the following syntax:

```
t.jobs.getJobSearchList(
    limit=5,
    orderBy='lastUpdated(desc),name(asc)',
    _tapis_query_parameters={'key': 'value'})
```

16.1.5 Additional Tapipy Parameters

The tapipy package allows for spec file customization in `Tapis` client initialization:

- **`resource_set`: str in ["tapipy", "prod", "staging", "dev"]**
  - Determines which set of resource to use, defaults to `tapipy`.
  - **Important to note that if a `custom_spec_dictionary` is used, it is appended to this resource_set.**
    - For example, you would set `prod` and then specify a custom specs that will be added on.
- **`custom_spec_dict`: {resource_name: str, resource_url: str}**
  - **Allows users to modify the base resource set urls.**
    - i.e. I can specify actor as a resource name and change the url.
  - **Also allows users to add new resources to the set.**
    - i.e. I can add a new resource named "test" with a custom url.
    - Any new specs will be downloaded and added cached without an explicit recache.
  - **Allows local resources.**
    - Specify an absolute path in the dict with local: prefixing it and tapipy will load in a local OpenAPI v3 yml spec file.
    - `custom_spec_dict={'cactus': 'local: /home/tapis/myfolder/cactusSpec.yml'}`
- **`download_latest_specs`: bool**
  - Allows users to re-download all specs regardless on if they already exist in the cache. Defaulted to False
  - **This will happen every time the `Tapis` client is initialized, it's a tad slower, and can cause live updates to specs.**
    - As such, be warned. There are functions to update spec files below.

- **spec_dir: str**
  - Allows users to specify folder to save specs to. Defaults to none which uses Tapipy's install folder.
  - If you are updating specs it's wise to use a different folder in order to not modify the base specs.

Warning

If your user account does not have permission to write to Tapipy's install directory you will encounter an error due to file system permissions. You can use `spec_dir` as highlighted above and use a directory you have permission to write to. (Important for Docker containers)

Example

The following is an example of some custom parameter setting. As you can see, the abaco resource will now use the spec at `URL#1`, overwriting the resource definition in the `tapipy` resource set, it'll download it if it doesn't exist. The same for the longhorn resource. This means that the `Tapis` client will now have access to all specs in `tapipy` like normal, but with a modified abaco and with a new longhorn resource. All of these are stored at the new spec_dir because you don't want to accidentally overwrite any packaged specs when call update_spec_cache() later (talked about in the next section).

```
from tapipy.tapis import Tapis

t = Tapis(base_url='https://admin.develop.tapis.io',
          username='username',
          password='password',
          resource_set='tapipy',
          custom_spec_dict={'abaco': 'URL#1',
                            'longhorn': 'URL#2'},
                            'cactus': 'local: /home/tapis/myfolder/cactusSpec.yml'},
          spec_dir='/home/username/tapipy_specs')
t.get_tokens()
```

16.1.6 Tapipy Instantiation with only a Tapis access token

Tapipy can be instantiated with only a Tapis access token. This is useful when you have a created token and don't want to provide username and password. The minimal instantiation requires the base_url and the token as follows:

```
from tapipy.tapis import Tapis

t = Tapis(base_url='https://admin.develop.tapis.io',
          access_token='tokenhere')
```

As the token is already created users do not have to run `t.get_tokens()`. You can use the client as normal at this point.

16.1.7 Refresh Tapipy's Cached Specs

Tapipy uses a cache to organize spec dictionaries as pickled files and has the ability to accept custom spec files. By default Tapipy keeps a set of base spec files in the `{tapipyInstallDir}/specs` folder. These specs are pre-pickled at package build time. To update specs,

Warning

If your user account does not have permission to write to Tapipy's install directory you will encounter an error due to file system permissions. You can use `spec_dir` as highlighted above and use a directory you have permission to write to. (Important for Docker containers)

Two Spec Refresh Methods

1. **`t.update_spec_cache()` - No inputs**
    - Will refresh client's `t.resource_set` resources and save to `t.spec_dir`.

```
from tapipy.tapis import Tapis
t = Tapis()
t.update_spec_cache()
```

2. **`tapipy.tapis.update_spec_cache()` - Takes `resources` and `spec_dir`**
    - Will refresh specified resources and cache to inputted `spec_dir`
    - Default to refreshing all resources from `tapipy` and saving to package install dir.

```
from tapipy.tapis import update_spec_cache
Resources = Dict[ResourceName, ResourceUrl]
update_spec_cache(resources: Resources = None, spec_dir: str = None)
```

16.1.8 Creating and Using Tapipy Refresh tokens

Tapipy will authenticate you to Tapis and generate a token, however to activate automatic token refreshing via refresh token you must also have a `Tapis` client created in order to provide `client_id` and `client_key` at Tapis initialization.

The following code will show you how to create a client if you don't already have one and then initialize a `Tapis` client with refresh capabilities by also providing `client_id` and `client_key`. If you already have a client created then you can jump to the `Fourth` step. Feel free to run just the code you need.

```
from tapipy.tapis import Tapis

# First - Create your normal Tapis client so we can run create_client.
t = Tapis(base_url = 'https://tacc.tapis.io',
          username = 'cgarcia',
          password = 'password')
t.get_tokens()

# Second - List your clients (check if you already have a client created)
t.authenticator.list_clients()

# Third - If you don't have a client, create one.
t.authenticator.create_client()

# Should result in:
'''
[
active: True
callback_url:
client_id: my_client_id
```

(continues on next page)

(continued from previous page)

```
client_key: my_client_key
create_time: Wed, 02 Nov 2022 20:18:01 GMT
description:
display_name: my_display_name
last_update_time: Wed, 02 Nov 2022 20:18:01 GMT
owner: cgarcia
tenant_id: icicle]
'''

# Fourth - Grab the client_id and client_key, add it to your new Tapis client␣
↪initialization
t2 = Tapis(base_url='https://tacc.tapis.io',
        username='cgarcia',
        password='pass',
        client_id='client_id',
        client_key='client_key')
t2.get_tokens()

# You should now be able to run refresh_tokens with no errors. The t2 object will␣
↪refresh when needed.
t2.refresh_tokens()
```

16.1.9 Using the Auto-retry Functionality

Note

Available for tapipy >= 1.6.2

Users to configure a python client to automatically retry failed api calls. Retries can be configured on a per-call basis at the Operation level. Users may provide the following keyword arguments to any tapipy operation to make use of this feature:

- `_retries: int = 0` - The total number of times that a call should be retried. The default is 0. Specifying a value of 1 for retries is equivalent to making 2 back-to-back api calls
- `_retry_delay_sec: Union[int,float] = 0` - Total number of seconds to delay between api call retires. Defaults to no delay; all calls are made immediately one after the other until one succeeds or the number retry limit is reached
- `_retry_on_exceptions: List[Exception] = [tapipy.errors.InternalServerError]` - A list of exceptions that trigger a retry of an api call. Defaults to a list containing a single item (`tapipy.errors.InternalServerError`). Any other exception could be used, but in practice, it is likely that the default error is the only one that really makes sense to retry.
- **`_retry_backoff_algo: Literal["constant", "exponential"] = "constant"` - The algorithm to use to calculate the duration from one retry to the next. Defaults to the value 'constant'. Note, this keyword argument has no effect if `_retries` == 0 or `_retry_delay_sec` == 0. Any values provided to this keyword argument that are not one of the two string literals listed below with raise a `TapyClientConfigurationError` exception.**
  - `constant` - the delay between api calls will not change. For example, if a user specifies a 5 second delay with 5 retries. There will be a 5 second delay between each retry.
  - `exponential` - the delay between api calls will change exponentially. For example, if a user specifies

a retry limit of 5 (6 calls in total including the initial call), and a 2 second delay, the delays between each api call will be 2, 4, 8, 16, 32, and 64 respectively.

16.1.10 Using `t.files.insert` with Tapipy for file upload.

Introduced in Tapipy 1.3.2, the library now has native support for `multipart/form-data` input data and headers. This allows us to natively support the `t.files.insert` operation. The following is an example of uploading a `filepath.ext` file to a specific `path` in a users Tapis `system_id` system.

```
with open("filepath.ext", "rb") as data_blob:
    res = t.files.insert(systemId=system_id, path=path, file = data_blob)
```

Important

The previous static workaround, `t.upload(filePath, systemId, local_file_path)`, is now deprecated and will be removed in a future release.

16.2 TapisService User Guide

16.2.1 What is TapisService

TapisService is a Python plugin for Tapipy which extends Tapipy with additional service oriented Python modules. This plugin is useful when creating a Tapis service or using a Tapis service account. The package is hosted on PyPi - tapisservice. You can install it with:

```
pip install tapisservice
```

The plugin adds systems for authentication, configs, logging, Tapis errors, TenantCache, alongside helper code for working in `Flask`, `FastAPI`, and `Django`, three popular Python web frameworks. Python based Tapis services should make use of `tapisservice` for at least the authentication in order to not unneccessary rereplicate code.

Links

PyPi Package: https://pypi.org/project/tapisservice
Github Repo: https://github.com/tapis-project/tapipy-tapisservice

16.2.2 Writing a New Service

Getting Started

When creating a new Tapis v3 service, use the following guidelines:

Create a git repository with the following:

```
+ service: directory for API source code.
+ migrations: directory for migration scripts.
- config-local.json: API config values for local development
- configschema.json: jsonschema definition of the API config.
- Dockerfile: Build recipe for the API service Docker image.
```

(continues on next page)

(continued from previous page)

```
- Dockerfile-migrations: Build recipe for the API service migrations Docker image.
- requirements.txt: Packages required for the API service or migrations scripts.
- docker-compose.yml: compose file for local development.
- CHANGELOG.md: Tracks releases for the service.
- README.md: High level description of the service.
- service.log: Mount into the API container to capture service logs during local␣
↪development.
```

Service configuration

A description of all possible service configurations should be provided in the `configschema.json` file or example, if your service requires a SQL database, you may have a property, `sql_db_url`, with the following definition in your `configschema.json`:

```
"sql_db_url": {
    "type": "string",
    "description": "full URL, including protocol and database, to the SQL db.",
    "default": "postgres://myservice:mypassword@postgres:5432/mydb"
},
```

The fields you define in your service's `configschema.json` file will complement those defined in the `configschema.json` file included in this repository. Any configs that will be used by all services, such as `service_password`, should be defined in the common `configschema.json`. It is currently possible to override the definition of a config provided in this repository with a new definition provided in your service's `configschema.json` file, but this is not recommended.

Provide values for the configs in the `config-local.json` file. When deploying to a remote environment, such as the Tapis develop environment, the `config-local.json` can be replaced with file mounted from a ConfigMap with different values.

- Add any packages required by your service to the `requirements.txt` file. Keep in mind that a number of packages are installed by this repository (such as jsonschema, pyjwt, etc.), so it is possible you will need to add few additional packages.
- Create a Dockerfile to build your service. The image name for your service should be `tapis/<service_name>-api`; for example, `tapis/tokens-api` or `tapis/tenants-api`. Here is a general template for the Dockerfile for your service:

```
# inherit from the flaskbase image:
FROM: tapis/flaskbase

# set the name of the api, for use by some of the tapisservice modules.
ENV TAPIS_API <api_name>

# install additional requirements for the service
COPY requirements.txt /home/tapis/requirements.txt
RUN pip install -r /home/tapis/requirements.txt

# copy service source code
COPY configschema.json /home/tapis/configschema.json
COPY config-local.json /home/tapis/config.json
COPY service /home/tapis/service
```

(continues on next page)

(continued from previous page)

```
# run service as non-root tapis user
RUN chown -R tapis:tapis /home/tapis
USER tapis
```

Migration skeleton

For services using Postgres, create migration skeleton. Migrations are based on the alembic package and must be initialized. Run the following commands from a terminal:

```
$ docker run -it --entrypoint=bash --network=<service>-api_<service> -v $(pwd):/home/
↪tapis/mig tapis/tenants-api
# inside the container:
$ cd mig; flask db init
$ flask db migrate
$ flask db upgrade
$ exit
```

Migrations Dockerfile

Create Dockerfile-migrations to containerize your migrations code. For simple cases, you may be able to just use the following after change <service> to the name of your service.

```
# image: tapis/<service>-api-migrations
from tapis/<service>-api

COPY migrations /home/tapis/migrations

ENTRYPOINT ["flask", "db"]
```

Service Dockerfile

Write a docker-compose.yml file to facilitate local development.

16.3 TapisService Modules

Code for a number of common tasks has been packaged into `tapisservice` modules. In this section, we give an overview of how to use some of the functionality in your service.

16.3.1 Accessing TapisService Modules

You can install `tapisservice` with `pip install tapisservice`. Services can import modules directly from this package; for example:

```
from tapisservice import auth
from tapisservice.utils import ok
```

16.3.2 Service Configuration and Initialization

Most services will need to do some initialization tasks before they are ready to respond to requests. For example, they may need to connect to a database or make some requests to some other Tapis services. Initialization also usually involves reading configuration data. `tapisservice` audits the supplied config file and

makes configration data available through a singelton object, `conf`, available from the `tapisservice.config` package. By importing the object, for instance, in the API's `__init__.py` module, the config file will be read and validated, and the resulting configurations transformed to Python objects. For example, if a service requires a configuration, `max_number_retries`, then it could use the following entry in its `configschema.json` file:

```
    "max_number_retries": {
    "type": "integer",
    "description": "Maximum number of times the service should retry some complicated␣
↪logic...",
    },
    . . . # additional properties
},
"required": ["max_number_retries", . . .]
```

and then, place the following code in its `__init__.py` file:

```
from tapisservice.config import conf

print(f"We'll be trying at most {conf.max_number_retries} times."
```

By the time the import has completed, the service is guaranteed that `conf` contains all required fields and that they conform to the requirements specified in the `configschema.json` file. In particular, the types of the attributes are the same as that specified in the `configchema.json` file.

16.3.3 Making Service Requests

The `tapisservice.auth` package provides a function, `get_service_tapis_client` which can be used to get a pre-configured `Tapis` client for making service requests. A common technique is to fetch the service client in the `__init__.py` module so that it is created at service initialization and available via import throughout the rest of the service code.

Within `__init__.py`:

```
from tapisservice.auth import get_service_tapis_client
from tapisservice.tenants import TenantCache

Tenants = TenantCache()

t = get_service_tapis_client(tenants=Tenants)
```

From within any other service modules:

```
from __init__ import t
. . .
# use the client within some method or function:
t.sk.getUsersWithRole(...)
```

16.3.4 Authentication

The `tapisservice.auth` module provides functions for resolving the authentication data contained in a reqeust to your service.

JWT Authentication

The most common and straight-forward case is when an endpoint in your service requires a JWT. For this use case, the `tapisservice.auth.authentication()` function can be used. This function does the following:

1. Checks for a JWT in the `X-Tapis-Token` header, and checks the other `X-Tapis-` headers.
2. Validates the JWT, including checking the signature and expiration, and sets the following on the global thread-local, `g`:
    a. `g.x_tapis_token` - the raw JWT.
    b. `g.token_claims` - the claims object associated with the JWT, as a python dictionary.
    c. `g.username` - the username from the JWT `username` claim.
    d. `g.tenant_id` - the tenant id from the JWT `tenant_id` claim.
    e. `g.account_type` - the account type (either `user` or `service`) from the JWT.
    f. `g.delegation` - whether the token was a delegation token (True or False).
    g. `g.x_tapis_tenant` - the value of the `X-Tapis-Tenant` header.
    h. `g.x_tapis_user` - the value of the `X-Tapis-User` header.
    i. `g.request_tenant_id` - **USE THIS** JWT `tenant_id` or, if service, `X-Tapis-Tenant`.
    j. `g.request_username` - **USE THIS** JWT `username` or, if service, `X-Tapis-User`.

Important

Use `g.request_tenant_id` and `g.request_username`. These both resolve to their respective values, obtained from the incoming request JWT Tapis token. However, the values change when service accounts send "on behalf of" (obo) requests. Only services within the same site may complete these request, this is checked by tapisservice.

This function raises the following exceptions:

1. `tapisservice.errors.NoTokenError` - if no token was found.
2. `tapisservice.errors.AuthenticationError` - the token was invalid.

Other Types of Authentication

Some services, such as the Authenticator, use other types of authentication, including HTTP Basic Auth. Several `tapisservice.auth` functions are provided to facilitate tasks related to alternative authentication methods. TODO

Logging

The `tapisservice.logs` module provides basic logger with a file and stdout handler. To use it, create an instance of the logger in each module where you want to add logs by calling the `get_logger` function with the module name; for example:

```
from tapisservice.logs import get_logger
logger = get_logger(__name__)
```

Then add log statements using the logger:

```
logger.debug(f"some debug message, x={x}."
```

Error Handling

For REST APIs, the `tapisservice.util` module provides the `TapisApi` class and the `flask_errors_dict` dict and `handle_error()` function. Use them by adding the following to your `api.py` module:

```
from flask import Flask
from tapisservice.utils import TapisApi, handle_error, flask_errors_dict

# basic flask "app" object:
app = Flask(__name__)

# TapisApi object; created with the app object and the flask_errors_dict to establish␣
↪the 4 stanza structure of error respones:
api = TapisApi(app, errors=flask_errors_dict)

# Set up error handling to use the handle_error() function -
api.handle_error = handle_error
api.handle_exception = handle_error
api.handle_user_exception = handle_error
```

Now, from within a controller, raise an exception of type `tapisservice.errors.BaseTapisError` (or any child class). When instantiating the exception to be raised, set values for the `msg` and `code` attributed to set the message and HTTP response code. For example, in your controller, you could have:

```
raise errors.MyServiceError(msg='Invalid sprocket; too many widgets.', code=400)
```

Then, as long as the `errors.MyServiceError` class descended from the `tapisservice.errors.BaseTapisError`, the HTTP response returned to the user would be:

```
{
    "message": "Invalid sprocket; too many widgets.",
    "status": "error",
    "version": "conf.version",
    "result": null
}
```

And the HTTP status code would be `400`.

Note that if your code (and, by extension, any code your code calls) raises an exception that does not descend from the `tapisservice.errors.BaseTapisError` then the HTTP response will still contain the 4-stanza JSON response above but the `message` field will contain "Unrecognized exception type.." to indicate that the exception was not a recognized exception. In general, your service should not raise exceptions of types other than `tapisservice.errors.BaseTapisError` and should instead handle all other exceptions and convert them to the appropriate Tapis Exceptions. This includes all exceptions from the Python standard library, such as KeyError, AttributeError, etc.

For example, if code you are writing could raise a KeyError, you should catch that and then translate it appropriately. Of course, different KeyErrors in different situations could translate into a user error, a service error, etc.

```
try:
    app_id = post_data['app_id']
except KeyError:
    raise errors.BadInputError(msg='The app_id parameter was missing. Please be sure to␣
↪pass app_id.')   # <- decends from BaseTapisError
```

16.4 Working with TapisService, Need to Knows

If you're developing with TapisService then please ensure you understand the following topics.

16.4.1 Framework Talk

Flaskbase is the original implementation of a `tapisservice` like scheme where we provide neccessary helper functions for dealing with Tapis. It was a library packaged in a Docker container which services based (and still can) their containers off of for a working coding environment. As the name says, this library was originally intended for `Flask` and overtime it was generalized for more frameworks and to be used even without any specified framework. Alongside `Flask`, `FastAPI` has full support of all `tapisservice` features. `Django` is also a supported framework, however all functionality is not yet at 100%.

These three frameworks already have Tapis services already built off of them. These can be great as example implementations to base your own services off of. Information is as follows:

- **Flask:**
  - Abaco. The repository's `/actors` folder contains `__init__.py` along with `auth.py`.
  - Abaco was the original user of Flaskbase and fully implements all `tapisservice` functionality.
- **FastAPI:**
  - Pod Service. The repository's `/service` folder contains `__init__.py` along with `auth.py`.
  - Pods was the first implementation of `FastAPI` with `tapisservice`, it fully implements all `tapisservice` functionality.
- **Django:**
  - PgREST. The repository's `/pgrest` folder contains `__init__.py` along with `views.py`.
  - PgREST implemented most features, however it's missing a full use of auth capabilities. Making use of only token resolution in the aforementioned `views.py`.

16.4.2 Framework Initialization

Flask

```
from __init__ import Tenants ## Expects "init.py" as described in "Making Service␣
↪Requests"
from flask import Flask
from tapisservice.tapisflask.utils import TapisApi
from tapisservice.auth import authn_and_authz

def authentication():
    ## You can write your own auth code here
    ## This will run after tapisservice validates token and sets all valuable g object␣
↪fields
    user = g.request_username
    tenant_id = g.request_tenant_id


def authorization():
    ## You can write your own auth code here
    ## This will run after tapisservice validates token and sets all valuable g object␣
↪fields
```

(continues on next page)

(continued from previous page)

```
app = Flask(__name__)
api = TapisApi(app)  ## Empty wrapper around "flask.Flask.api" for future use.

# tapisservice authn/authz
@app.before_request
def auth():
    # Provide callbacks for authn and authz which will have access to g object fields.
    # authentication runs, followed by authorization
    authn_and_authz(tenant_cache=Tenants,
                    authn_callback=authentication,
                    authz_callback=authorization)
```

FastAPI

```
from __init__ import Tenants ## Expects "init.py" as described in "Making Service␣
↪Requests"
from tapisservice.tapisfastapi.utils import GlobalsMiddleware
from tapisservice.tapisfastapi.auth import TapisMiddleware
from fastapi import FastAPI
from fastapi.middleware import Middleware

def authentication():
    ## You can write your own auth code here
    ## This will run after tapisservice validates token and sets all valuable g object␣
↪fields

def authorization():
    ## You can write your own auth code here
    ## This will run after tapisservice validates token and sets all valuable g object␣
↪fields

api = FastAPI(
    title="MyService",
    middleware=[
        Middleware(GlobalsMiddleware),
        Middleware(TapisMiddleware,
                   tenant_cache=Tenants,
                   authn_callback=authentication,
                   authz_callback=authorization)
    ])
```

16.4.3 Example of Usual API file imports

This applies most for FastAPI and Flask. Django can use some of this on a case by case basis.

```
from __init__ import t ## Expects "init.py" as described in "Making Service Requests"
from tapisservice.config import conf
from tapisservice.errors import BaseTapisError
from tapisservice.logs import get_logger
logger = get_logger(__name__)
```

(continues on next page)

(continued from previous page)

```
# Use precreated t object with service
t.systems.get_systems(_tapis_set_x_headers_from_service = True)

# Get config.json site_id key value
site_id_to_use = conf.site_id

# Use logger and Tapis error
try:
    logger.debug("Writing logs to file described in config.json and configschema.json.")
except BaseTapisError:
    logger.debug("Error found, will be given 400 status code and traditional Tapis 5␣
↪stanza output.")
```

16.4.4 Global g object - Flask and FastAPI

`tapisservice` uses request thread local `g` objects to expose data globally. This object is created per request and lasts the lifespan of said request. This allows `tapisservice` to write valuable information to it for reference by code. Key variables to use are: - `g.request_tenant_id` - Either incoming token `tenant_id` claim or "on behalf of" headers set by Tapis services. - `g.request_username` - Either incoming token `username` claim or "on behalf of" headers set by Tapis services. - `g.token_claims` - the claims object associated with the JWT, as a python dictionary.

Flask natively support a `g` object. `tapisservice` simply intercepts the object and writes to it during the `authn_and_authz` phase. FastAPI does not natively use a `g` object, but `tapisservice` introduces the variable. You can think of both versions of `g` to be the same in these docs, use it in both frameworks.

16.4.5 Use `g.request_tenant_id` and `g.request_username`

Hammering home the point, `tapisservice` exposes two sets of similar variables:

- `g.request_username` - `g.username` if not service and `g.x_tapis_user` otherwise
- `g.username` - incoming request token's `username` claim
- `g.x_tapis_user` - the value of the `X-Tapis-Tenant` header

and

- `g.request_tenant_id` - `g.tenant_id` if not service and `g.x_tapis_tenant` otherwise
- `g.tenant_id` - incoming request token's `tenant_id` claim
- `g.x_tapis_tenant` - the value of the `X-Tapis-User` header

Use `g.request_tenant_id` and `g.request_username`. These both resolve to the values obtained from the incoming request JWT Tapis token. However, the values change (to the `x_tapis_*` variants) when service accounts send "on behalf of" (obo) requests. This allows services to reference only one variable versus choosing to use `g.username` or `g.x_tapis_user` at runtime. To note, only services within the same site may complete these OBO requests, this is checked by tapisservice.

The remaining variables are useful for debugging or defining exactly who sent a request and exactly how.

16.4.6 Use Tapipy as a Service Account

A service account can still make use of the `tapipy` client, some changes will apply though.

```
from __init__ import t ## Expects "init.py" as described in "Making Service Requests"

# Uses tenant and username from client, meaning service's tenant and username.
t.systems.get_systems(_tapis_set_x_headers_from_service = True)

# Use specified _x_tapis_user and _x_tapis_tenant_id rather than service's values.
t.systems.get_systems(_x_tapis_user = "cgarcia",
                      _x_tapis_tenant_id = "tacc")
```

CHAPTER

SEVENTEEN

SPECIALIZED SERVICES - OVERVIEW

A collection of api and endpoints that provide specialized or extended functionality to core Tapis services

- *Tapis ETL*: Service for building automatic ETL pipelines
- Tapis Globus-Proxy: Service abstracting the Globus API for interaction with Tapis

CHAPTER

EIGHTEEN

TAPIS ETL

Tapis ETL is a convenience layer on top of Tapis Workflows that enables users to create automated ETL pipelines with a single request. Tapis ETL leverages Tapis Workflow resources (pipeline and tasks) to manage the entire ETL process; from ingesting data files from the configured source system, to tracking their status as they are processed by user-defined batch computing jobs and transferring the results to the configured destination system.

18.1 Overview

18.1.1 Introduction

An ETL pipeline (Extract, Trasform, Load) is a workflow pattern commonly used in scientific computing wherein data is extracted from one or more sources, transformed by some set of processes, and output data produced by those processes are archived to some final destination.

Tapis ETL is a framework built on top of Tapis Workflows that enables users to create fully automated and parallel ETL Pipelines in the Tapis ecosystem. Once a pipeline is configured, simply drop off data and manifest files on the source system. Tapis ETL will run the HPC jobs according to the manifests and archive the results to the configured destination system. For an example of how to get started, jump to the *quickstart* section.

18.1.2 Glossary

Here we introduce the Tapis ETL standard terminology for ETL pipelines

Note

Local denotes any resource (systems, files, etc) situated at TACC. **Remote** denotes any resource (systems, files, etc) **NOT** situated at TACC.

- *Manifests* - Files that track the progress of one or more data files through the various phases of an ETL Pipeline. They are responsible for managing and mantaining the state between pipeline runs.
- *Remote Outbox* - The source system that contains the data files you want to process. This system is commonly a Globus endpoint or S3 bucket.
- *Local Inbox* - The system where data is processed
- *ETL Jobs* - Batch computing jobs that will be run sequentially on HPC/HTC systems against the data files in a manifest.
- *Local Outbox* - The system where the output will be staged for transfer to the destination system
- *Remote Inbox* - The destination system where output data will be archived

- **ETL System Configuration** - A collection of systems responsible for storage of data and manifest files. It is the general term for remote and local inboxes and outboxes.
- **Data System** - Component system of an ETL System Configuration responsible for the storage of data.
- **Manifests System** - Component system of an ETL System Configuration responsible for the storage of manifests.
- **Phase** - A distinct stage of a single Tapis ETL Pipeline run: `ingress`, `transform`, `egress`
- **Resubmission** - Rerunning a specific phase or phases of an ETL pipeline for a given manifest.
- **Pipeline Run** - A single run of an ETL pipeline

18.1.3 How it works

Step 0 must be completed by a user or other external workflow asynchronous Tapis ETL.

0. Data files to be processed are placed in the data directory of the configured Remote Outbox. A manifest is generated for those files (by a user or a script) and placed in the Remote Outbox's manifest directory

Steps 1-4 are managed by Tapis ETL:

1. Tapis ETL then checks the Remote Outbox manifests directory for new manifests and transfers them to the manifests directory of the Local Inbox. The files enumerated in the new manifests on the Local Inbox are then transferred over to the data directory of the Local Inbox
2. A single unprocessed manifest is chosen by the ETL Pipeline (according to the manifest priority) and the files therein are then staged as inputs to the first batch computing jobs. Each batch computing job definied in the ETL Pipeline definition will then be submitted to the Tapis Jobs API; the first of which processes the data files in the manifest.
3. After all jobs run to completion, a manifest is generated for each of the output files found in the data directory of the Local Outbox.
4. All data files enumerated in that manifest are then transferred to the data directory of the Remote Inbox.

18.1.4 Phases

Tapis ETL Pipelines run in 3 seperate phases. The Ingress, Transform, and Egress Phases.

1. **Ingress Phase** - Tapis ETL inspects the Remote Outbox for new data files and transfers them over to the Local Inbox.
2. **Transform Phase** - A single data file or batch of data files is processed by a series of user-defined ETL Jobs
3. **Egress Phase** - The output data files from the ETL Jobs are tracked and transferred from the Local Outbox to the Remote Inbox

---

18.2 User Guide

This section shall serve as the primary technical reference and comprehensive step-by-step guide for the seting up, creating, and managing ETL Pipelines with Tapis ETL.

18.2.1 0. Prerequisites

If you have already satisfied the prerequisites, you can skip to the *ETL Systems Configuration* section.

0.1 Data Center Storage and Compute Allocations

Your project may need a storage allocation on some storage resource (for example at TACC: Corral, Stockyard, or one of our Cloud-based storage resources) to serve as the project's Local Inbox and Outbox. The size of the required allocation greatly depends on the size of the files that will be processed in the pipeline.

Your project may also need one or more allocations on a compute system (for example at TACC: such as Frontera, Stampede2, Lonestar5, or one of cloud computing systems. ETL Jobs run by Tapis ETL will use the allocations specified in the ETL Job definitions.

0.2 Users and Allocations

When Tapis ETL runs an ETL Job, it does so **as the owner** of the ETL Pipeline. This Tapis user must be mapped to a valid user with a valid allocation on the underlying compute system or the Transform Phase of the ETL Pipeline will always fail.

0.3 TapisV3 Client Setup

Throughout the User Guide, we will be creating TapisV3 objects. For all TapisV3 requests, both `cURL` and `tapipy` instructions will be available. Tapipy usage and installation instructions can be found at https://pypi.org/project/tapipy/.

0.4 Tapis Workflows Group

Tapis ETL uses Tapis Workflows' Groups to manage access (creating, deleting, running) ETL Pipelines. If you do not own or belong to a group, create it following the instructions below.

Creating a Group (createGroup)

Create a file in your current working directory called `group.json` with the following json schema:

```
{
  "id": "<group_id>",
  "users": [
      {
        "username":"<user_id>",
        "is_admin": false
      }
  ]
}
```

Note

You do not need to add your own Tapis id to the users list. The owner of the Group is added by default.

Replace <*group_id*> with your desired group id and <*user_id*> in the user objects with the ids of any other Tapis users that you want to have access to your workflows resources.

Warning

Users with `is_admin` flag set to `true` can perform every action on all Workflow resources in a Group except for deleting the Group itself (only the Group owner has those permissions)

Submit the definition.

Note

Section 0 Punch List

- Valid allocation for your project on the compute and storage resources
- Valid user with a valid allocation on the compute resources
- Set up a Tapis client (Bash or Python)
- Create or belong to a valid Tapis Workflows Group

18.2.2 1. ETL Systems Configurations

In this section, we discuss ETL System Configurations. At the end of this section, we will explain how to create and configure all Remote and Local ETL System Configruations for an ETL Pipeline.

Note

ETL System Configurations are by far the most complex part of creating an ETL Pipeline. Do not be discouraged! The initial complexity is a small price to pay for the ease with which ETL Pipelines are operated with Tapis ETL.

An **ETL System Configuration** is a collection of systems that are responsible for the storage of data files (Data System) and the storage of manifest files (Manifest System).

There are 4 ETL System Configurations for every ETL Pipeline:

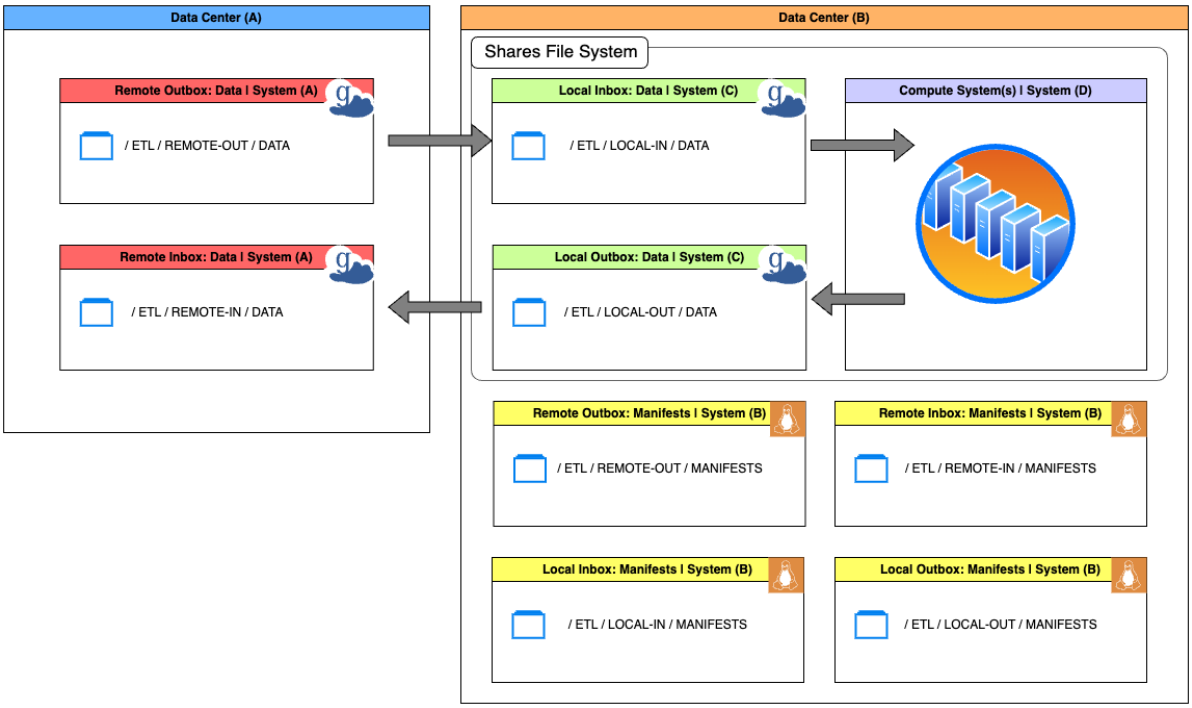
- **The Remote Outbox** - Where input data files to be processed are staged
- **The Local Inbox** - Where input data files are processed
- **The Local Outbox** - Where output data files are staged
- **The Remote Inbox** - Where output data files are transferred

Every ETL System Configuration is composed of 2 systems: A Data System and a Manifests System. These systems generally have a one-to-one correspondence with a *Tapis Systems*.

Note

There are many different possible ETL System Confguration setups for an ETL Pipeline. It is entirely possible to have a single Tapis System representing **all** systems in an ETL Pipeline. How you set up your ETL System Configurations depends entirely upon where the data is and how you want to process it.

Common ETL System Configurations Setup

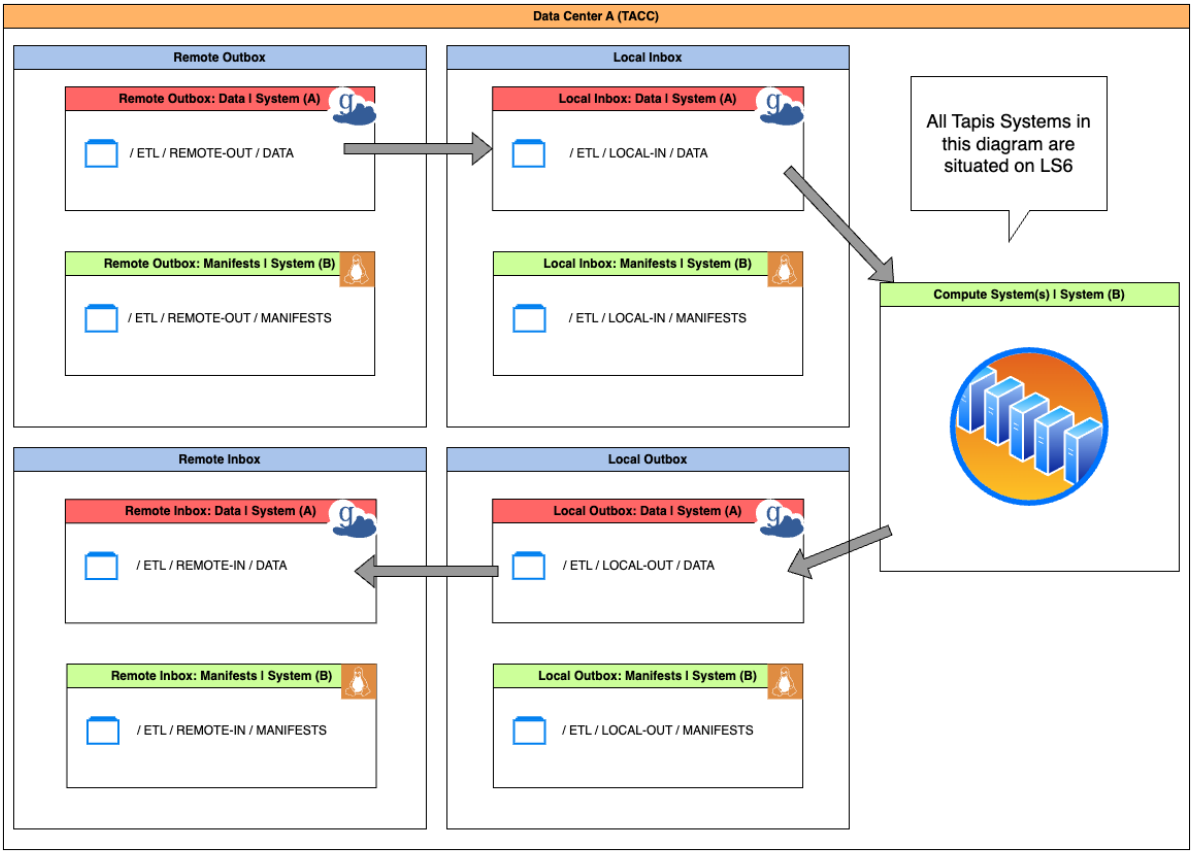


Above is an illustration of the most common ETL System Configuration setup. You will notice that there are four distinct Tapis Systems; System(A) System(C) are Globus endpoints, System(B) is a Linux system, and System(D) is the ETL Job execution system (also Linux) that shares a file system with Systems(C). It is worth noting that some of the *Data and Manifests Systems* for some ETL System Configurations (Remote Inbox, Remote Outbox, etc.) have corresponding Tapis System that are located at entirely different data centers.

1.1 Create Tapis Systems for the ETL System Configurations

Before we discuss ETL System Configurations in depth, we will first create all of the Tapis Systems that will be used by Tapis ETL to run our ETL Pipeline. We will be using a simplified single-data center model for this pipeline. In this setup, the Local Inbox, Local Outbox, and compute system will use LoneStar6 (LS6) as the host and the Remote Inbox and Remote Outbox will use the LS6 Globus Endpoint as the host.

Simplified ETL System Configurations Setup



Follow the instructions below to create the necessary Tapis Systems for your ETL Pipeline.

1.2 ETL System Confguration Components

Every ETL System Configuration is composed of two parts. The Data System and the Manifests System.

1.2.1 Data System

The Data System is a collection of properties that define where where data files are located (or where data files are supposed to be transferred) as well as how to perform integrity checks on that data. Below is an example of the Data System of a Remote Inbox ETL System Configuration (Manifests System ignored to simplify the schema)

```
{
    "remote_outbox": {
        "data": {
            "system_id": "etl.userguide.systema.<user_id>",
            "path": "/ETL/REMOTE-OUTBOX/DATA",
            "integrity_profile": {
                "type": "done_file",
                "done_files_path": "/ETL/REMOTE-OUTBOX/DATA",
                "include_patterns": ["*.md5"],
                "exclude_patterns": []
            },
            "include_patterns": ["*.txt"],
            "exclude_patterns": []
```

(continues on next page)

(continued from previous page)

```
        }
        "manifests": {}
    }
}
```

- `system_id` - The ID of Tapis System on which:
  - data files are stored (Outbox systems)
  - data files are to be transferred (Inbox systems)
- `path` - The path to the directory on the Tapis System where data files are staged
- `include_patterns` - An array of glob patterns that are used to filter filenames. All files matching the glob patterns are considered by Tapis ETL to be data files.
- `exclude_patterns` - An array of glob patterns that are used to filter filenames. All files matching the glob patterns are considered by Tapis ETL to be **non**-data files.

1.2.1.1 Data Integrity Profile

Every Data System has a Data Integrity Profile. The Data Integrity Profile is a set of instructions that informs Tapis ETL on how it should check the validity of all data files in the data path. There are 3 ways in which Tapis ETL can perform data integrity checks:

- `byte_check` - Checks the actual size (in bytes) of a data file against it's recored size in a manifest
- `checksum` - Takes the hash of a data file according to the specified hashing algorithm and checks that value against the value of the checksum in a manifest. This is an expensive process for large data files. The data integrity check type should only be used if data files are small or if absolutely necessary.
- `done_files` - Checks that a data file has a corresponding done file in which:
  - the done file matches a set of glob patterns
  - the done file's filename contains the corresponding data file's filename as a substring

Below are some example schemas of Data Systems with different Data Integrity Profiles

1.2.2 Manifests System

The Manfests System is a is a collection of properties that define where manifests for a given system are located as well as how to generate manifests. Below is an example of the Manifests System of a Remote Inbox ETL System Configuration (Data System ignored to simplify the schema)

```
{
    "remote_outbox": {
        "data": {}
        "manifests": {
            "system_id": "etl.userguide.systema.<user_id>",
            "generation_policy": "auto_one_per_file",
            "priority": "oldest",
            "path": "/ETL/REMOTE-OUTBOX/MANIFESTS",
            "include_patterns": [],
            "exclude_patterns": []
        }
    }
}
```

- `system_id` - The ID of Tapis System on which manifest files are stored or created

- `path` - The path to the directory on the Tapis System where the manifest files are stored or created
- `include_patterns` - An array of glob patterns that are used to filter filenames. All files matching the glob patterns are considered by Tapis ETL to be manifest files.
- `exclude_patterns` - An array of glob patterns that are used to filter filenames. All files matching the glob patterns are considered by Tapis ETL to be **non**-manifest files.
- `generation_policy` - Indicates how manifests will be generated for data files on the Manifests System. Must be one of the following values:
  - `manual` - A user must manually generate the manifests for the data files that they want their ETL Pipeline to process
  - `auto_one_per_file` - Generates one manifest per data file found in the data directory
  - `auto_one_for_all` - Generates one manifests for all data files found in the data directory
- `priority` - The order in which manifests should be processed. Must be one of the following values
  - `oldest` - Tapis ETL will process the oldest manifest in the manifests path
  - `newest` - Tapis ETL will process the newest manifest in the manifests path
  - `any` - Tapis ETL determine which manifests in the manifests path to process first

Note

Section 1 Punch List

- Created 1 Globus system with Tapis
- Registered credentials with Tapis for the Globus system
- Successully peformed a file listing with Tapis on the Globus system
- Created 1 Linux system with Tapis
- Registered credentials with Tapis for the Linux system
- Successully peformed a file listing with Tapis on the Linux system

---

18.2.3 2. ETL Pipeline Creation

In this section we create our ETL Pipeline. Once it's created, we will discuss each part of the ETL Pipeline schema in detail.

Note

This schema is built using data from resources that were created during the earlier sections of this User Guide.

2.1 Create the ETL Pipeline

Create an ETL Pipeline (createETLPipeline)

Save the following ETL pipeline definiton to a file named `etl-pipeline.json` in your current working directory.

```
{
    "id": "etl-userguide-pipeline-<user_id>",
    "before": null,
    "remote_outbox": {
        "data": {
            "system_id": "etl.userguide.systema.<user_id>",
            "path": "/ETL/REMOTE-OUTBOX/DATA",
        },
        "manifests": {
            "system_id": "etl.userguide.systemb.<user_id>",
            "generation_policy": "auto_one_per_file",
            "path": "/ETL/REMOTE-OUTBOX/MANIFESTS"
        }
    },
    "local_inbox": {
        "control": {
            "system_id": "etl.userguide.systemb.<user_id>",
            "path": "/ETL/LOCAL-INBOX/CONTROL"
        },
        "data": {
            "system_id": "etl.userguide.systema.<user_id>",
            "path": "/ETL/LOCAL-INBOX/DATA"
        },
        "manifests": {
            "system_id": "etl.userguide.systemb.<user_id>",
            "path": "/ETL/LOCAL-INBOX/MANIFESTS"
        }
    },
    "jobs": [
        {
            "name": "sentiment-analysis",
            "appId": "etl-sentiment-analysis-test",
            "appVersion": "dev",
            "nodeCount": 1,
            "coresPerNode": 1,
            "maxMinutes": 10,
            "execSystemId": "test.etl.ls6.local.inbox",
            "execSystemInputDir": "${JobWorkingDir}/jobs/${JobUUID}/input",
            "archiveSystemId": "test.etl.ls6.local.inbox",
            "archiveSystemDir": "ETL/LOCAL-OUTBOX/DATA",
            "parameterSet": {
                "schedulerOptions": [
                    {
                        "name": "allocation",
                        "arg": "-A TACC-ACI"
                    },
                    {
                        "name": "profile",
                        "arg": "--tapis-profile tacc-apptainer"
                    }
                ],
                "containerArgs": [
                    {
```

(continues on next page)

(continued from previous page)

```
                        "name": "input-mount",
                        "arg": "--bind $(pwd)/input:/src/input:ro,$(pwd)/output:/src/
↪output:rw"
                    }
                ],
                "archiveFilter": {
                    "includes": [],
                    "excludes": ["tapisjob.out"],
                    "includeLaunchFiles": false
                }
            }
        }
    ],
    "local_outbox": {
        "data": {
            "system_id": "etl.userguide.systema.<user_id>",
            "path": "/ETL/LOCAL-OUTBOX/DATA"
        },
        "manifests": {
            "system_id": "etl.userguide.systemb.<user_id>",
            "path": "/ETL/LOCAL-OUTBOX/MANIFESTS"
        }
    },
    "remote_inbox": {
        "data": {
            "system_id": "etl.userguide.systema.<user_id>",
            "path": "/ETL/REMOTE-INBOX/DATA"
        },
        "manifests": {
            "system_id": "etl.userguide.systemb.<user_id>",
            "path": "/ETL/REMOTE-INBOX/MANIFESTS"
        }
    },
    "after": null
}
```

Then submit the definition.

Once created, you can now fetch and run the pipeline

2.2 Remote Outbox

The Remote Outbox is the ETL System Configuration that tells Tapis ETL where data files to be processed are staged. Any data files placed on the Data System in the data directory (after applying the include and exclude pattern filters) are the files that will be processed during runs of an ETL Job.

How many get processed and in what order is determined by the manifests generated for those data files. Consider the schema below, specifically the Manifest Configuration.

```
{
    "data": {
        "system_id": "etl.userguide.systema.<user_id>",
        "path": "/ETL/REMOTE-OUTBOX/DATA"
```

(continues on next page)

(continued from previous page)

```
    },
    "manifests": {
        "system_id": "etl.userguide.systemb.<user_id>",
        "generation_policy": "auto_one_per_file",
        "path": "/ETL/REMOTE-OUTBOX/MANIFESTS"
    }
}
```

This Manifest Configuration tells Tapis ETL that the user wants Tapis ETL to handle manifests generation. According to the manifest `generation policy` of `auto_one_per_file`, Tapis ETL will generate one manifest for each data file that is not currently being tracked in another manifest.

Warning

Using automatic manifest generation policies without specifying a data integrity profile can beak a pipeline. The preferred manifest generation policy for the Remote Inbox is `manual`. Instructions for generating manifests will come in the following sections. For more info on other possible configurations, see the *Manifests Configuration* section

Tapis ETL will perform this operation for every untracked data file for every run of the pipeline. Whether this step runs or not has no effect on the actual data processing phase of the pipeline run (the phase where the ETL Jobs are run). If there are no data files to be tracked, Tapis ETL will simply move on to the next step; looking for an unprocessed manifest with a status of `pending` and submitting it for processing.

2.3 Local Inbox

Tapis ETL will transfer all of the data files from the Remote Outbox to the data directory of the Local Inbox for processing.

Note

When configuring your Local Inbox, consider that your ETL Jobs should run on a system that has a shared file system with the Local Inbox. The data path in the Local Inboxes Data Configuration should be accessible to the first ETL Job.

Notice that this ETL System Configuration has an addition property, `control`. This is simply a place that Tapis ETL will write accounting files to ensure the pipeline runs as expected. It is recommended that you use the same system as in Local Inbox's Manifest System, however any system to which Tapis ETL can write files to would work.

```
{
    "control": {
        "system_id": "etl.userguide.systemb.<user_id>",
        "path": "/ETL/LOCAL-INBOX/CONTROL"
    },
    "data": {
        "system_id": "etl.userguide.systema.<user_id>",
        "path": "/ETL/LOCAL-INBOX/DATA"
    },
    "manifests": {
        "system_id": "etl.userguide.systemb.<user_id>",
        "path": "/ETL/LOCAL-INBOX/MANIFESTS"
```

(continues on next page)

(continued from previous page)

```
    }
}
```

2.4 ETL Jobs

ETL Jobs are an ordered list Tapis Job definitions that are dispatched and run serially during a pipeline's transform phase. Tapis ETL dispatches these Tapis Jobs and uses the data files in a manifest as the inputs for the job. Once all of the ETL Jobs complete, the tranform phase ends.

```
[
    {
        "name": "sentiment-analysis",
        "appId": "etl-sentiment-analysis-test",
        "appVersion": "dev",
        "nodeCount": 1,
        "coresPerNode": 1,
        "maxMinutes": 10,
        "execSystemId": "test.etl.ls6.local.inbox",
        "execSystemInputDir": "${JobWorkingDir}/jobs/${JobUUID}/input",
        "archiveSystemId": "test.etl.ls6.local.inbox",
        "archiveSystemDir": "ETL/LOCAL-OUTBOX/DATA",
        "parameterSet": {
            "schedulerOptions": [
                {
                    "name": "allocation",
                    "arg": "-A TACC-ACI"
                },
                {
                    "name": "profile",
                    "arg": "--tapis-profile tacc-apptainer"
                }
            ],
            "containerArgs": [
                {
                    "name": "input-mount",
                    "arg": "--bind $(pwd)/input:/src/input:ro,$(pwd)/output:/src/
↪output:rw"
                }
            ],
            "archiveFilter": {
                "includes": [],
                "excludes": ["tapisjob.out"],
                "includeLaunchFiles": false
            }
        }
    }
]
```

Every ETL Job is furnished with the following envrionment variables which can be accessed at runtime by your Tapis Job.

- `TAPIS_WORKFLOWS_TASK_ID` - The ID of the Tapis Workflows task in the pipeline that is currently being executed

- `TAPIS_WORKFLOWS_PIPELINE_ID` - The ID of the Tapis Workflows Pipeline that is currently running
- `TAPIS_WORKFLOWS_PIPELINE_RUN_UUID` - A UUID given to this specific run of the pipeline
- `TAPIS_ETL_HOST_DATA_INPUT_DIR` - The directory that contains the data files for inital ETL Jobs
- `TAPIS_ETL_HOST_DATA_OUTPUT_DIR` - The directory to which output data files should be persisted
- `TAPIS_ETL_MANIFEST_FILENAME` - The name of the file that contains the manifest
- `TAPIS_ETL_MANIFEST_PATH` - The full path (including the filename) to the manifest file
- `TAPIS_ETL_MANIFEST_MIME_TYPE` - The MIME type of the manifest file (always `application/json`)

In addition to the envrionment variables, a *fileInput* for the manfiest file is added to the job definition to ensure that is available to the ETL Jobs runtime. In your application code, you can use the envrionment variables above to locate the manifest file and inspect its contents. This is useful if your application code does not know where to find its input data. The `local_files` array property of the manifest contains the list of input data files. The files are represented as objects in the `local_files` array and take the following form.

```
{
    "mimeType": null,
    "type": "file",
    "owner": "876245",
    "group": "815499",
    "nativePermissions": "rwxr-xr-x",
    "url": "tapis://etl.userguide.systemb.<user_id>/ETL/LOCAL-INBOX",
    "lastModified": "2024-02-26T22:51:31Z",
    "name": "data1.txt",
    "path": "ETL/LOCAL-INBOX/DATA/data1.txt",
    "size": 66
}
```

2.5 Local Outbox

Once the Transform Phase ends, all of the output data files should be in the data directory of the Local Inbox. These files are then tracked in manifests to be transferred in the Egress Phase of the pipeline.

```
{
    "data": {
        "system_id": "etl.userguide.systema.<user_id>",
        "path": "/ETL/LOCAL-OUTBOX/DATA"
    },
    "manifests": {
        "system_id": "etl.userguide.systemb.<user_id>",
        "path": "/ETL/LOCAL-OUTBOX/MANIFESTS"
    }
}
```

2.6 Remote Inbox

During the Egress Phase, all of the output data produced by the ETL Jobs is transferred from the Local Outbox to the Remote Inbox. Once all of the data files from an Egress Manifest are successfully transferred over, Tapis ETL will then transfer that manifest to the manifests directory of the Remote Inbox system to indicate that files transfers are complete.

```
{
    "data": {
```

(continues on next page)

(continued from previous page)

```
        "system_id": "etl.userguide.systema.<user_id>",
        "path": "/ETL/REMOTE-INBOX/DATA"
    },
    "manifests": {
        "system_id": "etl.userguide.systemb.<user_id>",
        "path": "/ETL/REMOTE-INBOX/MANIFESTS"
    }
}
```

Note

Section 2 Punch List

- Created an ETL Pipeline
- Can fetch the pipeline's details from Tapis Workflows

18.2.4 3. Running an ETL Pipeline

In this section we will make our final preperations and perform our first pipeline run. This ETL Pipeline is configured to run HPC jobs that perform sentiment analysis on our data files. It is safe to trigger multiple pipeline runs with Tapis ETL. Tapis ETL has a system for locking down ETL resources (such as manifests) to prevent race conditions for data files. In other words, multiple pipelines can be running in parallel, all of which can be guaranteed to be processing entirely different data files.

3.1 Staging data files to the Remote Outbox

Before running an ETL Pipeline, you must first ensure that there are data files in the data directory of the Remote Outbox. Running a pipeline before there is any data staged to it would result in a "no-op" for each of the 3 phases of the ETL pipeline.

3.2 Manifests

Manifests

Manifests are JSON files that track one or more data files on a Data System as well as those data file's progress through the various ETL Pipeline Phases. These manifest files contain a single manifest object that conforms to the schema below.

```
{
    "status": "pending",
    "phase": "ingress",
    "local_files": [],
    "transfers": [],
    "remote_files": [
        {
            "mimeType": null,
            "type": "file",
            "owner": "876245",
            "group": "815499",
            "nativePermissions": "rwxr-xr-x",
```

(continues on next page)

(continued from previous page)


```
            "url": "tapis://etl.userguide.systemb.<user_id>/ETL/LOCAL-INBOX",
            "lastModified": "2024-02-26T22:51:31Z",
            "name": "data1.txt",
            "path": "ETL/LOCAL-INBOX/DATA/data1.txt",
            "size": 66
        },
        {
            "mimeType": null,
            "type": "dir",
            "owner": "876245",
            "group": "815499",
            "nativePermissions": "rwxr-xr-x",
            "url": "tapis://etl.userguide.systemb.<user_id>/ETL/LOCAL-INBOX/data2.txt",
            "lastModified": "2024-02-26T22:51:38Z",
            "name": "data2.txt",
            "path": "ETL/LOCAL-INBOX/DATA/data2.txt",
            "size": 47
        }
    ],
    "jobs": [],
    "logs": [
        "2024-04-01 19:16:49.378611 Created"
    ],
    "created_at": 1711999009.378607,
    "last_modified": 1711999009.378607,
    "metadata": {}
}
```


3.3 Generating Manifests

For each data file or set of data files in the Remote Outbox that you want to run an ETL Job against, you will need to generate a manifest for them. Each manifest in the Remote Inbox corresponds with a single Transform, or ETL Job. Each data file in a single manifest will be used as input files for a single ETL Job.

There are two ways to generate manifest files. Manually and automatically. Automatic manifest generation is the simplest manifest generation policy to use but the least flexible. With automatic manifest generation, you can use the `auto_one_for_all` policy and generate a manifest for **all** untracked files in the data directory, or the `auto_one_per_file` and generate **one** manifest per file in the data directory.

For all situations in which you need to generate manifests for arbitrary files in the Remote Inbox, you must use the `manual` manifest generation policy

Note

Coming Soon - Manual manifest generation

If you are following the tutorial in the user guide, we previously configured our Remote Inbox to have a manifest generation policy of `auto_one_per_file` so Tapis ETL will generate the manifest for our `data1.txt` data file that we created in the previous step.

3.4 Run the Pipeline

Now that we have a data file (`data1.txt`) and a manifest tracking that data file, we can trigger the first run of our ETL Pipeline.

Run an ETL Pipeline (runPipeline)

To run a pipeline, you must:

1. Provide the `id` of the group that owns the pipeline
2. Provide the `id` of the pipeline
3. Belong to the group that owns the pipeline

Pipeline Arguments

When running an ETL Pipeline with Tapis ETL, you must provide the following arguments:

- `TAPIS_USERNAME` - Username of the Tapis user running the pipeline
- `TAPIS_PASSWORD` - Password of the user
- `TAPIS_BASEURL` - The URL of the Tapis Tenant in which your pipeline should run (should be the same as the base URL you used to create your pipeline)

Save the following Pipeline Args definiton to a file named `etl-pipeline-args.json` in your current working directory.

```
{
    "args": {
        "TAPIS_USERNAME": {
            "type": "string",
            "value": "<tapis_username>"
        },
        "TAPIS_PASSWORD": {
            "type": "string",
            "value": "<tapis_password>"
        },
        "TAPIS_BASE_URL": {
            "type": "string",
            "value": "https://tacc.tapis.io"
        }
    }
}
```

Then submit the definition.

3.5 Check the status of the Pipeline Run

Fetch Pipeline Run Details (getPipelineRun)

Fetch the details of a single pipeline run. You will need the following data. * `group_id` - id of the group that owns the pipeline * `pipeline_id` - id of the pipeline * `pipeline_run_uuid` - UUID of the pipeline run

3.6 Pipeline Run Complete

Once the pipeline has gone through the 3 phases, the ETL Pipeline will enter a terminal state of either `completed` or `failed`.

Here are the most common reasons why an ETL Pipeline may fail.

- Ingress transfer failed (Remote Outbox to Local Inbox)
- Egress transfer failed (Local Outbox to Remote Inbox)
- Malformed manifest - This is a critical and unrecoverable error
- One of the batch compute jobs (ETL Jobs) exited with a non-zero exit code

---

18.2.5 4. Resubmitting a Pipeline Run

Coming Soon

18.3 Quick Start

In this section we will be constructing an ETL pipeline that:

1. Detects the presence of data files and manifests on some remote system
2. Transfers those manifests and data files to a local system
3. Performs sentiment analysis on the text content of the data files and generates a result file for each analysis
4. Transfers the results files to a remote system for archiving

CHAPTER

NINETEEN

DEPLOYMENT & ADMINISTRATION GUIDE - OVERVIEW

Note

This guide is for users wanting to deploy Tapis software in their own datacenter. Researchers who simply want to make use of the Tapis APIs do not need to deploy any Tapis components and can ignore this guide.

The Deployment & Administration Guide for Tapis provides documentation for system operators wishing to run Tapis services in their own datacenter.

- *Considerations and Prerequisites*: Covers considerations and prerequisites before embarking on a Tapis deployment.
- *Tapis Deployer*: Covers using Tapis Deployer software to generate deployment scripts.
- *Tapis Vault*: Describes HashiCorp Vault integration.
- *Managing Secrets with SkAdmin*: Describes how to use SkAdmin to manage secrets.
- *Tapis Administration*: Describes common administrative tasks that must be performed when operating Tapis.
- *Upgrading Tapis*: Describes the process of upgrading an existing Tapis installation.
- *Tapis Artifacts & Architecture*: Provides a detailed overview of the Tapis system architecture.

CHAPTER

TWENTY

CONSIDERATIONS AND PREREQUISITES

Note

This guide is for users wanting to deploy Tapis software in their own datacenter. Researchers who simply want to make use of the Tapis APIs do not need to deploy any Tapis components and can ignore this guide.

If you are planning to deploy Tapis software at your own institution, there are a number of considerations that should be thought through and prerequisites that should be met before beginning. Administering a Tapis installation is a significant commitment and will require substantial computational resources and human effort to be successful. Proper planning and design upfront will reduce the time required to get to a working Tapis installation that meets your institution's requirements.

Warning

We strongly recommend consulting with the Tapis core team to plan your deployment and operations.

20.1 Site Type

Tapis supports geographically distributed deployments where different components are running in different data centers and managed by different institutions. These physically isolated installations of Tapis software are referred to as *sites*. There is a single *primary site* and zero or more *associate sites* within a Tapis installation.

When deploying Tapis software to a new institution, one must decide whether to deploy a primary site or an associate site. There are several aspects to consider:

- Primary sites must deploy all Tapis services, which increases the administrative burden and computational resource requirement significantly.
- Associate sites can run a few, critical services on-premise while deferring to the primary site for all other services. Associate sites still maintain full administrative control over services running at their site and full administrative control over tenants that they own.
- Primary sites must manage all site and tenant configuration, including signing keys.
- Primary sites are completely independent from any other Tapis software. Associate sites depend on a primary site to be functional – if a primary site goes offline, the associate site will not function.

The Texas Advanced Computing Center hosts a primary site for the "main" Tapis installation at the tapis.io domain. If you are interested in deploying an associate site as part of the main Tapis installation, please contact us.

20.2 Kubernetes

The official Tapis installation scripts target Kubernetes for container orchestration. In the future, support for using Docker Compose instead of Kubernetes will be added. For now, a Kubernetes cluster is required to deploy Tapis using the official installation software.

Successful deployment and operation of Tapis requires the operator(s) to have strong working knowledge of Kubernetes concepts and abstractions, including Jobs, Deployments, PVCs, and Services, among others. Introduction to and administration of Kubernetes is beyond the scope of this document.

20.3 Environments and Capacity Planning

Deploying Tapis requires dedicated capacity in your Kubernetes cluster. A minimum of two Tapis installations for each site is required to allow updates to be applied in a pre-production environment before impacting production services.

The following are minimum requirements for each Tapis environment/installation:

- 1+ control plane node. 2 cores/8GB Mem/32 GB disk for containers (/var/lib/containerd or /var/lib/docker or similar)
- 2+ worker nodes. 4 cores/16 GB mem/64 GB disk for containers
- **Must have the ability to create PVC. Tapis will need to know the storage class name.**
  - Both Ceph & NFS have been used successfully for this purpose
- Each Tapis installation must be deployed entirely within a single Kubernetes namespace, and the installation scripts require full administrative access within their namespace to create pods, PVCs, services, etc. No special or elevated privileges on the Kubernetes cluster level are required.
- Outbound networking is required for all Tapis services.
- **Inbound networking is only required for the external-facing IP which is then proxied to the rest of the Tapis services inside Kubernetes.**
  - This "ingress" can be handled various ways; at TACC we use a manual haproxy server.
  - In traditional Kubernetes setups a combo of Load Balancer & Ingress services may also work
- Based on Remote site firewall config, the Tapis Kube cluster may require special rules to be able to talk to "local" resources (clusters, storage, instruments, etc.)

Note that for primary sites and associate sites that receive significant usage, the following compute requirements are strongly recommended:

- 2+ worker nodes. 16 cores/64 GB mem/128 GB disk for containers

20.3.1 Namespaces

As mentioned above, each Tapis environment/installation must be deployed into its own, dedicated Kubernetes namespace. Uniqueness assumptions made by the deployment architecture and scripts imply that attempting to deploy multiple Tapis instances into the same Kubernetes namespace will result in failures.

20.3.2 Roles, Permissions and Service Accounts

The Tapis deployment scripts utilize the `kubectl` command-line utility to register jobs, pods PVCs, secrets, services, and other objects. Registering these objects requires certain permissions at the Kubernetes level. Additionally, some, but not all, Tapis jobs and pods use the Kubernetes API as part of their operation. In the spirit of least privilege, we recommend the following:

1. Create a clusterRole (e.g., "tapis-manager") with sufficient privileges to create and manage the Kubernetes objects associated with the Tapis installation (jobs, pods, PVCs, secrets, services, etc).
2. Create a Kubernetes service account for each namespace/Tapis installation.
3. Create a roleBinding for the namespace and service account to grant the service account the privileges in that particular namespace.

Note

It is important in Step 3 that you use a roleBinding, not a clusterRoleBinding, as the service account only needs the privileges in the specific namespace.

Note

We explicitly recommend that a new service account be created, as described in step 2, and that the roleBindings in step 3 be made to this new service account, not the default account. See the definition of the `skadmin_sk_privileged_sa` variable, described here, for more details.

For even better security, we suggest explicitly differentiating the privileges needed for the deployment of Tapis (that use the Deployer burnup and burndown scripts) versus the privileges needed by the Kubernetes jobs and deployments (i.e., the service account described in Step 2 above).

20.3.3 Public IP Addresses, Domains and TLS Certificates

All Tapis services respond to HTTP requests made to a configurable domain assigned to a site, called the "site domain". By default, each tenant is defined to a subdomain of the site domain. For example, the primary site at TACC has domain tapis.io, and each tenant is assigned the subdomain of the form <tenant_id>.tapis.io (e.g., designsafe.tapis.io for the DesignSafe project and cyverse.tapis.io for the CyVerse project, etc.).

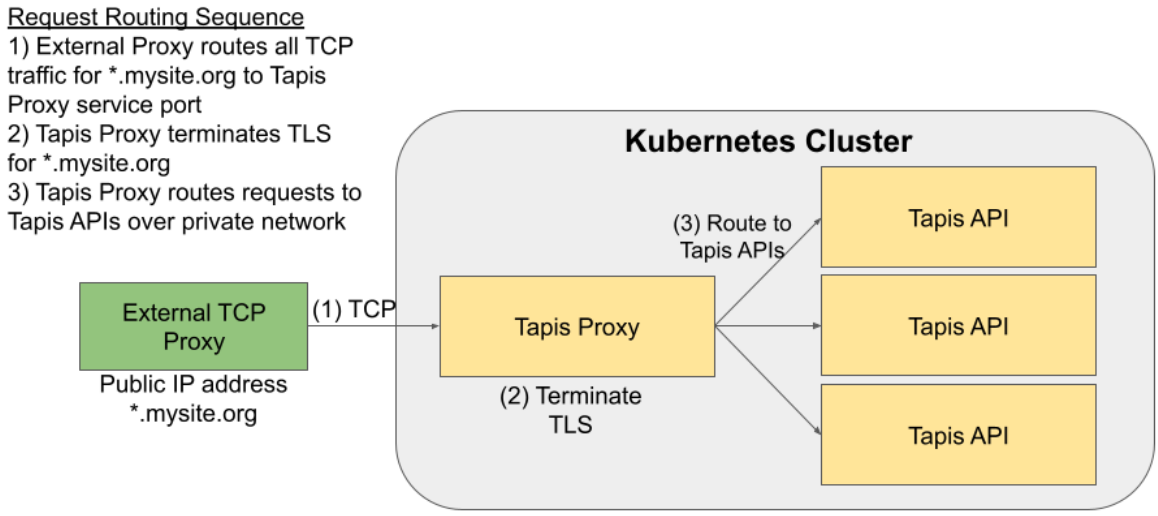
The official Tapis deployment tools will deploy and configure a special HTTP proxy, called Tapis Proxy, to handle TLS negotiation and service request routing for all tenants owned by the site. The official Tapis Proxy, or an HTTP reverse proxy with equivalent functionality, is strictly required for the Tapis services to function. In order for the Tapis Proxy to be configured and deployed properly, the following must be available and provided:

- A site domain, owned by the institution, resolvable by DNS to a public IP address in the site's datacenter.
- A wildcard TLS certificate used for encryption for all top-level subdomains of the site domain. For example, if the site domain is `mysite.org`, a wildcard certificate for `*.mysite.org` must be provided. In this case, tenants belonging to the site will use `<tenant_id>.mysite.org` as the base URL for making HTTP requests to Tapis.
- Assignment in DNS to the public IP address for all subdomains `*.mysite.org`.
- A basic TCP reverse proxy deployed to and listening on the public IP address, to route all HTTP traffic to the Tapis Proxy running in the Kubernetes cluster. HAProxy, nginx, Apache HTTPd, or similar software can be used for this purpose.

A key point is that the Tapis Proxy does **not** typically listen directly on the public IP address. This is because the Tapis Proxy is deployed as a pod to the Kubernetes cluster, and a standard Kubernetes installation does not have a way of assigning a public IP address to a pod. The typical request routing is depicted in the following figure:

Note

One must typically deploy the external reverse proxy outside of Kubernetes.



Support for deploying the Tapis Proxy with standard certificates for each domain (instead of a single, wildcard certificate) will be added in a future release. Additionally, support for having Tapis Proxy generate its own certificates using LetsEncrypt will be added in a future release.

20.4 Tenants & Authenticators

Every site must include a minimum of two tenants to function: an administrative tenant for the site, where the Tapis services running at that site authenticate and manage authorization (roles, permissions, etc.) and other service data, and one or more user tenants where actual users interact with Tapis.

The Tapis user tenants require an external facing "authenticator" to allow users to generate Tapis JWTs, and the Tapis project includes a "default" Authenticator service which can be used for one or more user tenants. Still, configuring authenticators, including the default Tapis Authenticator, is non-trivial and requires planning.

The Tapis Authenticator can work with the following ways:

1) Use an externally deployed LDAP server and configured organizational unit (ou) to check user credentials directly. This will require: a) creating an LDAP record within the Tenants service and assigning the ldap identifier to the `user_ldap_connection_id` attribute on the tenant object; and b) creating a secret in the Tapis Security Kernel for the authenticator to use to bind to the LDAP.
2) Use a third-party OIDC provider, such as GitHub, Google, or Globus. This option requires additional configuration.
3) Use a test LDAP server that the Tapis deployment tools can deploy; this option is not appropriate for external-facing users, but it can be used for testing and evaluation purposes. This option requires minimal configuration.

At a high level, institutions have the following options:

1) Use the Tapis Authenticator service for all user tenants.
2) Use a third-party authentication mechanism for all user tenants.
3) Use the Tapis Authenticator for some user tenants and a third-party authentication mechanism for others.

By default, the Tapis deployment tools will deploy the Tapis Authenticator configured to be used for all of the site's user tenants, and it will deploy the test LDAP server. Moreover, Tapis Authenticator will be configured to make use of the test LDAP server for a special user tenant, referred to as the site's "dev" tenant. The tenant id for the site's "dev" tenant can be provided to the deployment tools.

20.5 Deployer

The official Tapis installation scripts are based on the Ansible project and are available in the Tapis Deployer repository. Deployment of Tapis using the official scripts involves the following high-level steps:

1. Check out the Tapis Deployer repository
2. Provide some configuration for your site
3. Run the generate script that will generate a set of "deployment files" that will be used to start and manage the running Tapis services. These deployment files should be checked into a git repository so that they can be versioned as the files are regenerated using newer versions of deployer.
4. If necessary, check out the deployment files to the deployment environment (for example, the machine that has access to the Kubernetes API).
5. Run deployment scripts to start/update the Tapis services.

20.6 Additional Requirements for an Initial Associate Site Deployment

Some additional setup is required at the primary site before an associate site can be created. These steps include:

1) Creating the associate site record on the primary site table.

2) Creating the Associate site tenant records (in DRAFT mode) on primary tenants table.

Contact the administrators of the primary site to have these steps completed before beginning your associate site deployment.

CHAPTER

TWENTYONE

TAPIS DEPLOYER

Note

This guide is for users wanting to deploy Tapis software in their own datacenter. Researchers who simply want to make use of the Tapis APIs do not need to deploy any Tapis components and can ignore this guide.

In this section, we cover using Tapis Deployer to generate Tapis deployment files. We cover how to get the Tapis deployer software, create configuration for your site using Ansible host_vars, and running the generate playbook. Then we describe using the control scripts (`burnup` and `burndown`)

Keep in mind that the process of using Deployer involves the following high-level steps:

1. Check out the Tapis Deployer repository
2. Provide some configuration for your site
3. Run the generate script that will generate a set of "deployment files" that will be used to start and manage the running Tapis services. These deployment files should be checked into a git repository so that they can be versioned as the files are regenerated using newer versions of deployer.
4. If necessary, check out the deployment files to the deployment environment (for example, the machine that has access to the Kubernetes API).
5. Run deployment scripts to start/update the Tapis services.

Steps 1, 2 and 3 can be performed on a separate machine from the deployment environment. Steps 4 and 5 must be performed on a machine that has access to the Kubernetes API and the `kubectl` program where the Tapis services will be deployed.

21.1 Installing Deployer

The Tapis Deployer project is hosted on GitHub. Use the tags download page to download a specific version of the Deployer software. For example, to get version 1.3.5 of the Deployer software, we could do the following in a terminal:

```
# download the tar archive
wget https://github.com/tapis-project/tapis-deployer/archive/refs/tags/tapis-deployer-1.
↪3.1.tar.gz

# unpack the directory
tar -xf tapis-deployer-1.3.1.tar.gz

# produces a new directory, tapis-deployer-1.3.5, in the current working directory
ls -l tapis-deployer-1.3.5
```

(continues on next page)

(continued from previous page)

```
-rw-rw-r-- 1 jstubbs jstubbs 2340 Mar 22 10:13 CHANGELOG.md
drwxrwxr-x 3 jstubbs jstubbs 4096 Mar 22 10:13 inventory_example
drwxrwxr-x 3 jstubbs jstubbs 4096 Mar 22 10:13 playbooks
-rw-rw-r-- 1 jstubbs jstubbs 2014 Mar 22 10:13 README.md
```

Deployer is based on the Ansible project, and Ansible must be installed as well. For Debian/Ubuntu bases distributions, we recommend using the Ansible apt package:

```
apt-get install ansible
```

Refer to the official Ansible documentation to get Ansible installed on your machine.

21.1.1 Tapis Service & Deployer Versions

All Tapis software, including all API components, programming language SDKs and libraries, deployment jobs and utilities, are versioned using semantic versioning. Tapis Deployer itself is versioned using semantic versions, and each version of Deployer is built to work with a specific set of Tapis service versions. For this reason, Deployer releases occur regularly.

Warning

While it is technically possible to change the versions of the Tapis components that a specific version of Deployer deploys, this should be avoided. Any specific version of Deployer was tested with a specific set of Tapis software versions – trying to change the versions could result in errors.

21.2 Configuring Your Site

Deployer relies on a set of configuration files provided by you, the operator, to generate the deployment scripts for a given site. Configuring Deployer correctly can be challenging; to simplify the process, Deployer bundles a number of default configurations which are suitable for most but not all use cases. When planning a Tapis site deployment, be sure to review all required and optional configurations to ensure that your generated deployment scripts will be correct.

There are two primary configuration files – the Inventory file and the Host Vars file – that must be provided to Tapis Deployer to generate the deployment scripts. Together with additional supporting files, such as the TLS certificate files for the site domain, these files are then used to deploy Tapis components to the Kubernetes cluster.

21.2.1 Inventory File

The Tapis Deployer Inventory File is an Ansible inventory file specifying the main configuration file to use (called a "host vars" file) for each Tapis installation. Note that more than one Tapis installation can be specified in the inventory file. The inventory file also specifies the `tapisflavor` variable to use for each Tapis installation specified in the file. Currently, `tapisflavor` must be set to the value `kube` (for Kubernetes deployments), but a future version of Deployer will support additional types of deployment targets.

Copy and past the following code snippet into a file and change the highlighted `<tapis_installation_name>` to a name for your Tapis installation, such as "Tapis-test" or "Tapis-prod". Also, you can name the file anything you like, but a suitable name would be `tapis_installations.yml`.

```
1 tapis_installs:
2   hosts:
3     # Replace with a name for your Tapis installation; for example, "tapis-dev",
4     # "tapis-prod", etc. By default, Deployer uses this name for the directory
```

(continues on next page)

(continued from previous page)

```
    # where it writes its output files, though this can be changed.
     <tapis_installation_name>:
       ansible_connection: local
       tapisflavor: kube
    # Add additional installations here...
```

21.2.2 Host Vars File

Create a directory called `host_vars` in the same directory as the inventory file, and inside the `host_vars` directory, create a file with the same name as the `<tapis_installation_name>` used in the inventory file above. The file structure should look similar to the following, where we are using the name `tapis-test.yml` for the `<tapis_installation_name>`:

```
tapis_installations.yml
host_vars/
  * tapis-test.yml
```

The `<tapis_installation_name>` will hold all of the configuration, in the form of variables and values in YAML format, for that Tapis installation. Broadly, there are required fields that every site administrator must provide and there are optional fields that can be provided if the defaults set in Deployer are not appropriate. Required and optional fields depend, to some extent, on whether a primary or associate site is being deployed.

Below we include the required fields for both primary and associate sites as well as a few of the simplest optional fields that can be configured. The Advanced Configuration Options section goes into detail about additional advanced customizations that can be achieved.

21.3 Required Fields – All Sites

The following fields must be configured in the Host Vars file for all sites, including associate sites and primary sites.

- `global_tapis_domain` – Domain name for the site. Must be owned by the institution, resolvable by DNS to a public IP address in the site's datacenter. See the Public IP Addresses, Domains and TLS Certificates subsection of the Capacity Planning section for more details. Do not include "https://" at the beginning of the value.

  Examples:

  ```
  global_tapis_domain: tapis.io
  ```

  ```
  global_tapis_domain: develop.tapis.io
  ```

- `site_type` – Whether the site is a primary site or an associate site. The value of should be an integer: `1` for a primary site and `2` for an associate site.

  Examples:

  ```
  site_type: 1
  ```

  ```
  site_type: 2
  ```

- `global_site_id` – The Tapis id for the site being deployed. Notes: for associate sites, the site id must be agreed to with the primary site prior to installation, and the associate site record must be added to the primary site's site table.

  Examples:

```
global_site_id: tacc
```

```
global_site_id: uh
```

- `global_storage_class` – The storage class, in the Kubernetes cluster, that can be used for creating persistent volumes. Options such as `rbd` (for Ceph-based storage), `nfs`, `cinder`, etc. may be appropriate. The value should be recognized on your Kubernetes cluster.

  Examples:

```
global_storage_class: rbd
```

- `global_primary_site_admin_tenant_base_url` – The URL to the admin tenant for the primary site associated with the site being deployed. If deploying a primary site, this is likely to have the value `https://admin.{{ global_tapis_domain }}`; however, for associate sites, the value will use a different domain.

  Examples:

```
global_primary_site_admin_tenant_base_url: https://admin.tapis.io
```

- `proxy_nginx_cert_file` – Path to the wildcard certificate file to be used for the site domain and all subdomains. Note that this path should be a valid path on the deployment machine, i.e., the machine where the Tapis Deployer output files will be used to deploy the Tapis components to Kubernetes. Note also that this file should contain the host certificate as well as the full CA chain.

  Examples:

```
proxy_nginx_cert_file: $HOME/ssl/star.tapis.io.pem
```

- `proxy_nginx_cert_key` – Path to the wildcard certificate key file to be used for the site domain and all subdomains. Note that, just as with `proxy_nginx_cert_file`, this path should be a valid path on the deployment machine, i.e., the machine where the Tapis Deployer output files will be used to deploy the Tapis components to Kubernetes.

  Examples:

```
proxy_nginx_cert_key: $HOME/ssl/star.tapis.io.key
```

21.4 Optional Fields – All Sites

The following fields can optionally be provided in the Host Vars file.

- `tapisdir` – The path on the local machine where Deployer will write the deployment script directory.

  Default Value: `$HOME/.tapis/{{ inventory_name }}`

  Examples:

```
tapisdir: /home/cic/deployments/tapis-test
```

- `tapisdatadir` – The path on the local machine where Deployer & Tapis scripts will write important stateful data.

  Default Value: `$HOME/.tapis-data/{{ inventory_name }}`

  Examples:

```
tapisdirdata: /home/cic/deployments/tapis-test-data
```

- `vault_raft_storage` – Whether to use Raft storage for Vault.

  Default Value: `true`

  Examples:

```
vault_raft_storage: false
```

Warning

Using the *file* storage type for Vault is not considered viable for production environments. At the same time, changing the storage type from *file* to *Raft* requires a manual migration. Attempting to change the Tapis Vault with a different storage type without performing the manual migration could result in secret loss and permanent corruption of the Tapis installation.

- `skadmin_sk_privileged_sa` – The name of a service account to use when deploying certain Tapis components. If specified, this service account should have sufficient privileges to create and manage various Kubernetes API objects, including: jobs, pods, PVCs, and secrets. If this variable is not set, then no value will be specified for the `serviceAccountName` attribute and Kubernetes will fall back to using the `default` service account (in which case the default account must have sufficient privileges to create and manage the Tapis Kubernetes objects).

  Default Value: None (the value of `default` is supplied by Kuberentes).

  Examples:

```
skadmin_sk_privileged_sa: tapis-manager
```

21.5 Generating the Tapis Deployment Script Directory

Once the Tapis Deployer software and dependencies have been installed and the inventory and host vars files written, the Tapis deployment script directory can be generated. The deployment script directory contains the actual deployment scripts that will be used to deploy and manage Tapis components. Tapis Deployer will write the deployment scripts to the `tapisdir` path, which can optionally be set in the Host Vars file (see previous section).

Generate the Tapis deployment scripts directory using the following command:

```
ansible-playbook -i /path/to/inventory_file.yml /path/to/deployer/playbooks/generate.yml
```

For example, given a project structure like the following, with the Tapis Deployer installation in the same directory as the inventory file and host vars directory:

```
tapis_installations.yml
tapis-deployer-1.3.1/
  * CHANGELOG.md
  * playbooks/
  * inventory_example/
  * README.md
host_vars/
  * <tapis_installation_name>
```

we can execute the following command from within the project root directory to generate the Tapis deployment script directory:

```
ansible-playbook -i tapis_installations.yml tapis-deployer-1.3.1/playbooks/generate.yml
```

Note

When executing *ansible-playbook*, all Tapis installations defined in the inventory file will be generated. Use `-l <tapis_installation_name>` to only generate one installation.

Generating the deployment script directory takes quite a bit of some time. If you just need to generate (or regenerate) one directory within the deployment script directory, you can issue the following:

```
ansible-playbook -i /path/to/inventory_file.yml /path/to/deployer/playbooks/generate-
↪single-component.yml -e comp=<component>
```

For example, with the same file structure as above, we could regenerate just the *workflows* directory using:

```
ansible-playbook -i tapis_installations.yml tapis-deployer-1.3.1/playbooks/generate-
↪single-component.yml -e comp=workflows
```

21.6 Deployment Script Directory Structure

The deployment script directory is structured as follows:

```
actors/   # the Actors component
  burnup
  burndown
  # k8s .yml files for actors...

admin/   # the admin component, not a Tapis service
  backup/
  # . . .
  verification/

apps/   # the Apps component
  burnup
  burndown
  # k8s .yml files apps...

# ... additional component directories ...

burnup     # top-level burnup script
burndown   # top-level burndown script

# ... additional component directories ...
```

At the top level, there is a directory for every Tapis *component* that will be deployed. Note that most components are Tapis services, such as Actors and Apps, but some components, such as `admin`, `skadmin` and `vault` are not Tapis services but are instead components needed to make the deployment work.

Except for `admin`, each component contains a `burnup` and `burndown` script, together with some yaml files for defining the Kuberentes objects. The `burnup` BASH script is a convenience utility for creating the Kuberentes objects while the `burndown` script can be used to remove the objects. Similarly, there is a top-level `burnup` and `burndown` script to

create/remove all the Tapis objects. The top-level scripts call the individual component `burnup``and ``burndown` scripts, respectively.

21.7 Using the Deployer Control Scripts

As mentioned above, the deployment script directory contains bash scripts called `burnup` and `burndown`, referred to as the Deployer control scripts. These scripts provided convenience functions for managing entire sets of Tapis components at once. Deploying Tapis using the control scripts involves a three step process:

1. Initialize the Tapis Deployment
2. Deploy the Primary Tapis Services
3. Deploy the Secondary Tapis Services

We detail each step in the following subsections. We recommend proceeding in this order, ensuring that each step finishes to completion and verify that it works before moving onto the next step.

21.8 Initialize the Tapis Deployment

Start by creating the initial Kubernetes objects:

```
./burnup init
```

You will see a lot of outputs written to the screen. Kubernetes is a declarative system, where API calls are used to describe the *desired* state on the cluster and Kubernetes works to make the *actual* state converge to the desired state. In general there is no problem with re-running a control script step more than once, because we are simply re-declaring the desired state to be the same state we declared previously. As a result, you can see messages such as:

```
service/apps-api unchanged
```

This just means the command did not change anything about the desired state so Kuberentes made no update.

Also, it is quite normal to see Error messages indicating that some Kuberentes object was not found; for example:

```
Error from server (NotFound): secrets "vault-keys" not found
```

This could mean that one Kuberentes object definition references another object definition that has yet to finish creating.

Finally, you may see related errors such as:

```
Error from server (AlreadyExists): secrets "vault-token" already exists
```

Before moving onto the next step, we should validate that the initial objects all completed. Using `kubectl` we should check the output of the following commands:

Check the services:

```
kubectl get services
NAME                              TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)
actors-admin                      ClusterIP   10.105.126.200   <none>        5000/TCP
actors-events                     ClusterIP   10.110.114.165   <none>        5000/TCP
actors-grafana                    ClusterIP   10.105.16.173    <none>        3000/TCP
actors-mes                        ClusterIP   10.96.160.55     <none>        5000/TCP
actors-metrics                    ClusterIP   10.99.139.105    <none>        5000/TCP
actors-mongo                      NodePort    10.103.92.102    <none>
```

(continues on next page)

(continued from previous page)

```
↪27017:32340/TCP
actors-nginx                    NodePort    10.111.143.102   <none>        80:31633/
↪TCP
actors-prometheus               ClusterIP   10.109.2.194     <none>        9090/TCP
actors-rabbit                   NodePort    10.106.132.99    <none>        5672:31108/
↪TCP
actors-rabbit-dash              ClusterIP   10.105.199.16    <none>        15672/TCP
actors-reg                      ClusterIP   10.107.149.161   <none>        5000/TCP
apps-api                        NodePort    10.110.168.192   <none>        8080:32718/
↪TCP
apps-api-debug                  NodePort    10.100.5.250     <none>        8000:30225/
↪TCP
apps-pgadmin                    NodePort    10.102.221.245   <none>        80:31458/
↪TCP
apps-postgres                   ClusterIP   10.104.211.25    <none>        5432/TCP
authenticator-api               NodePort    10.97.35.247     <none>        5000:31167/
↪TCP
authenticator-ldap              ClusterIP   10.97.243.117    <none>        389/TCP
authenticator-postgres          ClusterIP   10.107.198.0     <none>        5432/TCP
chords-app                      NodePort    10.109.154.215   <none>        80:30156/
↪TCP
chords-influxdb2                ClusterIP   10.100.107.154   <none>        8086/TCP,
↪8083/TCP
chords-mysql                    ClusterIP   10.111.198.225   <none>        3306/TCP
files-api                       NodePort    10.101.53.166    <none>        8080:31557/
↪TCP
files-debug                     NodePort    10.107.253.44    <none>        8000:32367/
↪TCP
files-postgres                  ClusterIP   10.107.1.254     <none>        5432/TCP
files-rabbitmq                  ClusterIP   10.110.250.244   <none>        5672/TCP
globus-proxy                    ClusterIP   10.96.141.179    <none>        5000/TCP
jobs-api                        NodePort    10.110.93.52     <none>        8080:30577/
↪TCP
jobs-api-debug                  NodePort    10.100.250.144   <none>        8000:30813/
↪TCP
jobs-api-other                  NodePort    10.102.208.122   <none>        6157:30078/
↪TCP
jobs-api-ssl                    NodePort    10.105.51.28     <none>        8443:32513/
↪TCP
jobs-pgadmin                    NodePort    10.102.30.118    <none>        80:31786/
↪TCP
jobs-postgres                   ClusterIP   10.104.52.113    <none>        5432/TCP
jobs-rabbitmq                   ClusterIP   10.105.69.98     <none>        5672/TCP,
↪15672/TCP
jobs-rabbitmq-mgmt              NodePort    10.101.83.72     <none>
↪15672:30985/TCP
monitoring-exporter             NodePort    10.104.19.250    <none>        8000:32311/
↪TCP
monitoring-grafana              NodePort    10.105.48.54     <none>        3000:32088/
↪TCP
monitoring-prometheus           NodePort    10.101.27.134    <none>        9090:32204/
↪TCP
```

(continues on next page)

(continued from previous page)

```
notifications-api                NodePort    10.111.161.227   <none>        8080:31399/
↪TCP
notifications-pgadmin            NodePort    10.96.236.253    <none>        80:31703/
↪TCP
notifications-postgres           ClusterIP   10.99.47.18      <none>        5432/TCP
notifications-rabbitmq           ClusterIP   10.107.233.223   <none>        5672/TCP,
↪15672/TCP
notifications-rabbitmq-mgmt      NodePort    10.104.109.239   <none>        ␣
↪15672:32511/TCP
pgrest-api                       NodePort    10.107.91.195    <none>        5000:30084/
↪TCP
pgrest-postgres                  ClusterIP   10.101.255.95    <none>        5432/TCP
pgrest-postgres-nodeport         NodePort    10.103.193.222   <none>        5432:30525/
↪TCP
pods-api                         ClusterIP   10.106.237.143   <none>        8000/TCP
pods-postgres                    NodePort    10.100.171.106   <none>        5432:31128/
↪TCP
pods-rabbitmq                    ClusterIP   10.111.198.30    <none>        5672/TCP
pods-rabbitmq-dash               NodePort    10.111.90.160    <none>        ␣
↪15672:30061/TCP
pods-traefik                     ClusterIP   10.111.26.233    <none>        80/TCP
pods-traefik-dash                NodePort    10.105.118.198   <none>        8080:30146/
↪TCP
registry                         NodePort    10.97.98.114     <none>        5000:31275/
↪TCP
restheart                        ClusterIP   10.107.197.65    <none>        8080/TCP
restheart-debug                  NodePort    10.103.14.131    <none>        8080:32023/
↪TCP
restheart-mongo                  NodePort    10.109.224.10    <none>        ␣
↪27017:31792/TCP
restheart-security               NodePort    10.105.16.196    <none>        8080:30792/
↪TCP
site-router-api                  NodePort    10.102.33.197    <none>        8000:30063/
↪TCP
sk-api                           NodePort    10.107.235.138   <none>        8080:31645/
↪TCP
sk-api-debug                     NodePort    10.106.88.188    <none>        8000:31797/
↪TCP
sk-api-other                     NodePort    10.105.105.97    <none>        6157:31086/
↪TCP
sk-api-ssl                       NodePort    10.99.148.218    <none>        8443:30128/
↪TCP
sk-pgadmin                       NodePort    10.101.207.66    <none>        80:30046/
↪TCP
sk-postgres                      ClusterIP   10.96.73.92      <none>        5432/TCP
streams-api                      NodePort    10.98.10.161     <none>        5000:30552/
↪TCP
systems-api                      NodePort    10.108.23.253    <none>        8080:32072/
↪TCP
systems-api-debug                NodePort    10.97.231.157    <none>        8000:31973/
↪TCP
systems-pgadmin                  NodePort    10.108.234.139   <none>        80:30892/
```

(continues on next page)

(continued from previous page)

```
↪TCP
systems-postgres                    ClusterIP   10.101.21.137    <none>        5432/TCP
tapis-nginx                         NodePort    10.107.224.176   <none>        80:30175/
↪TCP,443:31864/TCP
tapisui-service                     NodePort    10.107.80.97     <none>        3000:31766/
↪TCP
tenants-api                         NodePort    10.109.125.21    <none>        5000:31327/
↪TCP
tenants-postgres                    ClusterIP   10.102.182.23    <none>        5432/TCP
tokens-api                          NodePort    10.110.229.6     <none>        5000:32706/
↪TCP
vault                               ClusterIP   10.101.97.112    <none>        8200/TCP
```

Note: The number of services will depend on the site type being deployed.

Check the PVCs:

```
kubectl get pvc
NAME                           STATUS   VOLUME                                     ␣
↪CAPACITY   ACCESS MODES   STORAGECLASS   AGE
actors-mongo-backup-vol01      Bound    pvc-fbb44e18-0256-4d0b-b799-a703b0f477b6   10Gi ␣
↪     RWO            rbd-new        8h
actors-mongo-vol01             Bound    pvc-d3c224eb-5930-4700-8b8a-5f1ae0f2a921   40Gi ␣
↪     RWO            rbd-new        8h
actors-rabbitmq-vol01          Bound    pvc-20dc6e04-e2bb-48b6-8647-ee28081eb0c3   20Gi ␣
↪     RWO            rbd-new        6h19m
apps-postgres-vol01            Bound    pvc-f1320b94-6069-41d3-a26a-91a3ebaaed21   20Gi ␣
↪     RWO            rbd-new        8h
authenticator-ldap-vol01       Bound    pvc-05ae6dbb-e46d-4383-af6b-f1d2726d6529   10Gi ␣
↪     RWO            rbd-new        4d4h
authenticator-postgres-vol01   Bound    pvc-4fa4b8b4-dd31-42f6-afcf-d3fe41eeb723   20Gi ␣
↪     RWO            rbd-new        4d4h
files-pgdata                   Bound    pvc-8c9b4e7b-feee-4823-96e2-ebe5631cd4ca   10Gi ␣
↪     RWO            rbd-new        8h
files-rabbitmq-data            Bound    pvc-0f060f7b-53a7-41e5-ae4a-4dcbdcf47eb1   10Gi ␣
↪     RWO            rbd-new        8h
jobs-postgres-vol01            Bound    pvc-62f74888-028c-4e5f-99c4-a3dd1f16881c   20Gi ␣
↪     RWO            rbd-new        8h
jobs-rabbitmq-vol01            Bound    pvc-08f6c91c-8797-4515-b6c7-28fb839fc1c2   10Gi ␣
↪     RWO            rbd-new        8h
notifications-postgres-vol01   Bound    pvc-d1a662de-60fc-4ead-a8c2-a84e4d302b2e   20Gi ␣
↪     RWO            rbd-new        8h
notifications-rabbitmq-vol01   Bound    pvc-e96e2d73-fe46-4e53-831e-48342715ae72   10Gi ␣
↪     RWO            rbd-new        8h
site-router-redis-vol01        Bound    pvc-dab2fdc8-d8e1-461a-902b-7f76026a278a   20Gi ␣
↪     RWO            rbd-new        4d4h
sk-postgres-vol01              Bound    pvc-e304ca96-143a-41e6-901f-b61d14590972   20Gi ␣
↪     RWO            rbd-new        4d4h
systems-postgres-vol01         Bound    pvc-14e58e3b-876e-4598-b9c2-a31447d3b530   20Gi ␣
↪     RWO            rbd-new        8h
tenants-postgres-vol01         Bound    pvc-65b83d4e-24a4-42cb-ae9f-5fce41109d4a   20Gi ␣
↪     RWO            rbd-new        4d4h
vault-vol01                    Bound    pvc-f39851c4-e140-4634-a4e4-441a5b143fd6   10Gi ␣
```

(continues on next page)

(continued from previous page)

```
↪      RWO            rbd-new        4d4h
```

Note: The number of PVCs will depend on the site type being deployed.

Check the jobs:

```
kubectl get jobs
NAME              COMPLETIONS   DURATION   AGE
renew-sk-secret   1/1           4s         30m
sk-admin-init     1/1           19s        30m
sk-presetup       1/1           3s         30m
```

Check the pods:

```
kubectl get pods

NAME                                READY   STATUS             RESTARTS   AGE
renew-sk-secret-zz8lm               0/1     Completed          0          2m33s
site-router-api-784ddbbcc-c456m     1/2     CrashLoopBackOff   4          2m46s
sk-admin-init-gpnnq                 0/1     Completed          0          2m29s
sk-presetup-nk8ht                   0/1     Completed          0          2m32s
tapis-nginx-55d47656f8-tvhfk        1/1     Running            0          2m48s
vault-67b44ff777-vwphn              1/1     Running            0          2m45s
```

It is expected that the site-router will be in CrashLoopBackOff state; this will automatically resolve once the primary services are deployed in the next step.

Warning

Quickly check that the initialization step compelted and move onto the next step. You have about 10 minutes to deploy the primary services (the topic of the next section) after the initialization. This is because a short-lived token for the Vault database is generated in this step and used in the next step.

21.9 Deploy the Primary Tapis services

Next, deploy the primary Tapis services:

```
./burnup primary_services
```

Similarly to the messages discussed in the Tapis initialization section, it is quite normal to see some messages like

```
error: timed out waiting for the condition on jobs/authenticator-migrations
```

Condition timeouts can happen when it is taking longer for Kubernetes to complete the deloyment of dependent objects, but these should resolve in due time. It is also quite normal to see sets of pods where the first several are in `Error` state while the last one `Completed`, for example:

```
notifications-init-db-25dk8                0/1     Error              0          109s
notifications-init-db-gq4lt                0/1     Completed          0          97s
notifications-init-db-zqhvt                0/1     Error              0          107s
```

The errors above are normal and could be caused for different reasons, but all of them amount to essentially the same thing: one or more of the Kubernetes objects that the pod depends on where not ready when the pod was launched, do

the pod crashed, hence the `Error` final state. Kubernetes continued to start a new instance of the pod until it finally reached the `Completed` state when all of the dependent objects where ready.

It could could several minutes (10 or 20 even) for the deployment to converge. Check that eventually there are no pods in CrashLoopBackOff using:

```
kubectl get pods
```

Then, check that a few critical services are healthy using the verification scripts:

```
cd admin/verification
```

Check that the Security Kernel is health (your output should be simialr that below):

```
./sk-test
hello
{"result":"Hello from the Tapis Security Kernel.","status":"success","message":"TAPIS_
↪FOUND hello found: 0 items","version":"1.3.0","commit":"ee1b3342","build":"2023-03-
↪01T15:42:55Z","metadata":null}
ready
{"result":{"checkNum":1,"databaseAccess":true,"vaultAccess":true,"tenantsAccess":true},
↪"status":"success","message":"TAPIS_READY Readiness check received by Security Kernel.
↪","version":"1.3.0","commit":"ee1b3342","build":"2023-03-01T15:42:55Z","metadata":null}
healthcheck
{"result":{"checkNum":1,"databaseAccess":true,"vaultAccess":true,"tenantsAccess":true},
↪"status":"success","message":"TAPIS_HEALTHY Health check received by Security Kernel.",
↪"version":"1.3.0","commit":"ee1b3342","build":"2023-03-01T15:42:55Z","metadata":null}
```

Check that the Tenants service is healthy:

```
./tenants-test
{
  "message": "Tenants retrieved successfully.",
  "metadata": {},
  "result": [
    {
      "admin_user": "admin",
      "authenticator": "https://admin.test.tapis.io/v3/oauth2",
. . .
```

Check that the Tokens service is healthy:

```
./tokens-test
{"message":"Token generation successful.","metadata":{},"result":{"access_token" . . .
. . .
```

Check that the Authenticator service is healthy:

```
./authenticator-test
{"message":"Token created successfully.","metadata":{},"result":{"access_token": . . .
. . .
```

21.10 Deploy the Secondary Tapis Services

Finally, deploy the secondary Tapis services:

```
./burnup secondary_services
```

21.10.1 Bootstrapping an Initial Primary Site Deployment

21.10.2 Bootstrapping an Initial Associate Site Deployment

21.11 Configuring Support for GLOBUS

Inlcuding support for GLOBUS is optional.

In order for a primary or associate site to support Tapis systems of type GLOBUS, a Globus project must be created and registered. This yields a Globus client ID that must be configured as part of the Tapis environment. For more information on creating a Globus project, please see the Globus Auth Developer Guide. Each Tapis installation can be configured with it's own Globus client ID.

The resulting client ID must be set in the host_vars file using the field `systems_globus_client_id`. This field is referenced as part of the deployment for the Systems and Files services. This is done by adding lines similar to the following to the host_vars file:

```
# Globus client ID for systems and files
systems_globus_client_id: 868c331e-ab77-4321-bd12-9c85cb0f12aa
```

To use Globus, an end-user will create a system in Tapis and follow an authentication flow to register credentials for the system. The Tapis client application uses the Globus OAuth2 Native App flow to obtain the initial access and refresh tokens for the end-user. Globus's support for the PKCE protocol is used to perform a three-legged OAuth2 authorization code grant.

For more information, please see Systems Support For Globus.

21.12 Configuring Support for TMS

Inlcuding support for the Trusted Manager System (TMS) is optional.

In order for a primary or associate site to support TMS, the deployment file for the Systems service must be updated to set the following 5 enviornment variables.

```
- name: TAPIS_TMS_ENABLED
  value: "True"
- name: TAPIS_TMS_SERVER_URL
  value: https://localhost:3000
- name: TAPIS_TMS_TENANT
  value: default
- name: TAPIS_TMS_CLIENT_ID
  value: tapis
- name: TAPIS_TMS_CLIENT_SECRET
  value: **********************
```

Each Tapis installation can be configured with it's TMS settings. The service must be restarted after updating the deployment file.

For more information, please see Systems Support For TMS.

21.13 Configuring Support for Email Notifications

Including support for notifications by EMAIL is optional.

In order for the Tapis Notifications service to support delivery of notifications by EMAIL, the service deployment files must be updated to include parameters for an SMTP relay. Parameters for the relay are set as environment variables to be picked up by the dispatcher service when it is started during a deployment. Each Tapis installation can be configured with it's own SMTP relay.

For more information on Notifications EMAIL support and a full list of relevant environment variables, please see Notification Delivery.

Site specific values must be set in the host_vars file. Values for environment variables `TAPIS_MAIL_PROVIDER`, `TAPIS_SMTP_HOST` must be set. Typically a value for `TAPIS_SMTP_PORT` is also included. These fields are referenced as part of the deployment for the Notifications service. This is done by adding lines similar to the following to the host_vars file:

```
# notifications
notifications_mail_provider: SMTP
notifications_mail_host: relay.example.com
notifications_mail_port: 25
```

21.14 Advanced Configuration Options

- Replacing the Vault with an "external" Vault
- Customizing routing in Tapis proxy
- Configuring custom LDAP servers
- Adding custom (i.e., external) authenticators

CHAPTER

TWENTYTWO

TAPIS VAULT

Note

This guide is for users wanting to deploy Tapis software in their own datacenter. Researchers who simply want to make use of the Tapis APIs do not need to deploy any Tapis components and can ignore this guide.

22.1 Introduction to Tapis Vault

Tapis stores all its secrets in an instance of Hashicorp Vault. It is critical that access to Vault is tightly controlled and its secret content safeguarded. The only Tapis component that interactes directly with Vault is the Security Kernel and its associated utility programs.

When planning a Tapis installation, one should consider the tradeoff between automation and robust secret management. In highly automated environments like Kubernetes, services are automatically restarted when they fail or need to be moved between nodes. This level of automation requires writing at least one secret on some software accessible disk. On the other hand, Vault initialization is geared toward having a human in the loop to execute its unseal protocol and to protect high value tokens.

Tapis supports two levels of Vault automation. When running in a Kubernetes cluster, Vault's unseal keys and a long-lived token are written to disk on the control plane. Kubernetes accesses these secrets when it initializes a new Vault instance or it needs to restart Vault. These secrets are protected by the operating system's account and access control mechanisms; if those mechanisms are compromised, the Vault's contents are vulnerable.

Tapis also supports deploying Vault outside a Kubernetes cluster while the rest of Tapis runs in the cluster. In this configuration, a data center can deploy Vault to meet its local security policies and standards, including limiting administrative access to Vault, running Vault in a physically secure location, employing a hardware security module, etc.

In the following two sections we discuss how the community version of Vault can be run inside Kubernetes and run on a VM outside of Kubernetes. In both cases, Vault needs to be installed, certains capabilities need to be enabled, Tapis-specific policies and roles need to be created, and administrative processes need to be put in place. Other configurations, such as running enterprise Vault or sharing Vault with other applications, are not covered, but information from this discussion can be adapted to other configurations.

22.2 Deploying Vault in Kubernetes

Using Tapis Deployer, Tapis's top-level burnup script configures and initializes Vault early in the deployment process. The Vault burnup script executes from the Kubernetes control plane and performs these tasks:

- Configures Vault's network and site location

- Configures Vault storage
- Creates and initializes Vault (new installations only)
  - Creates *vault* file containing unseal keys and root token
- Creates *vault-token* file containing root token from *vault* file (if necessary)
- Pushes unseal keys and root token to Kubernetes secrets
- Starts the Vault pod and unseals it (if necessary)

The deployer scripts detect the *vault* file to determine whether Vault has been installed. This file along with the *vault-token* file are placed in user's home directory by default, but this can be changed.

At this point, a Vault docker image is running in a Kubernetes pod and deployment scripts have written the unseal keys and the root token to files. These secrets are also written to Kubernetes secrets to make them available to pods and jobs. Kubernetes has all the secret information needed to restart Vault whenever it detects a failure or needs to move the pod. This automation comes at the cost of having sensitive Vault secrets on disk in the Kubernetes control plane.

The next phase initializes the Vault with Tapis roles, policies and secrets using the SkAdmin utility program. SkAdmin secrets management capabilities is discussed in depth in its own topic, but here we'll focus on its role during Vault initialization.

SkAdmin connects to Vault with administrative permissions so that is can set up Tapis's standard policies and its administrative policies. SkAdmin also sets up Tapis's standard roles and its administrative roles.

Once SkAdmin completes setting up Tapis's roles and policies, Vault is ready to accept Tapis service and user secrets. See the secrets discussion for details.

22.3 Deploying Vault Outside of Kubernetes

The procedure below describes how to build a Vault VM on Linux, which we'll call "**tapis-vault**". This procedure can also be used as a template for building Vault instances in other environments and follows the same general outline as dicussed above for Kubernetes environments.

Important security characteristics of the VM installation approach are:

- No Vault tokens are ever stored on the VM's disk.
- No unseal keys are ever stored on VM's disk.

The reason not to store these secrets on the Vault machine is because even if the root user is compromised, Vault secrets are inaccessible unless Vault is unsealed and the attacker has a valid token.

That said, automation inside Kubernetes will typically require a scoped token to start the Security Kernel and that token will most likely be saved in the control plane.

We assume the installation uses the open source version of Hashicorp Vault, so we do not have access to Vault's enterprise features. Tapis has been tested with Vault v1.8.3 and we assume a version compatible with that is being used.

The procedure can be split into two phases. The first phase requires command line access to the Vault host as root. The second phase can be performed remotely using the Vault REST APIs and a root token.

22.3.1 PHASE I - Command Line Execution

Step 1 - Acquire A VM

Acquire a VM with a TLS certificate installed. We'll use the fictitious domain "**mydomain.com**" for illustrative purposes.

Step 2 - Install Hashicorp Vault

SSH into target VM as root. Follow installation instructions for your package manager to natively install Vault:

https://learn.hashicorp.com/tutorials/vault/getting-started-install?in=vault/getting-started

https://developer.hashicorp.com/vault/docs/install

Step 3 - Change Private Key Access

Now that Vault is installed, change the group of the private key to "vault" and allow group read. Here are example commands:

chgrp vault /etc/pki/tls/private/tapis-vault-key.20230403
chmod 640 /etc/pki/tls/private/tapis-vault-key.20230403

It's also a good idea to create /etc/pki/tls/certs/README.VAULT explaining the steps you took to customize your VM.

Step 4 - Configure Vault for RAFT Storage

Save the original /etc/vault.d/vault.hcl. Update /etc/vault.d/vault.hcl to use the RAFT backend. Here are contents of an example vault.hcl file that can provide a template for your configuration:

```
# Full configuration options can be found at https://www.vaultproject.io/docs/
↪configuration

ui = true

disable_mlock = true

cluster_addr  = "https://tapis-vault.mydomain.com:8201"
api_addr      = "https://tapis-vault.mydomain.com:8200"

storage "raft" {
    path = "/opt/vault/data"
    node_id = "node_1"
}

# HTTPS listener
listener "tcp" {
    address       = "0.0.0.0:8200"
    tls_cert_file = "/etc/pki/tls/certs/certchain.pem"
    tls_key_file  = "/etc/pki/tls/private/tapis-vault-key.20230403"
    tls_client_ca_file = "/etc/pki/tls/certs/certchain.pem"
}
```

Vault information about using the RAFT protocol can be found here.

Step 5 - Start Vault

systemctl enable vault.service
systemctl start vault.service
systemctl status vault.service

Test the installation (customize for your hostname):

export VAULT_ADDR=https://tapis-vault.mydomain.com:8200
vault status

Step 6 - Initialize Vault

vault operator init

Five *unseal keys* and the *root token* will be written to the screen. DO NOT SAVE THESE DATA PERMANENTLY ON THE FILE SYSTEM. Instead, copy the information off the screen and save them securely off the VM.

Step 7 - Unseal Vault The Vault requires 3 out of the 5 of the unseal keys to unseal. Issue the operator unseal call 3 times, each time using a different key.

vault operator unseal
vault status

Step 8 - Export Root Token To avoid saving the root token to the command history file:

export HISTCONTROL=ignorespace
export VAULT_TOKEN=xxx

where the command has a leading space and xxx is the token output by the above operator init command.

Step 9 - Enable Authn Methods and Secrets Engines

vault secrets enable -version=2 -path=secret kv
vault auth enable approle
vault auth enable userpass

Step 10 - Check Remote Access

Before logging off, test remote access by running a status command that will be used in Phase II. On the remote machine, export the root token.

To avoid saving the root token to the command history file:

export HISTCONTROL=ignorespace
export VAULT_TOKEN=xxx
curl -H "X-Vault-Token: $VAULT_TOKEN" https://tapis-vault.mydomain.com:8200/v1/sys/health | jq

Step 11 - Logoff VM (optional)

All further configuration will be performed from the remote machine.

22.3.2 PHASE II - Remote Commands

Step 12 - Create SK Roles

On the remote machine terminal, export the root VAULT_TOKEN as shown in Step 10. Clone the tapis-vault-vm git repo into the current directory.

git clone https://github.com/tapis-project/tapis-vault-vm.git
cd tapis-vault-vm

curl -X POST -H "X-Vault-Token: $VAULT_TOKEN" --data @roles/sk-role.json
https://tapis-vault.mydomain.com:8200/v1/auth/approle/role/sk

curl -X POST -H "X-Vault-Token: $VAULT_TOKEN" --data @roles/sk-admin-role.json
https://tapis-vault.mydomain.com:8200/v1/auth/approle/role/sk-admin

Step 13 - Test SK Roles (optional)

curl -s -X POST -H "X-Vault-Token: $VAULT_TOKEN"
https://tapis-vault.mydomain.com:8200/v1/auth/approle/role/sk/secret-id | jq

curl -X GET -H "X-Vault-Token: $VAULT_TOKEN"
https://tapis-vault.mydomain.com:8200/v1/auth/approle/role/sk/role-id | jq

Step 14 - Create Roles and Policies

The tapis-vault/CreatePolicies.sh script encapsulates basic policy and role creation needed for Tapis to function. See comments in the script for details, but basically the script requires:

- The current directory to be tapis-vault.
- The VAULT_TOKEN environment variable be set to a root token.
- The DNS name of the new Vault VM be provided on the command line.
- Requirements 1 and 2 where already set in the previous two steps, so an invocation of the script looks like this (but with your VM):

./CreatePolicies.sh tapis-vault.mydomain.com

Step 15 - View Roles (optional) Each of the roles referenced in CreatePolicies.sh should be returned.

curl -H "X-Vault-Token: $VAULT_TOKEN" https://tapis-vault.mydomain.com:8200/v1/auth/approle/role/sk | jq

curl -H "X-Vault-Token: $VAULT_TOKEN" https://tapis-vault.mydomain.com:8200/v1/auth/approle/role/sk-admin | jq

Step 16 - View Policies (optional)

Each of the policies listed in CreatePolicies.sh should be returned.

curl -s -H "X-Vault-Token: $VAULT_TOKEN" https://tapis-vault.mydomain.com:8200/v1/sys/policy | jq

curl -s -H "X-Vault-Token: $VAULT_TOKEN" https://tapis-vault.mydomain.com:8200/v1/sys/policy/tapis/sk-acl | jq

Step 17 - Create tapisroot Token

The tapisroot token is a root token that should be used instead of the original root token generated by Vault. It tapisroot gets compromised it can easily be revoked and replaced.

Create a file named tapisroot.json with the content:

```
{
    "display_name": "tapisroot",
    "policies": [ "root" ],
    "ttl": 0
}
```

Run this command:

curl -X POST -s -H "X-Vault-Token: $VAULT_TOKEN" --data @tapisroot.json
https://tapis-vault.mydomain.com:8200/v1/auth/token/create | jq

Save the returned "client_token" in a secure place, such as stache or wherever you saved the original root token and unseal keys.

Step 18 - Test tapisroot Token (optional)

To avoid saving the root token to the command history file:

export HISTCONTROL=ignorespace
export TAPIS_ROOT_TOKEN=xxx

curl -X GET -H "X-Vault-Token: $TAPIS_ROOT_TOKEN"
https://tapis-vault.mydomain.com:8200/v1/auth/approle/role/sk/role-id | jq

curl -s -X POST -H "X-Vault-Token: $TAPIS_ROOT_TOKEN"
https://tapis-vault.mydomain.com:8200/v1/auth/approle/role/sk/secret-id | jq

Step 19 - Remove Secrets from History

Remove any commands that leaked secrets into the history file. Enter "history" to see the numbered history records. To remove by line number:

history -d <line number>

22.4 Vault Backup

Tapis configures Vault to run with the raft storage type by default, which allows Vault to operate normally while its database is backed up. Vault provides these two administrative commands to save and restore backups:

- *vault operator raft snapshot save <outfile>*
- *vault operator raft snapshot restore <infile>*

Tapis fills the gap in Vault's community edition support by automating periodic backups in Vault VM environments. The tapis-vaultbackup repository contains a backup utility program's source code and documentation. The program can be started in a secure manner to periodically takes snapshots of the Vault database (once a day by default). The program runs as a daemon until it's shutdown. Typically, a separate cron job is set up to copy the backup files from the VM to one or more remote data stores as local policy dictates.

The program is written in Java and packaged as a self-contained executable. The executable is then packaged into an rpm for use on operating systems that support that package manager. There are no plans to support other package managers or container runtimes, but everything needed for such support is available in the repository.

22.5 Vault Export

The SkExport utility program provides a quick way to extract many Tapis secrets from Vault. The output is written to stdout as either JSON data or key/value pairs. One use of this program is to acquire Tapis service secrets and then to inject them into docker containers as environment variables. SkExport source code is part of the Security Kernel library and is available as a docker image.

SkExport parameters:

```
SkExport [options...]
 -format (--format) [JSON | ENV] : JSON writes raw Vault data, ENV writes key=value␣
↪(default: ENV)
 -help (--help)                  : display help information (default: false)
 -nosan (--nosanitize)           : don''t replace unsupported characters with underscore␣
↪when -format=ENV (default is to sanitize)
 -noskip (--noskipusersecrets)   : don''t skip user secrets (default is to skip)
 -quote (--quoteenv)             : enclose secret values in single quotes when -
↪format=ENV (default: false)
 -v (--verbose)                  : output statistics in addition to secrets (default no␣
↪statistics)
 -vtok (--vaulttoken) VAL        : Vault token with proper authorization
 -vurl (--vaulturl) VAL          : Vault URL including port, ex: http(s)://host:32342
```

22.5.1 Running SkExport

The easiest way to execute SkExport is to run its docker image. The *-vtok* and *-vurl* parameters are required. Here's an example of how to export the tapis service secrets (user and system secrets are skipped) in environment variable format with the values single quoted:

```
export SKEXPORT_PARMS='-quote -vtok xxxx -vurl https://tapis-vault.mydomain.com:8200'
docker run --env SKEXPORT_PARMS tapis/securityexport
```

This example outputs JSON data:

```
export SKEXPORT_PARMS='-format=JSON -vtok xxxx -vurl https://tapis-vault.mydomain.
↪com:8200'
docker run --env SKEXPORT_PARMS tapis/securityexport
```

Since a token with at least as much authorization as the Security Kernel's token must be used to extract secrets from Vault, and since secrets are being output in the clear, it's important to take proper security precautions when using SkExport. These precautions include not leaving tokens or secrets in files and deleting sensitive information from the command line history file.

22.6 Revoke / Regenerating Vault Root Tokens

Revoking and regenerating Vault Root Tokens is standard practice for Vault.

Thanks to these sources for providing details for the procedures.

- Identifying Active Hashicorp Vault Root Tokens
- Generate Root Token

Examples of UUIDs, tokens, passwords are randomized below.

Step 1 - Make a Backup

Ensure you have a good backup of your Vault before proceeding.

Step 2 - Generate New Root Token

Generate a one-time-password:

```
# vault operator generate-root -init
A One-Time-Password has been generated for you and is shown in the OTP field.
You will need this value to decode the resulting root token, so keep it safe.
Nonce         8af5de52-0da3-4896-ab9f-6f0f291f31dc
Started       true
Progress      0/3
Complete      false
OTP           199273364d410af3c520c9460d
OTP Length    26
```

Use the OTP & unseal keys to generate an Encoded Token. You must enter 3 different unseal keys:

```
vault operator generate-root

# vault operator generate-root
Operation nonce: 8af5de52-0da3-4896-ab9f-6f0f291f31dc
Unseal Key (will be hidden):
Nonce       8af5de52-0da3-4896-ab9f-6f0f291f31dc
Started     true
Progress    1/3
Complete    false

# vault operator generate-root
Operation nonce: 8af5de52-0da3-4896-ab9f-6f0f291f31dc
Unseal Key (will be hidden):
Nonce       8af5de52-0da3-4896-ab9f-6f0f291f31dc
Started     true
```

(continues on next page)

(continued from previous page)

```
Progress    2/3
Complete    false

# vault operator generate-root
Operation nonce: 8af5de52-0da3-4896-ab9f-6f0f291f31dc
Unseal Key (will be hidden):
Nonce            8af5de52-0da3-4896-ab9f-6f0f291f31dc
Started          true
Progress         3/3
Complete         true
Encoded Token    cd7dc17b468d1c4d0754446b7fe6008aXy1
```

Decode the encoded token using the OTP:

```
# vault operator generate-root -decode=cd7dc17b468d1c4d0754446b7fe6008aXy1 -
↪otp=199273364d410af3c520c9460d
s.4f31f8b4380afdf78a0c4f2b
```

Note the New Root Token (*s.4f31f8b4380afdf78a0c4f2b*).

Step 3 - Find and Revoke the Old Root Token

In the vault container, list Accessors. There may be several.:

```
# vault list auth/token/accessors
Keys
----
31e7d2f9d581f43758635169
0a8511c58fdfffa7edea60e9
aa1faa246e2e4346bc6364d3
8fea0ebb035d3ffdcc8a5e36
```

You can use this to look the information for each accessor.:

```
# vault list -format json auth/token/accessors | jq -r .[] | xargs -I '{}' vault token␣
↪lookup -format json -accessor '{}' | jq -r 'select(.data.policies | any(. == "root"))'
```

For each accessor look for their creation time to identify the one you want to revoke:

```
# vault token lookup -format json -accessor 0a8511c58fdfffa7edea60e9 | grep creation
    "creation_time": 1607701202,
    "creation_ttl": 0,

# vault token lookup -format json -accessor aa1faa246e2e4346bc6364d3 | grep creation
    "creation_time": 1684427946,

$ date -d @1607701202
Fri Dec 11 09:40:02 CST 2021

$ date -d @1684427946
Thu May 18 11:39:06 CDT 2023
```

We see that token *0a8511c58fdfffa7edea60e9* is the 2-year-old one so that's the one to revoke.

Revoke the old token:

```
# vault token revoke -accessor 0a8511c58fdfffa7edea60e9
Success! Revoked token (if it existed)
```

Confirm the old one no longer works:

```
# vault list -format json auth/token/accessors
Error listing auth/token/accessors: Error making API request.

URL: GET http://127.0.0.1:8200/v1/auth/token/accessors?list=true
Code: 403. Errors:
* permission denied
```

Verify the new one works:

```
# export VAULT_TOKEN=s.4f31f8b4380afdf78a0c4f2b
# vault list -format json auth/token/accessors
[
  "aa1faa246e2e4346bc6364d3",
  "31e7d2f9d581f43758635169",
  "8fea0ebb035d3ffdcc8a5e36"
]
```

Note the old one is missing.

Step 4 - Setup Tapis to Use the new Root Token

- Use burndown script to stop vault.
- Move the old {tapisdatadir}/vault/vault-token out of the way (e.g. *mv vault-token vault-token-20230627-backup*)
- Update vault-token file with new root token.
- Delete vault secrets so they will be recreated. - kubectl delete secret vault-keys - kubectl delete secret vault-token
- Use burnup script to start vault.
- Restart Security Api. (Go to {tapisdir}/security/api and use the burndown & burnup scripts.)
- Confirm that the sk and vault secrets are just recently recreated:

```
# kubectl get secret | grep vault
tapis-sk-vault-secrets              Opaque                                2      15s
vault-keys                          Opaque                                3      ␣
↪2m19s
vault-token                         Opaque                                1      ␣
↪2m20s
```

CHAPTER

TWENTYTHREE

MANAGING SECRETS WITH SKADMIN

Note

This guide is for users wanting to deploy Tapis software in their own datacenter. Researchers who simply want to make use of the Tapis APIs do not need to deploy any Tapis components and can ignore this guide.

23.1 Introduction to Tapis Secrets

Tapis stores all secrets that it uses or manages in Hashicorp Vault. These secrets include:

- Service passwords
- Database credentials
- Signing key pairs
- Credentials for user systems
- Arbitrary passwords and other secrets

The only Tapis runtime component that can access the Vault is the Security Kernal (SK). There is, however, sometimes a need to read, write or delete secrets outside of SK, such as when installing or updating Tapis, rotating keys, or when manual action can quickly solve a problem. The SkAdmin utility program provides such capabilities.

23.2 The SkAdmin Utility

The SkAdmin command line program manages secrets when Tapis is running or when it's offline or only partially running, such as during start up.

SkAdmin does the following:

- Creates or updates secrets in SK.
- Creates or updates secrets directly in Vault without going through SK.
- Merges or replaces Kubernetes secrets with one or more values from SK.
- Merges or replaces Kubernetes secrets with one or more values directly from Vault.
- Generates passwords and key pairs on demand.
- Provides summary and detail information about work performed on a run.

Secret creation is independent of all Tapis services when going directly to Vault. When used in this mode, the only dependencies are Vault and, possibly, Kubernetes. This allows SkAdmin to bootstrap all secrets needed by Tapis before any services run. Whether secrets are created by going through SK or by going directly to Vault, the same secret path naming conventions are used.

23.2.1 Packaging

SkAdmin is written in Java and resides in the *securitylib* repository. SkAdminParameters defines the supported command line parameters.

To run SkAdmin directly as a Java program, use the shaded JAR file, *shaded-securitylib.jar*. For example, to see the help message, one would issue the following command from a terminal in which Java 17+ is configured and the JAR file is present:

```
java -cp shaded-securitylib.jar edu.utexas.tacc.tapis.security.commands.SkAdmin -help
```

The most common way to run SkAdmin is using a docker container. Here's how to display the help message using docker:

```
docker run --rm --env SKADMIN_PARMS=-help tapis/securityadmin
```

The value assigned to the container's SKADMIN_PARMS environment variable is passed to SkAdmin on the command line. See this DockerFile if you're interested in how this is done. Here is the SkAdmin help information:

```
SkAdmin for creating and deploying secrets to Kubernetes.

SkAdmin [options...]

-b (-baseurl) <base sk or vault url> : SK: http(s)://host/v3, Vault: http(s)://
↪host:32342)
-c (-create)                         : create secrets that don't already exist (default:␣
↪false)
-dm (-deployMerge)                   : deploy secrets to kubernetes, merge with existing␣
↪(default: false)
-dr (-deployReplace)                 : deploy secrets to kubernetes, replace any␣
↪existing (default: false)
-help (--help)                       : display help information (default: true)
-i (-input) <file path>              : the json input file or folder
-j (-jwtenv) VAL                     : JWT environment variable name
-kn (-kubeNS) VAL                    : kubernetes namespace to be accessed
-kssl                                : validate SSL connection to kubernetes (default:␣
↪false)
-kt (-kubeToken) VAL                 : kubernetes access token environment variable name
-ku (-kubeUrl) VAL                   : kubernetes API server URL
-o (-output) VAL                     : 'text' (default), 'json' or 'yaml' (default: text)
-passwordlen N                       : number of random bytes in generated passwords␣
↪(default: 16)
-u (-update)                         : create new secrets and update existing ones␣
↪(default: false)
-vr (-vaultRole) VAL                 : vault role-id
-vs (-vaultSecret) VAL               : vault secret-id


Use either the -c or -u option to change secrets in Vault. Use the -dm
```

(continues on next page)

(continued from previous page)

```
or -dr option to deploy secrets to Kubernetes.

Access to Vault secrets is always required. Use either the -j option
to access the secrets using the Security Kernel or the {-vr, -vs} options
to accress the secrets by going directly to Vault. Set the baseurl to
match the access method.  Set the {-kn, -kt, -ku} options when deploying
secrets to Kubernetes.
```

We'll use the containerized version of SkAdmin for the rest of this discussion.

23.2.2 Launching SkAdmin

To manage secrets, SkAdmin requires both of these parameters:

- -i (-input) - JSON file or directory containing one more JSON files that conform to the SkAdminInput.json schema.
- -b (-baseurl) - the SK or Vault server url.

And at least one of these *action* parameters:

- -c (-create) - create secrets only if they don't already exist in Vault.
- -u (-update) - write secrets to Vault even if they already exist.
- -dm (-deployMerge) - write the specified key/value pairs to Kubernetes secrets, merging with unspecified key/value pairs that may exist in any secret.
- -dr (-deplyReplace) - write the specified key/value pairs to Kubernetes secrets, completely replacing any secrets that may exist.

The *-c* and *-u* parameters are mutually exclusive, as are the *-dm* and *-dr* parameters. The *-c* option will never overwrite a secret that already exists in Vault, so it is non-destructive. If the secret doesn't exist, it will be created per the input file specification. On the other hand, while the *-u* option also creates secrets if they don't already exist in Vault, it will overwrite existing secrets according to the input file specification. The *-u* option can be destructive.

The *-dm* option deploys secrets from Vault to Kubernetes secrets in an additive manner. A Kubernetes secret can contain any number of key/value pairs. The *-dm* option preserves existing key/value pairs and adds any new ones that exist in Vault; it therefore is non-destructive. The *-dr* option, on the other hand, will replace all key/value pairs in a Kubernetes secret with the key/value pairs in Vault that are associated with the secret. The *-dr* option can be destructive.

Vault Parameters

Both of the following parameters are required when accessing Vault directly. Note that these parameters are mutually exclusive with the SK Parameters.

- -vr (-vaultRole) - the Vault role-id used to be acquire an authorized Vault token.
- -vs (-vaultSecret) - the Vault secret-id used to be acquire an authorized Vault token.

The role-id is the one assigned to the Security Kernel. The secret-id is a short-lived secret, usually with a 10 minute TTL, that can be thought of as a temporary password associated with the role-id. Details on how to obtain these values is beyond the scope of this discussion, but tapis-deployer's renew-sk-secret-script provides an example implementation.

SK Parameters

If transactions are going to occur using SK rather than going directly to Vault, the following parameter is required. Note that this parameter is mutually exclusive with all Vault Parameters.

- -j (-jwtenv) - the environment variable name whose value is a JWT authorized to access SK.

The JWT is usually a Security Kernel service JWT.

Kubernetes Parameters

Kubernetes parameters are only required if a deployment to Kubernetes secrets has been specified (*-dm* or *-dr*). All the following parameters are required to access Kubernetes.

- -kt (-kubeToken) - the environment variable name whose value contains an authorized Kubernetes token.
- -ku (-kubeUrl) - the URL to the Kubernetes API server.
- -kn (-kubeNS) - the Kubernetes namespace to access.

SkAdmin uses these values to call the Kubernetes secrets API.

General Parameters

These parameters are optional and have default values.

- -o (-output) - output format can be one of text, json, or yaml. The default is text.
- -passwordlen - the length of generated passwords. The default is 16.
- -help (–help) - show the SkAdmin help message (no value necessary).

SkAdmin Inputs

SkAdmin takes JSON input that conforms to the SkAdminInput.json schema. The required *-i* parameter can name a single JSON file or a directory that contains any number of JSON files. When a directory is specified, all JSON files that are immediate children of the directory are loaded and merged into a single set of secrets to be processed.

SkAdmin supports the following Security Kernel secret types:

- dbcredential - database credentials used by services
- servicepwd - the password used by services to obtain their JWTs
- jwtsigning - the asymmetric key pair used to sign and validate Tapis JWTs
- jwtpublic - the public part of a jwtsigning key pair
- user - arbitrary secret information associcated with a user

A short discussion on secret types can be found in the Security Kernel secrets section. The input files used to generate Tapis's initial set of secrets are in the initialLoad directory of tapis-deployer.

The values of passwords and keys fields can be specified using the distinguished value "<generate-secret>". SkAdmin will generate random passwords or asymmetric key pairs as required. The value of key fields can also be specified as "file:pathToPEMFile" where *pathToPEMFile* is a path, usually an absolute path, to a PEM file containing a public or private key. Alternatively, key values can be provided inline in the input json files, in which case they are required to be in PEM format.

When SkAdmin is directed to generate a new key pair, both the public and private parts are saved in SK, but only the private part is deployed to Kubernetes using JwtSigning input. To deploy the public key to Kubernetes, use separate JwtPublic input stanzas for each public key.

JwtPublic stanza can also be used independently of whether a key pair resides in SK. If the optional publicKey value is provided in the JwtPublic input stanza, then that value will be used without consulting SK. In addition, if that value starts with the "file:" string, the publicKey will be assigned the contents of the specified file.

23.2.3 Execution

SkAdmin execution consists of the following steps:

1. **Validate parameters** - Validate that a complete, non-conflicting set of parameters have be specified. The API used to perform all I/O on the secrets database will be either the SK or Vault interface.
2. **Load secret specifications** - The file or directory referenced by the *-input* parameter is loaded. In the directory case, the JSON files in the directory are loaded in alphabetic order. Each of the five secret types listed in the previous section are aggregated if there is more than one file.
3. **Validate secret specifications** - All five types of secrets are validated. If a value of *<generate-secret>* is encountered, then a password or key pair is generated depending on context. Any invalid input causes the program to abort.
4. **Create or update secrets** - If the *-create* parameter is set, then write the secrets to the Vault database only if they don't already exist there. If the *-update* parameter is set, always write the secrets to the Vault database.
5. **Deploy secrets** - If the *-deployMerge* parameter is specified, then any new secret keys that do not already exist in Kubernetes secrets will be deployed. If the *-deployReplace* parameter is specified, then all input secrets will be deployed to Kubernetes overwriting any existing secrets with the same name.
6. **Report results** - SkAdmin writes a report with both summary and detail information to stdout in the format specified by the *-output* parameter.

Result Reporting

An accounting of what actions were performed is printed to stdout when the program completes. Summary information include counts of secrets processed. Detailed information includes an outcome message for each secret action that includes success, failure and skipped outcomes. The default result format is text, but json and yaml can also be specified.

Creating Secrets with SkAdmin

SkAdmin is an integral part of Tapis deployment processing: It initializes Vault with Tapis roles and permissions, it creates secrets and, in Kubernetes environments, it deploys secrets to Kubernetes for subsequent injection into containers.

In an automated Kubernetes environment, SkAdmin would run in a pod, which complicates how values get passed to SkAdmin. Consult the top-level deployer burnup script to see when SkAdmin gets called during Tapis deployment and skadmin/burnup to see how parameters are passed into the SkAdmin pod.

For this discussion, however, we'll show how to create secrets by running an SkAdmin docker container directly. In the first scenario, we'll go to your deployment's *skadmin/initialLoad* directory, which contains the deployment's initial secret specifications. Here's how we invoke SkAdmin to only create the secrets that do not already exist in Vault:

```
cd $TAPIS_DIR/tapis-kube/skadmin/initialLoad
docker run -e SKADMIN_PARMS="-c -i /initialLoad -vr <VAULT_ROLEID> -vs <VAULT_SECRETID> -
↪b http://vault:8200" --mount type=bind,source=`pwd`,target=/initialLoad --rm tapis/
↪securityadmin
```

Going from left to right, we see that all the SkAdmin command line parameters are assigned to the SKADMIN_PARMS environment variable. In this case, SkAdmin is being asked to create secrets only if they don't already exist. It will read the JSON files in the */initialLoad* directory inside the container.

SkAdmin will communicate directly with Vault using values represented by <VAULT_ROLEID> and <VAULT_SECRETID> and the given Vault network location. Details on how to obtain these values is beyond the scope of this discussion, but tapis-deployer's renew-sk-secret-script provides an example implementation.

We bind mount the host's current directory into the */initialLoad* directory in the container. Any host directory that contains SkAdmin JSON input files could serve as the source directory; any target directory in the container that's referenced by the *-i* parameter could serve as the destination. We use the *–rm* flag to instruct docker to remove the container (and any secrets it contains) after execution.

Creating and Deploying Secrets with SkAdmin

If we add Kubernetes parameters to the command line shown in the previous section, we'll be able to both create secrets and deploy them to Kubernetes. The changes begin at the *-dm* parameter:

```
cd $TAPIS_DIR/tapis-kube/skadmin/initialLoad
docker run -e SKADMIN_PARMS="-c -i /initialLoad -vr <VAULT_ROLEID> -vs <VAULT_SECRETID> -
↪b http://vault:8200" --mount type=bind,source=`pwd`,target=/initialLoad --rm -dm -kt
↪<KUBE_TOKEN> -kn <KUBE_NAMESPACE> -ku https://kubernetes.default.svc.cluster.local␣
↪tapis/securityadmin
```

The *-dm* parameter instructs SkAdmin to merge the input secrets into Kubernetes secrets. The merge will not change any secret value already in Kubernetes. The <KUBE_TOKEN> and <KUBE_NAMESPACE> values allow access to the Kubernetes cluster running at the URL specified by the *-ku* parameter.

Updating Secrets with SkAdmin

It's common to add a secret after Tapis has already been deployed, such as when a new service is added to a site. The deployed SkAdmin directory contains an updateSecrets subdirectory that makes it easy make incremental changes to secrets in Vault and Kubernetes. The *updateSecrets/README* file contents explain how to do this:

```
Adding New Secrets
==================

This directory contains configuration and script files that allow new secrets to be␣
↪imported into SK and Kubernetes.  The general approach follows that of the initial␣
↪secrets loading process in the parent directory.  The main difference is that a␣
↪staging directory contains one or more SkAdmin input files specified for a particular␣
↪run.

Usage is as follows:

   1. Put one or more SkAdmin input json files into the ./updateFiles directory.
   2. ./burnup
   3. Remove your json files from the staging directory

Common use cases for this facility include creating secrets for new services, generating␣
↪new secrets for an existing service after deleting that service's secrets in SK, and␣
↪restoring secrets in Kubernetes that exist in SK.

This facility assumes that SK was previously configured and initialized with secrets by␣
↪running the burnup script in the parent directory.
```

Replacing Secrets

For safety, the process described in the above README calls SkAdmin with the create (*-c*) parameter just like in the two previous examples. This means that the procedure is non-destructive to secrets in Vault and Kubernetes. It also means the procedure cannot be used to replace existing secret values.

To replace an existing secret with a new one, follow these steps:

1. Use SK listSecret and readSecret APIs to understand what secrets currently exist in SK. Use *kubect get secret* calls to see what secrets are currently in Kubernetes and how SK maps them. When new to this process, consider temporarily saving the secrets you plan to replace off to the side.
2. Run an SkAdmin container from the command line using the update (*-u*) and/or deployReplace (*-dr*) parameters and one input file at a time to limit potential loss. SK saves up to 10 previous versions of a secret, so secrets are recoverable up to that limit.
3. Validate that everything went as expected and then delete any temporarily saved old secrets.

CHAPTER

TWENTYFOUR

TAPIS ADMINISTRATION

Note

This guide is for users wanting to deploy Tapis software in their own datacenter. Researchers who simply want to make use of the Tapis APIs do not need to deploy any Tapis components and can ignore this guide.

In this section, we cover common administration tasks that arise when operating Tapis software.

This section is under construction...

24.1 Monitoring Tapis

We recommend two approaches for monitoring the health and status of your Tapis installation:

1. Tapis Status Page/Dashboard: A simple web application that runs HTTP requests on a configurable time interval and checks the basic health of each service. The dashboard provides an overview page with green/red statuses allowing an administrator to get a quick look at the health of all services.
2. Monitoring Tests: A configurable suite of HTTP-based integration tests that can run on a time interval and perform more elaborate verification. For example, monitoring tests might ensure that Tapis functions can execute, small files can be transferred, and "hello world" jobs can be launched on specific systems.

While the needs for both monitorin items tend to be site-specific, the Tapis projects provides solutions for both 1) and 2) as open source repositories available to the community.

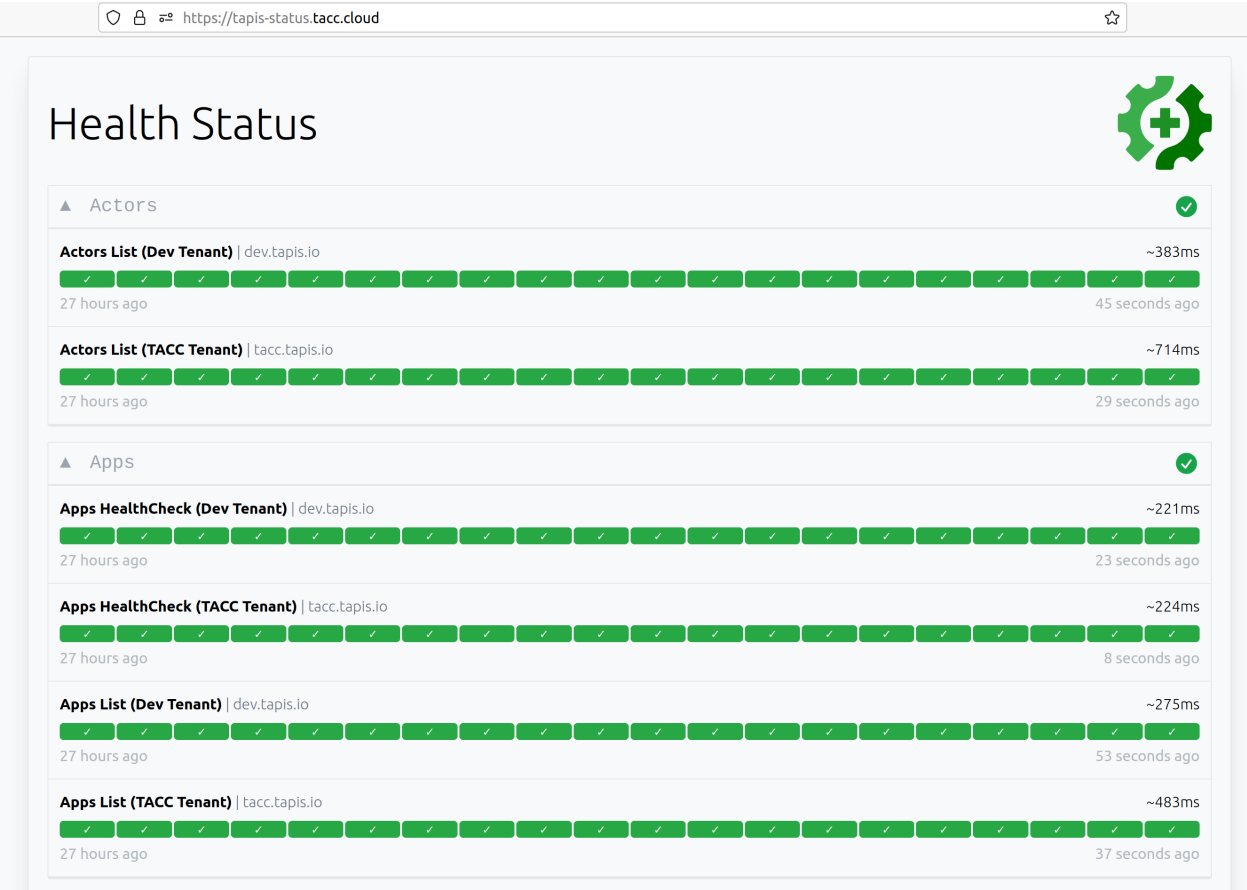
24.1.1 Tapis Status Page

A configurable Tapis Status application based on the Gatus project is available on GitHub; see the admin repository. We recommend deploying the status application outside of the network where Tapis itself is running so that any network issues will be detected. The status page for the TACC Tapis installation is hosted on AWS at https://tapis-status.tacc.cloud. Here is a screen shot of the status application:

Please see the README in the link included above for detailed instructions on configuring and deploying the Tapis status application.

24.1.2 Tapis Monitoring Tests

A configurable suite of tests is available on GitHub; see the tapis-tests repository. This repository contains test suites that can be used to validate a Tapis installation. There are two types of tests included in each of the subdirectories in the repository. The `smoke_tests` folder containers basic checks of the functionality of each service. The tests are written in BASH and utilize the curl HTTP client. There is significant overlap between these tests and the Tapis Status application, mentioned above, and in many cases using only one or the other is likely to be sufficient. Some projects may prefer the BASH/curl approach of the smoke tests to the Status App.


https://tapis-status.tacc.cloud
Health Status
Actors
Actors List (Dev Tenant) | dev.tapis.io
~383ms
27 hours ago
45 seconds ago
Actors List (TACC Tenant) | tacc.tapis.io
~714ms
27 hours ago
29 seconds ago
Apps
Apps HealthCheck (Dev Tenant) | dev.tapis.io
~221ms
27 hours ago
23 seconds ago
Apps HealthCheck (TACC Tenant) | tacc.tapis.io
~224ms
27 hours ago
8 seconds ago
Apps List (Dev Tenant) | dev.tapis.io
~275ms
27 hours ago
53 seconds ago
Apps List (TACC Tenant) | tacc.tapis.io
~483ms
27 hours ago
37 seconds ago

The other type of test included is contained in the `integration_tests` folder. These tests make more elaborate checks of the Tapis functionality than that checked by the smoke tests, including submitting jobs and monitoring that the jobs complete. These tests will require some configuration unique to each site (for example, an execution system to use to run the jobs). Also, unlike the smoke tests, the integration tests use Python and the Tapis Python SDK (tapipy). Please see the README in the repository for more details.

24.2 Troubleshooting Tapis Pods

Troubleshooting Tapis pods can be challenging. In this section, we provide some initial steps administrators can take to uncover the root cause of issues, but the core Tapis team is always available to help institutions troubleshoot problems with their Tapis installation.

Step 1. Running the verification scripts.

Step 2. Determine which Tapis pod might be causing the problem.

Step 3. Looking at the logs of a specific Tapis pod.

24.3 Backing Up Tapis

For any production deployment of Tapis, administrators should arrange for regular backups of all databases to occur. The frequency of the backup procedure, where to store backup files and how long to store them, among other aspects, are highly dependent on the requirements of the site. Nevertheless, all sites have one thing in common: backups are required to prevent catastrophic data loss.

Tapis uses a number of different database technologies, including InfluxDB, MySQL, MongoDB, Postgres, Redis, and Vault. Each technology requires its own backup and restoration procedure.

Except for Vault, which is treated separately in its own section, and MongoDB, for which we provide a dedicated GitHub repository due to its complexity, all of the database backup procedures follow a common pattern:

1. Fetch necessary secrets out of Kubernetes for direct access to the database.
2. Exec into the database container and execute a backup command to produce a dump file.
3. Save the dump file to a secure place.

At TACC, for 3) we have opted to use an AWS S3 bucket.

Below we provide templates of the backup command we use for each technology.

- InfluxDB (only used by Streams API): `influxd backup -portable /path/to/file`
- MySQL (only used by Streams and Workflows API): `mysqldump --all-databases > /path/to/file`
- MongoDB (only used by Meta and Actors API): See the GitHub repository tapis/mongodb-backup
- Postgres (used by Authenticator, Apps, Files, Jobs, Notifications, PgREST, Pods, SK, Systems, and Tenants API): `pg_dump --dbname={service_db} > /path/to/file`
- Vault (only used by SK API): See the Vault Backup section.

Note

Be aware that Deployer manipulates and stores Vault keys and a Vault root token in files. Please refer to the Vault guidelines on the safekeeping and handling of these. If starting the Vault with the Deployer "burnup" script, in the absence of the file containing the Vault keys, you will be prompted for Vault keys. However, a file containing a Vault root token will be required.

CHAPTER

TWENTYFIVE

UPGRADING TAPIS

Note

This guide is for users wanting to deploy Tapis software in their own datacenter. Researchers who simply want to make use of the Tapis APIs do not need to deploy any Tapis components and can ignore this guide.

In this section, we cover the process of upgrading an existing Tapis installation. In general, the process of upgrading a Tapis installation involves all of the same steps described in the Tapis Deployer section, but a careful comparison of the new version of the generated Tapis deployment script directory against the previous version must be done to verify that the changes will work in your environment. At a high level, the process is:

1. Check out the updated version of Tapis Deployer from the repository.
2. Check the Deployer CHANGELOG to compare the current version deployed in your site against the newer version. Breaking changes are identified explicitly as are newly required configuration variables.
3. Run the generate script to generate an updated version of the deployment script directory and compare the difference between this version and the currently deployed version of the deployment script directory. Note and review any differences.
5. Use the deployment control scripts to start/update the newer versions of the Tapis services in your pre-production environment first. Perform QA in the pre-production environment.
6. Deploy to production.

25.1 Breaking Deployment Changes

In this section, we call out some specific types of breaking changes to be aware of. This section does not discuss breaking code changes within the Tapis services themselves, but rather changes within a deployment configuration that could break an existing installation. Also, this list is not meant to be a comprehensive – operators should always review the entire list of changes in the Deployer and Tapis CHANGLOGs.

25.1.1 Vault Storage Type

There are two possible storage types for the Vault database – *file* and *Raft*. Early versions of Deployer defaulted to using "file" storage for Vault, however, this option proved to be suboptimal for different reasons (e.g., less amenable to automated backups). Newer versions of Deployer use the *Raft* storage type, and we recommend existing deployments that are using *file* to change to *Raft*. However, changing from *file* to *Raft* requires a special, manual process to be performed. We outline the steps involved in the Tapis Vault section.

The Tapis Deployer variable to look for is `vault_raft_storage`, a boolean, with default value *true*. If you need to use the *file* storage type, be sure to override the default in your host vars.

Warning

Attempting to deploy the Tapis Vault with a different storage type could result in secret loss and permanent corruption of the Tapis installation.

25.1.2 PVC Names

Tapis uses Persistent Volument Claims (PVCs) to persist data stored in data bases and message brokers. Each Tapis service has a set of these PVCs that it uses. You can see the set of PVCs in your deployment by using `kubectl`:

```
kubectl get pvc

NAME                            STATUS   VOLUME                                     ␣
↪CAPACITY   ACCESS MODES   STORAGECLASS   AGE
actors-mongo-backup-vol01       Bound    pvc-fbb44e18-0256-4d0b-b799-a703b0f477b6   10Gi ␣
↪     RWO            rbd-new        2d1h
actors-mongo-vol01              Bound    pvc-d3c224eb-5930-4700-8b8a-5f1ae0f2a921   40Gi ␣
↪     RWO            rbd-new        2d1h
actors-rabbitmq-vol01           Bound    pvc-20dc6e04-e2bb-48b6-8647-ee28081eb0c3   20Gi ␣
↪     RWO            rbd-new        47h
apps-postgres-vol01             Bound    pvc-f1320b94-6069-41d3-a26a-91a3ebaaed21   20Gi ␣
↪     RWO            rbd-new        2d1h
authenticator-ldap-vol01        Bound    pvc-05ae6dbb-e46d-4383-af6b-f1d2726d6529   10Gi ␣
↪     RWO            rbd-new        5d21h
. . .
```

The output above shows the name of each PVC in the first column, e.g., `actors-mongo-backup-vol01`, `apps-postgres-vol01`, `authenticator-ldap-vol01`, etc. It is very important to check that the names of the PVCs specified within the deployment script directory match those within your cluster.

Warning

Deploying Tapis services with different PVC names could result in data loss.

CHAPTER

TWENTYSIX

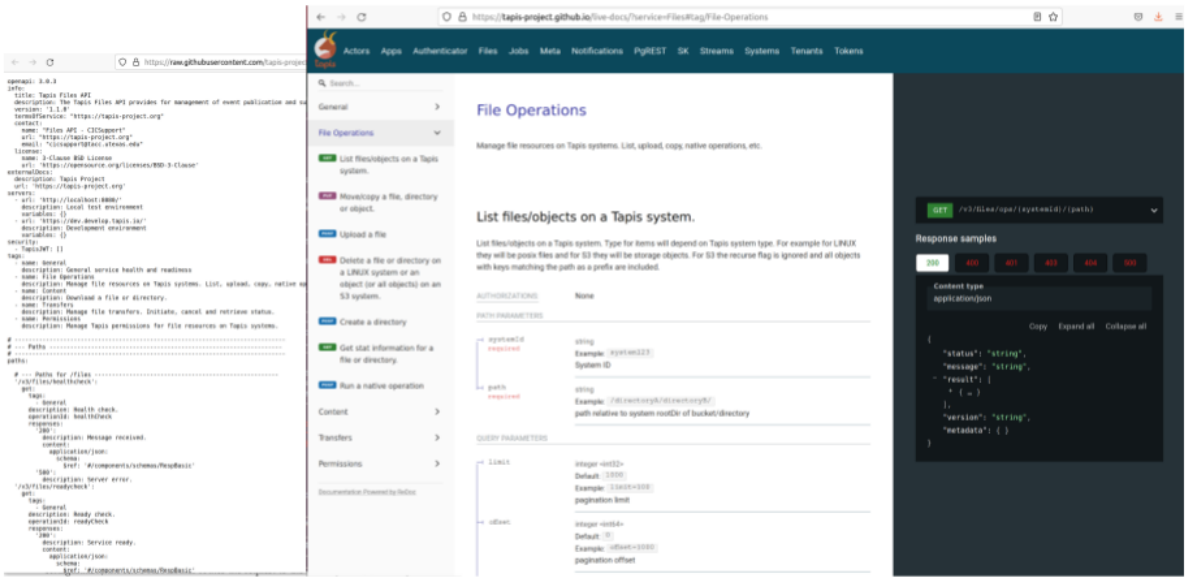
TAPIS ARTIFACTS & ARCHITECTURE

In this section we provide an overview of the Tapis system architecture and a reference to all Tapis artifacts and code repositories.

26.1 API Specification

Tapis consists of independent services communicating with each other via the HTTP protocol. Each Tapis service specifies its API contract using Open API v3:

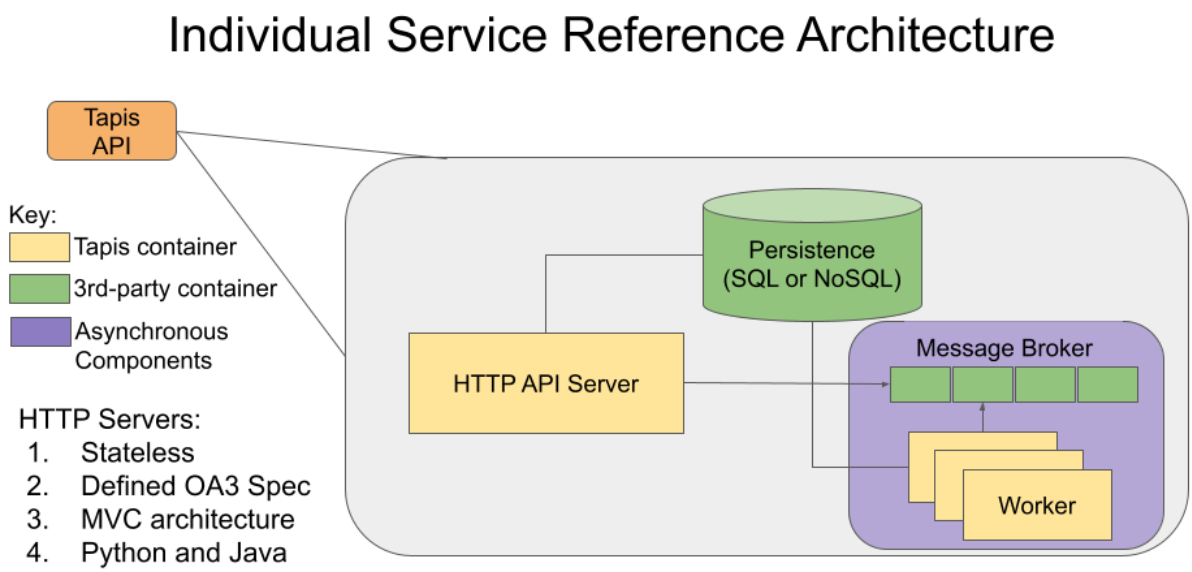
A complete listing of the API contracts for all Tapis services is available from the Tapis Live Docs website.



All Tapis services as well as the official tooling, including the Tapis CLI and Python SDK, make use of the service API specifications internally.

26.2 Micro-Service Architecture

While each Tapis service is developed independently, for the most part, services adhere to the following reference architecture:



26.3 Artifacts and Code Repositories

Below we list the primary Tapis service code repositories and their associate container images.

This section is a work in progress…

Service	Primary Repository	Additional Repositories	Container Image(s)
Apps	https://github.com/tapis-project/tapis-apps	https://github.com/tapis-project/tapis-shared-java	https://hub.docker.com/r/tapis/apps
Authenticator	https://github.com/tapis-project/authenticator	**https://github.com/tapis-project/tapipy** https://github.com/tapis-project/tapipy-tapisservice	https://hub.docker.com/r/tapis/authenticator https://hub.docker.com/r/tapis/authenticator-migrations
Tenants	https://github.com/tapis-project/tenants-api	**https://github.com/tapis-project/tapipy** https://github.com/tapis-project/tapipy-tapisservice	**https://hub.docker.com/r/tapis/tenants-api** https://hub.docker.com/r/tapis/tenants-api-migrations
Tokens	https://github.com/tapis-project/tokens-api	**https://github.com/tapis-project/tapipy** https://github.com/tapis-project/tapipy-tapisservice	https://hub.docker.com/r/tapis/tokens-api